

My Tool Box

Multi-Agent Reinforcement Learning Environments,
Algorithms and Research Utilities

User Manual, version 1.2

Dongming Wang

September 27, 2026

Contents

1	Introduction	1
1.1	What the package is built for	1
1.2	What the package provides	2
1.3	Design principles	5
1.4	Package layout	6
1.5	Conventions used in this manual	7
1.6	How this manual is organised	7
1.7	Licence and citation	8
2	Installation	9
2.1	Requirements	9
2.2	Installing from the repository	9
2.2.1	Optional extras	10
2.2.2	PyTorch	10
2.2.3	PettingZoo	10
2.2.4	Tk	11
2.3	Development installation	11
2.4	Verifying the installation	11
2.5	Optional dependency matrix	12
2.6	Upgrading and uninstalling	12
2.7	Building this manual	12
I	Environments	14
3	Environment Library Overview	15
3.1	Environments at a glance	15
3.2	Package structure	15
3.3	Creating environments	17
3.4	Registry and metadata	18
3.5	Common API conventions	19

3.5.1	Gymnasium interface	19
3.5.2	Joint multi-agent spaces	19
3.5.3	Info dictionaries	20
3.5.4	Termination and truncation	20
3.5.5	Seeding and reproducibility	21
3.5.6	Reset options	21
3.6	Vectorised simulation and the workflow layer	22
3.7	Graph utilities	23
3.8	Rendering	24
3.9	Recording episodes	25
3.10	Registered environments	26
4	Kuramoto Oscillator Networks	28
4.1	Model	28
4.2	Coupling modes and topologies	29
4.3	Spaces	30
4.4	Reward and episode end	30
4.5	Parameters	31
4.6	Info dictionary	33
4.7	PyTorch backend	33
4.8	Rendering	34
4.9	Registered ids	34
4.10	Usage	34
4.11	Native vector environment	36
4.12	Reproducibility and performance	36
4.13	Changes in 1.0	37
5	Networked Communication Environments	38
5.1	LineMsg	38
5.1.1	Model	38
5.1.2	Spaces	39
5.1.3	Parameters and info	39
5.1.4	Rendering	40
5.1.5	Usage	40
5.2	WirelessComm	41
5.2.1	Model	41
5.2.2	Spaces	42
5.2.3	Parameters and info	42
5.2.4	Rendering	42

5.2.5	Usage	43
5.3	Registered ids	44
5.4	Reproducibility and performance	44
5.5	Changes in 1.0	44
6	Pistonball	46
6.1	Arena and physics	46
6.2	Spaces	46
6.3	Reward and episode end	47
6.4	Parameters	48
6.5	Keyboard control	48
6.6	Rendering	49
6.7	Registered id	50
6.8	Usage	50
6.9	Reproducibility and performance	50
6.10	Changes in 1.0	51
7	Consensus and Formation Control	52
7.1	Model	52
7.2	Spaces	53
7.3	Reward and episode end	54
7.4	Parameters	54
7.5	Info dictionary and properties	55
7.6	Baseline controller	56
7.7	Rendering	56
7.8	Registered ids	57
7.9	Usage	57
7.10	Native vector environment	58
7.11	Reproducibility and performance	59
8	Power Grid Frequency Control	60
8.1	Model	60
8.2	Spaces	62
8.3	Reward and episode end	62
8.4	Parameters	63
8.5	Reset options, info dictionary and properties	64
8.6	Baseline controller	65
8.7	Vector environment	66
8.8	Rendering	66

8.9	Usage	67
8.10	Background	68
9	Vehicle Platoon Control	69
9.1	Model	69
9.2	Communication topologies	70
9.3	Spaces	71
9.4	Reward and episode end	71
9.5	Parameters	72
9.6	Info dictionary and properties	73
9.7	Baseline controllers and string stability	73
9.8	Vector environment	75
9.9	Rendering	75
9.10	Usage	76
9.11	Reproducibility	77
10	AJLATT: Active Joint Localisation and Target Tracking	78
10.1	Scenario and motion model	78
10.2	Sensing, communication and estimation	79
10.3	Spaces	80
10.4	Reward and episode end	80
10.5	Configuration	82
10.6	Target motion	84
10.7	Info dictionary and episode statistics	85
10.8	Team reward wrapper and legacy factory	85
10.9	Maps	86
10.9.1	Bundled maps	86
10.9.2	Map files	86
10.9.3	Map API	87
10.9.4	Building maps	88
10.9.5	Plotting maps	89
10.10	Rendering	89
10.11	Registered id	90
10.12	Usage	90
10.13	Reproducibility and performance	91
10.14	Changes in 1.0	92

11 Vectorised Simulation, Adapters, Baselines and Evaluation	93
11.1 Vector environments	93
11.2 Discovering environments	95
11.3 The <code>env-lib</code> command line	96
11.4 Wrappers and adapters	97
11.5 Baseline controllers	99
11.6 Evaluation and rollouts	99
11.7 A complete workflow	102
 II Algorithms	 103
12 Multi-Agent Reinforcement Learning Algorithms	104
12.1 Overview	104
12.2 Quick start	105
12.2.1 Training and evaluating from Python	105
12.2.2 The <code>marl-train</code> command line	108
12.3 Using the algorithms as baselines	110
12.3.1 In one call	110
12.3.2 From the command line	111
12.3.3 Recipes	112
12.4 Design of the core	112
12.4.1 The per-agent view: <code>MultiAgentSpec</code>	113
12.4.2 Collecting experience: <code>VectorRunner</code> and <code>Transition</code>	114
12.4.3 Buffers and generalised advantage estimation	114
12.4.4 Normalisation	115
12.4.5 Networks	116
12.4.6 Base classes and training loops	117
12.5 PPO: IPPO and MAPPO	119
12.6 MADDPG and MATD3	121
12.7 IQL, VDN and QMIX	123
12.8 Presets and results	124
12.9 The demonstration script	128
12.10 Adding an algorithm	129
12.11 References	132
 III Toolkit	 133
13 Toolkit Overview and Parameter Management	134

13.1	Structure of the toolkit	134
13.2	parakit: managing argparse parameters	135
13.2.1	Quick start	135
13.2.2	Headless API	135
13.2.3	File format	136
13.2.4	Parsing rules	137
13.2.5	Recommended workflow in training scripts	138
13.2.6	The ParameterTuner window	138
14	plotkit: Publication-Quality Plots	141
14.1	Overview	141
14.2	Input rules	142
14.3	Common parameters	143
14.4	Curves with uncertainty bands	143
14.4.1	plot_shadow_curve	143
14.4.2	plot_learning_curves	145
14.5	Lines, scatter plots, bars and histograms	145
14.5.1	plot_bar	145
14.5.2	plot_histogram	145
14.6	Heatmaps	145
14.7	Styles	146
14.8	Colour palettes	148
14.9	Saving figures	149
14.10	Recipes	149
14.10.1	Reinforcement-learning training curves	149
14.10.2	Comparing methods, including unfinished runs	150
14.10.3	Several panels with consistent colours	150
14.10.4	Distinguishing series without colour	151
14.10.5	Bar chart of final performance	151
14.10.6	Confusion and correlation matrices	151
14.10.7	Distributions of returns	151
14.10.8	Plotting tensors and data frames	152
14.11	Command-line gallery	152
15	neural_toolkit: Networks and Tabular Tools	153
15.1	Conventions	153
15.2	Building blocks	153
15.3	Network families	154
15.3.1	Constructor arguments	154

15.3.2 Architectures	154
15.4 Factories	157
15.5 NetworkUtils	158
15.6 Tabular tools	158
15.7 Example: a DQN agent for LineMsg	161
15.8 Changes in version 1.0	163
IV Practice	164
16 Examples, Benchmarks and Development	165
16.1 Example scripts	165
16.2 The <code>env-lib</code> command line	168
16.3 Writing an evaluation loop	169
16.4 Recording episodes	170
16.5 Benchmarks	171
16.6 Testing, linting and continuous integration	173
16.6.1 Running the tests	173
16.6.2 Linting and formatting	173
16.6.3 Continuous integration	173
16.6.4 Conventions for contributions	174
17 Troubleshooting	176
17.1 Rendering on machines without a display	176
17.2 No window appears in human mode	176
17.3 Missing optional dependencies	177
17.4 “Out of date” warnings for -v0 environment ids	178
17.5 AJLATT deprecation warnings	178
17.6 PyTorch, GPUs and devices	178
17.7 Video export	179
17.8 Vector environments	180
17.9 Warnings about unknown reset options	181
17.10 Reproducibility and seeding	181
17.11 Performance	182
17.12 Training with <code>marl_algorithms</code>	183
17.13 Other common errors	184
A API Reference	186
A.1 <code>env_lib</code>	186
A.1.1 Package functions	186

A.1.2	Registry records	187
A.1.3	Common environment interface	187
A.1.4	Kuramoto oscillator classes	188
A.1.5	LineMsgEnv and WirelessCommEnv	189
A.1.6	PistonballEnv	189
A.1.7	ConsensusEnv and ConsensusVectorEnv	190
A.1.8	PowerGridEnv and PowerGridVectorEnv	191
A.1.9	PlatoonEnv and PlatoonVectorEnv	192
A.1.10	AJLATTEnv and AJLATTConfig	192
A.1.11	AJLATT maps	193
A.2	env_lib.catalog	194
A.3	env_lib.baselines	194
A.4	env_lib.wrappers	195
A.5	env_lib.utils	196
A.5.1	Rendering and recording	196
A.5.2	Evaluation and trajectories	197
A.5.3	Graph utilities	197
A.5.4	Batched vector environments	198
A.6	The env-lib command line	199
A.7	marl_algorithms	199
A.7.1	Training, registry and presets	199
A.7.2	Baselines	200
A.7.3	Agent spec and experience collection	201
A.7.4	Buffers, normalisation, networks and configurations	201
A.7.5	Base classes and the training log	202
A.7.6	Algorithms and their configurations	203
A.7.7	The marl-train command line	204
A.8	toolkit.plotkit	205
A.9	toolkit.neural_toolkit	206
A.9.1	Layers	206
A.9.2	Policy and value networks	206
A.9.3	Q-networks	206
A.9.4	Encoders and decoders	207
A.9.5	Factories	207
A.9.6	NetworkUtils	208
A.9.7	Tabular tools	208
A.10	toolkit.parakit	209

B	Migration from Earlier Versions	210
B.1	From 1.1 to 1.2	210
B.2	From 1.0 to 1.1	211
B.3	From versions before 1.0	212
B.3.1	Checklist	213
B.3.2	Installation	213
B.3.3	Changes common to all environments	213
B.3.4	Kuramoto oscillators	214
B.3.5	LineMsg and WirelessComm	215
B.3.6	Pistonball	215
B.3.7	AJLATT	215
B.3.8	plotkit	217
B.3.9	neural_toolkit	217
B.3.10	parakit	218

Chapter 1

Introduction

My Tool Box is a Python package for research on networked multi-agent control and multi-agent reinforcement learning (MARL). It is distributed as a single installable project, `my-tool-box`, that provides three import packages:

env_lib Multi-agent environments that follow the Gymnasium API: eight environment families registered under 18 ids, among them coupled oscillators, consensus and formation control, power-grid frequency control, vehicle platoons and multi-robot target tracking. Around the environments it provides natively batched vector environments, a searchable catalogue and the `env-lib` command line, adapters for other interfaces, a classical baseline controller for every environment, evaluation tools and shared graph utilities.

marl_algorithms

Reference implementations of seven classical multi-agent reinforcement learning algorithms (IPPO, MAPPO, MADDPG, MATD3, IQL, VDN and QMIX) that train directly on the batched `env_lib` environments, with tuned presets, evaluation against the baseline controllers and the `marl-train` command line. It requires PyTorch.

toolkit Research utilities that do not depend on the environments: `plotkit` creates publication-quality figures, `neural_toolkit` provides PyTorch network building blocks and tabular reinforcement-learning tools, and `parakit` manages `argparse` parameters, with an optional graphical editor.

This manual describes version 1.2 of the package. It explains how to install it, documents every environment, the workflow layer that connects them to experiments, the learning algorithms and the toolkit modules, collects practical recipes and troubleshooting advice, and ends with an API reference and a migration guide for code written against earlier versions.

1.1 What the package is built for

The package is built for research on control and learning in networks of interacting agents. Its environments share five characteristics.

- *Networked agents.* Agents interact through an explicit graph: a communication topology, the lines of a transmission network or the radio links of a platoon. Each agent observes its own state and its neighbourhood, so the environments pose decentralised problems in which the graph matters.
- *Continuous physical dynamics.* Most environments have continuous states and actions in

physical units, with dynamics that come from established models: phase oscillators, single and double integrators, the swing equations of a power system, the longitudinal dynamics of vehicles with actuator lag, and robot kinematics with estimation. Models, observations, rewards and parameters are documented precisely enough to be cited in a paper.

- *Many agents.* Team sizes are constructor arguments, and dynamics, observations and rewards are computed with array expressions rather than loops over agents, so an environment with a hundred agents remains practical.
- *Large-batch simulation.* The networked continuous-control environments (Kuramoto, Consensus and Formation, PowerGrid, Platoon) have native vector implementations that advance thousands of copies with one set of array operations per step, which suits on-policy learning with large batches, Monte-Carlo evaluation and parameter sweeps.
- *Reproducibility.* All randomness is seeded, benchmark systems such as a power network or a heterogeneous platoon are fixed by their own seeds, a single environment and copy 0 of its vector environment share their code, and every environment ships a classical baseline controller that provides a reference result.

Two discrete networked benchmarks (message relaying and wireless access) and a cooperative physics game complement the continuous-control environments. The environments do not depend on any particular learning library; any algorithm that can drive a Gymnasium environment can use them, and the adapters of Chapter 11 provide flat single-agent and per-agent dictionary views of the same environments.

1.2 What the package provides

Table 1.1 lists the environments. Each one is available as a class in `env_lib` and under one or more registered Gymnasium ids; the version suffix of an id selects a preset configuration (for example `WirelessComm-v1` is the 4×4 grid), not a newer revision. `env_lib.list_envs()` returns all 18 ids, and `env-lib list` prints them as a table. Figure 1.1 shows the built-in visualisations.

Beyond the environments, `env_lib` provides the workflow layer of Chapter 11: `env_lib.make_vec` for batched simulation, `env_lib.catalog` and `env_lib.describe` to discover environments, the `env-lib` command line to list, run, record, evaluate and benchmark them without writing code, wrappers that flatten the joint spaces or expose per-agent dictionaries, `env_lib.baseline_policy` for the baseline controllers, and `env_lib.evaluate` and `env_lib.rollout` for evaluation and trajectory recording. The graph utilities of `env_lib.utils.graphs` (Section 3.7) build topologies and compute Laplacians, algebraic connectivity and k -hop neighbourhoods.

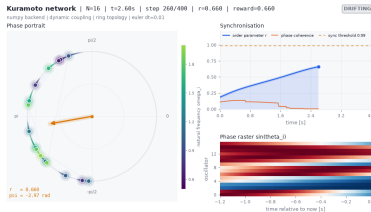
The `marl_algorithms` package (Chapter 12) trains cooperative multi-agent policies on these environments: IPPO and MAPPO for continuous and discrete actions, MADDPG and MATD3 for continuous actions, and IQL, VDN and QMIX for discrete actions, all built on one core that reads the per-agent structure of any environment, collects experience from the batched vector environments and evaluates the trained policies against random actions and the baseline controllers. `marl_algorithms.train` and the `marl-train` command line train a tuned preset in at most about two minutes on one CPU core.

The toolkit package contains three independent subpackages:

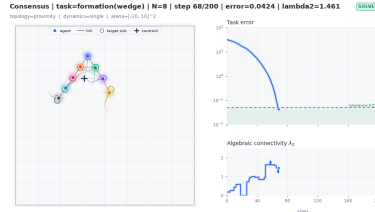
- `toolkit.plotkit` (Chapter 14): learning curves with confidence bands and smoothing, grouped bar charts, histograms, annotated heatmaps, style presets, colour-blind-safe palettes and multi-format export. It depends on `matplotlib` only.

Table 1.1: Environments provided by `env_lib`. “Batch” marks the environments with a native vector implementation (Chapter 11); the last column names the optional dependency group (Chapter 2) required in addition to the core installation.

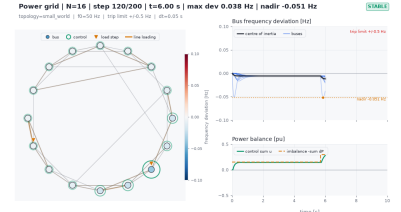
Registered ids	Task	Ch.	Batch	Extra
<i>Continuous states and actions</i>				
<code>KuramotoOscillator-*</code> (4 ids)	Drive a network of coupled phase oscillators to synchronisation (NumPy).	4	yes	–
<code>KuramotoOscillatorTorch-*</code> (5 ids)	The same model on PyTorch tensors, with many systems simulated in a batch.	4	–	torch
<code>Consensus-v0</code> , <code>Formation-v0</code>	Rendezvous or form a shape over a communication graph.	7	yes	–
<code>PowerGrid-v0</code>	Arrest and damp frequency deviations of a power network after load changes.	8	yes	–
<code>Platoon-v0</code>	Keep a vehicle platoon at safe, string-stable spacing behind a leader.	9	yes	–
<code>AJLATT-v0</code>	Robots localise themselves and jointly track a moving target.	10	–	–
<code>Pistonball-v0</code>	Pistons cooperate to roll a ball to the goal (2-D physics; continuous or discrete actions).	6	–	pistonball
<i>Discrete states and actions</i>				
<code>LineMsg-v0</code>	Relay a message along a line of agents with unreliable links.	5	–	–
<code>WirelessComm-v0</code> , <code>-v1</code>	Share wireless access points on a grid without collisions.	5	–	–



(a) Kuramoto oscillators



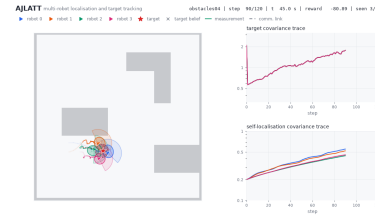
(b) Formation control



(c) PowerGrid



(d) Platoon



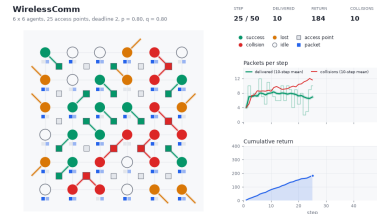
(e) AJLATT target tracking



(f) Pistonball



(g) LineMsg



(h) WirelessComm

Figure 1.1: Frames rendered in "rgb_array" mode by the environments of `env_lib`. Each renderer is a dashboard that combines the scene with the quantities that matter for the task, such as the order parameter of the oscillators, the frequency nadir of a power grid, the spacing errors along a platoon, per-agent outcomes or covariance traces.

- `toolkit.neural_toolkit` (Chapter 15): policy, value and Q-networks with MLP, convolutional, recurrent and transformer trunks, encoders and decoders (including a variational autoencoder), factories, network utilities and tabular reinforcement-learning tools. It requires PyTorch.
- `toolkit.parakit` (Chapter 13): saving, loading, validating and applying the defaults of an `argparse.ArgumentParser`, and a Tk window for editing them.

1.3 Design principles

One Gymnasium contract. Every environment subclasses `gymnasium.Env`. The method `reset` accepts a `seed` and an `options` dictionary as keyword arguments and returns the pair `(observation, info)`. The method `step` returns the observation, the reward, the flags `terminated` and `truncated`, and the `info` dictionary, where `terminated` signals a task-defined end (success or failure) and `truncated` the episode limit. The whole team is exposed through *joint* spaces whose leading axis indexes the agents: observations are `float32` arrays of shape `(n_agents, obs_dim)` contained in `observation_space`, and the action covers all agents at once. `step` returns the team reward, and every environment reports the per-agent rewards in `info["agent_rewards"]`. Each environment enforces its own episode limit through a constructor argument, and `env_lib.make(..., max_episode_steps=N)` sets that argument instead of adding a second limit. Chapter 3 describes these conventions in detail.

Batch-first kernels shared by single and vector environments. The dynamics of the environments with a native vector implementation are written once, as array operations on a leading batch dimension B . The single environment runs them with $B = 1$ and the native vector environment with $B = \text{num_envs}$, so both follow exactly the same model, and copy 0 of a vector environment reproduces a single environment started from the same state (the test suite checks this). Their steps contain no Python loop over agents or copies. The Kuramoto model additionally has a PyTorch backend that simulates many systems on the CPU or a GPU. Section 16.5 and Chapter 11 report measured step times and throughput.

The interaction graph as data. Who observes, communicates with or is coupled to whom is explicit data rather than a hidden detail of the code. The networked continuous-control environments expose the graph as a matrix (`adjacency` of Consensus, PowerGrid and Platoon, `topology_matrix` and `coupling_matrix` of the Kuramoto environments), together with the Laplacian or the line susceptances where they matter, and document the layout of the observation in an `observation_layout` property. The shared functions of `env_lib.utils.graphs` build standard topologies (PowerGrid draws its transmission network with them) and compute Laplacians, algebraic connectivity and k -hop neighbourhoods of any of these matrices.

Baselines as functions of the observation. Every environment ships a classical controller, decentralised wherever the environment has several agents: Laplacian consensus, droop control, cooperative adaptive cruise control and others (Section 11.5). Each controller is a pure function of the observation that accepts any leading batch shape, so the same object drives a single environment and all copies of a vector environment, and a decentralised controller computes the action of an agent from that agent's row of the observation only. `env_lib.baseline_policy(env)` returns the controller for any environment, which makes a reference result one line of code.

Light core, lazy extras. `import env_lib`, `import marl_algorithms` and `import toolkit` load only the core dependencies (NumPy, SciPy, matplotlib, Gymnasium and PyYAML). Environment classes, renderers, algorithms and toolkit subpackages are imported on first use, and PyTorch, pygame, pymunk, imageio and Tk are imported only when a feature that needs them is used; a missing dependency produces an error message that names the package extra to install, except in `marl_algorithms`, which always needs PyTorch and reports PyTorch's own `ModuleNotFoundError`. Other packages are optional integrations, never requirements: the parallel adapter of Section 11.4, for example, works without the library whose interface it implements.

Reproducibility. All randomness of an environment comes from its own random generator (`self.np_random`), which is seeded by `reset(seed=...)`. No library code seeds or draws from the global NumPy, Python or PyTorch random state, and plotting functions never modify the global matplotlib configuration unless asked to. Section 17.10 lists everything that must be seeded for an experiment to be repeatable.

Headless rendering. Rendering is optional and never performed unless requested. In "rgb_array" mode figures are drawn off-screen and returned as (H, W, 3) uint8 arrays, so episodes can be recorded to GIF or MP4 on machines without a display, for example on a compute cluster or in continuous integration. In "human" mode an environment renders itself at the end of every `reset()` and `step()`, as Gymnasium prescribes.

1.4 Package layout

The repository uses the standard `src` layout:

<code>My_Tool_Box/</code>	
<code>pyproject.toml</code>	packaging metadata, optional extras, tool settings
<code>CHANGELOG.md</code>	release notes and migration notes
<code>LICENSE</code>	MIT licence
<code>src/</code>	
<code>env_lib/</code>	environments (import name <code>env_lib</code>)
<code>registration.py</code>	ids and metadata, <code>make()</code> , <code>make_vec()</code> , <code>get_spec()</code>
<code>catalog.py</code>	<code>catalog()</code> , <code>describe()</code>
<code>baselines.py</code>	baseline controllers, <code>baseline_policy()</code>
<code>__main__.py</code>	the env-lib command line
<code>errors.py</code>	<code>ResetNeededError</code>
<code>wrappers/</code>	<code>FlattenJointSpaces</code> , <code>TeamReward</code> , <code>ParallelEnvAdapter</code>
<code>utils/</code>	rendering, recording, graphs, vector base class, <code>evaluate()</code> and <code>rollout()</code>
<code>kos_env/</code>	Kuramoto oscillators (NumPy, PyTorch, vector)
<code>consensus_env/</code>	Consensus and Formation (single and vector)
<code>power_grid_env/</code>	PowerGrid (single and vector)
<code>platoon_env/</code>	Platoon (single and vector)
<code>ajlatt_env/</code>	AJLATT (configuration, estimation, maps, rendering)
<code>pistonball_env/</code>	Pistonball
<code>linemsg_env/</code>	LineMsg
<code>wireless_comm_env/</code>	WirelessComm
<code>marl_algorithms/</code>	learning algorithms (import name <code>marl_algorithms</code>)
<code>core/</code>	agent spec, runner, buffers, networks, base classes
<code>algorithms/</code>	one module per algorithm, by family: <code>ppo/</code> (IPPO, MAPPO), <code>ddpg/</code> (MADDPG, MATD3), <code>q_learning/</code> (IQL, VDN, QMIX)

presets/	tuned settings for the environments
registry.py	train(), algorithm records
baselines.py	compare(): the algorithms as baselines
__main__.py	the marl-train command line
toolkit/	research utilities (import name toolkit)
plotkit/	plotting
neural_toolkit/	PyTorch networks and tabular tools
parakit/	argparse parameter management
examples/	runnable, documented example scripts: getting_started/, environments/, algorithms/, toolkit/
benchmarks/	performance measurements
tests/	pytest suite
docs/manual/	this manual (LaTeX sources)

A first program creates an environment through the registry, runs its baseline controller for one episode, and then evaluates the same controller on 256 copies of the environment simulated as one batch:

```
import env_lib

env = env_lib.make("PowerGrid-v0")           # 16 buses, one agent per bus
policy = env_lib.baseline_policy(env)         # decentralised droop control
obs, info = env.reset(seed=0)                 # obs: (16, 10) float32
while True:
    obs, reward, terminated, truncated, info = env.step(policy(obs))
    if terminated or truncated:
        break
print(info["peak_abs_omega"], info["agent_rewards"].shape) # rad/s, (16,)

envs = env_lib.make_vec("PowerGrid-v0", num_envs=256)      # native batch
result = env_lib.evaluate(envs, env_lib.baseline_policy(envs),
                          n_episodes=256, seed=0)
print(result.mean_return, result.ci95)                    # mean return, 95% CI half-width
```

The same can be done without code: `env-lib run PowerGrid-v0` runs one episode of the baseline and `env-lib evaluate PowerGrid-v0 --num-envs 64` compares it with random actions (Section 11.3).

1.5 Conventions used in this manual

Python identifiers are set in monospace, package and module names in sans-serif, and registered environment ids as `Consensus-v0`. Shell commands are meant to be run from the repository root in a POSIX shell; on Windows, replace `export VAR=value` by `set VAR=value (cmd)` or `$env:VAR="value"` (PowerShell). Code examples use the public API of version 1.2 and were executed with it. Array shapes are written as tuples, for example `(n_agents, obs_dim)`; `B` denotes a batch size (the number of copies of a vector environment) and `T` a sequence length.

1.6 How this manual is organised

- Chapter 2 covers requirements, installation with optional extras, verification and building this manual.

- Part I documents the environments: the common interface, the registry and the graph utilities (Chapter 3), the Kuramoto oscillators (Chapter 4), LineMsg and WirelessComm (Chapter 5), Pistonball (Chapter 6), consensus and formation control (Chapter 7), power-grid frequency control (Chapter 8), vehicle platoons (Chapter 9) and AJLATT (Chapter 10). It closes with the workflow layer: vectorised simulation, the catalogue and the command line, adapters, baseline controllers and evaluation (Chapter 11).
- Part II documents the multi-agent reinforcement learning algorithms of `marl_algorithms`: their shared core, the equations of every method, the configuration, the presets with their results, and how to add an algorithm (Chapter 12).
- Part III documents the toolkit: its structure and `parakit` (Chapter 13), `plotkit` (Chapter 14) and `neural_toolkit` (Chapter 15).
- Part IV is practical: the example scripts, recording, benchmarks and the contributor workflow (Chapter 16), and troubleshooting (Chapter 17).
- Appendix A is a compact API reference and Appendix B explains how to migrate code written for version 1.0 and for the versions before it.

1.7 Licence and citation

My Tool Box is free software distributed under the MIT licence; the full text is in the file `LICENSE` at the root of the repository. The source code is hosted at https://github.com/EricDMW/My_Tool_Box, where bugs and feature requests can be reported as issues.

If the package contributes to published work, please cite it as follows:

```
@software{wang2026mytoolbox,
  author  = {Wang, Dongming},
  title   = {My Tool Box: Multi-Agent Reinforcement Learning
            Environments and Research Utilities},
  version = {1.2.1},
  year    = {2026},
  url     = {https://github.com/EricDMW/My_Tool_Box}
}
```

When results depend on a particular environment, please also cite the original description of the underlying model or benchmark (for example the Kuramoto model of coupled oscillators), as given in the corresponding environment chapter.

Chapter 2

Installation

My Tool Box is a standard Python project described by a single `pyproject.toml` file. It is installed from its source repository with `pip`; heavy dependencies are optional and grouped into *extras*.

2.1 Requirements

- Python 3.9 or newer. The package is tested with CPython 3.9 to 3.13; continuous integration runs the test suite on each of these versions.
- Any operating system supported by the dependencies (Linux, macOS and Windows). Development and continuous integration use Linux.
- The core dependencies below, installed automatically by `pip`.

Table 2.1: Core dependencies (always installed).

Package	Minimum version	Used for
numpy	1.23	all numerical code
scipy	1.9	AJLATT estimation, plotkit smoothing
matplotlib	3.6	environment renderers, plotkit
pillow	9.0	GIF export
cycler	0.11	plotkit styles
gymnasium	1.1	environment API, registration, vector environments
pyyaml	6.0	AJLATT map files, map specifications, parakit YAML files

2.2 Installing from the repository

The recommended procedure creates a virtual environment, clones the repository and installs the package with all optional features:

```
git clone https://github.com/EricDMW/My_Tool_Box.git
cd My_Tool_Box
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
pip install "[all]"
```

`pip` can also install directly from the Git URL without a local clone, which is convenient for a project that only uses the package:

```
pip install "my-tool-box[all] @ git+https://github.com/EricDMW/My_Tool_Box.git"
```

With `conda`, create the environment first and then use `pip` inside it:

```
conda create -n mytoolbox python=3.11
conda activate mytoolbox
conda install tk                      # only for Tk windows (see below)
pip install ".[all]"
```

2.2.1 Optional extras

The extras of Table 2.2 are selected in square brackets and can be combined, for example `pip install ".[pistonball,video]"`. The quotes prevent shells such as `zsh` from interpreting the brackets.

Table 2.2: Optional extras defined in `pyproject.toml`.

Extra	Installs	Enables
<code>pistonball</code>	<code>pygame</code> $\geq$ 2.1, <code>pymunk</code> $\geq$ 6.4	<code>PistonballEnv</code> (<code>pymunk</code> for the physics, <code>pygame</code> for rendering and keyboard control)
<code>torch</code>	<code>torch</code> $\geq$ 2.0	<code>marl_algorithms</code> and <code>marl-train</code> , <code>KuramotoOscillatorEnvTorch</code> , <code>toolkit.neural_toolkit</code>
<code>video</code>	<code>imageio</code> $\geq$ 2.20, <code>imageio-ffmpeg</code> $\geq$ 0.5.1	MP4, WebM, AVI, MOV and MKV export in <code>save_animation</code> and <code>record_episode</code> (GIF export needs no extra)
<code>all</code>	the three extras above	every feature of the package
<code>dev</code>	<code>pytest</code> $\geq$ 7, <code>pytest-cov</code> $\geq$ 4, <code>ruff</code> $\geq$ 0.5	running the test suite and the linter; combine with <code>all</code>

2.2.2 PyTorch

The `torch` extra installs the default PyTorch wheel for the platform. To choose a specific build, install PyTorch first, following the selector at <https://pytorch.org/get-started/>, and then install the package; `pip` keeps the PyTorch that is already present. For example, a CPU-only build (much smaller, and the one used in continuous integration) is obtained with

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install ".[all]"
```

GPU use is covered in Section 17.6. The algorithms of `marl_algorithms` run on the CPU by default (argument `device`) and were tuned there; the CPU build is sufficient for them.

2.2.3 PettingZoo

The adapter `env_lib.wrappers.to_parallel` exposes every environment through the PettingZoo Parallel API (Section 11.4). PettingZoo is neither a dependency nor an extra: the adapter implements the protocol itself. When PettingZoo is installed, the adapter subclasses

`pettingzoo.utils.env.ParallelEnv`, so `isinstance` checks and PettingZoo's own utilities, such as `parallel_api_test`, accept it. Install it only if your multi-agent library needs it:

```
pip install pettingzoo
python -c "import env_lib.wrappers as w; print(w.PETTINGZOO_AVAILABLE)"
```

2.2.4 Tk

Tk is not installable with `pip`. It is needed only for the `parakit` editor window (Section 13.2.6) and for `render_mode="human"` with matplotlib's TkAgg backend; everything else, including off-screen rendering, works without it. Install it with the system package manager (`sudo apt-get install python3-tk` on Debian and Ubuntu, `sudo dnf install python3-tkinter` on Fedora, `brew install python-tk` with Homebrew on macOS) or with `conda install tk`. The official Python installers for Windows and macOS already include Tk.

2.3 Development installation

Contributors install the package in *editable* mode, so that changes to the sources under `src/` take effect without reinstalling, together with the development tools:

```
git clone https://github.com/EricDMW/My_Tool_Box.git
cd My_Tool_Box
python -m venv .venv && source .venv/bin/activate
pip install -e ".[all,dev]"
```

The console scripts and the version number are generated at installation time; after changing `pyproject.toml` (for example adding an entry point), run the `pip install -e` command again. The testing and linting workflow is described in Section 16.6.

2.4 Verifying the installation

The following commands check, from quickest to most thorough, that the package is installed correctly:

```
# 1. Versions of the three import packages
python -c "import env_lib, marl_algorithms as m, toolkit; \
print(env_lib.__version__, m.__version__, toolkit.__version__)"

# 2. Table of the registered environments (the env-lib console script)
env-lib list
python -m env_lib list           # the same without the console script

# 3. Create every environment and take a few random steps
python examples/getting_started/quickstart.py --steps 5

# 4. The algorithms (needs the torch extra) and a few seconds of training
marl-train list
python examples/algorithms/marl_training_demo.py --quick --plot ''

# 5. Full test suite (requires the dev extra)
pytest -q
```

The first command prints 1.2.1 1.2.1 1.2.1. `env-lib list` creates every environment once and prints one row per registered id (18 rows) with its family, number of agents, observation shape, action type and shape, whether `env-lib.make_vec` uses a native batched implementation, and the optional dependency it requires; an environment whose extra is not installed is marked (missing) instead of failing. `env-lib list --no-inspect` skips creating the environments. The command line is described in Section 11.3. The quick-start script prints one line per environment and skips, with a message, environments whose optional dependencies are missing. `import marl_algorithms` succeeds without PyTorch (it loads only the version number), so the fourth step is the check that the algorithms can be used: `marl-train list` prints the seven algorithms, and the demonstration script trains four of them for a few seconds each (Section 12.9); without PyTorch both fail with a `ModuleNotFoundError` for `torch`. The test suite skips tests of missing optional dependencies in the same way.

The installation provides five console scripts:

Command	Purpose
<code>env-lib</code>	list, run, evaluate and benchmark the environments (Section 11.3)
<code>marl-train</code>	list, train and evaluate the algorithms (Section 12.2.2)
<code>ajlatt-build-map</code>	build an AJLATT map from a YAML specification (Chapter 10)
<code>ajlatt-plot-map</code>	list and plot the bundled AJLATT maps (Chapter 10)
<code>plotkit-gallery</code>	gallery of plotkit figures (Section 14.11)

2.5 Optional dependency matrix

Table 2.3 summarises which features need which optional packages. Because the package imports optional dependencies lazily, a missing dependency only affects the feature that needs it; the error message names the extra to install.

2.6 Upgrading and uninstalling

To upgrade a clone, pull the latest sources and reinstall: `git pull` followed by `pip install ".[all]"` (or `pip install -e ".[all,dev]"` for an editable installation). Reinstalling also runs again the step that generates the console scripts, so an editable installation made with version 1.0 gains the `env-lib` command only after this step. Version 1.2.1 requires Gymnasium 1.1 or newer; `pip` upgrades an older Gymnasium automatically, which may affect other projects in the same environment (Section B.2). `pip uninstall my-tool-box` removes the package. Users of versions before 1.0, which were installed from two separate project folders, should first uninstall the old distributions and then follow Appendix B.

2.7 Building this manual

The manual is written in LaTeX and lives in `docs/manual/`. Building it requires a TeX distribution that provides `pdflatex` and `latexmk`, such as TeX Live or MacTeX. On Debian and Ubuntu the packages `latexmk`, `texlive-latex-extra`, `texlive-fonts-recommended` and `lmodern` are sufficient. The document uses only standard packages (`listings`, `booktabs`, `tabularx`, `hyperref`, `fancyhdr` and similar).

```
cd docs/manual
./build.sh                                # writes output/main.pdf
```

Table 2.3: Features and their optional requirements.

Feature	Requires	Provided by
NumPy environments (Kuramoto, LineMsg, WirelessComm, Consensus, PowerGrid, Platoon, AJLATT), <code>rgb_array</code> rendering, GIF export	core only	core
PistonballEnv	pymunk	pistonball
Pistonball rendering and ManualPolicy	pygame	pistonball
KuramotoOscillatorEnvTorch	torch	torch
<code>marl_algorithms</code> , <code>marl-train</code>	torch	torch
<code>toolkit.neural_toolkit</code>	torch	torch
MP4 and other video formats	imageio, imageio-ffmpeg	video
<code>toolkit.plotkit</code>	core only	core
Plotting pandas, PyTorch, TensorFlow or JAX data	the respective library	not required
<code>toolkit.parakit</code> headless API	standard library	core
<code>env-lib</code> command line, catalogue, vector environments, baselines, evaluation	core only	core
<code>to_parallel</code> as a <code>pettingzoo.utils.env.ParallelEnv</code> subclass	pettingzoo (the adapter works without it)	not required
<code>parakit</code> editor window	Tk and a display	system package
<code>render_mode="human"</code> (matplotlib environments)	an interactive backend (TkAgg, QtAgg, ...) and a display	system package or PyQt/PySide
Test suite and linting	pytest, pytest-cov, ruff	dev

```
# equivalently:
latexmk -pdf -interaction=nonstopmode -halt-on-error -outdir=output main.tex
latexmk -C -outdir=output # remove build products
```

`latexmk` runs `pdflatex` as often as needed to resolve cross references. All build products are written to `docs/manual/output/`, which is ignored by Git. The file `docs/manual/README.md` describes the source layout of the manual.

Part I

Environments

Chapter 3

Environment Library Overview

The `env_lib` package provides multi-agent environments for research on networked control, cooperative decision making and multi-robot estimation. All environments implement the Gymnasium API, are registered with Gymnasium when `env_lib` is imported, and share one registry, one rendering and recording layer and one set of graph utilities. This chapter describes what the environments have in common; the following chapters document each environment in detail:

- Kuramoto oscillator networks, NumPy and batched PyTorch backends (Chapter 4);
- LineMsg and WirelessComm, two discrete networked communication benchmarks (Chapter 5);
- Pistonball, a cooperative physics game (Chapter 6);
- Consensus and formation control on communication graphs (Chapter 7);
- PowerGrid, frequency control of a networked power system (Chapter 8);
- Platoon, cooperative adaptive cruise control of a vehicle platoon (Chapter 9);
- AJLATT, active joint localisation and target tracking with a robot team, including its map tools (Chapter 10).

Chapter 11 then describes the layer that connects the environments to experiments: native vector environments, the catalogue and the `env-lib` command line, adapters, baseline controllers and evaluation.

3.1 Environments at a glance

Table 3.1 summarises the environments in their default configuration. N is the number of oscillators, M the number of coupling edges and n the number of agents. The NumPy Kuramoto, Consensus, Formation, PowerGrid and Platoon environments have native vector implementations (Section 3.6).

3.2 Package structure

The package lives in `src/env_lib` of the repository:

Table 3.1: Environments in their default configuration.

Environment	Agents	Action space	Observation	Reward	Requires
Kuramoto (NumPy)	1 controller, $N = 10$ oscillators	continuous $\text{Box}((N+M,))$, $N + M = 55$	$(75,)$, i.e. $3N + M$	scalar synchronisation measure, bonus on synchronisation	none
Kuramoto (PyTorch)	<code>n_agents</code> parallel systems (1)	as NumPy; batched (<code>n_agents</code> , $N+M$) accepted	$(75,)$ of system 0	per system in info	torch
LineMsg	$n = 10$	$\text{Discrete}(2**n)$ or $\text{MultiBinary}(n)$	$(10, 3)$	sum of per-agent rewards	none
WirelessComm	$n = 36$ (6×6)	$\text{MultiDiscrete}([5]*n)$	$(36, 18)$	packets delivered (sum)	none
Pistonball	$n = 20$ pistons	$\text{Box}(-1, 1, (n,))$ or $\text{MultiDiscrete}([3]*n)$	$(20, 7)$	sum of local rewards	pymunk , pygame
Consensus / Formation	$n = 8$	$\text{Box}(-1, 1, (n, 2))$	$(8, 16)$	mean of local rewards, success bonus	none
PowerGrid	$n = 16$ buses	$\text{Box}(-0.1, 0.1, (n,))$: injection in per unit	$(16, 10)$	sum of local rewards, trip penalty	none
Platoon	$n = 8$ followers	$\text{Box}(-6, 3, (n,))$: acceleration command in m/s^2	$(8, 13)$	sum of local rewards, collision and break-up penalties	none
AJLATT	$n = 4$ robots	$\text{Box}((n, 2))$: $v \in [0, 2]$, $\omega \in [-\pi/4, \pi/4]$	$(4, 24)$	per-robot array	none

```

src/env_lib/
__init__.py      public names, lazily imported classes and functions
registration.py  ids and metadata (EnvSpec), make(), make_vec(), get_spec()
catalog.py       catalog(), describe(), EnvInfo
baselines.py     baseline controllers, baseline_policy(), list_baselines()
__main__.py     the env-lib command line (python -m env_lib)
errors.py        ResetNeededError
wrappers/        FlattenJointSpaces, TeamReward, ParallelEnvAdapter
utils/           themes, renderers, recording, graphs, BatchedVectorEnv,
                  evaluate() and rollout()

kos_env/         Kuramoto oscillators (NumPy, PyTorch, native vector)
consensus_env/   consensus and formation control (single, native vector)
power_grid_env/  PowerGrid: frequency control (single, native vector)
platoon_env/     Platoon: cooperative adaptive cruise control
ajlatt_env/      AJLATT: multi-robot localisation and target tracking
  maps/          occupancy maps, map builder and map plotting
pistonball_env/  Pistonball: cooperative physics game (pymunk, pygame)
linemsg_env/     LineMsg: message relay along a line of agents
wireless_comm_env/ WirelessComm: random access to shared access points

```

Every environment subpackage contains the environment module and a **rendering** module. The renderer is imported on the first call to `render()`, so an environment that is never rendered does not create any figure or window.

Lazy imports. `import env_lib` imports neither PyTorch nor pygame, pymunk or tkinter. The environment classes of Table 3.2 are attributes of the package that are imported on first

access, for example `env_lib.ConsensusEnv`; the same holds for `env_lib.baseline_policy`, `env_lib.evaluate`, `env_lib.rollout`, `env_lib.describe` and `env_lib.EnvInfo` and for the subpackages `env_lib.wrappers`, `env_lib.baselines` and `env_lib.catalog`. An optional dependency is therefore only needed when the environment that uses it is created (Table 3.3).

Table 3.2: Environment classes exported by `env_lib`.

Class	Module	Ch.
<code>KuramotoOscillatorEnv</code>	<code>env_lib.kos_env.kuramoto_env</code>	4
<code>KuramotoOscillatorEnvTorch</code>	<code>env_lib.kos_env.kuramoto_env_torch</code>	4
<code>KuramotoOscillatorVectorEnv</code>	<code>env_lib.kos_env.vector</code>	11
<code>LineMsgEnv</code>	<code>env_lib.linemsg_env.linemsg_env</code>	5
<code>WirelessCommEnv</code>	<code>env_lib.wireless_comm_env.wireless_comm_env</code>	5
<code>PistonballEnv</code>	<code>env_lib.pistonball_env.pistonball_env</code>	6
<code>ConsensusEnv</code>	<code>env_lib.consensus_env.consensus_env</code>	7
<code>ConsensusVectorEnv</code>	<code>env_lib.consensus_env.vector</code>	11
<code>PowerGridEnv</code> , <code>PowerGridVectorEnv</code>	<code>env_lib.power_grid_env.power_grid_env</code>	8
<code>PlatoonEnv</code> , <code>PlatoonVectorEnv</code>	<code>env_lib.platoon_env.platoon_env</code>	9
<code>AJLATTEnv</code> , <code>AJLATTConfig</code>	<code>env_lib.ajlatt_env</code>	10

Table 3.3: Optional dependencies of the environments. The extras are installed with `pip install -e ".[extra]"` from the repository root.

Extra	Packages	Needed for
<code>pistonball</code>	<code>pymunk</code> , <code>pygame</code>	Pistonball: <code>pymunk</code> to create the environment, <code>pygame</code> to render it and for keyboard control
<code>torch</code>	<code>torch</code>	the PyTorch backend of the Kuramoto environments
<code>video</code>	<code>imageio</code> , <code>imageio-ffmpeg</code>	<code>save_animation</code> to <code>.mp4</code> and other video formats (GIF export needs only <code>Pillow</code>)

`PettingZoo` is not an extra: the parallel adapter of Section 11.4 works without it and subclasses `PettingZoo`'s `ParallelEnv` when it is installed (Section 2.2.3).

3.3 Creating environments

There are three equivalent ways to create an environment:

```
import gymnasium as gym
import env_lib

env = env_lib.make("WirelessComm-v0")           # registered id
env = gym.make("WirelessComm-v0")               # after importing env_lib
env = env_lib.WirelessCommEnv(grid_x=6, grid_y=6) # direct construction

print(len(env_lib.list_envs()))                 # 18 registered ids
```

The function `env_lib.make` registers the environments if necessary and calls `gymnasium.make` with the id and the keyword arguments, which are passed to the constructor and override the configuration of the id, for example `env_lib.make("LineMsg-v0", num_agents=20, max_iter=100)`.

`gymnasium.make` returns the environment inside Gymnasium’s standard wrappers (order enforcement and, except for AJLATT-v0, the passive environment checker). Environment-specific methods and attributes such as `laplacian_policy()`, `adjacency` or `observation_layout` are reached through `env.unwrapped`.

Episode length. `env_lib.make(env_id, max_episode_steps=N)` sets the environment’s own limit argument (`max_steps`, `max_iter`, `max_cycles` or `max_episode_steps`, recorded in `EnvSpec.limit_kwarg`) to `N` instead of adding a `TimeLimit` wrapper, so there is always exactly one limit. Passing both `max_episode_steps` and a different value of the limit argument raises `ValueError`. `gymnasium.make` itself does not translate the argument (Section 3.5.4).

Vector environments. `env_lib.make_vec(env_id, num_envs)` creates `num_envs` copies as one Gymnasium vector environment: the environment’s native batched implementation when it has one, and `gymnasium.vector.SyncVectorEnv` otherwise (Section 3.6). It takes the same keyword arguments as `make`, including `max_episode_steps`.

```
import env_lib

env = env_lib.make("Platoon-v0", max_episode_steps=300)
print(env.unwrapped.max_steps, env.spec.max_episode_steps)    # 300 None

envs = env_lib.make_vec("Platoon-v0", num_envs=64)            # native batch
print(type(envs).__name__)                                     # PlatoonVectorEnv
print(envs.observation_space.shape)                            # (64, 8, 13)
envs = env_lib.make_vec("LineMsg-v0", num_envs=4)             # no native version
print(type(envs).__name__)                                     # SyncVectorEnv
```

3.4 Registry and metadata

Every registered id is described by an `EnvSpec` record (class `env_lib.registration.EnvSpec`) in `env_lib.registration.ENV_SPECS`; `env_lib.get_spec(env_id)` returns the record of one id and raises `KeyError` for an unknown id. `EnvSpec` is a frozen dataclass, distinct from Gymnasium’s `gymnasium.envs.registration.EnvSpec` (which `env.spec` returns); its fields are listed in Table 3.4. The catalogue (`env_lib.catalog()`, Section 11.2) and the `env-lib` command line are built on these records, so an environment can be selected by its properties instead of by name.

```
import env_lib

spec = env_lib.get_spec("PowerGrid-v0")
print(spec.family, spec.action_type)    # power_grid continuous
print(spec.limit_kwarg)                  # max_steps
print(spec.vector_entry_point)
# env_lib.power_grid_env.power_grid_env:PowerGridVectorEnv

native = env_lib.catalog(native_vector=True, inspect=False).ids()
print(len(native))                       # 8 ids with a native batch
```

Table 3.4: Fields of `env_lib.registration.EnvSpec`.

Field (default)	Content
<code>id</code>	registered Gymnasium id
<code>entry_point</code>	"module:Class" of the single environment
<code>kwargs ({})</code>	constructor arguments of this configuration
<code>description ("")</code>	one-line summary
<code>family ("")</code>	environment family: the subpackage name without <code>_env</code> ("kuramoto", "linemsg", "wireless_comm", "pistonball", "consensus", "power_grid", "platoon", "ajlatt")
<code>observation_type,</code> <code>action_type</code> <code>("continuous")</code>	"continuous", "discrete" or, for actions that can be either (Pistonball), "continuous discrete"
<code>requires (None)</code>	optional-dependency extra needed by the environment ("torch" or "pistonball")
<code>vector_entry_point</code> <code>(None)</code>	"module:Class" of the native batched implementation used by <code>make_vec</code>
<code>limit_kwarg</code> <code>("max_steps")</code>	name of the constructor argument that sets the episode length; <code>make</code> and <code>make_vec</code> translate <code>max_episode_steps</code> into it
<code>per_agent_rewards</code> <code>(False)</code>	rewards and termination flags are per-agent arrays (AJLATT); <code>make_vec</code> then wraps Sync and Async copies in <code>TeamReward</code>
<code>disable_env_checker</code> <code>(False)</code>	skip Gymnasium's passive checker (<code>True</code> for AJLATT, whose rewards are per-robot arrays)

3.5 Common API conventions

3.5.1 Gymnasium interface

Every environment implements

- `reset(*, seed=None, options=None)` returning `(observation, info)`;
- `step(action)` returning `(observation, reward, terminated, truncated, info)`;
- `render()` and `close()`; `close()` may be called several times.

Observations are `float32` NumPy arrays contained in `observation_space`. Calling `step()` before the first `reset()` raises `env_lib.ResetNeededError`, which derives from both `RuntimeError` and `gymnasium.error.ResetNeeded`, so either can be caught; the native vector environments raise it as well, also when the first `reset()` resets only some copies. Invalid actions (wrong shape, values outside a discrete range, NaN or infinite values) raise `ValueError`, and continuous actions are clipped to the action bounds. AJLATT is the exception: it only checks the action shape, and clipping is opt-in (`clip_actions=True`, Section 10.3).

3.5.2 Joint multi-agent spaces

Each environment exposes the whole team through a single Gymnasium interface. The action space covers all agents at once (for example `Box(-1, 1, (8, 2))` for eight planar agents, `Box(-0.1, 0.1, (16,))` for the 16 buses of a power grid, or `MultiDiscrete([5] * 36)` for 36 wireless agents), and observations have one row per agent. A multi-agent algorithm splits the joint arrays along the first axis; a centralised algorithm uses them directly. The wrappers of Section 11.4 provide both views ready-made: a flat vector for single-agent libraries and per-agent dictionaries for the PettingZoo Parallel API.

The scalar `reward` returned by `step()` is the team reward. The per-agent rewards are always available as `info["agent_rewards"]`, a `float64` array of shape `(n_agents,)`, which every `step()` returns. How the team reward is formed from the per-agent rewards depends on the environment (Table 3.1): it is their sum for LineMsg, WirelessComm, Pistonball, PowerGrid, Platoon and AJLATT and their mean (plus a success bonus) for Consensus. The Kuramoto environments control one or several complete oscillator networks, so their “agents” are systems: the NumPy backend reports `n_agents` copies of its reward and the PyTorch backend one reward per simulated system. AJLATT returns the per-robot rewards and per-robot collision flags directly as arrays; `TeamReward` (Section 11.4) and the older `TeamRewardWrapper` (Section 10.8) convert them to a scalar reward and a scalar `terminated` flag.

3.5.3 Info dictionaries

The info dictionary returned by `reset()` contains `"agent_rewards"` (all zeros) for LineMsg, WirelessComm, Consensus, PowerGrid, Platoon and AJLATT. The Kuramoto and Pistonball environments do not include the key in their reset info. Table 3.5 lists the other keys of the step info; the environment chapters define them.

Table 3.5: Keys of the info dictionary returned by `step()` besides `"agent_rewards"`.

Environment	Keys	Details
Kuramoto	<code>order_parameter</code> , <code>phase_coherence</code> , <code>phases</code> , <code>natural_frequencies</code> , <code>coupling_matrix</code> , <code>dphases_dt</code> , <code>step_count</code> , <code>device</code> , <code>n_agents</code>	Sec. 4.6
LineMsg	<code>none</code>	Sec. 5.1.3
WirelessComm	<code>outcomes</code> , <code>ap_load</code>	Sec. 5.2.3
Pistonball	<code>local_rewards</code> , <code>total_reward</code>	Sec. 6.4
Consensus	<code>error</code> , <code>algebraic_connectivity</code> , <code>adjacency</code> , <code>success</code> , <code>step</code>	Sec. 7.5
PowerGrid	<code>omega</code> , <code>rocof</code> , <code>max_abs_omega</code> , <code>frequency_nadir</code> , <code>peak_abs_omega</code> , <code>coi_omega</code> , <code>frequency_spread</code> , <code>control_effort</code> , <code>order_parameter</code> , <code>total_disturbance</code> , <code>tripped</code> , <code>step</code> , <code>time</code>	Sec. 8.5
Platoon	<code>spacing_errors</code> , <code>gaps</code> , <code>relative_speeds</code> , <code>speeds</code> , <code>accelerations</code> , <code>commands</code> , <code>peak_spacing_errors</code> , <code>error_amplification</code> , <code>leader_speed</code> , <code>leader_acceleration</code> , <code>min_gap</code> , <code>collision</code> , <code>breakup</code> , <code>step</code> , <code>time</code>	Sec. 9.6
AJLATT	<code>team_reward</code> , <code>collisions</code> , <code>episode_collisions</code> , <code>target_observed</code> , <code>communication</code> , <code>robot_cov_trace</code> , <code>target_cov_trace</code> , <code>step</code> , <code>numerical_error</code>	Sec. 10.7

In a vector environment every info value gains a leading `num_envs` dimension and comes with Gymnasium’s boolean mask `"_key"` (Section 11.1).

3.5.4 Termination and truncation

`terminated` signals a task-specific terminal state; `truncated` signals the end of the time budget. Every environment enforces its own episode limit through a constructor argument (Table 3.6). The registrations do not add a `TimeLimit` wrapper (`env.spec.max_episode_steps` is `None`), so the limit argument is the only limit. To change the episode length, pass the constructor argument, for example `env_lib.make("Consensus-v0", max_steps=400)`, or pass `max_episode_steps` to `env_lib.make` or `env_lib.make_vec`, which set the same argument. Only `gymnasium.make(..., max_episode_steps=N)` called directly adds an independent `TimeLimit` wrapper on top of the environment’s own limit; the shorter of the two then applies.

Table 3.6: Episode limits and termination conditions.

Environment	Limit argument	Default	<code>terminated</code> is set when
Kuramoto	<code>max_steps</code>	1000	the order parameter exceeds <code>sync_threshold</code>
LineMsg	<code>max_iter</code>	50	never (always <code>False</code>)
WirelessComm	<code>max_iter</code>	50	never (always <code>False</code>)
Pistonball	<code>max_cycles</code>	125	the ball reaches the left wall
Consensus / Formation	<code>max_steps</code>	200	the task error drops below <code>tolerance</code>
PowerGrid	<code>max_steps</code>	200	a frequency deviation exceeds the trip limit, $\max_i \omega_i > 2\pi \text{frequency_limit}$
Platoon	<code>max_steps</code>	600	a follower collides with its predecessor or its gap exceeds <code>breakup_distance</code>
AJLATT	<code>max_episode_steps</code>	120	per robot: an obstacle is closer than <code>obstacle_collision_distance</code> (array of flags)

3.5.5 Seeding and reproducibility

- `reset(seed=s)` seeds the environment’s generator `self.np_random`. All randomness of an episode (initial state, stochastic transitions, noise, load disturbances, leader profiles) is drawn from this generator, and `reset()` without a seed continues its stream. With the same seed, the same actions and the same library versions an episode is reproduced exactly.
- No environment reads or modifies the global NumPy or PyTorch random state. The PyTorch Kuramoto backend owns a `torch.Generator` that is seeded from `self.np_random` at every reset.
- `env.action_space.sample()` uses the space’s own generator; seed it separately with `env.action_space.seed(s)`.
- Some structure is fixed at construction and has its own seed: `topology_seed` (random Kuramoto topology, default 42), `graph_seed` (Erdős–Rényi graph of the consensus environment), `network_seed` (PowerGrid network and physical parameters, default 0), `vehicle_seed` (Platoon vehicle lags and lengths, default 0) and `AJLATTConfig.seed` (seed of the first AJLATT reset when none is given). With `randomize_network=True` (PowerGrid) or `randomize_vehicles=True` (Platoon) this structure is instead resampled from `self.np_random` at every reset.
- A native vector environment draws all copies from one generator, so `reset(seed=s)` makes the batch reproducible, but copy i does not reproduce a single environment seeded with $s + i$ (Section 11.1).
- The legacy `env.seed()` methods of LineMsg, WirelessComm and Pistonball still work but emit a `DeprecationWarning`; use `reset(seed=...)` instead.

3.5.6 Reset options

`reset(options=...)` can override parts of the sampled initial state (Table 3.7). The values of the listed keys are validated, and a wrong shape or a non-finite entry raises `ValueError`. *Unknown keys are ignored with a `UserWarning` that names them and the supported keys; they do not raise*, so generic tools that pass options of their own (such as PettingZoo’s `parallel_api_test`) work with every environment. LineMsg, WirelessComm, Pistonball and AJLATT have no reset

options and ignore `options` altogether. The native vector environments accept the same keys, as arrays for one copy (applied to every reset copy) or with a leading `num_envs` dimension, and in addition `"reset_mask"` (Section 11.1).

Table 3.7: Reset options.

Environment	Keys
Kuramoto	"phases", "natural_frequencies", "coupling_strengths" (dynamic coupling only)
Consensus / Formation	"positions", "velocities", each (n, 2)
PowerGrid	"theta", "omega", "disturbance", each (n,)
Platoon	"positions" and "velocities" (given together), "accelerations", "time_constants", "lengths", each (n + 1,) with the leader first; "leader_command" (max_steps,)
others	none; options is ignored

```
import warnings
import numpy as np
import env_lib

env = env_lib.make("PowerGrid-v0")
omega = np.full(16, 0.1) # 0.1 rad/s at every bus
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    obs, info = env.reset(seed=0, options={"omega": omega, "tag": "run-1"})
print(info["omega"][:3]) # [0.1 0.1 0.1]
print(caught[0].category.__name__) # UserWarning ("tag" is ignored)
```

3.6 Vectorised simulation and the workflow layer

`env_lib.make_vec` returns a Gymnasium vector environment in which observations, rewards and flags have a leading `num_envs` dimension. For the NumPy Kuramoto ids, `Consensus-v0`, `Formation-v0`, `PowerGrid-v0` and `Platoon-v0` it is a native implementation (`KuramotoOscillatorVectorEnv`, `ConsensusVectorEnv`, `PowerGridVectorEnv`, `PlatoonVectorEnv`) that advances all copies with one set of array operations and shares its dynamics code with the single environment. The native classes derive from `env_lib.utils.BatchedVectorEnv`, which implements seeding, action validation, the three autoreset modes and the info masks on top of three array hooks. All other ids are wrapped in `gymnasium.vector.SyncVectorEnv` (`vectorization_mode="async"` gives an `AsyncVectorEnv`).

Chapter 11 documents this layer in full: vector environments, autoreset and seeding (Section 11.1), the catalogue and the command line (Sections 11.2 and 11.3), the wrappers `FlattenJointSpaces`, `TeamReward` and `to_parallel` (Section 11.4), the baseline controllers (Section 11.5) and `evaluate` and `rollout` (Section 11.6).

3.7 Graph utilities

The module `env_lib.utils.graphs` collects the graph computations of networked control. A graph is a square adjacency matrix: agent i interacts with agent j when entry (i, j) is non-zero, and weighted matrices (such as the line susceptances of PowerGrid) are accepted where weights make sense. Every random topology takes a `numpy.random.Generator`, so graphs are reproducible when the generator is seeded. PowerGrid builds its transmission network with `make_graph`, and the functions apply equally to the matrices exposed by the other environments (`adjacency` of Consensus and Platoon, `topology_matrix` of Kuramoto).

Table 3.8: Functions of `env_lib.utils.graphs`.

Function	Result
<code>make_graph(topology, n, *, rng=None, ...)</code>	symmetric boolean (n, n) adjacency of one of TOPOLOGIES: "ring", "line", "star" (node 0 is the hub), "complete", "grid", "erdos_renyi", "small_world" (Watts–Strogatz), "random_geometric"; random topologies are resampled until connected (<code>connected=True</code>); <code>return_positions=True</code> also returns node coordinates
<code>is_connected(adjacency)</code>	whether the undirected graph is connected
<code>laplacian(adjacency)</code>	$L = D - A$ (weighted for a weighted matrix)
<code>algebraic_connectivity(adjacency)</code>	Fiedler value $\lambda_2(L)$ of the symmetrised graph; positive exactly for a connected graph, and the rate of linear consensus
<code>k_hop_adjacency(adjacency, k, *, include_self=False)</code>	boolean matrix of the pairs within k hops: the neighbourhoods of κ -hop (truncated) policies and critics
<code>degree, neighbor_lists, edge_list</code>	weighted degrees, neighbour indices per node, undirected edges (i, j) with $i < j$
<code>grid_shape(n)</code>	rows and columns of the most square grid with n nodes
<code>circular_layout(n)</code> , <code>spring_layout(adjacency)</code>	node positions for plotting (a circle, or a deterministic force-directed layout in $[-1, 1]^2$)

```
import numpy as np
import env_lib
from env_lib.utils import graphs

adj = graphs.make_graph("small_world", 12, rng=np.random.default_rng(0))
print(adj.shape, adj.dtype, graphs.is_connected(adj)) # (12, 12) bool True
print(len(graphs.edge_list(adj)), graphs.degree(adj).max()) # 24 6.0
lap = graphs.laplacian(adj) # L = D - A
print(round(graphs.algebraic_connectivity(adj), 3)) # 1.35
two_hop = graphs.k_hop_adjacency(adj, 2) # pairs within two hops
print(two_hop.sum(axis=1)) # [11 9 9 11 8 11 11 10 8 9 10 11]
xy = graphs.spring_layout(adj) # (12, 2) in [-1, 1]

grid = env_lib.make("PowerGrid-v0").unwrapped # the benchmark grid
print(round(graphs.algebraic_connectivity(grid.adjacency), 3)) # 0.668
print(round(graphs.algebraic_connectivity(grid.susceptance), 3)) # 0.424
```

The last line weights every line of the benchmark grid with its susceptance B_{ij} , which gives the algebraic connectivity of the weighted Laplacian that governs the electro-mechanical modes of the grid.

3.8 Rendering

The render mode is chosen at construction with the `render_mode` argument:

None	(default) No rendering. <code>render()</code> returns <code>None</code> and warns once.
"rgb_array"	<code>render()</code> returns the current frame as a (H, W, 3) uint8 array. Frames are drawn off-screen and need no display.
"human"	A window is opened on the first frame and updated automatically at the end of every <code>reset()</code> and <code>step()</code> , following the Gymnasium convention; do not call <code>render()</code> yourself in this mode. The frame rate is capped at <code>metadata["render_fps"]</code> .
"ansi"	(LineMsg and WirelessComm) <code>render()</code> returns a plain-text summary of the agents' actions and states.

The native vector environments accept `render_mode` as well and draw copy 0; a `SyncVectorEnv` returns one frame per copy.

Table 3.9: Render modes and frame sizes (width $\times$ height in pixels).

Environment	Modes	Frame size	Drawing backend, <code>render_fps</code>
Kuramoto	human, rgb_array	1000 $\times$ 560	matplotlib, 30
LineMsg	human, rgb_array, ansi	1000 $\times$ 560	matplotlib, 10
WirelessComm	human, rgb_array, ansi	1000 $\times$ 560	matplotlib, 10
Pistonball	human, rgb_array	$(160 + 40n) \times 560$	pygame, 20
Consensus / Formation	human, rgb_array	1000 $\times$ 560	matplotlib, 20
PowerGrid	human, rgb_array	1000 $\times$ 560	matplotlib, 20
Platoon	human, rgb_array	1000 $\times$ 560	matplotlib, 10
AJLATT	human, rgb_array	1100 $\times$ 620	matplotlib, 8

Renderer design. The matplotlib renderers are subclasses of `MatplotlibRenderer` from `env_lib.utils`. They create their axes and artists once and only update the artists' data on later frames. In "rgb_array" mode the static part of the figure (axes, ticks, labels, legends) is rasterised once and cached, and each frame redraws only the registered dynamic artists (blitting), which is typically three to six times faster than a full redraw. The renderers never modify `matplotlib.rcParams`. Pistonball draws with pygame surfaces.

Human mode and headless machines. Human mode needs an interactive matplotlib backend (for example TkAgg or QtAgg) or, for Pistonball, a display for pygame. With a non-interactive backend the environment issues a `RuntimeWarning` and shows no window. On servers and in continuous integration set

```
export MPLBACKEND=Agg SDL_VIDEODRIVER=dummy
```

and use `render_mode="rgb_array"`.

Themes. Two colour themes ship with the package: "dark" (the default) and "light", which is intended for papers and slides and was used for all figures in this manual. Select a theme with `env_lib.utils.set_theme`. The theme is read when a renderer is created, that is at the

first `render()` call of an environment, so set it before rendering starts. Custom palettes are `env_lib.utils.Theme` instances registered with `register_theme`.

```
import env_lib
from env_lib.utils import set_theme

set_theme("light")          # renderers created from now on use the light theme
env = env_lib.make("Consensus-v0", render_mode="rgb_array")
obs, info = env.reset(seed=0)
for _ in range(100):
    action = env.action_space.sample()
    obs, reward, terminated, truncated, info = env.step(action)
    if terminated or truncated:
        break
frame = env.render()        # numpy array of shape (560, 1000, 3), dtype uint8
env.close()
set_theme("dark")          # back to the default theme
```

3.9 Recording episodes

`env_lib.utils` provides two functions to turn rollouts into animations:

```
record_episode(env, policy=None, path=None, *, max_steps=None, seed=None,
               fps=None, render_every=1) -> list[numpy.ndarray]
save_animation(frames, path, fps=30.0, *, loop=0) -> pathlib.Path
```

`record_episode` resets the environment with `seed`, renders the initial state and then steps the environment with `policy` (a callable mapping an observation to an action; uniformly random actions from `env.action_space` if omitted), rendering one frame every `render_every` steps. The rollout stops when any entry of `terminated` or `truncated` is true or after `max_steps` steps. The function returns the list of frames and, when `path` is given, writes them with `save_animation` at `fps` frames per second (default `env.metadata["render_fps"]`). The environment must use `render_mode="rgb_array"`.

`save_animation` writes a sequence of equally sized RGB frames and returns the path. The format follows the file suffix: `.gif` is written with Pillow (always available); `.mp4`, `.webm`, `.avi`, `.mov` and `.mkv` are written with `imageio` and `imageio-ffmpeg` (extra video); frames are cropped to even dimensions for video codecs. `loop` is the GIF loop count (0 loops forever). Missing parent directories are created.

```
import env_lib
from env_lib.utils import record_episode, save_animation

env = env_lib.make("Formation-v0", render_mode="rgb_array")
policy = env_lib.baseline_policy(env)          # Laplacian controller
frames = record_episode(env, policy, "renders/formation.gif",
                        max_steps=150, seed=0, render_every=2)
save_animation(frames[:2], "renders/formation_fast.gif", fps=20)
env.close()
```

The command `env-lib run Formation-v0 --gif renders/formation.gif` records the same kind of episode without writing code (Section 11.3).

3.10 Registered environments

Table 3.10 lists all 18 ids returned by `env_lib.list_envs()` with the constructor arguments each registration sets; all other arguments keep their defaults. The last column shows how `env_lib.make_vec` vectorises the id: with a native implementation or with `SyncVectorEnv`. The version suffixes follow the historical ids of the project and denote preset configurations, not revisions: `WirelessComm-v1`, for example, is the 4×4 grid and not a newer version of `WirelessComm-v0`. Gymnasium may therefore emit an “out of date” `DeprecationWarning` when a `-v0` id is created with `gymnasium.make`; the warning is harmless, and `env_lib.make` suppresses it.

Table 3.10: Registered Gymnasium ids and their configuration.

Id	Class and configuration	Vector
KuramotoOscillator-v0	KuramotoOscillatorEnv; defaults (10 oscillators, dynamic coupling)	native
KuramotoOscillator-v1	KuramotoOscillatorEnv; <code>n_oscillators=5</code>	native
KuramotoOscillator-Constant-v0	KuramotoOscillatorEnv; <code>n_oscillators=6</code> , <code>coupling_mode="constant"</code>	native
KuramotoOscillator-FreqSync-Constant-v0	KuramotoOscillatorEnv; <code>n_oscillators=6</code> , <code>coupling_mode="constant"</code> , <code>reward_type="frequency_synchronization"</code> , <code>target_frequency=2.0</code>	native
KuramotoOscillatorTorch-v0	KuramotoOscillatorEnvTorch; defaults (10 oscillators, one system)	sync
KuramotoOscillatorTorch-v1	KuramotoOscillatorEnvTorch; <code>n_oscillators=8</code> , <code>n_agents=4</code> , <code>device="cpu"</code>	sync
KuramotoOscillatorTorch-v2	KuramotoOscillatorEnvTorch; <code>n_oscillators=10</code> , <code>n_agents=1</code> , <code>device="auto"</code> , <code>integration_method="rk4"</code>	sync
KuramotoOscillatorTorch-Constant-v0	KuramotoOscillatorEnvTorch; <code>n_oscillators=6</code> , <code>coupling_mode="constant"</code>	sync
KuramotoOscillatorTorch-FreqSync-Constant-v0	KuramotoOscillatorEnvTorch; <code>n_oscillators=6</code> , <code>coupling_mode="constant"</code> , <code>reward_type="frequency_synchronization"</code> , <code>target_frequency=2.0</code>	sync
LineMsg-v0	LineMsgEnv; defaults (10 agents)	sync
WirelessComm-v0	WirelessCommEnv; defaults (6×6 grid)	sync
WirelessComm-v1	WirelessCommEnv; <code>grid_x=4</code> , <code>grid_y=4</code>	sync
Pistonball-v0	PistonballEnv; defaults (20 pistons); needs pymunk (and pygame to render)	sync
Consensus-v0	ConsensusEnv; defaults (8 agents, rendezvous on a ring)	native
Formation-v0	ConsensusEnv; <code>task="formation"</code> (circle formation)	native
PowerGrid-v0	PowerGridEnv; defaults (16 buses, small-world network fixed by <code>network_seed=0</code>)	native

Id	Class and configuration	Vector
Platoon-v0	<code>PlatoonEnv</code> ; defaults (8 followers, predecessor following, mixed leader scenario)	native
AJLATT-v0	<code>AJLATTEnv</code> ; defaults (4 robots on <code>obstacles04</code>); passive environment checker disabled because rewards and <code>terminated</code> are per-robot arrays	sync

Chapter 4

Kuramoto Oscillator Networks

The Kuramoto environments simulate a network of coupled phase oscillators that a controller has to synchronise. They are used to study learning-based control of network synchronisation: shaping the coupling of a network (coupling design), steering oscillators with bounded inputs, tracking a common frequency, and comparing learned controllers with classical results such as the synchronisation transition of the Kuramoto model. Two interchangeable backends implement the same model:

- `KuramotoOscillatorEnv` (module `env_lib.kos_env.kuramoto_env`): NumPy, one network per environment;
- `KuramotoOscillatorEnvTorch` (module `env_lib.kos_env.kuramoto_env_torch`): PyTorch, `n_agents` independent networks simulated in parallel on the CPU or a GPU; requires the `torch extra`.

With identical initial states and actions both backends produce the same trajectories up to `float32` rounding.

4.1 Model

The state consists of the phases $\theta_1, \dots, \theta_N$ of N oscillators with natural frequencies ω_i . The controlled dynamics are

$$\dot{\theta}_i = \omega_i + a_i + c \sum_{j=1}^N K_{ij} \sin(\theta_j - \theta_i), \quad i = 1, \dots, N, \quad (4.1)$$

where a_i is the control input of oscillator i , K the coupling matrix and $c = 1$, or $c = 1/N$ with `normalize_coupling=True` (the classical K/N scaling). Positive K_{ij} are attractive and negative K_{ij} repulsive couplings; the diagonal of K has no effect.

Integration. One environment step advances the system by $\Delta t = dt$ with the explicit Euler method, $\theta \leftarrow \theta + \Delta t f(\theta)$, or with the classical fourth-order Runge–Kutta method (`integration_method="rk4"`),

$$\begin{aligned} \theta &\leftarrow \theta + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4), \\ k_1 &= f(\theta), \quad k_2 = f(\theta + \frac{\Delta t}{2} k_1), \quad k_3 = f(\theta + \frac{\Delta t}{2} k_2), \quad k_4 = f(\theta + \Delta t k_3), \end{aligned}$$

where f is the right-hand side of (4.1) with the action held constant during the step. If `noise_std` = $\sigma > 0$, independent Gaussian noise $\mathcal{N}(0, \sigma^2)$ is added to every phase after the integration step (the noise is not scaled by $\sqrt{\Delta t}$). Finally the phases are wrapped to $[-\pi, \pi)$.

Synchronisation measures. The complex order parameter

$$r e^{i\psi} = \frac{1}{N} \sum_{j=1}^N e^{i\theta_j} \quad (4.2)$$

has magnitude $r \in [0, 1]$: $r = 1$ for identical phases and $r \approx 0$ for incoherent phases; ψ is the mean phase. The phase coherence is

$$C = \exp(-\text{Var}(\theta)), \quad (4.3)$$

where Var is the population variance (normalised by N) of the phases wrapped to $[-\pi, \pi)$. For frequency tracking the environment uses the mean absolute frequency error

$$E_f = \frac{1}{N} \sum_{i=1}^N |\dot{\theta}_i - \Omega|, \quad (4.4)$$

with target frequency $\Omega = \text{target_frequency}$; $\dot{\theta}_i$ is evaluated at the start of the step, with the new action applied (for RK4 this is k_1).

4.2 Coupling modes and topologies

The network topology is an undirected graph with adjacency matrix A . Only connected pairs are coupled. The available topologies are listed in Table 4.1; `adj_matrix` overrides `topology`, which is then reported as `"custom"`.

Table 4.1: Network topologies. M is the number of undirected edges.

topology	Edges	M
"fully_connected"	every pair of oscillators	$N(N-1)/2$
"ring"	i is connected to $i \pm 1 \bmod N$	N (for $N \geq 3$)
"star"	oscillator 0 is connected to all others	$N-1$
"random"	every ordered pair is drawn with probability 1/2 and the matrix is symmetrised with a logical OR (a pair is connected with probability 3/4); the draw uses a local generator seeded with <code>topology_seed</code>	random
"custom"	non-zero entries of <code>adj_matrix</code> (either direction)	varies

Dynamic coupling. With `coupling_mode="dynamic"` (default) the coupling strength of every edge is part of the action. The edges are listed in row-major order of the upper triangle of A ; the list is available as `env.coupling_indices` (pairs (i, j) with $i < j$) and its length as `env.n_couplings` = M . The strength k_e of edge $e = (i, j)$ sets $K_{ij} = K_{ji} = k_e$. A non-symmetric `adj_matrix` triggers a warning, because every connected pair is one undirected edge with a single strength.

Constant coupling. With `coupling_mode="constant"` the coupling matrix is fixed and only the control inputs are actions. K is the matrix passed as `constant_coupling_matrix`, with its diagonal set to zero, or $K = \text{coupling_strength} \cdot A$ when no matrix is given. A matrix passed in dynamic mode is ignored with a warning.

4.3 Spaces

Action. A `float32 Box` with the layout of Table 4.2. Actions are clipped to the bounds; an action of the wrong shape or with non-finite values raises `ValueError`.

Table 4.2: Action layout.

Indices	Content	Bounds
0 : N	control inputs $a_1, \dots, a_N$	<code>control_input_range</code> , default $[-1, 1]$
N : N+M	coupling strengths $k_1, \dots, k_M$ (dynamic mode only)	<code>coupling_range</code> , default $[0, 5]$

Observation. An unbounded `float32 Box` of size $3N + M$ in dynamic mode and $3N$ in constant mode (Table 4.3). The default `KuramotoOscillator-v0` ($N = 10$, fully connected, $M = 45$) has an action of size 55 and an observation of size 75.

Table 4.3: Observation layout (dynamic mode; in constant mode the coupling block is absent and the control inputs occupy $2N : 3N$).

Indices	Content
0 : N	phases θ_i , wrapped to $[-\pi, \pi)$
N : 2N	natural frequencies ω_i
2N : 2N+M	coupling strengths of the last action (sampled at reset)
2N+M : 3N+M	control inputs of the last action (zero after reset)

4.4 Reward and episode end

The reward is computed after the step from the new phases. It has three ingredients: a *signal* S that measures synchronisation, a synchronisation bonus, and a *reward mode* that decides how the two are paid over the episode.

Signal. The signal is selected with `reward_type`:

$$\begin{aligned}
 \text{"order_parameter": } S &= r \in [0, 1], \\
 \text{"phase_coherence": } S &= C \in (0, 1], \\
 \text{"combined": } S &= r + C \in (0, 2], \\
 \text{"frequency_synchronization": } S &= -E_f \leq 0,
 \end{aligned}$$

with r , C and E_f from (4.2)–(4.4). $S_{\max}$ denotes the largest value of the signal (1, 1, 2 and 0).

Synchronisation and episode end. The state is *synchronised* when r exceeds `sync_threshold` (default 0.99; a threshold above 1 disables it), reported as `info["synchronized"]`. A synchronised step earns the bonus $b = \text{sync_bonus}$ (default 10) for every reward type except `"frequency_synchronization"`, and with `terminate_on_sync=True` (default) it ends the episode (`terminated=True`). With `terminate_on_sync=False` the episode always runs to `max_steps`, and the bonus is paid on every synchronised step, which turns the task into holding synchronisation. `truncated` becomes `True` after `max_steps` steps.

Reward modes. `reward_mode` decides how the signal and the bonus b_t of step t (zero when the step is not synchronised) are paid (Table 4.4). `control_cost` ≥ 0 subtracts $c \cdot \frac{1}{N} \sum_i a_i^2$ from every reward in every mode.

Table 4.4: Reward modes of the Kuramoto environments (S_t is the signal after step t , S_0 that of the initial state).

Mode	Reward of step t	Use
"dense" (default)	$S_t + b_t$	The original reward of every registered id.
"penalty"	$S_t - S_{\max} + b_t$	Never positive before the bonus: every step before synchronisation costs, so synchronising sooner earns more. Recommended for training.
"progress"	$S_t - S_{t-1} + b_t$	Potential-based shaping: the undiscounted return is $S_T - S_0$ plus the bonuses, so it rewards reaching synchronisation, and speed only through discounting.
"terminal"	$S_T + b_T$ on the last step T of the episode, 0 before	A terminal reward only: the quality of the final state.
"sparse"	b_t	Success only (not with <code>"frequency_synchronization"</code>).

Choosing a mode. With the default `"dense"` mode and `terminate_on_sync=True`, the signal is paid at every step and synchronisation ends the episode, so a policy that keeps r just below the threshold for the whole episode (up to 1000 steps of about 0.98) earns far more than one that synchronises (a few steps and the bonus of 10). The registered ids keep this reward for compatibility, and `env-lib describe` prints a caution. For learning, use `"penalty"`, `"progress"` or `"terminal"`, or `terminate_on_sync=False`. On `KuramotoOscillator-v0` (64 seeded episodes), random actions earn more than the classical controller in `"dense"` mode (24.0 against 17.2), because they synchronise in 21 steps instead of 10, and less in `"penalty"` mode (3.2 against 6.9).

```
import env_lib

env_id = "KuramotoOscillator-v0"
env = env_lib.make(env_id, reward_mode="penalty")
env = env_lib.make(env_id, reward_mode="terminal")           # final reward only
env = env_lib.make(env_id, terminate_on_sync=False,          # hold synchronisation
                    sync_bonus=0.0, control_cost=0.1)
```

4.5 Parameters

Both backends accept the parameters of Table 4.5; all have defaults. Invalid values raise `ValueError` or `TypeError`.

Table 4.5: Constructor parameters of `KuramotoOscillatorEnv` and `KuramotoOscillatorEnvTorch`.

Parameter	Default	Description
<code>n_oscillators</code>	10	Number of oscillators N (≥ 2).
<code>n_agents</code>	1	PyTorch: number of systems simulated in parallel. NumPy: only sets the length of <code>info["agent_rewards"]</code> .
<code>dt</code>	0.01	Integration time step (> 0).
<code>max_steps</code>	1000	Episode length; reaching it sets <code>truncated</code> .
<code>coupling_range</code>	(0.0, 5.0)	Bounds of the coupling actions and of the random initial coupling strengths.
<code>control_input_range</code>	(-1.0, 1.0)	Bounds of the control inputs a_i .
<code>natural_freq_range</code>	(0.5, 2.0)	Natural frequencies are drawn uniformly from this range at every reset.
<code>device</code>	"cpu"	PyTorch: "cpu", "cuda", "cuda:k", a <code>torch.device</code> , or "auto" (CUDA when available). NumPy: ignored.
<code>render_mode</code>	None	None, "rgb_array" or "human".
<code>integration_method</code>	"euler"	"euler" or "rk4".
<code>reward_type</code>	"order_parameter"	"order_parameter", "phase_coherence", "combined" or "frequency_synchronization".
<code>noise_std</code>	0.0	Standard deviation of the phase noise added after each step.
<code>topology</code>	"fully_connected"	"fully_connected", "ring", "star", "random" or "custom".
<code>adj_matrix</code>	None	Custom (N, N) adjacency matrix (array or tensor); overrides <code>topology</code> .
<code>target_frequency</code>	1.0	Target Ω of the frequency-synchronisation reward.
<code>coupling_mode</code>	"dynamic"	"dynamic" (couplings are actions) or "constant".
<code>constant_coupling_matrix</code>	None	Fixed (N, N) coupling matrix of constant mode.
<code>coupling_strength</code>	1.0	Uniform strength of constant mode without an explicit matrix ($K = \text{coupling_strength} \cdot A$).
<code>normalize_coupling</code>	False	Divide the coupling sum by N .
<code>topology_seed</code>	42	Seed of the local generator that builds the "random" topology.
<code>sync_threshold</code>	0.99	Order parameter above which the state is synchronised.
<code>sync_bonus</code>	10.0	Reward bonus of a synchronised step.
<code>reward_mode</code>	"dense"	"dense", "penalty", "progress", "terminal" or "sparse" (Table 4.4).
<code>terminate_on_sync</code>	True	End the episode when the state synchronises.
<code>control_cost</code>	0.0	Weight of the control-effort penalty $\frac{1}{N} \sum_i a_i^2$.
<code>render_agent</code>	0	PyTorch only: index of the system shown by <code>render()</code> .

Initial state and reset options. `reset()` draws phases uniformly from $[-\pi, \pi)$, natural frequencies uniformly from `natural_freq_range` and, in dynamic mode, coupling strengths uniformly from `coupling_range`; the control inputs start at zero. Any part can be fixed with

`reset(options=...)` using the keys `"phases"`, `"natural_frequencies"` (shape $(N,)$) and `"coupling_strengths"` (shape $(M,)$, dynamic mode only). The PyTorch backend also accepts arrays of shape $(n_agents, size)$ with one row per system.

4.6 Info dictionary

Table 4.6: Keys of the info dictionary. The PyTorch backend returns per-system NumPy arrays with a leading dimension `n_agents` for the quantities marked with an asterisk.

Key	Content
<code>order_parameter*</code>	order parameter r after the step
<code>phase_coherence*</code>	phase coherence C after the step
<code>synchronized*</code>	whether $r > \text{sync_threshold}$ after the step (not in the reset info)
<code>phases*</code>	wrapped phases, shape $(N,)$
<code>natural_frequencies*</code>	natural frequencies ω_i
<code>coupling_matrix*</code>	coupling matrix K used in the step, shape (N, N)
<code>dphases_dt*</code>	phase velocities $\dot{\theta}$ at the start of the step (not in the reset info)
<code>step_count</code>	number of steps taken in the episode
<code>device</code>	device of the simulation ("cpu" for NumPy)
<code>n_agents</code>	value of <code>n_agents</code>
<code>agent_rewards</code>	float64 array of shape $(n_agents,)$; NumPy: copies of the reward; PyTorch: the reward of every system, including its bonus (not in the reset info)

4.7 PyTorch backend

`KuramotoOscillatorEnvTorch` simulates `n_agents` independent networks with the same topology in one batch of tensors. All state tensors have a leading batch dimension.

- `step(action)` accepts an action of shape $(act_dim,)$, applied to every system, or (n_agents, act_dim) with one row per system, as a NumPy array or a tensor.
- The returned observation, scalar reward and `terminated` flag refer to system 0; `truncated` is shared. The per-system values are in `info` (Table 4.6).
- `get_batch_observations()` returns the observations of all systems as a tensor of shape (n_agents, obs_dim) on `env.device`.
- `get_batch_rewards(dphases_dt=None, include_bonus=False)` returns the per-system signal S of the current state as a tensor of shape $(n_agents,)$; `include_bonus=True` adds the bonus, which equals `info["agent_rewards"]` in the "dense" mode without control cost.
- Systems are not reset individually: a system that synchronises keeps evolving until the whole batch is reset, and its rewards follow the per-step rule of the reward mode (with `terminate_on_sync`, it earns its bonus, and in "terminal" mode its final reward, on every step it is synchronised).

Computations run in `float32`. For large batches or networks select a GPU with `device="cuda"` or `device="auto"`.

4.8 Rendering

Figure 4.1 shows the dashboard (1000×560 pixels). The phase portrait on the left places every oscillator on the unit circle at angle θ_i , coloured by its natural frequency (colour bar), with a glow and a fading trail of its recent phases. Couplings are drawn as chords whose width and opacity scale with $|K_{ij}|$; repulsive couplings use the theme’s negative colour. The arrow from the origin is the order parameter $re^{i\psi}$. On the right, the upper panel shows $r(t)$ and the phase coherence over the episode with the synchronisation threshold as a dashed line, and the lower panel a raster of $\sin \theta_i$ over the most recent steps. The header reports the step, time, r and the reward. The PyTorch backend shows system `render_agent`.

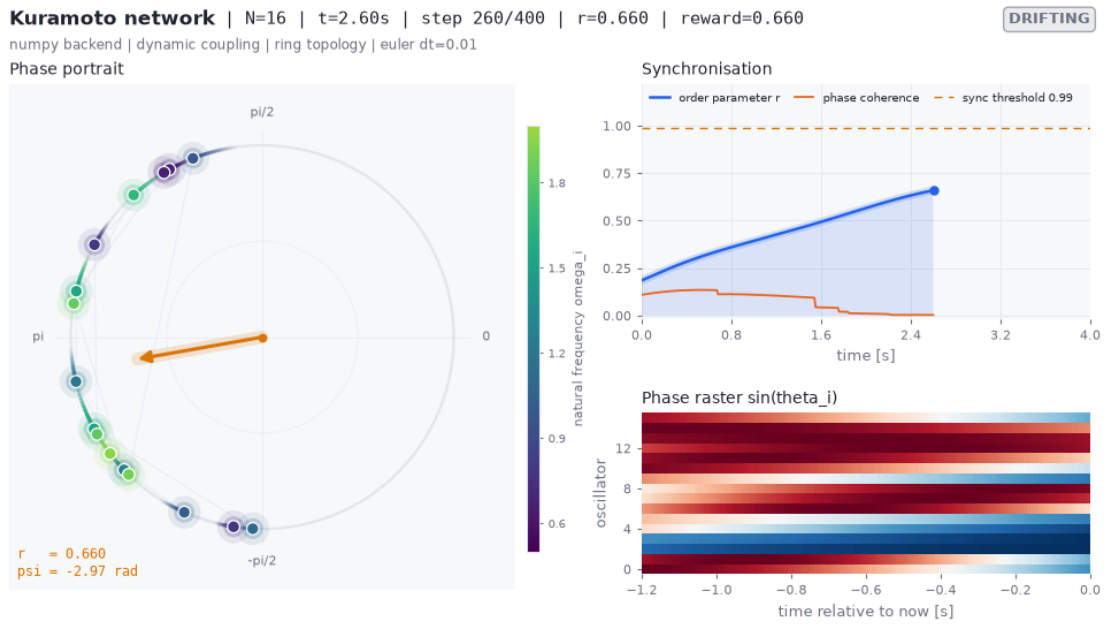


Figure 4.1: Kuramoto dashboard (16 oscillators on a ring, dynamic coupling, light theme). Left: phase portrait with coupling chords, trails and the order-parameter vector. Right: order parameter and phase coherence over time, and the phase raster $\sin \theta_i$.

4.9 Registered ids

Table 4.7 lists the registered Kuramoto ids. The `KuramotoOscillatorTorch-*` ids require the `torch` extra; keyword arguments of `env_lib.make` override their configuration.

4.10 Usage

The first example drives a network with constant coupling to synchronisation with a proportional phase-alignment input $a_i = r \sin(\psi - \theta_i)$ computed from the observation:

```
import numpy as np
import env_lib

env = env_lib.KuramotoOscillatorEnv(
    n_oscillators=12, coupling_mode="constant", coupling_strength=0.5
)
obs, info = env.reset(seed=0)
```

Table 4.7: Kuramoto ids (all other parameters keep their defaults).

Id	Configuration
KuramotoOscillator-v0	NumPy, $N = 10$, dynamic coupling
KuramotoOscillator-v1	NumPy, $N = 5$, dynamic coupling
KuramotoOscillator-Constant-v0	NumPy, $N = 6$, constant coupling
KuramotoOscillator-FreqSync-Constant-v0	NumPy, $N = 6$, constant coupling, frequency reward, $\Omega = 2$
KuramotoOscillatorTorch-v0	PyTorch, $N = 10$, one system
KuramotoOscillatorTorch-v1	PyTorch, $N = 8$, 4 systems, CPU
KuramotoOscillatorTorch-v2	PyTorch, $N = 10$, RK4, device="auto"
KuramotoOscillatorTorch-Constant-v0	PyTorch, $N = 6$, constant coupling
KuramotoOscillatorTorch-FreqSync-Constant-v0	PyTorch, $N = 6$, constant coupling, frequency reward, $\Omega = 2$

```

n = env.n_oscillators
while True:
    phases = obs[:n].astype(np.float64)
    z = np.exp(1j * phases).mean() # r exp(i psi)
    action = np.abs(z) * np.sin(np.angle(z) - phases) # phase alignment
    action = action.astype(np.float32)
    obs, reward, terminated, truncated, info = env.step(action)
    if terminated or truncated:
        break
print(info["step_count"], info["order_parameter"]) # 51, above 0.99
env.close()

```

The second example runs four systems in parallel on the PyTorch backend, one coupling level per system:

```

import numpy as np
import env_lib

env = env_lib.make("KuramotoOscillatorTorch-v1") # 8 oscillators, 4 systems
core = env.unwrapped
obs, info = env.reset(seed=0)
print(core.get_batch_observations().shape) # torch.Size([4, 52])

n, act_dim = core.n_oscillators, core.action_space.shape[0]
actions = np.zeros((core.n_agents, act_dim), dtype=np.float32)
actions[:, n:] = np.linspace(1.0, 4.0, core.n_agents)[:, None]
for _ in range(300):
    obs, reward, terminated, truncated, info = env.step(actions)
print(info["order_parameter"]) # one value per system
print(info["agent_rewards"]) # per system, with bonus
env.close()

```

The example script `examples/environments/kuramoto_demo.py` ramps the coupling strengths and adds phase alignment, and records the transition from incoherence to synchronisation:

```

python examples/environments/kuramoto_demo.py --save renders/kuramoto.gif
python examples/environments/kuramoto_demo.py --backend torch --n-agents 4 --theme
light
python examples/environments/kuramoto_demo.py --mode constant --topology

```

```
two_clusters \
--render-mode human
```

4.11 Native vector environment

The NumPy ids (`KuramotoOscillator-*`) have a native batched implementation, `KuramotoOscillatorVectorEnv`, which `env_lib.make_vec(id, B)` returns. It shares the array kernels of `KuramotoOscillatorEnv`, takes the same constructor arguments plus `num_envs` and `autoreset_mode`, and every copy reproduces a single environment started from the same state bit for bit, for both coupling modes, all reward types and both integrators. `reset(options=...)` accepts "phases", "natural_frequencies" and, in the dynamic coupling mode, "coupling_strengths", either for all copies or with a leading B axis, and combines with "reset_mask"; the batched state is available as `envs.unwrapped.phases` of shape (B, N) . The PyTorch backend of Section 4.7 is an alternative when the batch should live on a GPU. Chapter 11 covers autoreset modes, seeding and throughput.

```
import env_lib

envs = env_lib.make_vec("KuramotoOscillator-v0", num_envs=256)
obs, _ = envs.reset(seed=0) # (256, 75)
obs, reward, terminated, truncated, info = envs.step(envs.action_space.sample())
reward.shape, envs.unwrapped.phases.shape # (256,), (256, 10)
```

4.12 Reproducibility and performance

Seeding. `reset(seed=s)` seeds `self.np_random`, from which the NumPy backend draws the initial state and the phase noise. The PyTorch backend seeds its own `torch.Generator` from `self.np_random` at every reset; neither backend touches the global random state. The random topology is built once from `topology_seed`. The two backends draw from different generators, so the same seed gives different initial states; to compare them, pass the initial state through `reset(options=...)`.

Performance. Table 4.8 lists step times measured with `benchmarks/benchmark_envs.py` on the development machine (random actions, no rendering). The coupling term is evaluated with two matrix–vector products instead of N^2 sine evaluations. Rendering in "rgb_array" mode takes about 25–35 ms per frame and dominates the cost of recorded rollouts; use `render_every` in `record_episode` to thin out long episodes. Use Euler integration for speed and RK4 when accuracy at larger `dt` matters, and a GPU for large batches.

Table 4.8: Step time in milliseconds on the development machine.

Case	Before 1.0	Version 1.0
NumPy, $N = 10$	0.17	0.04
NumPy, $N = 50$	2.36	0.07–0.11
NumPy, RK4, $N = 50$	7.1	0.1
PyTorch (CPU), $N = 50$, 8 systems	10.3	0.3–0.4

4.13 Changes in 1.0

The behaviour of the Kuramoto environments changed in several ways; see Appendix B for migration advice.

- The PyTorch backend had a sign error that made the coupling repulsive ($\sin(\theta_i - \theta_j)$ instead of $\sin(\theta_j - \theta_i)$). Both backends now implement (4.1) and produce the same trajectories; normalisation by N is opt-in for both.
- The coupling uses the full matrix; the former NumPy loop skipped entries $K_{ij} \leq 0$. Default settings are unaffected.
- Synchronisation sets `terminated` and the step limit sets `truncated` (previously both were reported as `terminated`); `sync_threshold` and `sync_bonus` are configurable.
- Constant mode without a matrix uses `coupling_strength · A`, so the `*-Constant-v0` ids work (they raised before).
- The environments no longer call `np.random.seed(42)` or `torch.manual_seed(42)`; the random topology uses a local generator (`topology_seed`) and is unchanged.
- The PyTorch backend accepts `device="auto"`, and its phase coherence and bonus rules match the NumPy backend.
- The dynamics are vectorised (20 times faster at $N = 50$, 65 times with RK4) and the renderer was rewritten; the former `"rgb_array"` path failed with matplotlib 3.10 and later.
- `env_lib.kos_env.register()` is deprecated; the ids are registered when `env_lib` is imported.

Chapter 5

Networked Communication Environments

This chapter documents two discrete multi-agent benchmarks in which agents on a network cooperate under local observations:

- **LineMsg** (`env_lib.linemsg_env.LineMsgEnv`): agents on a line relay a message from a source to a sink;
- **WirelessComm** (`env_lib.wireless_comm_env.WirelessCommEnv`): agents on a grid send deadline-constrained packets to shared access points and must avoid collisions.

Both environments have small discrete state and action spaces, local observation windows controlled by `n_obs_neighbors`, per-agent rewards and a fixed episode length. They are designed for research on decentralised and networked multi-agent reinforcement learning: exploiting the locality of interactions (policies and value functions that depend on a κ -hop neighbourhood), credit assignment, communication and coordination protocols, scalability in the number of agents, and robustness to failing agents. The WirelessComm environment is the Gymnasium joint-action port of an earlier PettingZoo implementation used in the DSDP project.

5.1 LineMsg

5.1.1 Model

N agents sit on a line. Agent $N - 1$ at the right end is the source, agent 0 at the left end is the sink. Each agent holds a binary state $s_i \in \{0, 1\}$ (1: it holds the message) and chooses a binary action $a_i \in \{0, 1\}$ at every step. All states are updated synchronously from the previous states:

$$\begin{aligned} s_0 &\leftarrow s_1, \\ s_i &\leftarrow a_i \wedge (s_{i+1} \vee [u_i < 0.8]), \quad 0 < i < N - 1, \quad u_i \sim \mathcal{U}(0, 1), \\ s_{N-1} &\leftarrow a_{N-1}. \end{aligned} \tag{5.1}$$

An active middle agent ($a_i = 1$) therefore receives the message from its right neighbour for sure and otherwise obtains one with probability 0.8; an inactive agent loses its message. The sink copies agent 1 and its own action has no effect; the source holds the message exactly when it acts. At reset all agents hold the message ($s_i = 1$).

Reward. Agent i receives

$$r_i = w_i s_i, \quad w_0 = 1.0, \quad w_i = 0.1 \quad (i > 0), \quad (5.2)$$

evaluated on the new states, and the team reward is $R = \sum_i r_i$. The maximum per step is $1 + 0.1(N - 1)$, which the “always relay” policy ($a_i = 1$ for all i) attains at every step.

Episode end. `terminated` is always `False`; `truncated` becomes `True` after `max_iter` steps.

5.1.2 Spaces

Action. The joint action can be given in two formats, both of which `step()` always accepts:

- an integer in `Discrete(2**N)` whose bit i is the action of agent i (the declared space for `action_space_type="discrete"`, the default; only possible for $N \leq 62$);
- an array of N values in $\{0, 1\}$ (integer or boolean), the declared space `MultiBinary(N)` for `action_space_type="multibinary"`, which has no limit on N .

Out-of-range integers, arrays of the wrong shape and values other than 0 and 1 raise `ValueError`.

Observation. `Box(0, 3, (N, 2k+1), float32)` with $k = \text{n_obs_neighbors}$. The state vector is padded with k boundary cells on each side, and row i is the window of the $2k + 1$ cells centred on agent i :

Table 5.1: LineMsg observation (row i).

Column	Content
0 : k	states of agents $i - k, \dots, i - 1$ (left neighbours, towards the sink)
k	own state s_i
k+1 : 2k+1	states of agents $i + 1, \dots, i + k$ (right neighbours, towards the source)
Values	0: no message, 1: message, 2: boundary cell beyond the end of the line

5.1.3 Parameters and info

Table 5.2: Constructor parameters of `LineMsgEnv`.

Parameter	Default	Description
<code>num_agents</code>	10	Number of agents N (≥ 2 : a sink and a source, plus any number of relays).
<code>n_obs_neighbors</code>	1	Observation radius k ; 0 is treated as 1 with a warning.
<code>max_iter</code>	50	Episode length; reaching it sets <code>truncated</code> .
<code>render_mode</code>	<code>None</code>	<code>None</code> , <code>"human"</code> , <code>"rgb_array"</code> or <code>"ansi"</code> .
<code>action_space_type</code>	<code>"discrete"</code>	Declared action space: <code>"discrete"</code> (<code>Discrete(2**N)</code> , $N \leq 62$) or <code>"multibinary"</code> (<code>MultiBinary(N)</code>).

The info dictionary of `reset()` and `step()` contains a single key, `"agent_rewards"`: the float64 per-agent rewards r_i (zeros at reset). The attribute `state` holds the padded state vector (agent i is `state[k + i]`), `actions` the last joint action with the same padding (2 before the first step), and `num_moves` the number of steps taken. `step()` before `reset()` raises `env_lib.ResetNeededError`.

5.1.4 Rendering

The dashboard (Figure 5.1) shows the line of agents at the top, coloured by state, with a halo under agents that act and arrows on the active links, which point in the direction of message flow towards the sink. Below it a space-time raster (one row per step, one column per agent) makes message fronts visible, and the right panel traces the team reward with its maximum. The header reports the step, the last reward, the episode return and the number of agents holding the message. `render_mode="ansi"` returns a text line with the action and state of every agent.



Figure 5.1: LineMsg dashboard (12 agents, step 30 of 50). Top: agent states and active links. Bottom left: space-time raster of the agent states. Bottom right: team reward per step.

5.1.5 Usage

```
import numpy as np
from env_lib.linemsg_env import LineMsgEnv

env = LineMsgEnv(num_agents=10, action_space_type="multibinary")
obs, info = env.reset(seed=0)  # obs: (10, 3) float32
rng = np.random.default_rng(0)
episode_return = 0.0
while True:
    action = (rng.random(env.num_agents) < 0.9).astype(np.int64)  # duty cycle
    obs, reward, terminated, truncated, info = env.step(action)
    episode_return += reward
    if terminated or truncated:
        break
print(episode_return, info["agent_rewards"])

env.reset(seed=1)
env.step(0b111111110)  # integer action: all agents but the sink act
```

The example script compares a random policy, duty-cycled relays and the always-relay policy and can record an episode:

```
python examples/environments/linemsg_demo.py --policy relay --save
    renders/linemsg.gif
python examples/environments/linemsg_demo.py --num-agents 30 --episodes 50
python examples/environments/linemsg_demo.py --render-mode ansi
```

Over 20 episodes with $N = 10$ and `max_iter=50` the mean returns are 44.1 for uniformly random actions, 85.2 for duty-cycled relays that act with probability 0.9, and 95.0 for the always-relay policy (the maximum).

5.2 WirelessComm

5.2.1 Model

$G_x \times G_y$ agents sit on a grid (`grid_x` rows, `grid_y` columns); agent (i, j) has index $iG_y + j$. Access points sit at the $(G_x - 1)(G_y - 1)$ interior corners of the grid: access point (a, b) lies between agents (a, b) , $(a + 1, b)$, $(a, b + 1)$ and $(a + 1, b + 1)$. Every agent holds a packet queue with $D = \text{ddl}$ deadline slots; slot 0 holds the packet with the earliest deadline. Each agent chooses one of five actions per step (Table 5.3).

Table 5.3: WirelessComm actions of agent (i, j) . A direction without an access point (at the border of the grid) is a failed attempt.

Action	Meaning	Access point
0	stay idle	none
1	transmit up-left	$(i - 1, j - 1)$
2	transmit down-left	$(i, j - 1)$
3	transmit up-right	$(i - 1, j)$
4	transmit down-right	(i, j)

Transition. Let ℓ_m be the load of access point m , the number of agents that transmit to it in the step; every transmitting agent counts, whether or not it holds a packet. The transmission of agent k is *eligible* when the agent transmits, holds at least one packet, its access point exists and that access point has load exactly $\ell_m = 1$. Each eligible transmission succeeds with probability q ; one uniform draw is made per eligible agent, in agent order. A success removes the agent's earliest-deadline packet and earns reward 1. Afterwards every queue shifts one slot towards the deadline (packets in slot 0 expire and are lost), and a new packet arrives in the last slot with probability p . At reset every queue slot is occupied independently with probability $1/2$.

Reward and outcomes. The per-agent reward is 1 for a delivered packet and 0 otherwise; the team reward is the number of packets delivered in the step. The outcome of every agent is reported in `info["outcomes"]` (Table 5.5): idle (0), success (1), collision (2: the access point exists and has load ≥ 2) or lost (3: any other failed attempt, that is no packet, no access point or a channel loss).

Episode end. `terminated` is always `False`; `truncated` becomes `True` after `max_iter` steps.

5.2.2 Spaces

Action. `MultiDiscrete([5] * n)` with $n = G_x G_y$. An action of shape $(n,)$ or $(\text{grid_x}, \text{grid_y})$ is accepted; other shapes and values outside $0, \dots, 4$ raise `ValueError`.

Observation. `Box(0, 2, (n, D * w^2), float32)` with $w = 2k+1$ and $k = \text{n_obs_neighbors}$. The queues form a (D, G_x, G_y) array that is padded with k cells of value 2 on every side of the grid. Row m of the observation is the (D, w, w) window centred on agent m , flattened in C order (slot, then row, then column). Values are 0 (empty slot), 1 (packet) and 2 (outside the grid). The agent's own slot d is column $dw^2 + kw + k$. The default configuration ($D = 2, k = 1$) has 18 columns.

5.2.3 Parameters and info

Table 5.4: Constructor parameters of `WirelessCommEnv`.

Parameter	Default	Description
<code>grid_x, grid_y</code>	6, 6	Number of agent rows and columns (each ≥ 2 , so that at least one access point exists).
<code>ddl</code>	2	Deadline horizon D : queue slots per agent (≥ 1).
<code>packet_arrival_probability</code>	0.8	Arrival probability p of a new packet per agent and step.
<code>success_transmission_probability</code>	0.8	Success probability q of an eligible transmission.
<code>n_obs_neighbors</code>	1	Observation radius k (Chebyshev distance, ≥ 0).
<code>max_iter</code>	50	Episode length; reaching it sets <code>truncated</code> .
<code>render_mode</code>	None	None, "human", "rgb_array" or "ansi".

Table 5.5: `WirelessComm` info dictionary (the same keys, all zero, at reset).

Key	Content
<code>agent_rewards</code>	<code>float64</code> array $(n,)$: 1 for a delivered packet, else 0
<code>outcomes</code>	<code>int8</code> array $(n,)$: 0 idle, 1 success, 2 collision, 3 lost (constants <code>OUTCOME_IDLE</code> , <code>OUTCOME_SUCCESS</code> , <code>OUTCOME_COLLISION</code> , <code>OUTCOME_LOST</code> and <code>OUTCOME_LABELS</code> in <code>env_lib.wireless_comm_env.wireless_comm_env</code>)
<code>ap_load</code>	<code>int64</code> array $(\text{grid_x} - 1, \text{grid_y} - 1)$: number of agents that transmitted to each access point

The environment also exposes `last_outcomes` and `last_ap_load` (the values of the last step), `access_point_mapping(i, j, action)`, which returns the access point (a, b) reached by agent (i, j) or $(\text{None}, \text{None})$, and `check_transmission_fail(a, b, load)`. `step()` before `reset()` raises `env_lib.ResetNeededError`.

5.2.4 Rendering

The dashboard (Figure 5.2) draws the agent grid on the left. Every agent is a circle coloured by the outcome of its last action, with a small queue bar below it (one cell per deadline slot; the left cell expires first). Access points are squares at the interior corners, coloured by their load (idle, one transmitter, collision), and a line joins every transmitting agent to the access point it chose,

coloured by the outcome; the line of an attempt towards a missing access point ends outside the grid. The right panels show the packets delivered per step with a rolling mean and the rolling collision count, and the cumulative return. `render_mode="ansi"` returns the action and queue of every agent as text.

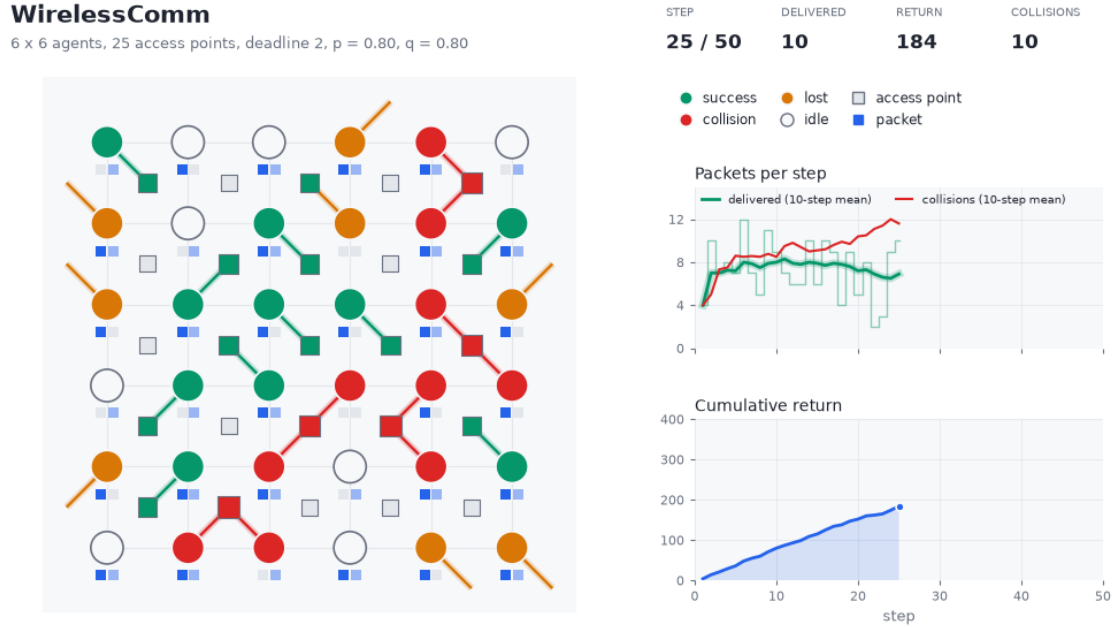


Figure 5.2: WirelessComm dashboard (6 × 6 agents, 25 access points, step 25 of 50). Left: agents coloured by outcome with their packet queues, access points coloured by load and transmission links. Right: packets delivered and collisions per step (10-step means) and the cumulative return.

5.2.5 Usage

The following policy is a collision-free rotating schedule: at step t every agent that holds a packet uses the same direction $(4, 1, 2, 3)[t \bmod 4]$ if an access point exists in that direction. A common direction maps agents to access points one-to-one, so no collisions occur.

```
import numpy as np
from env_lib.wireless_comm_env import WirelessCommEnv

env = WirelessCommEnv(grid_x=6, grid_y=6)
obs, info = env.reset(seed=0) # obs: (36, 18) float32
k, w = env.n_obs_nghbr, env.window
own = [d * w * w + k * w + k for d in range(env.ddl)] # own queue columns
directions = (4, 1, 2, 3)
cells = [(i, j) for i in range(env.grid_x) for j in range(env.grid_y)]
reachable = {d: np.array([env.access_point_mapping(i, j, d)[0] is not None
                        for i, j in cells])
             for d in directions}
episode_return = 0.0
for t in range(env.max_iter):
    d = directions[t % 4]
    has_packet = obs[:, own].max(axis=1) > 0.5
    action = np.where(has_packet & reachable[d], d, 0)
    obs, reward, terminated, truncated, info = env.step(action)
    episode_return += reward
print(episode_return, np.bincount(info["outcomes"], minlength=4))
```

The example script compares uniformly random actions, p -persistent slotted ALOHA and the rotating schedule:

```
python examples/environments/wireless_comm_demo.py --policy schedule \
    --save renders/wireless.gif
python examples/environments/wireless_comm_demo.py --grid 8 8 --tau 0.5 --episodes
    50
```

Over 20 episodes on the 6×6 grid the mean returns are 385 for random actions, 398 for ALOHA with transmit probability 0.6 and 910 for the rotating schedule, which causes no collisions.

5.3 Registered ids

Table 5.6: Registered ids of the networked environments.

Id	Class	Configuration
LineMsg-v0	LineMsgEnv	defaults (10 agents)
WirelessComm-v0	WirelessCommEnv	defaults (6×6 grid, 36 agents)
WirelessComm-v1	WirelessCommEnv	grid_x=4, grid_y=4 (16 agents)

5.4 Reproducibility and performance

Both environments draw all randomness (relay successes in LineMsg; initial queues, transmission successes and packet arrivals in WirelessComm) from `self.np_random`, seeded by `reset(seed=...)`. Seeded trajectories are bit-identical to those of the implementation before version 1.0, so earlier results can be reproduced exactly.

Measured on the development machine without rendering, a LineMsg step with 10 agents takes 0.013 ms (0.019 ms before 1.0), and a WirelessComm step takes 0.05 ms on the 6×6 grid (0.154 ms before) and 0.046 ms on a 12×12 grid (0.56 ms before). Rendering a dashboard frame in "rgb_array" mode takes about 15 ms for either environment; the history needed by the dashboards is only recorded in the "human" and "rgb_array" modes.

5.5 Changes in 1.0

The seeded dynamics of both environments are unchanged. Interface changes are summarised below; see Appendix B for migration advice.

LineMsg.

- Action decoding and observations are vectorised.
- The joint action may be an integer or an array of N bits. The new option `action_space_type="multibinary"` declares a `MultiBinary(N)` space and supports more than 62 agents.
- `step()` before `reset()` raises instead of resetting implicitly.
- The former text output is `render_mode="ansi"`; "human" and "rgb_array" show the new dashboard.

WirelessComm.

- `step()` is fully vectorised (up to 37 times faster on large grids).
- Fixed: the stored actions were never updated, so rendering showed no actions. Per-agent outcomes and access-point loads are reported in `info`.
- Arguments and actions are validated; "ansi" text mode and a new dashboard were added.
- `env_lib.wireless_comm_env.register()` is deprecated; the ids are registered when `env_lib` is imported.

Chapter 6

Pistonball

Pistonball is a cooperative physics game. A row of pistons forms the floor of a walled arena and a ball is dropped near the right wall; every piston is an agent that moves up or down, and together the pistons must roll the ball to the left wall. Each piston observes the ball only when it is close to it, and the reward of a piston depends on the ball's progress while it observes the ball. The environment is used to study cooperation under local observability, credit assignment with local rewards, scalability in the number of agents and the effect of the observation radius κ on learning. The physics is simulated with `pymunk`; `pygame` is only needed for rendering and keyboard control (extra `pistonball`).

The class is `env_lib.pistonball_env.PistonballEnv`; the package also exports `ManualPolicy`, `PistonballRenderer` and `PistonballLayout`.

6.1 Arena and physics

All coordinates are pygame screen pixels with y growing downwards. With n pistons the screen is $W = 160 + 40n$ pixels wide and 560 pixels high; the walls are 80 pixels from each screen edge, so the arena spans $x \in [80, W - 80]$ and $y \in [80, 480]$. The left wall is the goal.

- **Pistons.** Piston i is 40 pixels wide with its left edge at $x = 80 + 40i$. Its height y_i (the y coordinate of the piston head) moves by 4 pixels per unit action and is limited to $y_i \in [387, 451]$, a travel of 64 pixels around the middle position 419. At reset every piston starts at $451 - d_i$ with d_i drawn uniformly from $\{0, 4, \dots, 28\}$.
- **Ball.** A disc of radius 40 pixels with mass `ball_mass`, friction `ball_friction` and elasticity `ball_elasticity`. It starts at $x = W - 150 + \delta_x$, $y = 370 + \delta_y$, where $\delta_x \in \{-30, \dots, 30\}$ and $\delta_y \in \{-15, \dots, 15\}$ are uniform integers when `random_drop=True` and zero otherwise, with an angular velocity drawn uniformly from $[-6\pi, 6\pi]$ rad/s when `random_rotate=True`.
- **Simulation.** Gravity is 750 px/s² downwards, walls and pistons have friction 0.64, and one environment step is one physics step of $\Delta t = 1/20$ s. The pistons are kinematic bodies that are moved to their new height before the physics step; the ball speed is capped at 1000 px/s and its position is kept inside the screen.

6.2 Spaces

Action. One value per piston, stacked in an array of shape `(n,)`:

- continuous (`continuous=True`, default): `Box(-1, 1, (n,), float32)`; the value is clipped to $[-1, 1]$ and moves the piston by $4a_i$ pixels, where $+1$ moves it up;
- discrete (`continuous=False`): `MultiDiscrete([3] * n)` with $0 = \text{down}$, $1 = \text{stay}$ and $2 = \text{up}$ (4 pixels).

Actions of the wrong shape, continuous actions with NaN or infinite values, and discrete values outside $\{0, 1, 2\}$ raise `ValueError`.

Observation. `float32` array of shape $(n, 7)$ in an unbounded `Box`. Row i is the observation of piston i (Table 6.1). The piston under the ball has index

$$p(x) = \text{clip}(\lfloor (x - 85)/40 \rfloor, 0, n - 1), \quad (6.1)$$

and piston i observes the ball when $|i - p(x)| \leq \kappa$, where x is the ball centre and $\kappa = \text{kappa}$. Columns 2–6 are zero for pistons that do not observe the ball. The method `observable_mask()` returns the boolean mask of the observing pistons; its optional argument `ball_x` evaluates the mask for another ball position.

Table 6.1: Pistonball observation (row i). (x, y) is the ball centre, (v_x, v_y) its velocity in px/s and $\dot{\phi}$ its angular velocity.

Column	Value	Meaning
0	$(y_i - 419)/32$	piston height in $[-1, 1]$; $+1$ is the lowest position
1	$(5 + 40i)/(40n)$	piston position along the arena in $[0, 1]$
2	$(x - 80)/(40n)$	ball x position (observers only)
3	$(y - 80)/400$	ball y position (observers only)
4	$v_x/15$	ball x velocity (observers only)
5	$v_y/8$	ball y velocity (observers only)
6	$\dot{\phi}/8$	ball angular velocity (observers only)

6.3 Reward and episode end

Let $b_t = \lfloor x_t - 40 \rfloor$ be the x coordinate of the ball’s left edge after step t and $\Delta b = b_{t-1} - b_t$ its leftward displacement. The ball term rewards leftward progress at half rate and penalises rightward motion in full:

$$g_t = \begin{cases} 0.5 \Delta b, & \Delta b > 0, \\ \Delta b, & \text{otherwise.} \end{cases} \quad (6.2)$$

The local reward of piston i is

$$r_i = \tau + g_t \mathbf{1}[i \in O(b_{t-1}) \cup O(b_t)] + \rho_T \mathbf{1}[\text{goal}, i \in O(x_t)] + \rho_L \mathbf{1}[\text{goal}, i = 0] \\ + \mu \frac{|\Delta y_i|}{4} \mathbf{1}[|\Delta y_i| > \epsilon_\mu], \quad (6.3)$$

where $O(z) = \{i : |i - p(z)| \leq \kappa\}$ is the set of pistons within κ hops of the piston under position z (the ball term uses the left-edge positions b , the goal bonus the ball centre x_t), $\tau = \text{time_penalty}$, $\rho_T = \text{termination_reward}$, $\rho_L = \text{leftmost_piston_reward}$, $\mu = \text{movement_penalty}$, $\epsilon_\mu = \text{movement_penalty_threshold}$ and Δy_i the movement of piston i in pixels. The team reward returned by `step()` is $R = \sum_i r_i$.

Goal and episode end. With `terminated_condition=True` (default) the goal is reached when the ball's left edge will reach the left wall within one step, $x_t - 40 + v_x \Delta t \leq 80.5$; the episode then terminates. `truncated` becomes `True` after `max_cycles` steps. For example, with `leftmost_piston_reward=1.0` and the default termination reward, piston 0 receives $0.5 + 1.0 = 1.5$ on reaching the goal, the other pistons that observe the ball receive 0.5, and the remaining pistons receive no bonus.

6.4 Parameters

Table 6.2: Constructor parameters of `PistonballEnv`.

Parameter	Default	Description
<code>n_pistons</code>	20	Number of pistons (agents), ≥ 1 .
<code>time_penalty</code>	-0.1	Reward τ added to every piston at every step.
<code>continuous</code>	<code>True</code>	Continuous actions in $[-1, 1]$; <code>False</code> for <code>MultiDiscrete([3] * n)</code> .
<code>random_drop</code>	<code>True</code>	Randomise the initial ball position (± 30 px horizontally, ± 15 px vertically).
<code>random_rotate</code>	<code>True</code>	Random initial angular velocity in $[-6\pi, 6\pi]$ rad/s.
<code>ball_mass</code>	0.75	Ball mass (> 0).
<code>ball_friction</code>	0.3	Ball friction (≥ 0).
<code>ball_elasticity</code>	1.5	Ball elasticity (≥ 0).
<code>max_cycles</code>	125	Episode length; reaching it sets <code>truncated</code> .
<code>render_mode</code>	<code>None</code>	<code>None</code> , <code>"human"</code> (pygame window) or <code>"rgb_array"</code> .
<code>movement_penalty</code>	0.0	Reward μ per 4 pixels of piston movement, typically negative; 0 disables it.
<code>movement_penalty_threshold</code>	0.01	Movements of at most this many pixels are not penalised.
<code>kappa</code>	1	Observation radius κ in pistons (≥ 0).
<code>terminated_condition</code>	<code>True</code>	End the episode when the ball reaches the left wall.
<code>leftmost_piston_reward</code>	0.0	Bonus ρ_L for piston 0 when the goal is reached.
<code>termination_reward</code>	0.5	Bonus ρ_T for every piston that observes the ball when the goal is reached.

Physics presets. The ball parameters change the character of the task considerably. A heavy, damped ball (`ball_mass=2.0`, `ball_friction=0.8`, `ball_elasticity=0.5`) rolls slowly and rewards coordinated ramps; a light, bouncy ball (`ball_mass=0.5`, `ball_friction=0.1`, `ball_elasticity=2.0`) is harder to control.

Info dictionary. `step()` returns `"local_rewards"` and `"agent_rewards"` (both the `float64` local rewards r_i , shape `(n,)`) and `"total_reward"` (their sum, equal to the returned reward). The info dictionary of `reset()` is empty. `step()` before `reset()` raises `env_lib.ResetNeededError`.

6.5 Keyboard control

`ManualPolicy(env, agent_id=0, show_obs=False)` lets a person control one piston at a time in a window. Create the environment with `render_mode="human"` so that the pygame window

exists and receives the key presses, and call the policy once per step: **W** or **Up** raises the selected piston, **S** or **Down** lowers it (hold the key to keep moving), **A/Left** and **D/Right** select the previous or next piston, **Backspace** resets the environment, and **Esc** or closing the window sets `policy.running` to `False`. All other pistons stay in place.

```
from env_lib.pistonball_env import ManualPolicy, PistonballEnv

env = PistonballEnv(n_pistons=8, render_mode="human")
policy = ManualPolicy(env)
obs, info = env.reset(seed=0)
while policy.running:
    obs, reward, terminated, truncated, info = env.step(policy(obs))
    if terminated or truncated:
        obs, info = env.reset()
env.close()
```

6.6 Rendering

The renderer draws with pygame and needs no image assets (Figure 6.1). It shows a gradient background with shaded walls and a highlighted goal strip on the left wall, the pistons as rounded rectangles (pistons that observe the ball in the accent colour with a glow), the ball with shading, a rotation marker and a fading motion trail, a heads-up display with the step counter, episode return, ball x velocity and number of observing pistons, and a progress bar with the ball's distance to the goal. Frames have the screen size, $(160 + 40n) \times 560$ pixels. The `"rgb_array"` mode never initialises the display and works on headless machines; `"human"` mode opens a window capped at 20 frames per second. Colours follow the active theme (Section 3.8).



Figure 6.1: Pistonball with 12 pistons (light theme) at the moment the ball reaches the goal. The three pistons that observe the ball ($\kappa = 1$) are highlighted; the ramp formed by the pistons rolls the ball to the left wall.

6.7 Registered id

Pistonball-v0 creates `PistonballEnv` with its defaults (20 pistons, continuous actions, $\kappa = 1$). Creating it requires `pymunk`; rendering also requires `pygame`.

6.8 Usage

The following heuristic reads the ball position from the observing pistons and shapes the pistons into a ramp that falls towards the left: pistons right of the ball move up, pistons left of it move down.

```
import numpy as np
from env_lib.pistonball_env import PistonballEnv

env = PistonballEnv(n_pistons=12, kappa=1)
obs, info = env.reset(seed=0)           # obs: (12, 7) float32
episode_return = 0.0
while True:
    action = np.zeros(env.n_pistons, dtype=np.float32)
    observers = np.any(obs[:, 2:] != 0.0, axis=1)
    if observers.any():
        ball_x = obs[observers, 2].mean()      # (x - 80) / (40 n)
        offset = (obs[:, 1] - ball_x) * env.n_pistons # in piston widths
        action = np.clip(2.0 * offset, -1.0, 1.0).astype(np.float32)
    obs, reward, terminated, truncated, info = env.step(action)
    episode_return += reward
    if terminated or truncated:
        break
print(env.frames, terminated, episode_return)  # goal after 99 steps
env.close()
```

The example script evaluates a similar heuristic against random actions, records episodes and offers keyboard control:

```
python examples/environments/pistonball_demo.py --save renders/pistonball.gif
python examples/environments/pistonball_demo.py --discrete --kappa 2
    --movement-penalty -0.05
python examples/environments/pistonball_demo.py --manual --n-pistons 8
```

Over 20 episodes with the default configuration the heuristic of the script reaches a mean return of 857.1 ± 254.3 and brings the ball to the goal in 95% of the episodes, whereas random actions obtain -290.5 and never reach the goal.

6.9 Reproducibility and performance

`reset(seed=...)` seeds `self.np_random`, from which the initial piston heights, the ball position and its angular velocity are drawn; the physics itself is deterministic. The random draws are made in the same order as before version 1.0, so seeded episodes start from the same states as in earlier versions.

Measured on the development machine with 20 pistons, a step without rendering takes 0.06 ms (1.11 ms before 1.0, when the screen was drawn at every step). Rendering an `"rgb_array"` frame takes about 3 ms.

6.10 Changes in 1.0

See Appendix B for migration advice.

- Nothing is drawn unless rendering is requested; pygame is initialised lazily. Together with vectorised observations and rewards this makes steps 16 to 30 times faster.
- In discrete mode the declared action space is `MultiDiscrete([3] * n)`, one entry per piston. The per-piston action arrays accepted by `step()` are unchanged; the former single-integer `Discrete(3**n)` declaration was removed.
- Actions are validated (shape, range, NaN), and `step()` before `reset()` raises.
- New vector renderer (shaded pistons, highlighted κ -hop observers, ball trail, heads-up display, progress bar); the PNG sprites were removed. The former drawing methods (`draw()`, `draw_background()`, `draw_pistons()`, `enable_render()`) are deprecated no-ops.
- Compatible with pymunk 6 and 7.

Chapter 7

Consensus and Formation Control

`env_lib.consensus_env.ConsensusEnv` is a networked multi-agent control benchmark, new in version 1.0. N agents move in a bounded plane and must either agree on a common position (*consensus* or *rendezvous*) or assume a geometric formation up to translation (*formation control*). Every agent only sees the agents it is connected to in a communication graph, which is either fixed or rebuilt every step from the agents' distances. The environment connects multi-agent reinforcement learning with the classical theory of distributed control: the Laplacian consensus protocol is included as a baseline controller, the algebraic connectivity of the graph is reported at every step, and the difficulty can be tuned through the topology, the dynamics (single or double integrators), noise and the formation shape. Typical research uses are learning distributed controllers from local observations, studying the effect of graph connectivity and time-varying graphs on learning, and comparing learned policies with model-based protocols.

7.1 Model

Agent i has position $x_i \in \mathbb{R}^2$, velocity $v_i \in \mathbb{R}^2$ and a formation offset d_i . The offsets are zero for consensus and the slots of the chosen formation shape, centred at the origin, for formation control. The task is solved when the shifted positions $y_i = x_i - d_i$ agree. The task error

$$e = \frac{1}{N} \sum_{i=1}^N \|y_i - \bar{y}\|^2, \quad \bar{y} = \frac{1}{N} \sum_{j=1}^N y_j, \quad (7.1)$$

measures the disagreement; the task is solved when $e < \text{tolerance}$.

Dynamics. The action u_i of agent i is clipped to $[-u_{\max}, u_{\max}]^2$ with $u_{\max} = \text{max_control}$. With time step $\Delta t = \text{dt}$, noise intensity $\sigma = \text{noise_std}$, $w_i \sim \mathcal{N}(0, I_2)$ and the arena $[-a, a]^2$ ($a = \text{arena_size}$):

- single integrator (`dynamics="single"`, the action is a velocity):

$$x_i \leftarrow \text{clip}_{[-a, a]}(x_i + \Delta t u_i + \sigma \sqrt{\Delta t} w_i); \quad (7.2)$$

the reported velocity is the realised one, $v_i = (x_i^+ - x_i)/\Delta t$;

- double integrator (`dynamics="double"`, the action is an acceleration, damping $c = \text{damping}$):

$$v_i \leftarrow v_i + \Delta t (u_i - c v_i) + \sigma \sqrt{\Delta t} w_i, \quad x_i \leftarrow \text{clip}_{[-a, a]}(x_i + \Delta t v_i) \quad (7.3)$$

(semi-implicit Euler); a velocity component is set to zero when the arena wall stops the agent.

Communication graph. The agents communicate over an undirected graph with adjacency matrix A and neighbour sets $\mathcal{N}_i$ (Table 7.1). Static graphs are built once at construction; the proximity graph is rebuilt after every step. The environment reports the algebraic connectivity $\lambda_2(L)$, the second smallest eigenvalue of the graph Laplacian $L = D - A$ (D the diagonal degree matrix). It is positive exactly when the graph is connected and bounds the convergence rate of linear consensus.

Table 7.1: Communication topologies.

topology	Graph
"ring"	i is connected to $i \pm 1 \bmod N$
"line"	i is connected to $i \pm 1$ (a path)
"star"	agent 0 is the hub connected to all others
"complete"	all pairs are connected
"erdos_renyi"	every pair is connected independently with probability <code>edge_probability</code> ; the graph is resampled (up to 1000 times) until it is connected, using a generator seeded with <code>graph_seed</code>
"proximity"	disk graph: i and j are connected when $\ x_i - x_j\ \leq \text{comm_radius}$; rebuilt after every step and may disconnect during an episode

Formation shapes. `make_formation(shape, n_agents, radius)` returns the offsets d_i , centred at the origin; all shapes have a characteristic extent of $2R$ with $R = \text{formation_radius}$:

- "circle": regular polygon of radius R , agent 0 at the top;
- "line": evenly spaced on a horizontal segment of length $2R$;
- "grid": row-major grid with $\lceil \sqrt{N} \rceil$ columns whose longer side spans $2R$;
- "wedge": V shape pointing up, agent 0 at the apex and the others alternating between the two arms (half opening angle 35 degrees, arm length $2R$).

A formation that does not fit into the arena raises `ValueError`.

Initial state. Positions are drawn uniformly from $[-0.8a, 0.8a]^2$ and velocities are zero. For the proximity topology the draw is conditioned on a connected initial graph (rejection sampling in vectorised batches); if `comm_radius` is so small that no connected placement is found, the agents are placed sequentially, each within 0.95comm_radius of a previously placed agent, and a warning is issued once. `reset(options={"positions": ..., "velocities": ...})` starts from a given state (arrays of shape $(N, 2)$; positions are clipped to the arena).

7.2 Spaces

Action. `Box(-max_control, max_control, (N, 2), float32)`: one velocity command (single integrator) or acceleration (double integrator) per agent. Any array with $2N$ entries is reshaped to $(N, 2)$; values are clipped, and NaN or infinite values raise `ValueError`.

Observation. An unbounded `Box` of shape $(N, 6 + 5K)$, `float32`, where $K = \text{max_neighbors}$ is the number of neighbour slots. Row i has the layout of Table 7.2. The slots hold the neighbours $j \in \mathcal{N}_i$ sorted by physical distance $\|x_j - x_i\|$ (ties by index); unused slots are zero with mask 0. If an agent has more neighbours than slots, the farthest ones are left out of the observation; they

still count in the reward and in the baseline controller. The property `env.observation_layout` returns the column slices ("position", "velocity", "offset", "neighbors"), and `obs[:, layout["neighbors"]].reshape(N, K, 5)` gives the slots, whose columns are described by `ConsensusEnv.neighbor_slot_layout`.

Table 7.2: Consensus observation (row i ; slot $s = 0, \dots, K - 1$).

Columns	Layout key	Content
0:2	position	x_i
2:4	velocity	v_i
4:6	offset	d_i (zeros for consensus)
6+5s : 6+5s+2	neighbors	slot s : $(x_j - d_j) - (x_i - d_i) = y_j - y_i$
6+5s+2 : 6+5s+4	neighbors	slot s : $v_j - v_i$
6+5s+4	neighbors	slot s : valid mask (1 or 0)

K defaults to the maximum degree of a static graph (2 for the default ring, so the default observation has shape (8, 16)) and to $\min(6, N - 1)$ for the proximity graph.

7.3 Reward and episode end

Agent i receives the negative mean squared disagreement with its neighbours and a quadratic control cost,

$$r_i = -\frac{1}{\max(|\mathcal{N}_i|, 1)} \sum_{j \in \mathcal{N}_i} \|y_i - y_j\|^2 - c_u \|u_i\|^2, \quad (7.4)$$

with $c_u = \text{control_cost}$ and the clipped action u_i , evaluated after the step on the new graph (isolated agents only pay the control cost). The team reward is $R = \frac{1}{N} \sum_i r_i$, plus `success_bonus` on the step on which the task error first drops below `tolerance`.

`terminated` is `True` when $e < \text{tolerance}$; `truncated` is `True` once `max_steps` steps were taken. Both can be `True` on the last step.

7.4 Parameters

All constructor arguments are keyword-only.

Table 7.3: Constructor parameters of `ConsensusEnv`.

Parameter	Default	Description
<code>n_agents</code>	8	Number of agents N (≥ 2).
<code>task</code>	"consensus"	"consensus" (rendezvous) or "formation".
<code>topology</code>	"ring"	"ring", "line", "star", "complete", "erdos_renyi" or "proximity".
<code>dynamics</code>	"single"	"single" (velocity input) or "double" (acceleration input with damping).
<code>dt</code>	0.1	Integration time step (> 0).
<code>max_steps</code>	200	Episode length before truncation.
<code>arena_size</code>	10.0	Positions are clipped to $[-a, a]^2$ (> 0).
<code>max_control</code>	1.0	Bound of every action component (> 0).
<code>control_cost</code>	0.01	Weight c_u of the control penalty (≥ 0).

Parameter	Default	Description
<code>noise_std</code>	0.0	Noise intensity σ ; each step adds noise with standard deviation $\sigma\sqrt{\Delta t}$ to the positions (single) or velocities (double).
<code>tolerance</code>	0.05	Success threshold on the task error (> 0 , squared distance units).
<code>success_bonus</code>	10.0	Team-reward bonus paid once, on the step the task is solved.
<code>formation_shape</code>	"circle"	"circle", "line", "grid" or "wedge" (formation task only).
<code>formation_radius</code>	3.0	Scale R of the formation (> 0).
<code>comm_radius</code>	4.0	Communication range of the proximity topology (inclusive, > 0).
<code>edge_probability</code>	0.3	Edge probability of the Erdős–Rényi topology, in $(0, 1]$.
<code>graph_seed</code>	None	Seed of the Erdős–Rényi graph, sampled once at construction; None uses fresh entropy, so two instances may differ in graph and observation size.
<code>max_neighbors</code>	None	Number of neighbour slots K (≥ 1); defaults to the maximum degree of a static graph and to $\min(6, N - 1)$ for the proximity graph.
<code>damping</code>	0.5	Linear velocity damping c of the double integrator (≥ 0).
<code>render_mode</code>	None	None, "human" or "rgb_array".

7.5 Info dictionary and properties

Table 7.4: Consensus info dictionary (returned by `reset()` and `step()`).

Key	Content
<code>agent_rewards</code>	float64 array (N,) of the rewards r_i (zeros at reset)
<code>error</code>	task error e (7.1)
<code>algebraic_connectivity</code>	$\lambda_2(L)$ of the current graph
<code>adjacency</code>	boolean adjacency matrix (N, N) of the current graph
<code>success</code>	True once the task has been solved in the episode (sticky)
<code>step</code>	number of steps taken in the episode

The state is available through read-only properties (Table 7.5); `obs_dim` and `max_neighbors` are plain attributes.

The module `env_lib.consensus_env` also exports the helpers used by the environment, which are useful for analysis and for reference controllers:

- `make_topology(topology, n_agents, *, edge_probability=0.3, rng=None)`: adjacency matrix of a static graph;
- `proximity_adjacency(positions, comm_radius)`: disk-graph adjacency;
- `make_formation(shape, n_agents, radius=1.0)`: formation offsets;
- `algebraic_connectivity(adjacency)` and `is_connected(adjacency)`;
- the tuples `TASKS`, `TOPOLOGIES`, `DYNAMICS` and `FORMATION_SHAPES` of valid option values.

Table 7.5: Read-only properties of `ConsensusEnv` (arrays are copies).

Property	Content
<code>positions, velocities</code>	x_i and v_i , shape $(N, 2)$
<code>formation_offsets</code>	offsets d_i , shape $(N, 2)$ (zeros for consensus)
<code>adjacency, laplacian</code>	adjacency matrix A and Laplacian $L = D - A$ of the current graph
<code>algebraic_connectivity</code>	$\lambda_2(L)$ of the current graph
<code>task_error</code>	task error e
<code>step_count</code>	number of steps taken in the episode
<code>observation_layout</code>	column slices of an observation row

7.6 Baseline controller

`env.laplacian_policy(gain=1.0, velocity_gain=None)` returns the classical distributed consensus and formation feedback

$$u_i = -k \sum_{j \in \mathcal{N}_i} ((x_i - d_i) - (x_j - d_j)), \quad \text{that is} \quad u = -k L (x - d), \quad (7.5)$$

with gain k , clipped to the action bounds. For the double integrator the velocity feedback $-k_v v_i$ is added; the default $k_v = \max(0, \sqrt{k} - c)$ makes the total damping $c + k_v$ equal to $\sqrt{k}$. The feedback uses the full neighbour sets, even when the observation is truncated by `max_neighbors`. The discrete-time single-integrator loop is stable when $k \Delta t \lambda_{\max}(L) < 2$; the method warns once otherwise.

Table 7.6 reports the success rate of the controller with gain 1 for all topologies and task/dynamics combinations (default parameters otherwise; 100 episodes with reset seeds 0–99 and `graph_seed` equal to the reset seed). The controller solves every episode on connected static graphs. On the line, the slowest graph, some episodes run out of time, and on the proximity graph agents can lose contact.

Table 7.6: Success rate of `laplacian_policy(gain=1.0)` over 100 episodes, with the mean episode length in steps (truncated episodes count 200).

Topology	Consensus		Formation (circle)	
	single	double	single	double
ring	1.00 (84)	1.00 (81)	1.00 (97)	1.00 (86)
line	0.82 (160)	0.94 (154)	0.53 (183)	0.72 (174)
star	1.00 (73)	1.00 (80)	1.00 (85)	1.00 (85)
complete	1.00 (65)	1.00 (81)	1.00 (78)	1.00 (87)
Erdős–Rényi	1.00 (90)	1.00 (90)	1.00 (104)	1.00 (97)
proximity	0.83 (72)	0.70 (109)	0.55 (135)	0.12 (190)

7.7 Rendering

The dashboard (1000×560 pixels, Figures 7.1 and 7.2) shows the arena on the left: agents as coloured markers with halos, communication links whose opacity decreases with their length, fading trails of the recent positions, the formation slots around the current centroid (hollow markers, formation task only) and the centroid (cross). On the right, the task error is plotted on

a logarithmic scale with the tolerance as a dashed line over a shaded success band, and below it the algebraic connectivity λ_2 . The header reports the task, the team size, the step, the error, λ_2 and a status badge.

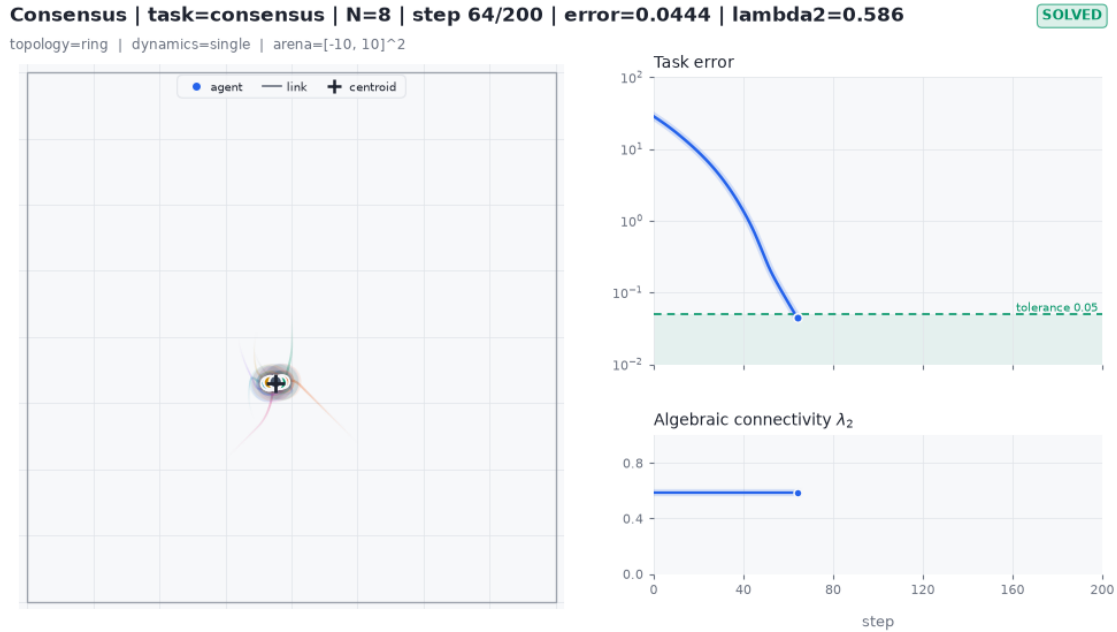


Figure 7.1: Consensus (rendezvous) of 8 single integrators on a ring, solved at step 64 by the Laplacian controller. The agents have gathered at the centroid; the task error has fallen below the tolerance and λ_2 of the static ring is constant.

7.8 Registered ids

Table 7.7: Registered ids of the consensus environment.

Id	Class	Configuration
Consensus-v0	ConsensusEnv	defaults (rendezvous, 8 agents on a ring)
Formation-v0	ConsensusEnv	task="formation" (circle of radius 3)

Table 7.7 lists the two ids. Keyword arguments of `env_lib.make` override the registered configuration, for example `env_lib.make("Formation-v0", formation_shape="wedge", topology="proximity")`.

7.9 Usage

```
import env_lib

env = env_lib.make("Formation-v0", formation_shape="wedge",
                  topology="proximity", dynamics="double")
core = env.unwrapped
obs, info = env.reset(seed=0)  # obs: (8, 36), K = 6 slots
layout = core.observation_layout
n, k = core.n_agents, core.max_neighbors
slots = obs[:, layout["neighbors"]].reshape(n, k, 5)  # neighbour slots
```

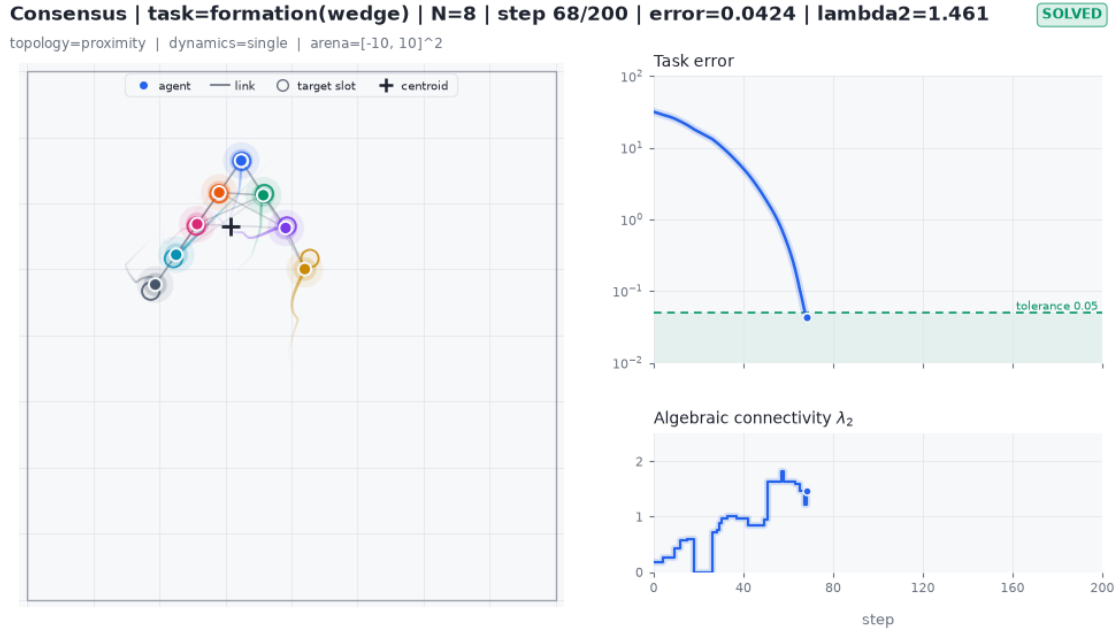


Figure 7.2: Wedge formation of 8 agents on a proximity graph, solved at step 68. Hollow markers are the formation slots around the centroid; λ_2 changes as links appear and disappear and drops to zero while the graph is disconnected.

```
episode_return = 0.0
while True:
    action = core.laplacian_policy(gain=1.0) #  $u = -gain \cdot L \cdot (x - d) - k_v \cdot v$ 
    obs, reward, terminated, truncated, info = env.step(action)
    episode_return += reward
    if terminated or truncated:
        break
print(info["step"], info["error"], info["success"], episode_return)
env.close()
```

The example script compares random actions with the Laplacian controller and records the controller:

```
python examples/environments/consensus_demo.py
python examples/environments/consensus_demo.py --task formation --save
    renders/formation.gif
python examples/environments/consensus_demo.py --task formation --shape wedge \
    --topology proximity --dynamics double --save renders/wedge.gif
```

7.10 Native vector environment

`env_lib.make_vec("Consensus-v0", B)` and `make_vec("Formation-v0", B)` return a `ConsensusVectorEnv`, which simulates B copies with the same array kernels as the single environment. It takes the constructor arguments of `ConsensusEnv` plus `num_envs` and `autoreset_mode`, supports every task, topology (proximity graphs are rebuilt per copy and step), dynamics and formation shape, and returns observations of shape (B, N, d) , team rewards of shape $(B,)$ and batched info arrays with a leading B axis. Each copy reproduces a single environment started from the same state bit for bit:

```

import numpy as np
import env_lib

envs = env_lib.make_vec("Formation-v0", num_envs=4)
obs, _ = envs.reset(seed=0) # (4, 8, 16)
single = env_lib.make("Formation-v0").unwrapped
first, _ = single.reset(options={"positions": envs.unwrapped.positions[0]})
assert np.array_equal(first, obs[0])

action = envs.unwrapped.laplacian_policy() # batched baseline, (4, 8, 2)
obs, reward, terminated, truncated, info = envs.step(action)
assert np.array_equal(single.step(action[0])[0], obs[0])

```

`reset(options=...)` accepts "positions" and "velocities" either for all copies (shape $(N, 2)$) or per copy (shape $(B, N, 2)$), and combines with "reset_mask". Erdos-Renyi graphs are sampled once per copy from `numpy.random.default_rng(graph_seed)`; copy 0 has the graph of a single environment with the same `graph_seed`. The default `max_neighbors` is then the maximum degree over all copies, so pass `max_neighbors` explicitly when observations must have the same width as a single environment's. `render()` draws copy 0. Chapter 11 covers autoreset modes, seeding and throughput.

7.11 Reproducibility and performance

The initial positions and the process noise are drawn from `self.np_random`, seeded by `reset(seed=...)`. The Erdős–Rényi graph is sampled once at construction from `graph_seed`; pass a seed to obtain the same graph (and observation size) in every instance, for example in vectorised training environments.

Measured on the development machine without rendering, a step takes about 0.1 ms with the default 8 agents and 0.18 ms for a grid formation of 32 agents on a proximity graph. Rendering an "rgb_array" frame takes about 16–21 ms.

Chapter 8

Power Grid Frequency Control

`env_lib.power_grid_env.PowerGridEnv` (registered as `PowerGrid-v0`, new in version 1.1) is a benchmark for decentralised frequency control of a networked power system. The n buses of a transmission network, synchronous generators or inverter-based resources, are the agents. They are coupled through the power lines and obey the classical swing equations. Step load changes and random load fluctuations push the bus frequencies away from their nominal value and excite lightly damped electro-mechanical oscillations; every agent injects a bounded, fast frequency response (for example from storage or inverter headroom) to arrest the frequency drop and damp the oscillations, using only local measurements and aggregates over its neighbourhood in the network.

The environment is designed for research on networked multi-agent control: the interaction graph and all physical parameters are explicit and reproducible, the reward is a sum of per-agent terms, the benchmark system is fixed by a seed while `randomize_network=True` resamples topology and parameters for robustness studies, and the dynamics are written once for a batch of grids, so `PowerGridVectorEnv` simulates thousands of grids at once (Section 8.7). The decentralised droop controller is included as a baseline (Section 8.6). Typical uses are learning decentralised controllers that outperform droop control (for example by adapting gains to the local inertia or by exploiting neighbour information), studying the effect of the network topology and of low inertia, and comparing learned policies with model-based designs.

8.1 Model

Bus i has the voltage (or rotor) angle θ_i in radians and the frequency deviation ω_i in rad/s from the nominal angular frequency $\omega_0 = 2\pi f_0$ ($f_0 = \text{nominal_frequency}$); the frequency deviation in hertz is $\Delta f_i = \omega_i/2\pi$. On a lossless, Kron-reduced network (the load buses are eliminated, so every node is an active bus) with line susceptances $B_{ij} \geq 0$ (per unit), the swing equations read

$$\dot{\theta}_i = \omega_i, \quad M_i \dot{\omega}_i = P_i(t) - D_i \omega_i - \sum_j B_{ij} \sin(\theta_i - \theta_j) + u_i, \quad (8.1)$$

with the inertia coefficient $M_i = 2H_i/\omega_0$ (H_i the inertia constant in seconds), the damping coefficient $D_i = d_i/\omega_0$ (d_i the per-unit damping, which lumps the frequency sensitivity of the loads and mechanical damping), the net injection $P_i(t) = P_i^0 + \Delta P_i(t)$ and the control injection $u_i \in [-u_{\max,i}, u_{\max,i}]$. The sum $P_i^e = \sum_j B_{ij} \sin(\theta_i - \theta_j)$ is the electrical power exported by bus i . Powers are in per unit of a common base, time in seconds.

Network and operating point. The line graph is built with the function `make_graph` of `env_lib.utils.graphs` (topologies in Table 8.1); H_i , d_i and the susceptance of every line are drawn

uniformly from `inertia_range`, `damping_range` and `susceptance_range`. The operating point is constructed so that it is an exact and stable equilibrium with a prescribed stress. A raw injection pattern $z_i \sim \mathcal{U}(-1, 1)$, shifted to zero mean, is solved with the linearised (DC) flow equations $L_B \theta^{\text{DC}} = z$ (L_B the susceptance-weighted Laplacian, $\sum_i \theta_i^{\text{DC}} = 0$); the angles are scaled so that the most loaded line operates at the loading $\ell = \text{line_loading}$,

$$\theta^* = s \theta^{\text{DC}}, \quad s = \frac{\arcsin \ell}{\max_{(i,j) \in \mathcal{E}} |\theta_i^{\text{DC}} - \theta_j^{\text{DC}}|}, \quad P_i^0 = \sum_j B_{ij} \sin(\theta_i^* - \theta_j^*), \quad (8.2)$$

and the nominal injections are the exact flows at these angles (they sum to zero). Hence $(\theta^*, 0)$ is a synchronous equilibrium of (8.1) without disturbance and control, to machine precision. Since every line angle satisfies $|\theta_i^* - \theta_j^*| < \pi/2$, the linearisation couples the buses through a Laplacian with positive weights $B_{ij} \cos(\theta_i^* - \theta_j^*)$, and the equilibrium is stable (up to the rotational symmetry of the angles). A test verifies that Newton's method on the flow equations, started from the DC solution, converges to the same angles. Every episode starts from this equilibrium, so only the disturbances drive the system away from it. The benchmark network is sampled once at construction from `network_seed` (default 0), so the default system is the same for every user.

Table 8.1: Network topologies (`topology`); see `make_graph` for details.

topology	Graph
"small_world" (default)	Watts–Strogatz ring lattice, every bus linked to its 4 nearest buses, each link rewired with probability 0.1 (resampled until connected)
"ring", "line", "star", "complete"	deterministic graphs (bus 0 is the hub of the star)
"grid"	4-neighbour lattice of the most square shape with n nodes
"erdos_renyi"	independent edges with probability <code>edge_probability</code> , by default $\min(1, 2 \ln n/n)$, resampled until connected
"random_geometric"	buses uniform in the unit square, linked when closer than a radius about twice the connectivity threshold

Parameters of `make_graph` (`edge_probability`, `neighbors`, `rewire_probability`, `radius`) are passed through `topology_kwargs`.

Disturbances. The injection disturbance is $\Delta P_i = S_i + X_i$. The step component S_i receives step load changes: at reset, `initial_disturbances` distinct buses are hit; afterwards, at every step boundary an event occurs with probability $1 - e^{-\lambda \Delta t}$ ($\lambda = \text{disturbance_rate}$, $\Delta t = \text{dt}$) at a uniformly random bus. Step sizes $|\Delta P|$ are uniform in `disturbance_magnitude`; a step is a load increase (a negative change of the injection) with probability `load_increase_prob`. Steps persist until the end of the episode. The optional Ornstein–Uhlenbeck fluctuation X_i has the stationary standard deviation $\sigma = \text{noise_std}$ and the correlation time $\tau = \text{noise_tau}$; it starts from its stationary distribution and is updated exactly at every step boundary,

$$X_i(t_{k+1}) = a X_i(t_k) + \sigma \sqrt{1 - a^2} \xi_{i,k}, \quad a = e^{-\Delta t/\tau}, \quad \xi_{i,k} \sim \mathcal{N}(0, 1). \quad (8.3)$$

All random draws come from `self.np_random`, in a vectorised way. The disturbance sequence of a seeded episode does not depend on the actions, so different controllers can be compared on identical disturbances.

Integration. The action and the disturbance are held constant over the control interval Δt (zero-order hold). Equation (8.1) is integrated with the classical fourth-order Runge–Kutta scheme and `substeps` sub-steps of length $h = \Delta t/\text{substeps}$. The fastest electro-mechanical

mode is bounded by $\omega_{\max}^2 \leq \max_i 2 \sum_j B_{ij}/M_i$ (Gershgorin); the environment warns when $h \omega_{\max}$ exceeds 2.5, close to the stability limit of RK4 on the imaginary axis. On the default system, after 5 seconds of an undisturbed recovery from a step load change (largest $|\omega_i|$ 0.354 rad/s), the largest deviation from a reference solution with 32 sub-steps was $1.3 \cdot 10^{-3}$ rad/s with one sub-step, $8.7 \cdot 10^{-5}$ rad/s with two (the default) and $5.5 \cdot 10^{-6}$ rad/s with four: the expected fourth-order convergence.

Energy function. Without disturbance and control, the classical energy function of the swing equations,

$$V = \frac{1}{2} \sum_i M_i \omega_i^2 - \sum_i P_i^0 (\theta_i - \theta_i^*) + \frac{1}{2} \sum_{i,j} B_{ij} (\cos(\theta_i^* - \theta_j^*) - \cos(\theta_i - \theta_j)), \quad (8.4)$$

satisfies $\dot{V} = -\sum_i D_i \omega_i^2 \leq 0$. The tests check that V decreases monotonically in simulation and is conserved without damping.

The default system. The default benchmark has 16 buses and 32 lines (small-world graph, bus degrees 2 to 5), inertia constants between 2.5 and 7.6 seconds and nominal injections between -0.62 and 0.50 per unit. Its linearisation around θ^* has 15 oscillatory modes between 0.55 and 2.41 Hz with damping ratios between 1.4 % and 4.4 %, and a centre-of-inertia mode with the time constant $\sum_i M_i / \sum_i D_i = 3.1$ seconds. Without control a load step therefore produces a slow drift of the system frequency with visible, slowly decaying inter-area oscillations; with the droop baseline ($k = 0.25$) the damping ratios of the oscillatory modes rise to between 0.35 and 0.95.

8.2 Spaces

Action. `Box(-u_max, u_max, (n,), float32)`: the control injection u_i of every bus in per unit (default bound 0.1). Any array with n entries is accepted; values are clipped to $[-u_{\max,i}, u_{\max,i}]$, and NaN or infinite values raise `ValueError`.

Observation. `Box(-inf, inf, (n, 10), float32)`. Row i holds quantities a controller at bus i can measure locally, aggregates over its κ -hop neighbourhood $\mathcal{N}_i$ ($\kappa = \text{neighbourhood}$, default 1, bus i excluded) and two local parameters (Table 8.2). Every feature is multiplied by a fixed factor (`OBSERVATION_SCALE`), chosen so that typical values are of order one; the factors do not depend on the constructor arguments, so a policy can recover the physical values. `env.observation_layout` returns the column slice of every key and `env.observation_scale` the factors.

8.3 Reward and episode end

Agent i is penalised for its frequency deviation, its rate of change of frequency $\rho_i = (\omega_i(t_{k+1}) - \omega_i(t_k))/\Delta t$ and its control effort, all evaluated at the end of the step,

$$r_i = -(w_f \omega_i^2 + w_\rho \rho_i^2 + w_u u_i^2) \Delta t - c_{\text{trip}} \mathbb{I}[\text{trip}], \quad (8.5)$$

with $w_f = \text{w_freq}$ (default 1), $w_\rho = \text{w_rocof}$ (0.1), $w_u = \text{w_u}$ (10) and $c_{\text{trip}} = \text{trip_penalty}$ (100). The team reward returned by `step()` is $R = \sum_i r_i$; the per-agent rewards are in `info["agent_rewards"]`. The weights were chosen so that all three terms matter for a good controller: over 1000 default episodes with the droop baseline, the three terms contributed on

Table 8.2: PowerGrid observation (row i). The RoCoF is zero on the first observation of an episode.

Col.	Key	Content	Unit	Scale
0	<code>omega</code>	frequency deviation ω_i	rad/s	1
1	<code>rocof</code>	rate of change of frequency over the last step, $(\omega_i(t) - \omega_i(t - \Delta t))/\Delta t$	rad/s ²	0.1
2	<code>disturbance</code>	local injection disturbance ΔP_i for the next interval	pu	10
3	<code>power_flow</code>	change of the exported power, $P_i^e - P_i^0$	pu	10
4	<code>action</code>	previous applied control u_i	pu	10
5	<code>neighbour_omega_mean</code>	mean of ω_j over $\mathcal{N}_i$	rad/s	1
6	<code>neighbour_omega_max</code>	maximum of $ \omega_j $ over $\mathcal{N}_i$	rad/s	1
7	<code>neighbour_action_mean</code>	mean previous control u_j over $\mathcal{N}_i$	pu	10
8	<code>capacity</code>	control bound $u_{\max,i}$	pu	10
9	<code>inertia</code>	inertia constant H_i	s	0.1

average 0.27, 0.31 and 0.17 to the team cost of an episode (frequency, RoCoF, control), whereas without control the frequency term dominates (68 on average) and with random actions the RoCoF term does (330).

`terminated` is `True` when the largest deviation exceeds the trip limit, $\max_i |\omega_i| > 2\pi f_{\text{lim}}$ with $f_{\text{lim}} = \text{frequency_limit}$ (default 0.5 Hz), which stands for loss of stability or a protection trip; every agent then receives the extra penalty $-c_{\text{trip}}$. The default penalty exceeds the frequency cost of operating at the limit for a whole default episode ($w_f(2\pi f_{\text{lim}})^2 \cdot 10 \text{ s} \approx 99$ per agent), so tripping early never pays off through the frequency term. `truncated` is `True` after `max_steps` steps.

8.4 Parameters

All constructor arguments are keyword-only. The defaults make an episode 10 seconds of simulated time (200 steps of 50 ms, a realistic update period of fast frequency response), a disturbance a sizeable fraction of a bus injection, and the uncontrolled system stable but slow to settle.

Table 8.3: Constructor parameters of `PowerGridEnv` (also accepted by `PowerGridVectorEnv`).

Parameter	Default	Description
<code>n_buses</code>	16	Number of buses (agents) $n \geq 2$; tested up to several hundred.
<code>topology</code>	"small_world"	Line graph (Table 8.1).
<code>topology_kwargs</code>	None	Extra <code>make_graph</code> arguments.
<code>network_seed</code>	0	Seed of the benchmark network, sampled once at construction; <code>None</code> uses fresh entropy.
<code>randomize_network</code>	False	Resample topology and all physical parameters from <code>self.np_random</code> on every reset (per copy in the vector environment).
<code>inertia_range</code>	(2.0, 8.0)	Range of the inertia constants H_i in seconds (low-inertia to conventional units).
<code>damping_range</code>	(2.0, 6.0)	Range of the per-unit damping d_i ; $D_i = d_i/\omega_0$.
<code>susceptance_range</code>	(0.3, 0.8)	Range of the line susceptances B_{ij} (per unit); sets the modes to about 0.5–2.5 Hz.

Parameter	Default	Description
line_loading	0.5	Loading $\max \sin(\theta_i^* - \theta_j^*) $ of the most loaded line, in $[0, 1]$; higher values weaken the synchronising coupling.
nominal_frequency	50.0	f_0 in Hz.
u_max	0.1	Control bound $u_{\max,i}$ (per unit), scalar or array of shape $(n,)$.
dt	0.05	Control interval Δt in seconds.
substeps	2	RK4 sub-steps per interval.
max_steps	200	Episode length before truncation.
initial_disturbances	1	Number of distinct buses with a step load change at reset.
disturbance_rate	0.1	Rate (1/s) of further step load changes.
disturbance_magnitude	(0.05, 0.2)	Range of the step sizes $ \Delta P $ (per unit).
load_increase_prob	0.5	Probability that a step is a load increase.
noise_std	0.0	Stationary standard deviation σ of the Ornstein–Uhlenbeck load fluctuation (per unit); 0 disables it.
noise_tau	1.0	Correlation time τ of the fluctuation in seconds.
frequency_limit	0.5	Trip limit f_{lim} in Hz.
w_freq, w_rocof, w_u	1.0, 0.1, 10.0	Reward weights w_f , w_ρ , w_u .
trip_penalty	100.0	Penalty c_{trip} added to every agent's reward on a trip.
neighbourhood	1	Number of hops κ of the observed neighbourhood.
render_mode	None	None, "human" or "rgb_array".

Invalid values raise `ValueError` and arguments of the wrong type `TypeError`.

8.5 Reset options, info dictionary and properties

`reset(*, seed=None, options=None)` starts from the equilibrium with zero frequency deviation and random initial steps. The options "theta", "omega" and "disturbance" (arrays of shape $(n,)$) set the initial angles, frequency deviations and step disturbance instead; their values are validated. Other keys are ignored with a `UserWarning`, so generic tools that pass their own options keep working.

Table 8.4: PowerGrid info dictionary (returned by `reset()` and `step()`; frequencies in rad/s).

Key	Content
agent_rewards	float64 array $(n,)$ of the rewards r_i (zeros at reset)
omega, rocof	arrays $(n,)$ of ω_i and ρ_i
max_abs_omega	current $\max_i \omega_i $
frequency_nadir	lowest ω_i over buses and time in the episode so far
peak_abs_omega	largest $ \omega_i $ over buses and time in the episode so far
coi_omega	centre-of-inertia frequency $\sum_i M_i \omega_i / \sum_i M_i$
frequency_spread	standard deviation of ω_i over the buses
control_effort	$\sum_i u_i $ of the applied control (per unit)
order_parameter	phase order parameter $ \frac{1}{n} \sum_i e^{j\theta_i} $
total_disturbance	$\sum_i \Delta P_i$ (per unit; negative for a net load increase)
tripped, step, time	trip flag, steps taken, simulated time in seconds

Table 8.5: Read-only properties of `PowerGridEnv` (arrays are copies).

Property	Content
<code>adjacency, edges</code>	boolean line graph $(\mathbf{n}, \mathbf{n})$ and the lines (i, j) , $i < j$, as $(\mathbf{E}, 2)$
<code>susceptance</code>	weighted adjacency B_{ij} , $(\mathbf{n}, \mathbf{n})$
<code>inertia, inertia_constants, damping</code>	M_i , H_i and D_i
<code>nominal_injections, equilibrium</code>	P_i^0 and θ^*
<code>u_max, omega_limit</code>	control bounds and the trip limit in rad/s
<code>theta, omega, disturbance</code>	current state and ΔP_i (after <code>reset()</code>)
<code>line_flows</code>	$B_{ij} \sin(\theta_i - \theta_j)$ in the order of <code>edges</code>
<code>time, step_count</code>	simulated time and steps of the episode
<code>positions</code>	plotting layout of the buses in $[-1, 1]^2$
<code>observation_layout, observation_scale</code>	column slices and scale factors of an observation row

8.6 Baseline controller

`droop_policy(obs, gain=0.25)` (module-level function in `env_lib.power_grid_env`) implements decentralised droop control, the classical primary frequency response,

$$u_i = \text{clip}(-k \omega_i, -u_{\max,i}, u_{\max,i}), \quad (8.6)$$

with the gain k in per unit power per rad/s. It reads ω_i and $u_{\max,i}$ from agent i 's own observation row and works for any leading batch shape, $(\dots, \mathbf{n}, 10) \rightarrow (\dots, \mathbf{n})$, so the same function drives the single and the vector environment. `env_lib.baseline_policy(env)` returns it for `PowerGrid-v0`.

The droop gain is limited by the sampling of the controller. Over one control interval, the local loop of bus i behaves approximately like $\omega_i(t_{k+1}) \approx (1 - k\Delta t/M_i) \omega_i(t_k)$, which is non-oscillatory for $k\Delta t < M_i$ and unstable for $k\Delta t > 2M_i$ (the control then chatters between its bounds). With the default ranges ($H_i \geq 2$ s at 50 Hz, $\Delta t = 0.05$ s) the default gain 0.25 satisfies the first condition for every admissible network. Table 8.6 compares zero control, uniformly random actions and droop control with several gains over 1000 default episodes; droop control improves the mean return by two orders of magnitude and never trips. On the default network (smallest inertia constant 2.5 s) the best uniform gain lies near 0.5, while $k = 0.75$ violates the sampling condition at the low-inertia buses: the RoCoF term explodes. A learned policy can use the `inertia` and `rocof` features to go beyond a uniform gain.

Table 8.6: Baselines on `PowerGrid-v0` (default parameters; one episode on each of the 1000 copies of `PowerGridVectorEnv(1000)` reset with seed 0; nadir and largest deviation are episode values averaged over the episodes).

Controller	mean return	std	nadir [Hz]	max $ \Delta f $ [Hz]	trips
zero control	-80.61	143.32	-0.090	0.156	0.4 %
random actions	-566.75	468.71	-0.291	0.387	9.4 %
droop, $k = 0.1$	-2.05	1.80	-0.044	0.069	0 %
droop, $k = 0.25$ (default)	-0.75	0.61	-0.033	0.053	0 %
droop, $k = 0.4$	-0.58	0.47	-0.029	0.046	0 %
droop, $k = 0.5$	-0.55	0.45	-0.028	0.045	0 %
droop, $k = 0.75$	-39.32	2.36	-0.039	0.047	0 %

8.7 Vector environment

The native vector environment

```
PowerGridVectorEnv(num_envs=1, *, autoreset_mode="next_step", render_mode=None,
**kwargs)
```

accepts every keyword argument of `PowerGridEnv` and simulates `num_envs` grids as one batch; `env_lib.make_vec` returns it for `PowerGrid-v0`. Observations have shape `(num_envs, n, 10)` and actions `(num_envs, n)`. The reward of a copy is its team reward, and every info key of Table 8.4 is returned as a batched array; for example, `infos["agent_rewards"]` has shape `(num_envs, n)`. The info of a copy that was reset automatically within a call describes its new episode, as in Gymnasium’s `SyncVectorEnv`. The reset options accept arrays of shape `(n,)` (broadcast to all copies) or `(num_envs, n)`, together with `"reset_mask"`. `render()` draws copy 0.

The dynamics are implemented once for a batch of grids (the single environment is the batch of size one), with the state stored node-major and the network coupling computed as $P^e = \sin \theta \odot (B \cos \theta) - \cos \theta \odot (B \sin \theta)$ with one sparse matrix product, so the cost per step grows with the number of lines rather than with n^2 . Copy 0 of the vector environment reproduces a single environment started from the same state (injected with `reset(options=...)`) and driven by the same actions; the tests check this. With the default fixed network all copies share it; with `randomize_network=True` every copy owns its network (a block-diagonal coupling matrix), resampled when the copy resets.

Table 8.7 reports the measured throughput with constant actions (best of three runs on a 4-core x86-64 development machine shared with other jobs), compared with Gymnasium’s `SyncVectorEnv` of single environments. A single environment step takes about 0.27 ms with 16 buses and 0.38 ms with 128 buses (a reset about 0.2 ms).

Table 8.7: Throughput of `PowerGrid-v0`: native `PowerGridVectorEnv` versus `SyncVectorEnv` (env-steps and agent-steps per second).

<i>n</i>	copies	PowerGridVectorEnv			SyncVectorEnv	
		ms/step	env-steps/s	agent-steps/s	env-steps/s	speed-up
16	1	0.285	3 514	56 230	3 435	1.0
16	64	0.782	81 836	1 309 369	3 411	24
16	1024	5.40	189 538	3 032 602	3 507	54
128	1	0.409	2 445	312 975	2 276	1.1
128	64	2.83	22 654	2 899 730	2 233	10
128	1024	54.4	18 829	2 410 050	2 232	8.4

Most of the time of a large batch is spent in the double-precision sine and cosine evaluations of the eight right-hand-side evaluations per step (two RK4 sub-steps); `substeps=1` halves the cost at a moderate loss of accuracy (Section 8.1).

8.8 Rendering

The dashboard (1000 × 560 pixels, Figure 8.1) shows the network on the left: buses coloured by their frequency deviation (diverging colour map with a colour bar in hertz, a symmetric range that only grows within an episode), a ring around every bus whose size shows its control effort

$|u_i|/u_{\max,i}$ (green when injecting, violet when absorbing power), lines coloured and widened with their loading $|\sin(\theta_i - \theta_j)|$ and triangles at buses with an active step load change (pointing down for a load increase). On the right, the upper panel shows the frequency deviation of every bus over time (thin traces), the centre-of-inertia frequency (bold), the frequency nadir (dotted line and marker) and the trip limits over a shaded band when they are within range; the lower panel shows the power balance, the net control injection $\sum_i u_i$ against the load imbalance $-\sum_i \Delta P_i$ it has to cover. The header reports the system size, step, time, largest deviation and nadir, and a status badge: STABLE, ALERT (deviation above half the limit) or TRIPPED. Both themes are supported. With blitting an "rgb_array" frame took about 20 ms (median) with 16 buses and 26 ms with 128 buses on the development machine; the few frames per episode that rebuild the cached static layer (when an axis range grows) took up to about 90 ms.

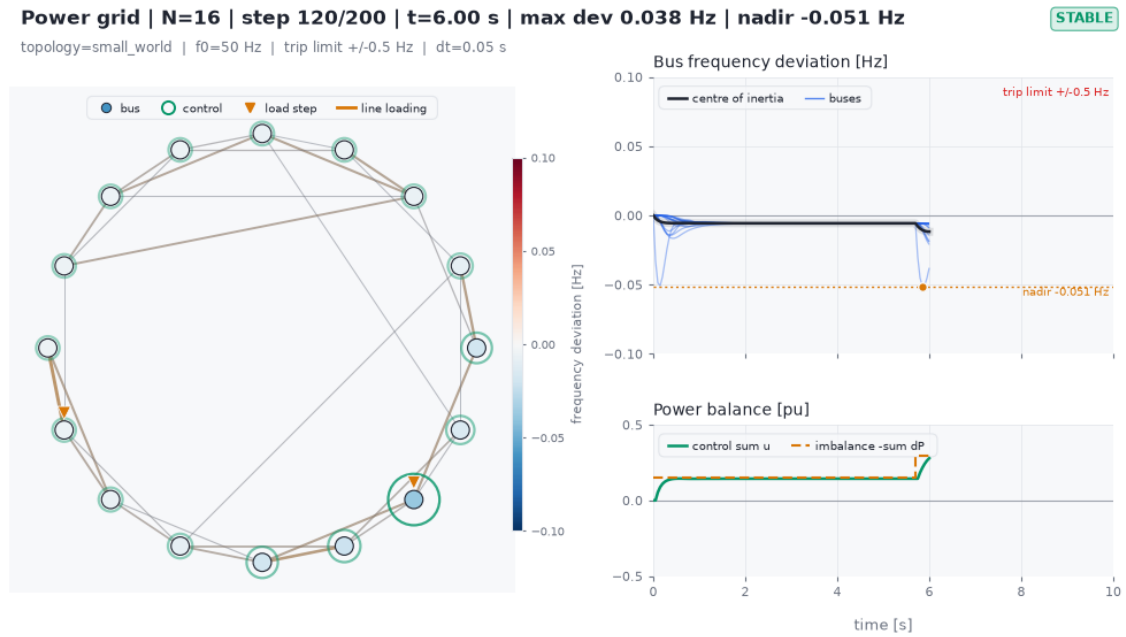


Figure 8.1: PowerGrid-v0 with the droop baseline (seed 16, light theme) at $t = 6$ s. A load increase at reset and a second one at $t = 5.7$ s (triangles) caused frequency dips of about 0.05 Hz at the affected buses; the controllers near the second event respond most (large rings), and the net control injection has risen to cover the new imbalance.

8.9 Usage

```
import env_lib
from env_lib.power_grid_env import droop_policy

env = env_lib.make("PowerGrid-v0", render_mode="rgb_array")
obs, info = env.reset(seed=0) # obs: (16, 10)
layout = env.unwrapped.observation_layout
episode_return = 0.0
while True:
    action = droop_policy(obs, gain=0.25) #  $u_i = -k \omega_i$ , clipped
    obs, reward, terminated, truncated, info = env.step(action)
    episode_return += reward
    if terminated or truncated:
        break
print(info["frequency_nadir"], info["peak_abs_omega"], episode_return)
frame = env.render() # (560, 1000, 3) uint8
```

```
env.close()

# 1024 grids at once, each with its own random network
envs = env_lib.make_vec("PowerGrid-v0", 1024, randomize_network=True)
obs, infos = envs.reset(seed=0)          # obs: (1024, 16, 10)
obs, rewards, terminated, truncated, infos = envs.step(droop_policy(obs))
```

The example script compares zero, random and droop control on a few seeded episodes and can record the droop controller:

```
python examples/environments/power_grid_demo.py
python examples/environments/power_grid_demo.py --buses 32 --topology ring
--episodes 5 \
--theme light --render renders/power_grid.gif
python examples/environments/power_grid_demo.py --render-mode human
```

8.10 Background

The swing equation and the per-unit conventions follow the standard treatment of power system stability and frequency control in textbooks such as Kundur's *Power System Stability and Control*. Network-reduced models with one oscillator per generator go back to the classical transient-stability literature; Bergen and Hill proposed the structure-preserving alternative, and Dörfler and Bullo analysed the Kron reduction of electrical networks and the relation between power-network synchronisation and non-uniform Kuramoto oscillators. The energy function of Section 8.1 is the classical Lyapunov function of the direct methods of transient-stability analysis. The view of power grids as networks of second-order phase oscillators was popularised in the physics literature, for example by Filatrella, Nielsen and Pedersen, by Rohden, Sorge, Timme and Witthaut and by Motter, Myers, Anghel and Nishikawa. Droop control is the classical decentralised primary frequency control; distributed averaging integral controllers, studied for instance by Simpson-Porco, Dörfler and Bullo and by Andreasson and co-authors, add secondary control that restores the nominal frequency using neighbour communication, which the neighbour features of the observation make possible. The default topology is the small-world model of Watts and Strogatz.

Chapter 9

Vehicle Platoon Control

`env_lib.platoon_env.PlatoonEnv` (registered as `Platoon-v0`, new in version 1.1) is a benchmark for cooperative adaptive cruise control (CACC). A leader vehicle is followed by n follower vehicles on a single lane. The followers are the agents: each commands its own acceleration and must keep a speed-dependent distance to its predecessor while the leader drives an exogenous speed profile with cruising phases, stop-and-go waves and random speed changes. The vehicles have heterogeneous drivetrain lags and exchange data over a vehicle-to-vehicle (V2V) communication topology, which determines what each agent observes.

The environment reproduces the central phenomenon of platoon control, *string stability*: with sensor-only adaptive cruise control (ACC) and a short time headway, spacing errors grow from vehicle to vehicle along the platoon until vehicles collide, while cooperative ACC, which also uses the acceleration communicated by the predecessor, attenuates them. Section 9.7 demonstrates this numerically. Typical research uses are learning decentralised controllers that are safe, comfortable and string stable, comparing communication topologies, and studying robustness to heterogeneous vehicles. The dynamics are written once for a batch of platoons, so `PlatoonVectorEnv` simulates thousands of platoons at once (Section 9.8).

9.1 Model

Vehicles. Vehicle 0 is the leader and vehicles $i = 1, \dots, n$ are the followers (agent index $k = i - 1$ in all arrays). Vehicle i has the position p_i of its front bumper, speed v_i , acceleration a_i , length L_i and a first-order actuator (drivetrain) lag with time constant τ_i :

$$\dot{p}_i = v_i, \quad \dot{v}_i = a_i, \quad \dot{a}_i = \frac{u_i - a_i}{\tau_i}, \quad (9.1)$$

where u_i is the commanded acceleration, the action of follower i , clipped to $[a_{\min}, a_{\max}]$. The command is held constant over a step of length Δt and (9.1) is discretised exactly (zero-order hold). With $\alpha_i = e^{-\Delta t/\tau_i}$ and $c_i = \tau_i \Delta t - \tau_i^2(1 - \alpha_i)$,

$$a_i^+ = \alpha_i a_i + (1 - \alpha_i) u_i, \quad (9.2)$$

$$v_i^+ = v_i + \tau_i(1 - \alpha_i) a_i + (\Delta t - \tau_i(1 - \alpha_i)) u_i, \quad (9.3)$$

$$p_i^+ = p_i + \Delta t v_i + c_i a_i + (\frac{1}{2} \Delta t^2 - c_i) u_i. \quad (9.4)$$

The update is vectorised over vehicles and platoons. Vehicles do not reverse: a vehicle whose speed would become negative within a step is held at standstill ($v = 0$, $a = \max(a, 0)$, position not decreased); the position error of this approximation is below $v\Delta t/2$ at the moment of stopping. The lags and lengths are drawn uniformly from `tau_range` and `length_range` with a

generator seeded by `vehicle_seed`, so every episode uses the same heterogeneous platoon, or from `self.np_random` at every reset when `randomize_vehicles=True`.

Spacing policy. Follower i keeps a constant *time headway* h to its predecessor. With the gap g_i (rear bumper of the predecessor to front bumper of follower i), the standstill distance r and the desired gap d_i^* ,

$$g_i = p_{i-1} - p_i - L_{i-1}, \quad d_i^* = r + h v_i, \quad e_i = g_i - d_i^*, \quad \Delta v_i = v_{i-1} - v_i, \quad (9.5)$$

where e_i is the spacing error and Δv_i the relative speed. Note that $\dot{e}_i = \Delta v_i - h a_i$: a platoon that tracks the spacing policy exactly has $\Delta v_i = h a_i \neq 0$ while it accelerates or brakes.

Leader. The leader has the same dynamics; its command follows a speed profile drawn per episode from `self.np_random`. The profile starts at the initial cruise speed $v_c \sim U[16, 25]$ m/s, which is held for 2–5 s, and continues with a sequence of events. Every event has two segments, each of which ramps the reference speed to a target with constant acceleration and then holds it (Table 9.1); segment boundaries are aligned with the time grid, and after the last event the speed is held. The commands are stored per step ($8 \times \text{max_steps}$ bytes per platoon). Because of the lag, the leader’s speed approaches every target from one side, without overshoot.

Table 9.1: Event types of the leader profile (uniform ranges).

Event	Segments (target speed, rate, hold time)
cruise	twice: $v_c \pm 2$ m/s at $0.3\text{--}0.8$ m/s ² , hold 4–10 s
stop-and-go	brake at $2\text{--}4$ m/s ² to $0\text{--}4$ m/s, hold 1–5 s; accelerate at $1\text{--}2$ m/s ² to $v_c \pm 3$ m/s, hold 4–10 s
random	twice: target in $[5, 30]$ m/s at $0.5\text{--}2.5$ m/s ² , hold 1–6 s

`scenario="cruise"`, `"stop_and_go"` and `"random"` use one event type throughout. The default `"mixed"` starts with a stop-and-go wave and draws every further event uniformly from the three types, so that every episode contains a hard braking manoeuvre in which the string instability of naive controllers becomes visible.

Initial state. The platoon starts near equilibrium at v_c : zero accelerations, the leader at $p_0 = 0$, follower speeds $v_c + \sigma_v \xi_i$ and gaps $r + h v_c + \sigma_g \xi'_i$ with standard normal ξ_i, ξ'_i truncated at three standard deviations ($\sigma_v = \text{init_speed_noise}$, $\sigma_g = \text{init_spacing_noise}$; gaps are kept above half the desired gap). `reset(options=...)` overrides parts of the episode; all arrays list the leader first: `"positions"` and `"velocities"` (shape $(n+1,)$, given together), `"accelerations"` (default zeros), `"leader_command"` (shape $(\text{max_steps},)$, commanded accelerations of the leader, clipped to the action bounds), `"time_constants"` and `"lengths"` (this episode only). Values of these keys are validated strictly; unknown keys are ignored with a `UserWarning` (PettingZoo’s API test passes a dummy option).

9.2 Communication topologies

`topology` selects the V2V links (Table 9.2). `env.adjacency` is a boolean (n, n) matrix over the followers: `adjacency[i, j]` is `True` when follower i receives data from follower j (rows are receivers, agent indices). The links from the leader are reported separately by `env.leader_links`, a boolean vector of length n . `platoon_adjacency(topology, n)` returns both.

Table 9.2: Communication topologies and the observation channels they provide.

topology	Links	Channels
"predecessor" (PF)	$i - 1 \rightarrow i$; the leader sends to follower 1	predecessor
"predecessor_leader" (PLF)	PF plus the leader to every follower	predecessor, leader
"bidirectional" (BD)	$i - 1 \rightarrow i$ and $i + 1 \rightarrow i$	predecessor, follower (not for the last one)
"none"	no V2V; radar only	none

9.3 Spaces

Action. `Box(accel_min, accel_max, (n,), float32)`, the commanded accelerations in m/s^2 (default $[-6, 3]$). Any array with n entries is accepted; it is cast to `float32` like the action space and clipped to the bounds; NaN or infinite values raise `ValueError`.

Observation. An unbounded `Box` of shape `(n, 13), float32`; row k belongs to follower $i = k+1$ (Table 9.3). Every feature is a physical quantity divided by a fixed scale (`FEATURE_SCALES`). Radar measurements and the own state are always present. Communicated channels that the topology does not provide are zero with availability flag 0, so the observation size does not depend on the topology. `env.observation_layout` maps every feature name (`OBSERVATION_FEATURES`) to its column slice.

Table 9.3: Platoon observation (row of follower i).

Col.	Layout key	Content	Scale
0	<code>spacing_error</code>	e_i (radar)	5 m
1	<code>relative_speed</code>	$v_{i-1} - v_i$ (radar)	5 m/s
2	<code>speed</code>	v_i	30 m/s
3	<code>acceleration</code>	a_i	3 m/s ²
4	<code>predecessor_available</code>	1 for PF, PLF and BD	–
5	<code>predecessor_acceleration</code>	a_{i-1} (V2V)	3 m/s ²
6	<code>predecessor_command</code>	u_{i-1} applied in the last step (V2V)	3 m/s ²
7	<code>leader_available</code>	1 for PLF	–
8	<code>leader_speed_error</code>	$v_0 - v_i$ (V2V)	5 m/s
9	<code>leader_acceleration_error</code>	$a_0 - a_i$ (V2V)	3 m/s ²
10	<code>follower_available</code>	1 for BD, except for the last follower	–
11	<code>follower_spacing_error</code>	e_{i+1} (V2V)	5 m
12	<code>follower_relative_speed</code>	$v_i - v_{i+1}$ (V2V)	5 m/s

9.4 Reward and episode end

Follower i receives, after the step,

$$r_i = -(w_e e_i^2 + w_v \Delta v_i^2 + w_u u_i^2 + w_j j_i^2) \Delta t - P_c [g_i \leq 0] - P_b [g_i > g_{\max}], \quad (9.6)$$

with the clipped command u_i , the jerk $j_i = (a_i^+ - a_i)/\Delta t$, the weights $w_e = \text{spacing_weight}$ (1.0), $w_v = \text{speed_weight}$ (0.1), $w_u = \text{control_weight}$ (0.02) and $w_j = \text{jerk_weight}$ (0.01), the

collision penalty $P_c = \text{collision_penalty}$ (500), the break-up penalty $P_b = \text{breakup_penalty}$ (250) and the break-up distance $g_{\max} = \text{breakup_distance}$ (100 m). The team reward returned by `step()` is $R = \sum_i r_i$.

The weights make tracking dominate while keeping comfort relevant. The relative-speed weight is small because $\Delta v_i \neq 0$ is inherent to the time-headway policy during manoeuvres (Section 9.1); the control and jerk terms mainly penalise aggressive or oscillating commands. Over 20 episodes of the default scenario, the four terms make up 5, 60, 33 and 2 % of the cost of the CACC baseline (which tracks almost perfectly) and 65, 31, 4 and 0.2 % of the cost of ACC with $h = 1.2$ s, whose cost is eight times higher. The collision penalty exceeds the cost of a whole ACC episode of the entire team (about 400), so for controllers of that quality ending an episode early by a collision does not pay off.

`terminated` is `True` after a collision ($g_i \leq 0$) or a break-up ($g_i > g_{\max}$) of any follower; `truncated` is `True` after `max_steps` steps.

9.5 Parameters

All constructor arguments are keyword-only; `env.config` returns them as an immutable `PlatoonConfig`.

Table 9.4: Constructor parameters of `PlatoonEnv`.

Parameter	Default	Description
<code>n_followers</code>	8	Number of followers (agents), ≥ 1 .
<code>topology</code>	"predecessor"	"predecessor", "predecessor_leader", "bidirectional" or "none".
<code>scenario</code>	"mixed"	Leader profile: "mixed", "cruise", "stop_and_go" or "random".
<code>dt</code>	0.1	Time step in seconds.
<code>max_steps</code>	600	Episode length (60 s).
<code>headway</code>	0.6	Time headway h in seconds (≥ 0).
<code>standstill_distance</code>	2.0	Standstill distance r in metres (> 0).
<code>accel_min, accel_max</code>	-6.0, 3.0	Command bounds in m/s^2 ($a_{\min} < 0 < a_{\max}$).
<code>tau_range</code>	(0.2, 0.4)	Range of the actuator lags τ_i in seconds.
<code>length_range</code>	(4.0, 5.0)	Range of the vehicle lengths in metres.
<code>vehicle_seed</code>	0	Seed of the fixed vehicle parameters (None: fresh entropy).
<code>randomize_vehicles</code>	False	Resample lags and lengths from <code>self.np_random</code> at every reset.
<code>init_spacing_noise</code>	0.1	Standard deviation σ_g of the initial gap perturbation (m).
<code>init_speed_noise</code>	0.05	Standard deviation σ_v of the initial speed perturbation (m/s).
<code>spacing_weight</code>	1.0	w_e in (9.6).
<code>speed_weight</code>	0.1	w_v .
<code>control_weight</code>	0.02	w_u .
<code>jerk_weight</code>	0.01	w_j .
<code>collision_penalty</code>	500.0	P_c , per colliding follower.
<code>breakup_distance</code>	100.0	Gap $g_{\max}$ (m) above which the platoon counts as broken up.
<code>breakup_penalty</code>	250.0	P_b , per follower beyond $g_{\max}$.

Parameter	Default	Description
<code>render_mode</code>	None	None, "human" or "rgb_array".

The defaults were chosen as follows. A headway of 0.6 s is a typical CACC setting and short enough that sensor-only ACC is string unstable. Lags of 0.2–0.4 s are heterogeneous yet within the range for which the CACC baseline is string stable (up to 0.43 s, Section 9.7). The command bounds allow emergency braking at twice the leader’s hardest braking (4 m/s^2). The initial perturbations are small compared with the errors caused by the leader, so that the peak-error statistics measure the propagation of disturbances rather than the initial condition.

9.6 Info dictionary and properties

Table 9.5: Platoon info dictionary (per-follower arrays have shape $(n,)$, float64).

Key	Content
<code>agent_rewards</code>	rewards r_i (zeros at reset)
<code>spacing_errors, gaps, relative_speeds</code>	e_i , g_i and Δv_i
<code>speeds, accelerations</code>	v_i and a_i of the followers
<code>commands</code>	applied (clipped) commands u_i
<code>peak_spacing_errors</code>	$\max e_i $ since the reset
<code>error_amplification</code>	peak error of the last follower divided by that of the first (the latter floored at 0.01 m); values above one indicate amplification along the platoon
<code>leader_speed, leader_acceleration</code>	v_0 and a_0
<code>min_gap</code>	$\min_i g_i$
<code>collision, breakup</code>	termination causes
<code>step, time</code>	steps taken and simulated time in seconds

Read-only properties give the parameters (`n_followers`, `n_agents`, `topology`, `scenario`, `dt`, `max_steps`, `headway`, `standstill_distance`, `obs_dim`, `config`), the graph (`adjacency`, `leader_links`, `observation_layout`) and copies of the state with the leader first (`positions`, `velocities`, `accelerations`, `time_constants`, `lengths`, `leader_command`) or per follower (`gaps`, `spacing_errors`), as well as `step_count`. Using the state before `reset()` raises `env_lib.ResetNeededError`.

9.7 Baseline controllers and string stability

`cacc_policy(obs, *, k_p=0.5, k_d=0.6, k_a=0.65, accel_min=-6.0, accel_max=3.0)` is the decentralised linear CACC law

$$u_i = k_p e_i + k_d (v_{i-1} - v_i) + k_a a_{i-1}, \quad (9.7)$$

a pure function of each follower’s own observation row that works for any leading batch shape, $(\dots, n, 13) \rightarrow (\dots, n)$. The predecessor acceleration a_{i-1} is taken from the V2V channel and multiplied by the availability flag, so without communication (`topology="none"`) and for `k_a=0` the law is sensor-only ACC. `env_lib.baseline_policy(env)` binds it with the action bounds of the environment.

Choice of the gains. The law has the structure of the CACC designs studied in the platooning literature: a PD feedback on the spacing error and the relative speed plus a feed-forward of the acceleration that the predecessor transmits over V2V. That literature shows that the feed-forward is what makes string stability possible at much shorter time headways than with ACC. For identical vehicles and the exact discretisation above, the commands of consecutive followers satisfy $U_i(z) = \Gamma(z) U_{i-1}(z)$ with

$$\Gamma = \frac{k_p H_p + k_d H_v + k_a H_a}{1 + k_p H_p + (k_p h + k_d) H_v}, \quad (9.8)$$

where H_p , H_v and H_a are the transfer functions from the command to the position, speed and acceleration of one vehicle; the same ratio relates consecutive positions, speeds and accelerations. The platoon is string stable when $\max_{\omega} |\Gamma(e^{j\omega\Delta t})| \leq 1$; since $\Gamma(1) = 1$ the peak is never below one. `string_stability_gain(k_p, k_d, k_a, *, headway, time_constant, dt)` evaluates this peak (and returns `inf` if the vehicle's own loop is unstable). The gains were chosen with it: for $k_p = 0.5 \text{ s}^{-2}$ and $k_d = 0.6 \text{ s}^{-1}$ the peak equals one for all lags in $[0.2, 0.4] \text{ s}$ at $h = 0.6 \text{ s}$ exactly when $k_a \in [0.575, 0.755]$, and $k_a = 0.65$ lies in the middle of this interval. A full feed-forward ($k_a = 1$) is not string stable with a lagged drivetrain (peak 1.25), because the measured acceleration of the predecessor lags its command. Table 9.6 lists the peak over the lag range for both controllers: CACC is string stable from $h \approx 0.56 \text{ s}$ on, ACC with the same feedback gains only from $h \approx 1.14 \text{ s}$ on; with the default gains CACC stays string stable at $h = 0.6 \text{ s}$ for lags up to 0.43 s .

Table 9.6: Peak string-stability gain $\max_{\omega} |\Gamma|$, worst case over $\tau \in [0.2, 0.4] \text{ s}$ ($\Delta t = 0.1 \text{ s}$, $k_p = 0.5$, $k_d = 0.6$; CACC: $k_a = 0.65$, ACC: $k_a = 0$). Values above one are string unstable.

Headway h [s]	0.4	0.5	0.6	0.8	1.0	1.2	1.5
CACC	1.136	1.042	1.000	1.000	1.000	1.000	1.000
ACC	1.625	1.491	1.377	1.197	1.062	1.000	1.000

Numerical demonstration. Table 9.7 compares random actions, ACC at the default headway, ACC at $h = 1.2 \text{ s}$ and CACC in 20 episodes per scenario (reset seeds 0–19, 8 followers, predecessor following, default parameters otherwise). ACC at $h = 0.6 \text{ s}$ is string unstable: in the cruise scenario the peak spacing error grows from the first to the last follower by a factor of 2.4 (median), and in every stop-and-go episode the amplified braking wave ends in a collision after about 11 s. ACC with the longer headway of 1.2 s is string stable again (amplification below one) but keeps twice the distance and tracks worse. CACC at $h = 0.6 \text{ s}$ never collides, attenuates the errors along the platoon (median last-to-first ratio 0.35) and keeps them below 0.42 m in all stop-and-go episodes. It clearly beats random actions, which collide within about 7 s.

The standard deviation of the CACC return in the mixed scenario is 13. In the cruise scenario the errors of CACC are so small (about 0.1 m) that the initial perturbations dominate the peaks, which is why its amplification ratio is less informative there. The example script reproduces such a comparison:

```
# random vs ACC vs CACC (default settings), then a GIF of CACC in stop-and-go
python examples/environments/platoon_demo.py
python examples/environments/platoon_demo.py --scenario stop_and_go --render
renders/platoon.gif
# a longer headway, bidirectional communication and 12 followers
python examples/environments/platoon_demo.py --headway 1.2 --topology bidirectional
--vehicles 12
```

Table 9.7: Baselines over 20 episodes per scenario: mean return, collisions, mean peak $|e_i|$ of the first and the last (8th) follower, and median error amplification (peak of the last divided by peak of the first follower). Episodes that end in a collision are cut short, so their peaks only cover the part of the platoon the wave reached.

Scenario	Controller	Return	Collisions	Peak $ e_1 $ [m]	Peak $ e_8 $ [m]	Ampl.
mixed	random	-4395	20/20	22.57	11.30	0.49
	ACC, $h = 0.6$	-844	20/20	4.16	1.28	0.10
	ACC, $h = 1.2$	-395	0/20	1.76	1.58	0.93
	CACC, $h = 0.6$	-50	0/20	0.26	0.10	0.35
stop-and-go	ACC, $h = 0.6$	-816	20/20	4.30	1.24	0.03
	ACC, $h = 1.2$	-529	0/20	1.94	1.66	0.86
	CACC, $h = 0.6$	-68	0/20	0.28	0.11	0.35
cruise	ACC, $h = 0.6$	-140	0/20	0.83	1.96	2.40
	ACC, $h = 1.2$	-8.2	0/20	0.35	0.23	0.67
	CACC, $h = 0.6$	-1.3	0/20	0.12	0.08	0.74

9.8 Vector environment

`PlatoonVectorEnv(num_envs, *, autoreset_mode="next_step", render_mode=None, **kwargs)` takes the keyword arguments of `PlatoonEnv` and simulates `num_envs` platoons with the same batched core, so a copy evolves exactly like a single environment started from the same state (the test suite checks bit-for-bit equality of copy 0 with a single environment). `env_lib.make_vec("Platoon-v0", num_envs)` creates it. Observations have shape `(num_envs, n, 13)`, actions `(num_envs, n)`, rewards are the team rewards of the copies and the info arrays gain a leading `num_envs` dimension (`agent_rewards` has shape `(num_envs, n)`). Reset options may have the single-environment shape (applied to every reset copy) or an extra leading `num_envs` dimension. With the default "next_step" autoreset, the info rows of the copies reset in a step describe their new initial state, as in `gymnasium.vector.SyncVectorEnv`. `render()` draws copy 0. All copies share one generator, so copy b does not reproduce a single environment seeded with `seed + b`.

Table 9.8 reports the measured throughput of `step()` (CACC actions, no resets during the measurement; policy time excluded) against `gymnasium.vector.SyncVectorEnv` of the single environment, on a 4-core x86-64 machine with Python 3.11 and NumPy 2.4 (one thread). A single `PlatoonEnv.step()` takes about $78 \mu\text{s}$ for 8 as for 64 followers. With random actions, which end episodes after about 65 steps and therefore reset many copies every step, the native environment reaches 0.65 M environment steps per second for $n = 8$ and $B = 1024$ and 0.13 M for $n = 64$.

9.9 Rendering

The dashboard (1000×560 pixels, Figure 9.1) shows the road at the top: a two-lane strip seen from above with a camera that follows the platoon (moving lane markings and distance posts). Vehicles are rounded rectangles drawn to scale in length; the leader is drawn in the text colour and labelled, the followers in a sequential colour ramp from the first to the last follower. Brake lights glow when a vehicle decelerates. The gap in front of every follower is shaded by its spacing error (red: too close, blue: too far, more opaque for larger errors), a tick marks the desired position, and the V2V links of the topology are drawn as arcs with moving data packets. Below,

Table 9.8: Throughput of `PlatoonVectorEnv` (native) and of `SyncVectorEnv` over single environments (sync): time per `step()` of the whole batch, environment steps and agent steps (n per environment step) per second, and the speed-up of native over sync.

n	B	native			sync	
		ms/step	env-steps/s	agent-steps/s	env-steps/s	speed-up
8	1	0.118	8 500	68 000	7 600	1.1
8	64	0.162	395 000	3.2 M	10 000	40
8	1024	0.743	1 380 000	11.0 M	9 000	153
64	1	0.128	7 800	0.50 M	7 100	1.1
64	64	0.309	207 000	13.3 M	8 500	24
64	1024	5.21	197 000	12.6 M	8 300	24

the spacing errors of all followers over time show how disturbances propagate, the speed panel shows the leader, its reference profile for the whole episode and the followers, and the bar chart gives the peak spacing error of every follower together with the last-to-first ratio. The header reports the team size, the topology, step, time, leader speed and minimum gap, with a status badge (CRUISING, COLLISION, BREAK-UP or COMPLETE). Both themes are supported. An "rgb_array" frame takes about 24 ms for 8 followers and 40 ms for 64 (mean over an episode; the first frame, which builds the figure, about 0.25 s).

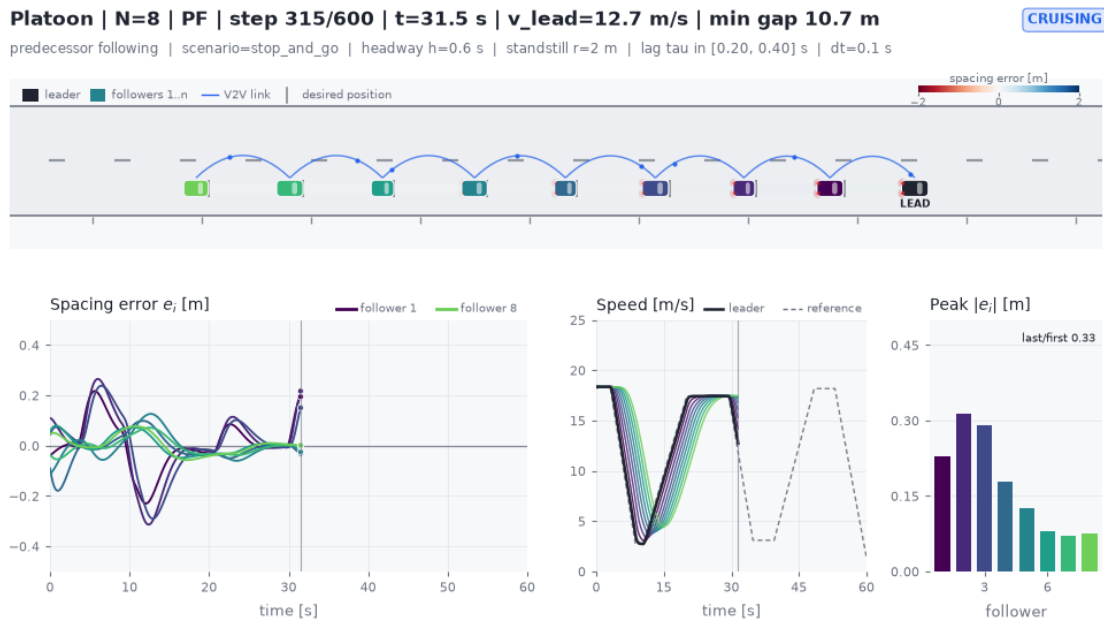


Figure 9.1: CACC in the stop-and-go scenario (seed 2, light theme) at $t = 31.5$ s: the leader has started the second braking manoeuvre and the braking wave (brake lights) travels down the platoon. The spacing errors stay below 0.35 m, and the peak errors of the last followers are much smaller than those of the first ones (last over first follower: 0.33).

9.10 Usage

```
import env_lib
from env_lib.platoon_env import cacc_policy
```

```

env = env_lib.make("Platoon-v0", topology="predecessor_leader",
                  scenario="stop_and_go")
obs, info = env.reset(seed=0)          # obs: (8, 13) float32
layout = env.unwrapped.observation_layout
print(obs[:, layout["leader_available"]].ravel()) # all ones for PLF
episode_return = 0.0
while True:
    action = cacc_policy(obs)           # ACC: cacc_policy(obs, k_a=0.0)
    obs, reward, terminated, truncated, info = env.step(action)
    episode_return += reward
    if terminated or truncated:
        break
print(info["collision"], info["peak_spacing_errors"], episode_return)
env.close()

# 1024 platoons in one batch; the same policy works on the batched observation.
envs = env_lib.make_vec("Platoon-v0", num_envs=1024)
obs, infos = envs.reset(seed=0)        # obs: (1024, 8, 13)
for _ in range(600):
    obs, rewards, terminated, truncated, infos = envs.step(cacc_policy(obs))
envs.close()

```

A custom leader profile is passed through the reset options, for example a constant speed with `env.reset(options={"leader_command": np.zeros(env.unwrapped.max_steps)})`; with `init_spacing_noise=0` and `init_speed_noise=0` the platoon then starts and stays in equilibrium under CACC.

9.11 Reproducibility

The leader profile, the initial perturbations and, with `randomize_vehicles=True`, the vehicle parameters are drawn from `self.np_random`, seeded by `reset(seed=...)`. With the default `randomize_vehicles=False` the heterogeneous platoon is fixed by `vehicle_seed` and identical in every instance, including every copy of a vector environment. The simulation itself is deterministic.

Chapter 10

AJLATT: Active Joint Localisation and Target Tracking

In AJLATT a team of mobile robots with range-bearing sensors has to localise itself and track a moving target on an occupancy grid map. Every robot runs an estimator for its own pose and for the target; estimates are exchanged within a limited communication range and fused with covariance intersection. The robots choose their velocities, and the reward drives the team to keep the target and its own poses well estimated while avoiding obstacles, the map border and each other. The environment is used to study active perception and information-driven control, cooperative localisation and distributed target tracking, and multi-agent reinforcement learning with continuous actions, partial observability and safety constraints.

The main entry points are exported by `env_lib.ajlatt_env`:

AJLATTEnv	the Gymnasium environment with joint multi-agent spaces (also available as <code>env_lib.AJLATTEnv</code>);
AJLATTConfig	a validated dataclass holding every parameter;
TeamRewardWrapper	a scalar-reward view for single-agent algorithms;
make	the backwards-compatible factory accepting the original keyword names;
maps	occupancy maps, the map builder and map plotting (Section 10.9).

10.1 Scenario and motion model

The team consists of `num_robots` robots (default 4) and `num_targets` targets (default 1). Target 0 moves according to a target policy (Section 10.6); further targets are static landmarks whose positions are estimated in 2-D. One step lasts $\Delta t = \text{sampling_period}$ (0.5 s), so the default episode of 120 steps covers 60 s. Robots and the target follow the unicycle model

$$\begin{bmatrix} p_x \\ p_y \\ \theta \end{bmatrix}_{k+1} = \begin{bmatrix} p_x \\ p_y \\ \theta \end{bmatrix}_k + \begin{bmatrix} v \Delta t \cos \theta_k \\ v \Delta t \sin \theta_k \\ \omega \Delta t \end{bmatrix}, \quad (10.1)$$

with the heading wrapped to $[-\pi, \pi)$. The true robot poses are propagated with noisy velocities $v \sim \mathcal{N}(v_{\text{cmd}}, \sigma_v^2)$, $\omega \sim \mathcal{N}(\omega_{\text{cmd}}, \sigma_\omega^2)$, where $(\sigma_v, \sigma_\omega) = (\text{sigma_vR}, \text{sigma_wR})$; with `process_noise_fixed=False` the noise scales with the speed, $\sigma_v = \sigma/\sqrt{2}$ and $\sigma_\omega = 2\sqrt{2}\sigma$ with $\sigma = \text{sigmaR} \cdot v_{\text{cmd}}$. The target moves exactly with the command of its policy, while the

robots predict their target beliefs with a noisy copy of that command (`sigma_vT`, `sigma_wT`, or `sigmapT.target_linear_velocity` in the speed-dependent case).

10.2 Sensing, communication and estimation

Beliefs. Robot i keeps a Gaussian belief $(\hat{x}_i, P_i)$ of its own pose and a belief $(\hat{x}_{i,t}^T, P_{i,t}^T)$ of every target t (a pose for target 0, a position for landmarks). At reset the means are drawn around the initial poses with the initial covariances: $P_i = \text{robot_init_cov}[i] I$ with heading variance 10^{-3} , and $P_{i,t}^T = \text{target_init_cov}[t] I$ with heading variance 0.1 for target 0.

Prediction. Every belief is predicted with an extended Kalman filter step for (10.1):

$$\hat{x} \leftarrow f(\hat{x}, u), \quad P \leftarrow \Phi P \Phi^\top + G Q G^\top, \quad Q = \text{diag}(\sigma_v^2, \sigma_\omega^2), \quad (10.2)$$

where Φ is the Jacobian of the motion with respect to the pose and $G = \Delta t \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix}$ its Jacobian with respect to the velocity noise.

Measurements. Robot ℓ measures the range and bearing $z = (\rho, \beta)$ of another robot or a target j when `sensor_r_min` $< \rho < \text{sensor_r_max}$, $|\beta| \leq \text{fov}/2$, the straight line between the two (true) positions does not cross an occupied cell of the grid (Bresenham ray casting), and robot ℓ 's estimated position lies inside the map. The measurement noise has standard deviations $\sigma_\rho = \text{sigma_p} \cdot \rho$ (with `range_noise_proportional=True`) or `sigma_r`, and $\sigma_\beta = \text{sigma_th}$. All three default to zero: by default the measurements are exact, and the uncertainty of the estimates stems from the motion noise and from the uncertainty of the measured party's own estimate.

Communication. Robots closer than `commu_r_max` (true distance) are neighbours; the neighbour set $\mathcal{N}_i$ of robot i includes i itself.

Update. With `use_update=True` (default) the robots are processed one after another in index order, and each update is applied in place, so later robots use the updated beliefs of earlier ones:

1. *Self-localisation.* Every measurement z of an entity j with belief $(\hat{x}_j, P_j)$ contributes the information pair

$$S = H_i^\top W H_i, \quad y = H_i^\top W (z - \hat{z} + H_i \hat{x}_i), \quad W = (R + H_j P_j H_j^\top)^{-1},$$

where $\hat{z}$ is the predicted measurement and H_i, H_j are the Jacobians of the range-bearing model with respect to the observer and the observed pose. These pairs and the prior $(P_i^{-1}, P_i^{-1} \hat{x}_i)$ are fused by covariance intersection.

2. *Target tracking.* For each target, the measurements of all neighbours $r \in \mathcal{N}_i$ that detect it contribute analogous information pairs (with the roles of observer and target exchanged); these are fused by covariance intersection in information form. The neighbours' prior beliefs of the target are fused in the same way, and the two results are combined by a final covariance intersection. If no neighbour detects the target, robot i 's belief is the fusion of the neighbours' priors.

With `use_update=False` the robots only dead-reckon.

Covariance intersection. Given information pairs (S_k, y_k) , $k = 1, \dots, K$, covariance intersection chooses weights c on the probability simplex that minimise the trace of the fused covariance,

$$c^* = \arg \min_{c \geq 0, \sum_k c_k = 1} \text{tr} \left(\left(\sum_k c_k S_k \right)^{-1} \right), \quad P = \left(\sum_k c_k^* S_k \right)^{-1}, \quad \hat{x} = P \sum_k c_k^* y_k. \quad (10.3)$$

The fused estimate is consistent without knowledge of the correlations between the sources. The solver is chosen with `ci_solver`: "newton" (default) is an active-set projected Newton method with the analytic gradient $g_k = -\text{tr}(PS_k P)$ and Hessian $H_{kl} = 2 \text{tr}(PS_k P S_l P)$; its objective is never worse than that of "slsqp", the original solver (SciPy SLSQP with finite-difference gradients), which reproduces trajectories of the implementation before version 1.0 to about 10^{-6} .

10.3 Spaces

Action. Box of shape `(num_robots, 2)`, one command (v, ω) per robot with $0 \leq v \leq v_{\max}$ and $|\omega| \leq \omega_{\max}$, where $v_{\max} = \text{max_linear_velocity}$ (default 2m/s) and $\omega_{\max} = \text{max_angular_velocity}$ (default $\pi/4$ rad/s). The per-robot space is `env.single_action_space`. Actions are not clipped unless `clip_actions=True` (the original environment did not clip). NumPy arrays and tensors are accepted; rows beyond `num_robots` are ignored with a warning, and fewer rows raise `ValueError`.

Observation. Box of shape `(num_robots, obs_dim)` with `obs_dim = 7 + 4(n_R - 1) + 5` for n_R robots (24 for the default team); the per-robot space is `env.single_observation_space`. Row i is computed from robot i 's beliefs; positions and headings of other entities are expressed in robot i 's estimated body frame. `env.observation_layout()` returns the slices of Table 10.1.

Table 10.1: AJLATT observation of robot i ($n = 6 + 4(n_R - 1)$).

Key of <code>observation_layout()</code>	Columns	Content
<code>target_relative_position</code>	0:2	believed target position in the robot frame
<code>target_relative_heading</code>	2	believed target heading minus own heading
<code>target_velocity</code>	3:5	commanded target velocity (v, ω) of the last step (zero after reset)
<code>target_cov_trace</code>	5	$\text{tr } P_{i,0}^T$
<code>neighbours</code>	6:n	for every other robot, in index order: relative x , relative y , relative heading (estimates), trace of its covariance
<code>closest_obstacle</code>	n:n+2	range and bearing of the closest obstacle cell within the field of view and sensor range; $(r_{\max}, \pi)$ if none; zeros while the robot is within <code>obstacle_sensing_margin</code> of the map border
<code>self_pose</code>	n+2:n+5	own pose estimate (x, y, θ) in the map frame
<code>self_cov_trace</code>	n+5	$\text{tr } P_i$

10.4 Reward and episode end

`step()` returns the per-robot rewards as an array of shape `(num_robots,)`. They are built from the tracking cost c_i and the penalty p_i of robot i in the step,

$$c_i = w_T \text{tr } P_{i,0}^T + w_R \text{tr } P_i, \quad p_i = \beta_B \mathbf{1}_{\text{border}}(i) + \beta_O \mathbf{1}_{\text{obstacle}}(i) + \beta_M \mathbf{1}_{\text{mutual}}(i), \quad (10.4)$$

and, in the default `reward_mode="cost"` (the original reward),

$$r_i = -c_i - p_i, \quad (10.5)$$

with $w_T = \text{target_cov_weight}$ (10), $w_R = \text{robot_cov_weight}$ (5) and the penalties

- $\beta_B = \text{boundary_penalty}$ (20) when the estimated position lies within 0.1 m of the map border or outside the map;
- otherwise $\beta_O = \text{obstacle_penalty}$ (1.3) when an obstacle cell is closer to the estimated pose than `obstacle_collision_distance` (0.2m, searched in all directions up to `sensor_r_max`); this also sets the robot's collision flag;
- $\beta_M = \text{mutual_collision_penalty}$ (0.5) when another robot's estimated position is closer than `mutual_collision_distance` (0.4m).

The team reward is $\sum_i r_i$ (`info["team_reward"]`), and `info["tracking_cost"]` holds c_i for evaluations that do not depend on the reward settings.

Reward settings. Four configuration fields change the reward (Table 10.2); they are applied in the order of the table.

Table 10.2: Reward settings of AJLATT (defaults first; the defaults give the original reward (10.5)).

Setting	Effect
<code>reward_mode="cost"</code>	$r_i = -c_i - p_i$: every reward is negative.
<code>reward_mode="bounded"</code>	$r_i = e^{-c_i/\sigma} - p_i/\sigma$ with $\sigma = \text{cost_scale}$ (20): at most 1, positive while the robot tracks well, so a collision that ends the episode forfeits the rewards of the remaining steps.
<code>terminate_on_collision</code> (True)	An obstacle collision ends the robot's episode (<code>terminated[i]</code>); with <code>False</code> the collision is only penalised and the episode continues.
<code>collision_termination_penalty</code> (0)	Subtracted from r_i on the step a collision ends robot i 's episode.
<code>team_reward_weight</code> w (0)	$r_i \leftarrow (1 - w) r_i + w \frac{1}{n_R} \sum_j r_j$: 0 keeps individual rewards, 1 gives every robot the team mean; the team sum is unchanged.

Choosing the settings. With the defaults every reward is negative and the first collision ends the episode (in the team view of `TeamRewardWrapper` and `env_lib.make_vec`), so a team can raise its return by colliding early: on 64 seeded episodes random actions reach -627 in 16 steps, the classical controller $-7,090$ in 115. The registered id keeps this reward for compatibility, and `env-lib describe` prints a caution. For learning, use `reward_mode="bounded"` (random 39.9, controller 223.6) or `terminate_on_collision=False` (random $-34,273$, controller $-7,415$), or a `collision_termination_penalty` larger than the cost of an episode.

```
import env_lib

# collisions penalised, the episode continues
env = env_lib.make("AJLATT-v0", terminate_on_collision=False)
env = env_lib.make("AJLATT-v0", reward_mode="bounded", team_reward_weight=0.5)
```

Episode end. `terminated` is a boolean array of shape `(num_robots,)` holding the obstacle collision flags (all `False` with `terminate_on_collision=False`). If an estimator diverges (non-finite covariance) all flags are set, a `RuntimeWarning` is issued and `info["numerical_error"]` is `True`. `truncated` is a `bool`, set after `max_episode_steps` steps. The environment keeps simulating after a collision flag; whether a flag ends the episode is left to the caller (`record_episode` and `TeamRewardWrapper` use `any(terminated)`).

10.5 Configuration

`AJLATTEnv(config=None, render_mode=None, **overrides)` takes an `AJLATTConfig` and/or individual fields as keyword arguments, which are applied on top of `config`. Unknown field names raise `TypeError`.

```
from env_lib import AJLATTConfig, AJLATTEnv

env = AJLATTEnv(map_name="obstacles05", num_robots=3)    # fields as keywords
config = AJLATTConfig(map_name="obstacles04", sensor_r_max=5.0, fov=120.0)
env = AJLATTEnv(config, render_mode="rgb_array")
env = AJLATTEnv(config.replace(ci_solver="slsqp"))      # validated copy
```

`AJLATTConfig` validates its values on construction (for example $0 \leq \text{sensor_r_min} < \text{sensor_r_max}$, $0 < \text{fov} \leq 360$, non-negative noise levels, enough initial poses and covariances for the team) and offers `replace(**changes)`, `to_dict()` and `from_dict(data)`, the property `observation_dim`, and `from_kwargs(strict=True, **kwargs)`, which also accepts the legacy names of Table 10.4 (as do `AJLATTEnv`, `env_lib.ajlatt_env.make` and the callable module `env_lib.ajlatt_env`). All fields are listed in Table 10.3.

Table 10.3: Fields of `AJLATTConfig` (keyword arguments of `AJLATTEnv`).

Field	Default	Description
<i>Scenario</i>		
<code>map_name</code>	"obstacles04"	Bundled map name or path to a user map (Section 10.9).
<code>num_robots</code>	4	Team size n_R (≥ 1).
<code>num_targets</code>	1	Number of targets; target 0 moves, the others are static landmarks.
<code>max_episode_steps</code>	120	Episode length; reaching it sets <code>truncated</code> .
<code>sampling_period</code>	0.5	Duration Δt of one step in seconds.
<code>margin2wall</code>	1.0	Safety margin of the map queries <code>in_bound</code> and <code>is_collision</code> .
<code>robot_init_pose</code>	six poses	(x, y, θ) per robot; the first four defaults are (7.5, 12.5, 0), (7.5, 10.5, 0), (10, 12.5, 0), (10, 10.5, 0).
<code>target_init_pose</code>	13 poses	(x, y, θ) per target; target 0 starts at (10.5, 11.5, 0).
<code>robot_init_cov</code>	(0.1,) * 6	Initial position variance per robot.
<code>target_init_cov</code>	(1.0,) + (10.0,) * 10	Initial position variance per target.
<i>Sensing and communication</i>		
<code>sensor_r_min</code>	0.0	Minimum sensing range (m).
<code>sensor_r_max</code>	3.0	Maximum sensing range (m); also the range of the obstacle reading.

Field	Default	Description
fov	180.0	Field of view in degrees, centred on the heading.
sigma_p	0.0	Range noise as a fraction of the range (proportional mode).
sigma_r	0.0	Absolute range noise (m) when <code>range_noise_proportional=False</code> .
sigma_th	0.0	Bearing noise (rad).
range_noise_proportional	True	Use <code>sigma_p</code> instead of <code>sigma_r</code> .
use_update	True	Run measurement updates (<code>False</code> : dead reckoning only).
commu_r_max	6.0	Communication range (m).
ci_solver	"newton"	Covariance-intersection solver, "newton" or "slsqp".
<i>Motion noise</i>		
process_noise_fixed	True	Fixed velocity noise; <code>False</code> scales it with the speed.
sigmapR	0.1	Robot noise factor in the speed-dependent mode.
sigma_vR	0.1	Robot linear-velocity noise (m/s).
sigma_wR	$0.5\pi/180$	Robot angular-velocity noise (rad/s, 0.5 degrees/s).
sigmapT	0.1	Target noise factor in the speed-dependent mode.
sigma_vT	0.2	Target linear-velocity noise used by the estimators (m/s).
sigma_wT	$0.5\pi/180$	Target angular-velocity noise used by the estimators (rad/s).
<i>Target motion</i>		
target_policy	"auto"	Target policy (Table 10.5).
target_linear_velocity	0.3	Speed of the "sine" policy; also scales the speed-dependent target noise.
target_omega_bound	$\pi/4$	Turn-rate bound of the "random" policy.
k_theta	0.5	Heading gain of the "sine" and "waypoints" policies.
k_p	1.0	Kept for compatibility; currently unused.
<i>Actions</i>		
max_linear_velocity	2.0	Upper bound of v (m/s); the lower bound is 0.
max_angular_velocity	$\pi/4$	Bound of $ \omega $ (rad/s).
clip_actions	False	Clip actions to the action space.
<i>Rewards and termination</i>		
target_cov_weight	10.0	Weight w_T of the target covariance trace.
robot_cov_weight	5.0	Weight w_R of the self-localisation covariance trace.
boundary_penalty	20.0	Penalty β_B near or beyond the map border (0.1 m margin).
obstacle_penalty	1.3	Penalty β_O for an obstacle closer than <code>obstacle_collision_distance</code> .
obstacle_collision_distance	0.2	Obstacle collision distance (m).
mutual_collision_penalty	0.5	Penalty β_M for robots closer than <code>mutual_collision_distance</code> .
mutual_collision_distance	0.4	Robot-robot distance threshold (m).

Field	Default	Description
<code>terminate_on_collision</code>	<code>True</code>	End a robot's episode (terminated) at an obstacle collision; <code>False</code> only penalises it.
<code>reward_mode</code>	<code>"cost"</code>	<code>"cost"</code> or <code>"bounded"</code> (Table 10.2).
<code>cost_scale</code>	<code>20.0</code>	Scale σ of the <code>"bounded"</code> reward.
<code>collision_termination_penalty</code>	<code>0.0</code>	Extra penalty when a collision ends a robot's episode.
<code>team_reward_weight</code>	<code>0.0</code>	Weight $w \in [0, 1]$ of the team mean in each robot's reward.
<code>obstacle_sensing_margin</code>	<code>1.0</code>	Robots closer than this to the map border report a zero obstacle reading (original behaviour).
<i>Miscellaneous</i>		
<code>seed</code>	<code>None</code>	Seed of the first <code>reset()</code> when none is passed.

The default poses and covariances cover up to six robots and eleven targets; larger teams need explicit `robot_init_pose` and `robot_init_cov` (and likewise for targets).

Table 10.4: Legacy parameter names; each emits a `DeprecationWarning`.

Legacy name	Replacement
<code>num_Robot, num_Target</code>	<code>num_robots, num_targets</code>
<code>T_steps</code>	<code>max_episode_steps</code>
<code>maxmum_run</code>	caps <code>max_episode_steps</code> at <code>maxmum_run + 1</code>
<code>SIGPERCENT</code>	<code>range_noise_proportional</code>
<code>useupdate</code>	<code>use_update</code>
<code>fix_seed</code>	<code>seed</code> (<code>fix_seed=True</code> sets <code>seed=1</code>)
<code>render=True</code>	<code>render_mode="human"</code>
obsolete keys (<code>figID</code> , <code>ssh_debug</code> , <code>episode_num</code> , ...)	ignored with a warning

Command-line integration. `AJLATTConfig.add_arguments(parser, prefix="")` registers every scalar field as an option `--<prefix><name>` on an `argparse` parser, and `AJLATTConfig.from_namespace(namespace, prefix="")` builds a configuration from the parsed arguments. Training scripts keep their own options next to the environment's:

```
import argparse
from env_lib import AJLATTConfig, AJLATTEnv

parser = argparse.ArgumentParser()
parser.add_argument("--lr", type=float, default=3e-4)      # training options
AJLATTConfig.add_arguments(parser, prefix="env.")          # --env.map_name, ...
args = parser.parse_args(["--env.map_name", "obstacles05",
                          "--env.num_robots", "3"])
env = AJLATTEnv(AJLATTConfig.from_namespace(args, prefix="env."))
```

10.6 Target motion

The target policy maps the true target pose to a unicycle command. Policies are per-environment objects that are reset with the environment's generator at every `reset()`.

Table 10.5: Target policies (`target_policy`).

Policy	Behaviour
"auto"	"sine" on the map empty, "waypoints" on obstacles04 and obstacles05, "circle" otherwise
"sine"	follows the slope field $dy/dx = 5 \cos x$ at speed <code>target_linear_velocity</code> with heading gain <code>k_theta</code> until $x \geq 32$ m, then circles with $v = 0.25$ m/s on a 4 m radius
"waypoints"	proportional heading control (gain <code>k_theta</code>) towards piecewise waypoints; built-in routes for <code>obstacles04</code> (0.1 m/s) and <code>obstacles05</code> (0.25 m/s); other routes with <code>env_lib.ajlatt_env.controllers.WaypointPolicy</code>
"circle"	constant $(v, \omega) = (0.25 \text{ m/s}, 0.15 \text{ rad/s})$
"random"	$v = 0.25$ m/s and a turn rate drawn uniformly from $[-\text{target_omega_bound}, \text{target_omega_bound}]$ at every step
"static"	the target does not move

10.7 Info dictionary and episode statistics

Table 10.6: AJLATT info dictionary ($n_R = \text{num_robots}$).

Key	Content
<code>agent_rewards</code>	float64 array ($n_R, $), equal to the returned rewards (zeros at reset)
<code>team_reward</code>	sum of the per-robot rewards
<code>collisions</code>	boolean array ($n_R, $): obstacle collisions in this step
<code>episode_collisions</code>	integer array ($n_R, $): number of border, obstacle and robot-robot penalty events in the episode
<code>target_observed</code>	boolean array ($n_R, $): robot measured target 0 in this step
<code>communication</code>	boolean matrix (n_R, n_R): communication links, including self-loops
<code>robot_cov_trace</code>	array ($n_R, $): $\text{tr } P_i$
<code>target_cov_trace</code>	array ($n_R, $): $\text{tr } P_{i,0}^T$
<code>tracking_cost</code>	array ($n_R, $): tracking cost c_i of (10.4) (<code>step()</code> only)
<code>step</code>	number of steps taken
<code>numerical_error</code>	True if an estimator diverged (<code>step()</code> only)

`env.episode_statistics()` returns per-step traces of the current episode as arrays of shape `(steps + 1, num_robots)`, starting with the state after reset: `robot_cov_trace`, `target_cov_trace`, `robot_error` and `target_error` (Euclidean position errors of the robots' own and target beliefs) and `reward`. The traces are cleared at every reset.

10.8 Team reward wrapper and legacy factory

`TeamRewardWrapper(env)` returns the team reward $\sum_i r_i$ as a float and `any(terminated)` as a bool, for algorithms that expect scalar values. The per-robot values remain available in the info dictionary under `"agent_rewards"` and `"agent_terminated"`.

```
from env_lib import AJLATTEnv
from env_lib.ajlatt_env import TeamRewardWrapper

env = TeamRewardWrapper(AJLATTEnv(map_name="obstacles04"))
obs, info = env.reset(seed=1)
obs, reward, terminated, truncated, info = env.step(env.action_space.sample())
```

The original factory remains available for existing code: `env_lib.ajlatt_env.make(**kwargs)`, or the callable module `env_lib.ajlatt_env(num_Robot=4, T_steps=100)`. It translates the legacy names of Table 10.4, ignores unknown keys with a warning and maps `render=True` to `render_mode="human"`. New code should create `AJLATTEnv` directly.

10.9 Maps

The subpackage `env_lib.ajlatt_env.maps` contains the occupancy maps and the tools to create and inspect them.

10.9.1 Bundled maps

`available_maps()` lists the bundled maps (Table 10.7); `ajlatt-plot-map --list` prints the same list. All maps start at the origin (`mapmin = (0,0)`). Figure 10.1 shows `obstacles05`.

Table 10.7: Bundled maps. Size in cells ($n_x \times n_y$) and metres.

Map	Cells	Cell size	Extent	Notes
<code>obstacles</code>	51×51	0.5 m	25.5 m	
<code>obstacles02</code> , <code>obstacles07</code> , <code>obstacles07-02</code> , <code>obstacles08</code>	181×181	0.2 m	36.2 m	
<code>obstacles03</code>	102×102	0.2 m	20.4 m	
<code>obstacles04</code>	181×181	0.2 m	36.2 m	default; waypoint route
<code>obstacles05</code>	181×181	0.4 m	72.4 m	waypoint route
<code>obstacles06</code> , <code>obstacles06-02</code> , <code>obstacles06-03</code> , <code>obstacles08-02</code>	181×181	0.1 m	18.1 m	
<code>example_complete_map</code> , <code>sample_map_config</code>	181×181	0.4 m	72.4 m	map builder examples
<code>empty</code>	82×62	0.5 m	41×31 m	no obstacles; sine target policy
<code>empty05</code>	181×181	0.4 m	72.4 m	no obstacles
<code>emptySmall</code>	135×135	0.2 m	27 m	no obstacles
<code>emptySmall05</code>	65×65	0.4 m	26 m	no obstacles
<code>dynamic_map</code> , <code>dynamic_map_2</code>	181×181	0.4 m	72.4 m	obstacles resampled at every reset

10.9.2 Map files

A static map consists of two files with the same name:

- `<name>.yaml`, the header, with the keys `mapdim` ($[n_x, n_y]$, cells along x and y), `mapres` (cell size $[r_x, r_y]$ in metres), `mapmin` and `mapmax` (corners of the map in metres) and optionally `origin`. Further keys written by older tools or the map builder (`datatype`, `mappath`, `origincells`, `storage`, the embedded specification) are ignored by the loader.
- `<name>.cfg`, the occupancy grid as whitespace-separated numbers, one text row per grid row: n_y rows of n_x values, where row 0 is the bottom row ($y = \text{mapmin}$) and 1 marks an occupied cell.

Cell (c_x, c_y) has its centre at $(c + 0.5)r + \text{mapmin}$. A grid with fewer cells than its header declares is padded with occupied cells and a warning. An empty (zero-byte) `.cfg` file denotes an obstacle-free map in which only the boundary blocks rays; a map whose name contains `empty` may omit

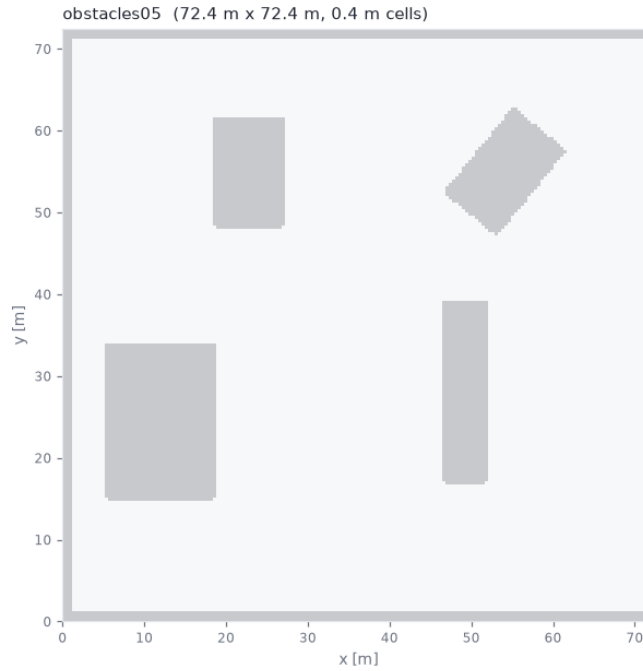


Figure 10.1: The bundled map `obstacles05` (72.4×72.4 m, 0.4 m cells) plotted with `ajlatt-plot-map obstacles05`; occupied cells are grey, including the one-cell wall around the map.

the file altogether, while a missing grid for any other static map raises `FileNotFoundError`. A dynamic map has only a header, which defines `submaporigin` (four (row, column) cell anchors, flattened to eight numbers) and `lib_path` (a directory of `.npy` obstacle masks relative to the header).

To use your own map, pass its path to `map_name`, with or without the `.yaml` suffix, for example `AJLATTEnv(map_name="maps/my_map")`; the grid file must sit next to the header. Check that the default initial poses (Table 10.3) are free cells of the map, or pass `robot_init_pose` and `target_init_pose`.

10.9.3 Map API

- `load_grid_map(name_or_path, margin2wall=0.5)` returns a `GridMap`, or a `DynamicMap` for a header with `submaporigin` and no grid. `load_map(name)` returns (`occupancy`, `mapmin`, `mapmax`) for plotting, and `resolve_map_path(name)` the file path without suffix.
- `GridMap` holds the grid as `map` (shape (n_y, n_x) , `None` for empty maps) together with `mapdim`, `mapres`, `mapmin` and `mapmax`; its queries are listed in Table 10.8. `GridMap.from_array(occupancy, mapmin=(0, 0), mapres=0.4)` builds a map from an array of shape (n_y, n_x) .
- `DynamicMap.generate_map(chosen_idx=None, rot_angs=None, rng=None)` places four distinct obstacles from its library at the four anchors, each rotated by a random multiple of 18 degrees, and closes the border. The environment calls it with its own generator at every reset.

Table 10.8: Queries of GridMap (positions and poses in metres and radians).

Method	Result
<code>is_blocked(start, end)</code>	True if the straight line crosses an occupied cell (exact Bresenham ray casting)
<code>is_blocked_batch(starts, ends)</code>	the same for arrays of shape (k, 2) or (k, 3)
<code>get_closest_obstacle(pose, ang_res=0.05, fov=pi, r_max=3.0)</code>	range and bearing of the closest occupied cell in the field of view, or None
<code>is_collision(pos, margin=None)</code>	True if <code>pos</code> is out of bounds or within <code>margin</code> (default <code>margin2wall</code>) of an occupied cell
<code>in_bound(pos)</code>	True if <code>pos</code> is at least <code>margin2wall</code> inside the map
<code>se2_to_cell(pos), cell_to_se2(cell)</code>	conversion between positions and cell indices

10.9.4 Building maps

The map builder (`env_lib.ajlatt_env.maps.builder`, command `ajlatt-build-map`) rasterises a YAML specification that describes the map in metres:

```
map_info:
  name: my_map
  width: 36.0          # metres along x
  height: 36.0         # metres along y
  resolution: 0.2      # cell size in metres
  boundary:
    enabled: true
    thickness: 0.2
    style: rectangle   # rectangle | circle | polygon
    rectangle: {margin: 0.0}
  obstacles:
    - {type: rectangle, center: [10, 10], width: 4, height: 2, angle: 0.3}
    - {type: circle, center: [25, 25], radius: 3, filled: false}
    - {type: polygon, points: [[5, 20], [9, 20], [7, 26]]}
    - {type: line, start: [0, 18], end: [12, 18], width: 0.4}
    - {type: random, num_obstacles: 5, min_size: 0.5, max_size: 1.5, seed: 3}
```

The grid has $n_x = \text{round}(\text{width}/\text{resolution})$ columns and $n_y = \text{round}(\text{height}/\text{resolution})$ rows; a cell is occupied when its centre lies inside a shape. The boundary (enabled by default) is a rectangular wall with an optional `margin`, the outside of a circle (`circle: {center, radius}`) or the outside of a polygon (`polygon: {points}`), `thickness` metres thick. The obstacle types are listed in Table 10.9; closed shapes accept `filled: false` for a one-cell outline.

The builder writes `<name>.yaml` (the grid header followed by the specification, so a map can always be rebuilt from its own header) and `<name>.cfg`, in the orientation the environment reads. The name is `map_info.name` unless `-n` is given:

```
ajlatt-build-map --create-sample my_map      # writes my_map.spec.yaml
ajlatt-build-map my_map.spec.yaml --png      # my_map.yaml, .cfg, .png
ajlatt-build-map spec.yaml -o maps/ -n arena # other directory and name
```

`--create-sample` sets `map_info.name` to the requested file name, so the second command writes `my_map.yaml`, `my_map.cfg` and `my_map.png`. The same functionality is available from Python: `build_map(spec, output_dir=None, *, name=None, png=False)` accepts a path or a dictionary and returns the paths of the header and grid, `rasterize(spec)` returns the occupancy array and the header, and `save_map` writes them.

Table 10.9: Obstacle types of the map builder (lengths in metres, angles in radians).

Type	Keys
rectangle	center, width, height, angle (default 0), filled
circle	center, radius, filled
triangle	vertices (three points), filled
hexagon	center, radius, angle, filled
polygon	points (at least three), filled
line	start, end, width (default: one cell)
maze	cell_size (5.0), wall_thickness (0.5) and gaps, a list of {type: horizontal vertical, position: <cell index>} walls to leave open
random	num_obstacles (10), min_size (1.0), max_size (3.0), seed: axis-aligned squares

```

from env_lib import AJLATTEnv
from env_lib.ajlatt_env.maps.builder import build_map

spec = {
    "map_info": {"name": "corridor", "width": 20.0, "height": 10.0,
                 "resolution": 0.2},
    "obstacles": [
        {"type": "line", "start": [10, 0], "end": [10, 6], "width": 0.4},
        {"type": "circle", "center": [15, 5], "radius": 1.5},
    ],
}
header, grid = build_map(spec, "maps", png=True)  # maps/corridor.yaml, ...
env = AJLATTEnv(map_name="maps/corridor",
                 robot_init_pose=[(2, 2, 0), (2, 4, 0), (4, 2, 0), (4, 4, 0)],
                 target_init_pose=[(5, 5, 0)], target_policy="static")

```

10.9.5 Plotting maps

`ajlatt-plot-map` draws a bundled or user map; without `--save` it writes `<map>.png` in the working directory. It uses the light theme unless `--theme dark` is given. Dynamic maps are drawn with a fixed obstacle sample.

```

ajlatt-plot-map --list                                # bundled map names
ajlatt-plot-map obstacles04 --save o4.png
ajlatt-plot-map maps/corridor --theme dark

```

From Python, `plot_map(map_or_name, ax=None, *, theme=None, title=None, save=None, dpi=150)` in `env_lib.ajlatt_env.maps.plotting` draws a map into existing axes (or a new off-screen figure) and returns the axes.

10.10 Rendering

The dashboard (1100 × 620 pixels, Figure 10.2) shows the map on the left with the robots' true poses as oriented arrowheads, their pose beliefs as covariance ellipses, their sensing sectors, fading trajectory trails, communication links, the active target measurements, the true target (star) and every robot's belief about the target (crosses). The two panels on the right plot the trace of each robot's target covariance and of its self-localisation covariance over the episode. The header

shows the map, the step, the simulated time, the team reward and how many robots currently see the target. Human mode requires an interactive matplotlib backend; TkAgg needs the tk bindings of the Python installation (python3-tk on Debian and Ubuntu, tk in conda).

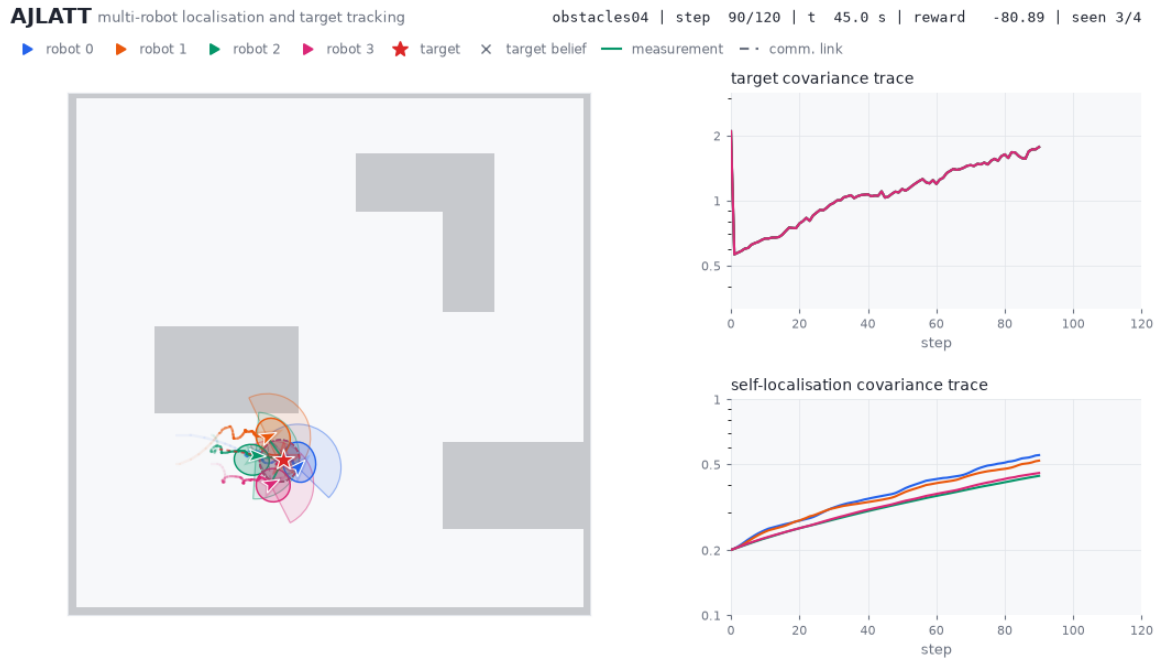


Figure 10.2: AJLATT dashboard on `obstacles04` (four robots running the encircling heuristic of the example script, step 90 of 120). The robots surround the target; the right panels show the trace of the target covariance and of the self-localisation covariances of the four robots.

10.11 Registered id

AJLATT-v0 creates `AJLATTEnv` with the default configuration (four robots on `obstacles04`). Gymnasium's passive environment checker is disabled for this id because rewards and `terminated` are per-robot arrays. Keyword arguments of `env_lib.make` are configuration fields, for example `env_lib.make("AJLATT-v0", map_name="obstacles05")`.

10.12 Usage

The following controller sends robot i to the i -th of n_R evenly spaced slots on a circle of radius 1.8m around its belief of the target, using only the observation:

```
import numpy as np
from env_lib import AJLATTEnv

def encircle(obs, layout, radius=1.8, gain=1.2):
    """Send robot i to the i-th slot of a circle around its target belief."""
    n = len(obs)
    pose = obs[:, layout["self_pose"]].astype(np.float64)  # x, y, theta
    rel = obs[:, layout["target_relative_position"]].astype(np.float64)
    c, s = np.cos(pose[:, 2]), np.sin(pose[:, 2])
    target = pose[:, :2] + np.column_stack([c * rel[:, 0] - s * rel[:, 1],
```

```

                                s * rel[:, 0] + c * rel[:, 1]])
    angle = 2 * np.pi * np.arange(n) / n
    slot = target + radius * np.column_stack([np.cos(angle), np.sin(angle)])
    delta = slot - pose[:, :2]
    heading = np.arctan2(delta[:, 1], delta[:, 0])
    error = np.angle(np.exp(1j * (heading - pose[:, 2]))) # wrapped
    speed = np.minimum(2.0, 0.5 * np.linalg.norm(delta, axis=1))
    v = speed * np.maximum(0.0, np.cos(error))
    w = np.clip(gain * error, -np.pi / 4, np.pi / 4)
    return np.column_stack([v, w])

env = AJLATTEnv(map_name="obstacles04", num_robots=4)
layout = env.observation_layout()
obs, info = env.reset(seed=0) # obs: (4, 24) float32
episode_return = np.zeros(env.num_robots)
while True:
    obs, reward, terminated, truncated, info = env.step(encircle(obs, layout))
    episode_return += reward # per-robot rewards
    if truncated: # collision flags ignored
        break
stats = env.episode_statistics() # arrays of shape (121, 4)
print(episode_return.sum(), info["episode_collisions"])
print(stats["target_cov_trace"][-1])
env.close()

```

The example script compares a similar heuristic with random actions and can record an episode:

```

python examples/environments/ajlatt_demo.py --episodes 3
python examples/environments/ajlatt_demo.py --save renders/ajlatt.gif
python examples/environments/ajlatt_demo.py --map obstacles05 --render-mode human

```

With the default configuration the heuristic of the script obtains a team return of about -7250 per episode without collision events and keeps the trace of the target covariance near 2, whereas random actions obtain returns of $-25\,000$ or lower with dozens of collision events and target covariance traces of 7 to 12 (the random actions are not seeded, so these values vary from run to run).

10.13 Reproducibility and performance

All randomness (initial beliefs, motion and measurement noise, the random target policy and dynamic maps) is drawn from `self.np_random`. If `reset()` is called without a seed for the first time and `config.seed` is set, that seed is used. With `ci_solver="slsqp"` trajectories match the implementation before version 1.0 to about 10^{-6} ; the default Newton solver finds weights that are at least as good and therefore produces slightly different, not worse, estimates.

Measured on the development machine without rendering, a step takes 5–9 ms on `obstacles04` (90 ms before 1.0) and 6–7 ms on `obstacles05` (67 ms before). The gains come from exact batched Bresenham ray casting (closest-obstacle queries are 7 to 19 times faster) and from the Newton covariance-intersection solver. Rendering a frame takes about 25 ms (about 110 ms before). The cost of a step grows with the number of robots, because every robot fuses the measurements and priors of its neighbours.

10.14 Changes in 1.0

AJLATT was rewritten as a Gymnasium environment. See Appendix B for migration advice.

- The environment class `AJLATTEnv` with its configuration dataclass `AJLATTConfig` replaces the former factory, which parsed the command line internally and therefore failed in scripts with their own command-line arguments. The legacy factory still works and translates the old names with a `DeprecationWarning`. Training options that used to live in the parameter module (PPO, LLM and debug settings) were removed; use `AJLATTConfig.add_arguments`.
- `reset()` returns `(observation, info)`. Rendering is off by default; pass `render_mode`.
- The spaces are joint multi-agent spaces with one row per robot, and rewards and `terminated` are per-robot arrays; the per-robot spaces are available as `single_action_space` and `single_observation_space`.
- The target-velocity entries of the observation contain the commanded target velocity instead of a configured constant.
- Fixed a memory leak: covariance traces accumulated across episodes (per-episode traces are now returned by `episode_statistics()`). Also fixed: target policies were shared between environments and never reset, unknown maps triggered an interactive prompt, and dynamic maps did not work; they now resample at every reset and no longer need scikit-image. Three bundled maps that were one row short of their header were padded with a wall row.
- About ten times faster (Section 10.13) and a new renderer.
- Map tools were consolidated into `env_lib.ajlatt_env.maps`; the builder now writes grids in the orientation the environment reads (the old builder produced transposed maps) and embeds its specification in the header.
- The id `AJLATT-v1`, an unusable alias of `AJLATT-v0`, was removed.

Chapter 11

Vectorised Simulation, Adapters, Baselines and Evaluation

The previous chapters describe the environments one by one. This chapter describes the layer that connects them to a research workflow, new in version 1.1: batched simulation of many environment copies, a catalogue and a command line to discover and try the environments, wrappers that plug them into single-agent and multi-agent reinforcement learning libraries, a classical baseline controller for every environment, and tools to evaluate policies and record trajectories. Everything is reachable from the top-level package:

```
import env_lib

print(env_lib.catalog(action_type="continuous"))      # discover
envs = env_lib.make_vec("Formation-v0", num_envs=256) # native batched copies
policy = env_lib.baseline_policy(envs)               # classical controller
result = env_lib.evaluate(envs, policy, n_episodes=256, seed=0)
print(result)                                        # mean return, 95% CI, ...
```

Table 11.1: Public names of the convenience layer.

Name	Purpose
<code>env_lib.make_vec</code>	B copies as one vector environment (Section 11.1)
<code>env_lib.catalog</code> , <code>env_lib.describe</code> , <code>env_lib.EnvInfo</code> <code>env-lib</code> (python <code>-m env_lib</code>)	discover environments (Section 11.2) command line (Section 11.3)
<code>env_lib.wrappers</code>	<code>FlattenJointSpaces</code> , <code>TeamReward</code> , <code>ParallelEnvAdapter</code> , <code>to_parallel</code> (Section 11.4)
<code>env_lib.baseline_policy</code> , <code>env_lib.baselines</code> <code>env_lib.evaluate</code> , <code>env_lib.rollout</code>	baseline controllers (Section 11.5) evaluation and trajectories (Section 11.6)

11.1 Vector environments

```
env_lib.make_vec(env_id, num_envs=1, vectorization_mode=None, *,
                 vector_kwargs=None, wrappers=None, **kwargs)
```

creates `num_envs` copies of a registered environment as one Gymnasium vector environment. Keyword arguments configure the environment exactly as in `env_lib.make`. With `vectorization_mode=None` the environment's native batched implementation is used when it has one and `gymnasium.vector.SyncVectorEnv` otherwise; `"sync"`, `"async"` and `"vector_entry_point"` force a mode. `vector_kwargs` and `wrappers` (applied to every copy) are only accepted by the Sync and Async modes.

Native batched environments. Native vector environments keep the state of all B copies in arrays with a leading batch dimension and advance them with one set of array operations per step, instead of stepping B Python objects one after another. They share their dynamics code with the single environment, so copy b reproduces a single environment started from the same state. Native implementations exist for `Consensus-v0`, `Formation-v0`, the NumPy Kuramoto Oscillator-* ids, `PowerGrid-v0` and `Platoon-v0`; `env_lib.catalog(native_vector=True)` lists them. Batched shapes, with n agents:

- observations ($B, n, \text{obs_dim}$) ($(B, \text{obs_dim})$ for Kuramoto), actions ($B, n, \text{act_dim}$) or (B, n);
- rewards ($B, \text{}$): the team reward of every copy;
- infos: arrays with a leading B dimension, among them `agent_rewards` of shape (B, n), each key k with the Gymnasium validity mask `"_k"`.

Autoreset. A copy whose episode ended is reset automatically according to the `autoreset_mode` argument of `make_vec`, see Table 11.2. It accepts `"next_step"`, `"same_step"`, `"disabled"`, Gymnasium's spellings (`"NextStep"`, ...) or a member of `gymnasium.vector.AutoresetMode`, and works for every vectorisation mode: `make_vec` passes it to the native class or converts it for Gymnasium's Sync and Async classes. Gymnasium 1.0, which has no `AutoresetMode`, supports only `"next_step"` for Sync and Async environments.

Table 11.2: Autoreset modes of vector environments.

Mode	Behaviour
<code>"next_step"</code> (default)	The copy is reset on the <i>next</i> call to <code>step()</code> : that call ignores its action and returns its first observation with reward 0 and both flags <code>False</code> . The last observation of the episode is the one returned together with <code>terminated/truncated</code> .
<code>"same_step"</code>	The copy is reset within the step that ended the episode; the returned observation already belongs to the new episode and the final observation and info are in <code>infos["final_obs"]</code> and <code>infos["final_info"]</code> .
<code>"disabled"</code>	No automatic reset; call <code>reset(options={"reset_mask": mask})</code> with a boolean array of shape ($B, \text{}$) to reset selected copies.

Seeding. `reset(seed=s)` makes a batch reproducible in every mode, with one difference. Sync and Async environments seed copy i with $s + i$, so copy i reproduces a single environment reset with seed $s + i$. Native environments draw the initial states of all copies from one shared generator, so their copies do *not* reproduce single environments seeded with $s + i$; to compare a copy with a single environment, start both from the same state through `reset(options=...)` (for example the `"positions"` option of the consensus environment). Automatic resets continue the generators and take no seed.

Which mode to use. Use the native implementation whenever one exists and many copies are needed (training with large batches, Monte-Carlo evaluation, parameter sweeps): its cost per step grows slowly with B . Use "sync" for environments without a native implementation, for small batches and for debugging (everything runs in one process and exceptions surface directly). Use "async", which steps the copies in worker processes, only when a single step is expensive compared with the inter-process communication, as for AJLATT and Pistonball on a machine with several cores. Sync and Async environments need a scalar reward per copy: for environments that return per-agent reward arrays (AJLATT, `EnvSpec.per_agent_rewards`) `make_vec` applies `TeamReward` to every copy unless `wrappers` is given; the per-agent rewards stay in `infos["agent_rewards"]` (Section 11.4).

`env-lib bench ID --num-envs 256` compares the three modes for one environment (Section 11.3), and `benchmarks/benchmark_vector.py` measures native and Sync throughput for 1 to 1024 copies. Table 11.3 lists reference values.

Table 11.3: Throughput in environment steps per second with random actions and automatic resets, measured with `benchmarks/benchmark_vector.py` on one core of an Intel Xeon at 2.8 GHz (OMP_NUM_THREADS=1, Python 3.11, NumPy 2.4). The last column counts agent-steps (copies times agents).

Environment	1 copy	native, $B = 256$	sync, $B = 256$	speed-up	agent-steps/s
Consensus-v0	7 864	345 252	8 695	39.7	2.76×10^6
Formation-v0	7 859	312 695	8 286	37.7	2.50×10^6
KuramotoOscillator-v0	6 895	315 715	9 625	32.8	3.16×10^6
PowerGrid-v0	3 286	133 637	3 074	43.5	2.14×10^6
Platoon-v0	7 128	295 176	7 370	40.1	2.36×10^6

The Sync throughput stays at a few thousand steps per second whatever the number of copies, because every copy is a separate Python object stepped in a loop. The native throughput grows with B until the arrays no longer fit in the processor caches; with 1024 copies it is between 1.1×10^5 (PowerGrid) and 5.3×10^5 (Platoon) steps per second.

11.2 Discovering environments

```
env_lib.catalog(*, family=None, action_type=None, observation_type=None,
               native_vector=None, available_only=False, inspect=True)
```

returns a `Catalog`, a list of `EnvInfo` records (Table 11.4) whose `str()` is an aligned text table; `to_markdown()` gives a Markdown table and `ids()` the list of ids. The filters combine: `family` takes one family name or several, `action_type` and `observation_type` take "continuous" or "discrete" (Pistonball, whose actions can be either, matches both), `native_vector` selects environments with or without a native vector implementation and `available_only` hides environments whose optional dependencies are missing. With `inspect=True` every environment is created once to read the number of agents and the space shapes; the results are cached for the session, and an environment that cannot be created is reported as unavailable with a `note` instead of raising. `inspect=False` skips this step (and the import of optional dependencies such as PyTorch).

`env_lib.describe(env_id)` returns a readable multi-line description: the summary of the class docstring, the spaces with their bounds, the observation layout (for environments with an `observation_layout`), all constructor parameters (values set by the registered id are marked with *), native vector support, the baseline controller and the optional dependency. An excerpt:

Table 11.4: Fields of `EnvInfo` (a frozen dataclass).

Field	Content
<code>id, family, description</code>	registered id, family (subpackage without <code>_env</code>), one-line summary
<code>observation_type, action_type</code> <code>requires, available</code>	"continuous", "discrete" or "continuous discrete" optional-dependency extra (or <code>None</code>); whether it and the environment module can be imported
<code>native_vector</code> <code>n_agents</code>	whether <code>make_vec</code> uses a native batched implementation leading dimension of the joint observation (1 for a flat observation)
<code>observation_shape, action_shape</code> <code>observation_space, action_space</code> <code>kwargs</code>	shapes of the joint spaces (<code>()</code> for a <code>Discrete</code> action) one-line descriptions of the spaces with their bounds constructor arguments of this registered configuration (read-only)
<code>entry_point, vector_entry_point, note</code>	implementation classes; reason for unavailability

```

Formation-v0 -- networked formation control

family      consensus
summary     Networked multi-agent consensus (rendezvous) and formation control.
agents      8
observation  continuous; Box(8, 16) float32, low -inf, high inf
action       continuous; Box(8, 2) float32, low -1, high 1
obs layout   position    columns 0:2
              velocity    columns 2:4
              offset      columns 4:6
              neighbors    columns 6:16
parameters  n_agents=8, task='formation*', topology='ring', dynamics='single', ...
vector       native batched implementation
              env_lib.consensus_env.vector:ConsensusVectorEnv (env_lib.make_vec)
baseline    Laplacian protocol  $u_i = k \sum_j (y_j - y_i) - k_v v_i$ 
requires    no optional dependencies

```

11.3 The `env-lib` command line

Installing the package provides the console script `env-lib` (equivalently `python -m env_lib`) with the sub-commands of Table 11.5. Constructor arguments are passed as `--kward key=value ...`, where values are Python literals (strings may be written without quotes). Output is plain aligned text; errors such as an unknown id are reported on standard error with exit status 2.

```

env-lib list --family consensus power_grid platoon
env-lib describe PowerGrid-v0
env-lib run Formation-v0 --kward formation_shape=wedge topology=proximity --gif
renders/wedge.gif
env-lib evaluate Formation-v0 --episodes 64 --num-envs 32
env-lib bench Consensus-v0 --num-envs 256 --steps 200

```

Output of `env-lib run Formation-v0 --seed 1 --kward n_agents=6`:

```

Formation-v0 (n_agents=6)

```

Table 11.5: Sub-commands of `env-lib`.

Command	Purpose
<code>list</code> [<code>--continuous --discrete</code>] [<code>--family F ...</code>] [<code>--native</code>] [<code>--available</code>] [<code>--markdown</code>] [<code>--no-inspect</code>]	catalogue table (Section 11.2)
<code>describe ID</code>	<code>env_lib.describe(ID)</code>
<code>baselines</code>	baseline controller of every family
<code>run ID</code> [<code>--steps N</code>] [<code>--policy</code> <code>baseline random</code>] [<code>--seed S</code>] [<code>--gif PATH</code>] [<code>--fps F</code>] [<code>--theme light dark</code>] [<code>--kwarg</code> <code>K=V ...</code>]	roll out one episode, print the team return, the per-agent returns and the final info; record a GIF (or MP4) with <code>record_episode</code>
<code>evaluate ID</code> [<code>--episodes N</code>] [<code>--policy</code> <code>both baseline random</code>] [<code>--num-envs N</code>] [<code>--seed S</code>] [<code>--kwarg K=V ...</code>]	compare random actions with the baseline using <code>env_lib.evaluate</code> , in parallel with <code>--num-envs</code>
<code>bench ID</code> [<code>--num-envs N</code>] [<code>--steps N</code>] [<code>--async</code>] [<code>--kwarg K=V ...</code>]	throughput of the single environment, the native vector environment (if any) and <code>SyncVectorEnv</code> (and <code>AsyncVectorEnv</code>) with a fixed random action

```

policy      baseline: Laplacian protocol  $u_i = k \sum_j (y_j - y_i) - k_v v_i$ 
seed        1
episode     60 steps, terminated (0.01 s)
team return -551.1200
agent return mean -561.1200, min -909.6050 (agent_4), max -305.6054 (agent_1)
  agent      return
  agent_0    -312.4189
  ...
  agent_5    -495.0546
final info   error=0.04559 algebraic_connectivity=1 success=True step=60

```

and of `env-lib evaluate Formation-v0 --episodes 64 --num-envs 32`:

```

Formation-v0: 64 episodes per policy, seed 0, 32 parallel copies
policy  mean return      std  95% CI +-  length  terminated  success    s
random   -18076.8171  5566.8354  1390.5541  200.0         0%         0%  0.29
baseline -1608.2215   635.6866   158.7898   92.7         100%       100%  0.09

```

11.4 Wrappers and adapters

The environments expose the whole team through one Gymnasium interface (Chapter 3): stacked observations, a joint action and a team reward with the per-agent rewards in `info["agent_rewards"]`. The package `env_lib.wrappers` converts this interface to the forms expected by single-agent and multi-agent libraries.

FlattenJointSpaces(env). The joint observation becomes a one-dimensional `float32` vector and the wrapper accepts a flat action, which it reshapes to the joint action shape. This

is the form single-agent libraries such as Stable-Baselines3 and CleanRL expect; they then learn one centralised policy for the team. `Box`, `MultiBinary` and `MultiDiscrete` observations are supported (discrete values become `float32` entries with matching bounds), as are `Box`, `MultiBinary`, `MultiDiscrete` and `Discrete` actions (`Discrete` is passed through); other spaces raise `TypeError`. Rewards and flags are passed through, so environments with per-agent reward arrays (AJLATT) are combined with `TeamReward`. The flattened environments pass Gymnasium's `check_env`: the test suite checks the flattened Consensus, Kuramoto, LineMsg and Wireless-Comm environments and `TeamReward` on AJLATT, and the flattened PowerGrid, Platoon and `TeamReward`-wrapped AJLATT environments pass as well. Stable-Baselines3 itself is not part of the test suite. A typical use (not executed here, since Stable-Baselines3 is not a dependency):

```
import env_lib
from env_lib.wrappers import FlattenJointSpaces, TeamReward
from stable_baselines3 import PPO          # not a dependency of env_lib

env = FlattenJointSpaces(env_lib.make("Formation-v0")) # obs (128,), act (16,)
model = PPO("MlpPolicy", env).learn(total_timesteps=200_000)

ajlatt = FlattenJointSpaces(TeamReward(env_lib.make("AJLATT-v0")))
```

`TeamReward(env, reduce="sum", source="reward")`. Returns a scalar team reward and scalar `terminated`/`truncated` flags. With `source="reward"` a per-agent reward array is reduced with `reduce` ("sum" or "mean") and a scalar reward, which already is the environment's team reward (possibly with a success bonus), is passed through; with `source="agent_rewards"` the team reward is always `reduce(info["agent_rewards"])`. `terminated` is true when any agent terminated. The per-agent values stay available in `info["agent_rewards"]` and `info["agent_terminated"]`. It generalises `env_lib.ajlatt_env.TeamRewardWrapper`.

`ParallelEnvAdapter(env)` and `to_parallel(env_or_id, **kwargs)`. These expose an environment through the PettingZoo Parallel API, in which every quantity is a dictionary keyed by agent name. PettingZoo is not a dependency: when it is installed the adapter subclasses `pettingzoo.utils.env.ParallelEnv`, otherwise it is a plain class with the same protocol. Agents are named "agent_0", "agent_1", ... in the order of the leading axis of the joint observation (a flat observation, as in Kuramoto, gives the single agent "agent_0").

- `observation_space(agent)` and `action_space(agent)` are slices of the joint spaces: `Box (n, d)` gives `Box (d,)`, `Box (n,)` gives `Box (1,)`, `MultiDiscrete` and `MultiBinary` give `Discrete`, and the joint `Discrete(2 ** n)` of `LineMsg` gives `Discrete(2)` per agent (bit i is agent i).
- `reset(seed, options)` returns observation and info dictionaries; `options` are forwarded to the environment unchanged.
- `step(actions)` takes one action per agent and returns observation, reward, termination, truncation and info dictionaries. Rewards come from `info["agent_rewards"]`; termination and truncation flags are per agent when the environment reports arrays (AJLATT) and shared otherwise. When the joint episode ends, agents that did not terminate are marked truncated and `agents` becomes empty.
- Info arrays whose leading dimension is the number of agents are sliced per agent (for example row i of the consensus adjacency matrix); other entries are shared. `state()` returns the joint observation (`state_space` is the joint observation space); `render()` and `close()` are forwarded.

```

import env_lib
from env_lib.wrappers import to_parallel

env = to_parallel("PowerGrid-v0", n_buses=24)
policy = env_lib.baseline_policy(env)          # dictionaries in, dictionaries out
observations, infos = env.reset(seed=0)
while env.agents:
    actions = policy(observations)              # {agent: action}
    observations, rewards, terminations, truncations, infos = env.step(actions)
print(len(rewards), sum(rewards.values()))     # 24 agents, last team reward
env.close()

```

PettingZoo's `parallel_api_test` passes for the adapted environment of every family: `tests/envs/test_wrappers.py` runs it on LineMsg (both action spaces), WirelessComm, AJLATT, Pistonball, Consensus, Kuramoto, PowerGrid and Platoon. The test resets with the dummy option `{"options": 1}`; the environments with reset options ignore such unknown keys with a `UserWarning` (Section 3.5.6), and the others ignore `options` altogether.

11.5 Baseline controllers

`env_lib.baseline_policy(env, **kwargs)` returns a classical controller for any environment of the package: a single environment, a vector environment (native, Sync or Async), wrapped or not, a `ParallelEnvAdapter` or a registered id. It dispatches on the type of the unwrapped environment and reads the parameters it needs (action bounds, dynamics, grid size, ...) from the environment. The returned `BaselinePolicy` maps an observation with any leading batch shape to an action, so one object drives a single environment and all copies of a vector environment; it also accepts PyTorch tensors (returning a tensor on the same device), the observation dictionaries of the parallel adapter (returning action dictionaries) and observations flattened by `FlattenJointSpaces`. Keyword arguments set the controller gains; `env_lib.baselines.list_baselines()` returns a short description per family and the underlying pure functions (`laplacian_consensus`, `kuramoto_feedback`, `linemsg_relay`, `wireless_schedule`, `pistonball_ramp`, `ajlatt_encircle`) are public. Table 11.6 lists the controllers.

Table 11.7 compares the baselines with uniformly random actions. Two environments need care when reading returns. The Kuramoto environments with an order-parameter reward terminate on synchronisation while paying a positive reward per step in their default `reward_mode="dense"`, so a faster controller collects *less* return; their task metric is the time to synchronise. With `reward_mode="penalty"`, which pays the signal minus its maximum, or with termination disabled, the baseline has the higher return (Section 4.4). AJLATT pays a negative reward per step and terminates on collisions, so random robots that collide early collect a *higher* return; with `reward_mode="bounded"` or `terminate_on_collision=False` the comparison is fair (Section 10.4).

11.6 Evaluation and rollouts

```

env_lib.evaluate(env, policy=None, *, n_episodes=10, seed=None,
                 max_steps=None, deterministic_seeds=True)

```

Table 11.6: Baseline controllers (defaults in brackets; all outputs are clipped to the action bounds).

Environment	Controller	Law
Consensus, Formation	Laplacian protocol	$u_i = k \sum_{j \in \mathcal{N}_i} (y_j - y_i) - k_v v_i$, $y = x - d$, from the neighbour slots of the observation [$k = 1$, $k_v = \max(0, \sqrt{k} - c)$ for the double integrator, else 0]; equals <code>laplacian_policy()</code> when all neighbours fit into the slots
Kuramoto (NumPy, PyTorch, vector)	frequency compensation and mean-field phase feedback	$a_i = (\Omega - \omega_i) + k \sin(\psi - \theta_i)$ with the mean phase ψ , Ω the mean natural frequency (or <code>target_frequency</code> for frequency tracking) [$k = 3$]; coupling strengths at the upper bound (dynamic mode)
LineMsg WirelessComm	always relay collision-free schedule	$a_i = 1$ for every agent (maximises the expected reward) agent (a, b) owns access point (a, b) and sends to it when it holds a packet; agents of the last row and column borrow an access point whose owner (and competing borrower) has an empty queue
Pistonball	decentralised ramp	pistons that see the ball track a height rising from fully down half a piston left of the ball to fully up half a piston right of it, $a_i = \text{clip}(k(h_i - h_i^*), -1, 1)$ [$k = 8$]; other pistons move down
AJLATT	encircle the target belief	robot i drives to the slot at angle $2\pi i/N$ and distance r from its target belief: $v = \min(v_{\max}, 0.5\ g\) \max(0, \cos e)$, $\omega = \text{clip}(ke)$, e the bearing error [$r = 1.8$ m, $k = 1.2$]
PowerGrid	droop control	$u_i = \text{clip}(-k\omega_i, -u_{\max,i}, u_{\max,i})$ (<code>droop_policy</code>) [$k = 0.25$]
Platoon	cooperative adaptive cruise control	$u_i = k_p e_i + k_d(v_{i-1} - v_i) + k_a a_{i-1}$ (<code>cacc_policy</code>) [$k_p = 0.5$, $k_d = 0.6$, $k_a = 0.65$]

Table 11.7: Mean return (95 % confidence half-width) and mean episode length of random actions and the baseline; 10 episodes with seeds 0–9 (5 for AJLATT), default parameters unless stated, measured with `env_lib.evaluate`.

Environment	Random		Baseline	
	return	length	return	length
Consensus-v0	-16923 ± 3372	200	-1456 ± 421	85
Formation-v0	-17673 ± 3470	200	-1676 ± 503	96
KuramotoOscillator-v0	24.4 ± 1.1	22	17.7 ± 0.8	12
reward_mode="penalty"	2.3 ± 2.4	22	6.2 ± 1.2	12
sync_threshold=1.5, max_steps=200	191.7 ± 3.0	200	196.2 ± 1.2	200
KuramotoOscillator-Constant-v0	65.7 ± 9.5	92	47.8 ± 3.8	58
reward_mode="penalty"	-26.6 ± 17.2	92	-10.6 ± 7.1	58
sync_threshold=1.5, max_steps=200	165.1 ± 14.4	200	179.4 ± 7.1	200
KuramotoOscillator-FreqSync-Constant-v0	-149.1 ± 25.7	92	-124.7 ± 14.2	61
LineMsg-v0	39.8 ± 3.3	50	95.0 ± 0.0	50
WirelessComm-v0	391.8 ± 12.1	50	923.6 ± 9.1	50
WirelessComm-v1	141.5 ± 9.0	50	336.3 ± 7.7	50
Pistonball-v0	-323 ± 88	125	919 ± 34	83
AJLATT-v0	-414 ± 317	13	-7025 ± 281	120
terminate_on_collision=False	-35347 ± 9948	120	-7025 ± 281	120
reward_mode="bounded"	33.5 ± 19.8	13	235.8 ± 4.4	120
PowerGrid-v0	-750.5 ± 469.3	195	-0.7 ± 0.3	200
Platoon-v0	-4957 ± 1673	69	-58.1 ± 7.5	600

runs a policy (any callable from observation to action; `None` samples uniformly random actions from a private, seeded copy of the action space) and returns an `EvaluationResult` (Table 11.8).

- Single environment: episode k is reset with seed `seed + k` (`deterministic_seeds=True`), or only the first reset is seeded.
- Vector environment (native, Sync or Async): the copies run in parallel after one `reset(seed=seed)`. Copy i contributes the first $\lfloor (N + i)/B \rfloor$ episodes it finishes (N episodes, B copies), so that short episodes are not over-represented. All autoreset modes are handled: the reset step of "next_step" is not counted, the final per-agent rewards of "same_step" are read from `infos["final_info"]`, and "disabled" copies are reset through the reset mask.
- `max_steps` caps the episode length; capped episodes count as truncated (vector copies are then reset through the reset mask).
- Per-agent reward arrays (AJLATT) are summed into the team return.

Table 11.8: Fields of `EvaluationResult` (a frozen dataclass).

Field	Content
<code>returns, lengths, terminated</code>	per-episode team return, length and termination flag (read-only arrays)
<code>agent_returns</code>	per-episode per-agent returns (<code>N, n</code>) from <code>info["agent_rewards"]</code> , or <code>None</code>
<code>successes</code>	<code>info["success"]</code> at the end of each episode, or <code>None</code>
<code>mean_return, std_return</code>	mean and sample standard deviation (<code>ddof = 1</code>) of the returns
<code>ci95</code>	half-width $t_{0.975, N-1} s / \sqrt{N}$ of the 95 % confidence interval of the mean return (<code>nan</code> for one episode)
<code>mean_length, termination_rate, success_rate</code>	mean length and fractions of terminated and successful episodes
<code>env_id, policy, n_episodes</code>	labels and count; <code>summary()</code> returns the scalars as a dictionary

`print(result)` gives a compact summary:

```
EvaluationResult: Formation-v0, policy baseline:consensus, 64 episodes
  return      -1608.222 +- 635.687 (std), 95% CI +- 158.790
  length      92.7 steps (min 59, max 113)
  terminated   100.0% of episodes
  success     100.0% of episodes
  agent return mean -1618.222, min -1863.161, max -1463.515 (8 agents)
```

```
env_lib.rollout(env, policy=None, *, n_steps=None, n_episodes=None, seed=None)
```

records the transitions of a single environment as a `Trajectory` of stacked arrays: `observations` and `next_observations` (`T, ...`), `actions`, the team rewards (`T,`), `agent_rewards` (`T, n`) (or `None`), `terminated`, `truncated` and `episode_ids` (`T,`). Without limits one episode is recorded; episode k is reset with seed `seed + k`. `Trajectory.save(path)` writes a compressed `.npz` file, `Trajectory.load(path)` reads it back, `episode(k)` selects one episode and `episode_returns()` returns the team return of each episode.

```
import env_lib

env = env_lib.make("Platoon-v0")
data = env_lib.rollout(env, env_lib.baseline_policy(env), n_episodes=20, seed=0)
data.save("data/platoon_cacc.npz")          # e.g. for offline RL
print(len(data), data.n_episodes, data.episode_returns().mean())
```

11.7 A complete workflow

examples/getting_started/workflow_demo.py combines the pieces of this chapter: it lists the environments with continuous actions, creates a native vector environment, evaluates random actions and the baseline on all copies in parallel, flattens the environment for a single-agent library, adapts it to the Parallel API and records a GIF of the baseline (about ten seconds in total):

```
python examples/getting_started/workflow_demo.py
python examples/getting_started/workflow_demo.py --env Consensus-v0 --num-envs 128
--episodes 128
```

Part II

Algorithms

Chapter 12

Multi-Agent Reinforcement Learning Algorithms

The package `marl_algorithms`, new in version 1.2, contains reference implementations of seven classical algorithms for cooperative multi-agent reinforcement learning: independent and multi-agent proximal policy optimisation (IPPO, MAPPO), multi-agent deep deterministic policy gradient and its twin-critic variant (MADDPG, MATD3), and independent Q-learning, value decomposition networks and QMIX (IQL, VDN, QMIX). All seven are written against one small core and train directly on the environments of `env_lib`: on the native batched vector environments where these exist, with the per-agent rewards that every environment reports, and with evaluation against the classical baseline controllers of Section 11.5.

The code is meant to be read next to the papers. Every algorithm module cites the method it implements, every class docstring states the loss it minimises, and every simplification or deviation from the original method is stated in the code and in this chapter. Tuned *presets* train each algorithm on one or more demonstration environments in at most about two minutes on a single CPU core. The package requires PyTorch (the `torch` extra, Section 2.2.1); `import marl_algorithms` itself loads nothing but the version number, and the first use of an algorithm imports PyTorch.

12.1 Overview

Three parts of the package connect the algorithms to `env_lib`:

1. *A per-agent view of the joint spaces.* The environments expose the whole team through joint spaces: observations stacked as `(n_agents, obs_dim)` and one joint action (Section 3.5.2). `MultiAgentSpec` reads the number of agents, the size of one agent's observation and the kind and size of one agent's action from these spaces, and converts observations to per-agent arrays and per-agent actions back to the joint action (Section 12.4.1).
2. *Training on batched copies.* `make_vector_env` creates `num_envs` copies of an environment with `env_lib.make_vec`, which uses the native batched implementation when there is one (Section 11.1), with the "same_step" autoreset mode, so that every transition is a real one and the final observation of a truncated episode is available for bootstrapping. A `VectorRunner` turns every vector step into a `Transition` of per-agent arrays, including the per-agent rewards of `info["agent_rewards"]` (Section 12.4.2).
3. *Evaluation with the env_lib tools.* The method `policy()` of a trained algorithm returns a function from environment observations to environment actions that works for single and

vector environments. The evaluation function `env_lib.evaluate` therefore compares the trained policy with random actions and with the baseline controller on the same seeded episodes, and the recording function `record_episode` of `env_lib.utils` turns an episode of it into an animation (Section 12.2).

Table 12.1 lists the algorithms as registered in `marl_algorithms.registry`; the function `list_algorithms()` returns these records and the command `marl-train list` prints them. Section 12.11 gives the full references cited in the source code.

Table 12.1: Registered algorithms. “Family” and “Actions” are the `family` and `action_kinds` fields of `AlgorithmInfo`.

Name	Family	Actions	Key idea	Reference
<code>ippo</code>	on-policy	continuous, discrete	PPO for every agent; decentralised critic $V(o_i)$ trained on the agent’s own reward	de Witt et al. (2020)
<code>mappo</code>	on-policy	continuous, discrete	decentralised PPO actors; centralised critic $V(s, i)$ on the global state, trained on the team reward	Yu et al. (2022)
<code>maddpg</code>	off-policy actor-critic	continuous	deterministic actors $\mu_i(o_i)$; centralised critics $Q_i(s, a_1, \dots, a_n)$ on the global state and the joint action	Lowe et al. (2017)
<code>matd3</code>	off-policy actor-critic	continuous	MADDPG with twin critics, target policy smoothing and delayed actor updates	Ackermann et al. (2019)
<code>iq1</code>	value-based	discrete	one DQN utility $Q_i(o_i, a_i)$ per agent, each trained on its own reward	Tan (1993)
<code>vdn</code>	value-based	discrete	team value $\sum_i Q_i(o_i, a_i)$, trained end to end on the team reward	Sunehag et al. (2018)
<code>qmix</code>	value-based	discrete	team value as a monotonic, state-conditioned mixture of the utilities (hypernetworks)	Rashid et al. (2018)

Every algorithm works on every environment whose action kind it supports. Table 12.2 shows the per-agent structure that `MultiAgentSpec` reads from the registered environments; every environment was trained briefly with an algorithm of each family that applies to it, and an algorithm given an environment of the wrong action kind raises `TypeError`. Environments whose observation is not stacked per agent, the Kuramoto environments, are treated as a single agent that observes and acts on the whole network; with one agent IPPO and MAPPO reduce to PPO, MADDPG and MATD3 to DDPG and TD3, and IQL to (double) DQN.

12.2 Quick start

12.2.1 Training and evaluating from Python

`marl_algorithms.train` trains a registered algorithm on batched copies of an environment and returns the trained algorithm together with its `TrainingLog`. The run below is deliberately short (2000 environment steps, a few seconds); the tuned MAPPO preset for `PowerGrid-v0` trains for 250 000 steps (Section 12.8).

```
import env_lib
from marl_algorithms import train

# A short run; the tuned MAPPO preset for PowerGrid-v0 trains for 250_000 steps.
algo, log = train("mappo", "PowerGrid-v0", total_steps=2_000, num_envs=8,
                  seed=0)
```

Table 12.2: Per-agent structure of the registered environments as read by `MultiAgentSpec` (default parameters) and the algorithms that apply. n is the number of agents and d the size of one agent's observation.

Environment	n	d	Action per agent	Algorithms
Consensus-v0, Formation-v0	8	16	continuous, 2-d	IPPO, MAPPO, MADDPG, MATD3
PowerGrid-v0	16	10	continuous, 1-d	IPPO, MAPPO, MADDPG, MATD3
Platoon-v0	8	13	continuous, 1-d	IPPO, MAPPO, MADDPG, MATD3
AJLATT-v0	4	24	continuous, 2-d	IPPO, MAPPO, MADDPG, MATD3
Pistonball-v0	20	7	continuous, 1-d; 3 choices with <code>continuous=False</code>	all seven, by action mode
LineMsg-v0 (per-agent bits)	10	3	2 choices	IPPO, MAPPO, IQL, VDN, QMIX
WirelessComm-v1 (-v0)	16 (36)	18	5 choices	IPPO, MAPPO, IQL, VDN, QMIX
KuramotoOscillator-v0 (other Kuramoto ids: other sizes)	1	75	continuous, 55-d	IPPO, MAPPO, MADDPG, MATD3

```
print(algo)
print(len(log.episodes), "training episodes, mean return",
      round(log.mean_return(), 1))

envs = env_lib.make_vec("PowerGrid-v0", 16)
for name, policy in [("random", None), ("mappo", algo.policy()),
                    ("droop", env_lib.baseline_policy(envs))]:
    result = env_lib.evaluate(envs, policy, n_episodes=32, seed=1)
    print(f"{name:8s}{result.mean_return:10.2f}")
envs.close()
```

```
MAPPO(16 agents, 10-d observations, continuous 1-d per agent, env_steps=2048)
8 training episodes, mean return -381.0
random      -613.22
mappo       -49.52
droop       -0.71
```

```
train(algorithm, env_id, total_steps, *, num_envs=16, seed=0, env_kwargs=None,
      device="cpu", callback=None, **config) -> (Algorithm, TrainingLog)
```

creates the copies with `make_vector_env`, builds the algorithm with the seed and the configuration overrides `**config` (field names of the algorithm's configuration class; an unknown name raises `TypeError`), trains it with `learn`, whose environment reset uses the same seed, and closes the copies. On-policy algorithms stop after the first complete rollout that reaches `total_steps`, so the run above collected 2048 steps (two rollouts of 64 steps on 8 copies). The training returns in the log are those of the exploratory policy; the evaluation uses the deterministic policy (the mean action), which `algo.policy()` returns by default.

The presets of Section 12.8 are looked up and trained with the functions of `marl_algorithms.presets`, which the top-level package also exports:

```

from marl_algorithms import get_preset, list_presets, train_preset

print(len(list_presets()), list_presets()[1:2])
print(get_preset("qmix", "LineMsg-v0"))

# The preset trains for 25_000 steps; a smoke test overrides the length.
algo, log = train_preset("qmix", "LineMsg-v0", total_steps=2_000,
                        warmup_steps=500)
print(algo.config.lr, algo.config.batch_size, algo.env_steps)

```

```

17 [('ippo', 'PowerGrid-v0'), ('ippo', 'Platoon-v0')]
{'num_envs': 16, 'total_steps': 25000, 'config': {'batch_size': 128,
'lr': 0.001, 'epsilon_decay_steps': 12500}, 'env_kwargs': {}}
0.001 128 2000

```

```

train_preset(algorithm, env_id, *, seed=0, total_steps=None, num_envs=None,
             env_kwargs=None, device="cpu", callback=None,
             **config) -> (Algorithm, TrainingLog)

```

calls `train` with the preset's values; `total_steps`, `num_envs` and configuration overrides replace them, and `env_kwargs` are merged into the preset's environment arguments. It raises `KeyError` when the pair has no preset. `get_preset` returns a copy of the preset (or `None`), so it can be modified freely.

A trained algorithm drives any environment of the same structure through `policy()`, and `save` and `Algorithm.load` store and restore it:

```

import numpy as np
import env_lib
from env_lib.utils import record_episode
from marl_algorithms import Algorithm, make_vector_env, train

algo, log = train("vdn", "LineMsg-v0", total_steps=2_000, warmup_steps=500)

# Record the greedy policy on a single environment with per-agent actions.
env = env_lib.make("LineMsg-v0", action_space_type="multibinary",
                  render_mode="rgb_array")
frames = record_episode(env, algo.policy(), "renders/vdn_linemsg.gif",
                      seed=1, render_every=2)
env.close()

# Save and restore: the class and the configuration are read from the file.
path = algo.save("runs/vdn_linemsg.pt")
restored = Algorithm.load(path)
envs = make_vector_env("LineMsg-v0", 4)
obs = restored.spec.agent_obs(envs.reset(seed=0)[0]) # (4, 10, 3)
assert np.array_equal(restored.act(obs, deterministic=True),
                    algo.act(obs, deterministic=True))
print(type(restored).__name__, restored.env_steps, len(frames))
envs.close()

```

The file written by `save` holds the registry name, the `MultiAgentSpec`, the configuration, the seed and the state (networks, optimisers, counters and normalisation statistics); `Algorithm.load` selects the class through the registry. The replay buffer of the off-policy methods is not saved. Checkpoints are read with `torch.load(..., weights_only=False)` because they contain more

than tensors, so load only files from trusted sources.

Training in stages. The method `learn` of an algorithm (Section 12.4.6) trains until the counter `algo.env_steps`, which persists across calls and through `save` and `load`, reaches `total_steps`; `total_steps` is therefore a cumulative target, not a number of additional steps. To continue a run, call `learn` again with a larger total and pass the previous log to append to it. Every call resets the copies with its `seed`, so episodes still running at the end of a call are not completed. The callback is called as `callback(algorithm, log)` after every update (on-policy) or every round of gradient steps (off-policy); returning `False` stops training. The environment copies passed to `learn` must use the "same_step" autoreset mode, which `make_vector_env` sets:

```
from marl_algorithms import MATD3, make_vector_env

def stop_when_solved(algo, log):
    """Stop once the last 20 training episodes average more than -1600."""
    if log.mean_return(20) > -1600:      # NaN (no episode yet) compares False
        return False

envs = make_vector_env("Consensus-v0", 16)
algo = MATD3(envs, seed=0, reward_scale=0.01, normalize_observations=True)
log = algo.learn(envs, 3_200, seed=0)      # one episode per copy
log = algo.learn(envs, 6_400, seed=1, log=log,      # 3_200 more steps
                 callback=stop_when_solved)
print(algo.env_steps, algo.num_updates, len(log.episodes))
log.to_csv("runs/matd3_consensus.csv")
envs.close()
```

12.2.2 The marl-train command line

Installing the package provides the console script `marl-train` (equivalently `python -m marl_algorithms`) with the sub-commands of Table 12.3. Values given with `--set` and `--env-kwarg` are parsed as Python literals (plain text otherwise) and override the preset's configuration and environment arguments.

```
marl-train list                # algorithms and references
marl-train presets             # tuned (algorithm, environment) pairs
marl-train run mappo PowerGrid-v0  # train with the preset, then evaluate
marl-train run qmix LineMsg-v0 --steps 50000 --set lr=1e-3
marl-train run maddpg Consensus-v0 --save runs/maddpg.pt --csv runs/maddpg.csv
marl-train evaluate runs/maddpg.pt Consensus-v0 --episodes 64
marl-train run ippo Formation-v0 --no-preset --steps 200000 --num-envs 32 \
    --set lr=1e-3 normalize_observations=False --env-kwarg formation_shape=wedge
```

The evaluation of `run` and `evaluate` uses `make_vector_env(env_id, min(episodes, 64))`, so up to 64 episodes run in parallel. Output of `marl-train run qmix LineMsg-v0` (the complete preset; wall-clock times depend on the machine):

```
Training qmix on LineMsg-v0 (preset): 25,000 env steps, 16 copies, seed 0
    2,512 steps mean return (last 20)      43.02      2.7 s
    5,008 steps mean return (last 20)      54.33      4.7 s
    ...
   22,512 steps mean return (last 20)      92.35     17.3 s
   25,008 steps mean return (last 20)      91.96     19.1 s
```

Table 12.3: Sub-commands of `marl-train` (defaults in brackets).

Command	Purpose
<code>list</code>	the registered algorithms with family, action kinds, summary and reference
<code>presets</code>	every (<code>algorithm</code> , <code>environment</code>) pair with a preset, with its steps and copies
<code>run ALGORITHM ENV_ID</code>	train (with the preset when there is one), print progress and a summary, optionally save the algorithm and the training episodes, then evaluate random actions, the trained policy and the baseline
<code>--steps N, --num-envs N</code>	environment steps and batched copies [preset; 100 000 and 16 without a preset]
<code>--seed S</code>	seed of training [0]; the evaluation uses $S + 1$
<code>--no-preset</code>	ignore the preset and use the defaults of the configuration class
<code>--set KEY=VALUE ...</code>	configuration overrides
<code>--env-kwarg KEY=VALUE ...</code>	environment arguments
<code>--eval-episodes N</code>	evaluation episodes [64]; 0 skips the evaluation
<code>--reports N</code>	progress lines during training [10]
<code>--save PATH, --csv PATH</code>	save the algorithm (<code>Algorithm.save</code>) and the training episodes (<code>TrainingLog.to_csv</code>)
<code>--threads N</code>	PyTorch threads [1]; 0 keeps the PyTorch default
<code>compare ENV_ID</code>	train baseline algorithms over one or more seeds and compare them with random actions and the classical controller on the same episodes (Section 12.3)
<code>--algos ALGO ...</code>	algorithms [every one with a preset for the environment]
<code>--seeds S ...</code>	training seeds [0]
<code>--steps N, --num-envs N</code>	budget and copies per run [preset]
<code>--episodes N, --eval-seed S</code>	evaluation episodes [64] and seed [1]
<code>--set, --env-kwarg</code>	as for <code>run</code>
<code>--markdown, --csv PATH</code>	print a Markdown table, save the report as CSV
<code>--save-dir DIR</code>	save every trained algorithm as <code>DIR/NAME_seedS.pt</code>
<code>--threads N</code>	PyTorch threads [1]
<code>evaluate CHECKPOINT ENV_ID</code>	evaluate a saved algorithm against random actions and the baseline
<code>--episodes N, --seed S</code>	evaluation episodes [64] and seed [1]
<code>--env-kwarg KEY=VALUE ...</code>	environment arguments (as used in training)

```
qmix on LineMsg-v0: 496 episodes, 25008 env steps, mean return (last 100) 92.18,
17.0 s
```

```
Evaluation on LineMsg-v0, 64 episodes (seed 1):
```

policy	mean return	95% CI +-	length
random	43.2	1.17	50.0
qmix	95	0	50.0
baseline	95	0	50.0

The progress lines report the mean return of the last 20 exploratory training episodes; the summary line is `TrainingLog.summary()`, with the environment steps taken and the time from the start of training to its last record. The evaluation columns are the mean team return of the deterministic policy, the half-width of its 95 % confidence interval and the mean episode length (Table 11.8).

12.3 Using the algorithms as baselines

A new method for one of the environments is judged against established ones. Three kinds of reference are available for every environment: uniformly random actions (the floor), the environment's classical controller (`env_lib.baseline_policy`, the standard of its field) and the seven learning algorithms, with tuned presets for the environments of Table 12.9. The function `marl_algorithms.compare` trains the learning baselines, over as many seeds as you ask for, and evaluates everything, your methods included, on the same seeded episodes.

12.3.1 In one call

```
import numpy as np
from marl_algorithms import compare

def my_policy(obs):
    """Your method: (num_envs, n_agents, obs_dim) -> (num_envs, n_agents)."""
    return np.zeros(obs.shape[:-1], dtype=np.float32) # replace me

report = compare("PowerGrid-v0", ["mappo", "ippo"], seeds=(0, 1),
                 policies={"my method": my_policy},
                 total_steps=4_096, num_envs=16) # small budget: seconds
print(report)
report.to_csv("runs/power_grid_comparison.csv")
```

Without `total_steps` and `num_envs` every algorithm trains with its preset. The script `examples/algorithms/baseline_comparison.py` runs exactly this comparison with a proposed method, a distributed droop controller that also reacts to the neighbours' mean frequency deviation, and the full presets (about six minutes on one core):

PowerGrid-v0: mean return over 64 evaluation episodes (seed 1); higher is better

method	kind	mean return	std	seeds	env steps	train [s]
-----	-----	-----	-----	-----	-----	-----
random	reference	-629.3	-	-	-	-
baseline	reference	-0.786	-	-	-	-
mappo	algorithm	-1.02	0.182	2	251,904	101.9
ippo	algorithm	-2.25	0.172	2	200,704	65.2
my method	policy	-0.675	-	-	-	-

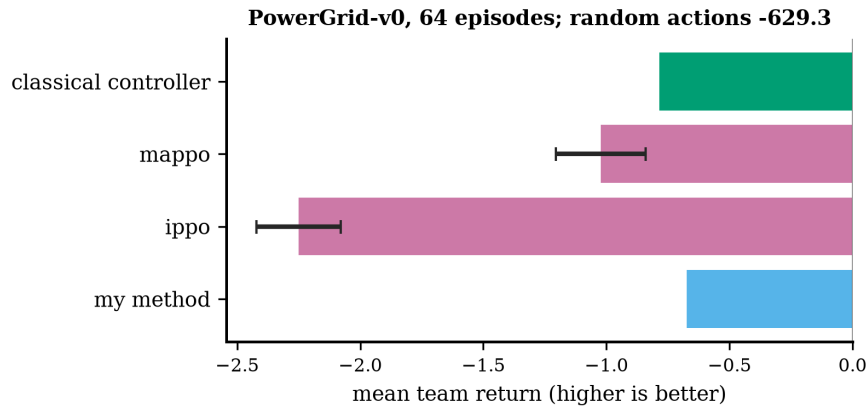


Figure 12.1: Output figure of `baseline_comparison.py`: mean return on PowerGrid-v0 over 64 evaluation episodes; the error bars are the standard deviation over two training seeds. Random actions (−629.3) are off the scale.

Figure 12.1 draws the same numbers. The rows are the references first, then the algorithms with the mean and the standard deviation over the training seeds, the environment steps and the wall-clock training time of one run, then your policies. `compare` works in five steps:

1. it checks every argument: the algorithm names, their support for the environment’s actions, the configuration overrides (applied to each algorithm’s preset), the environment arguments, the seeds and the budget, so that a mistake shows in seconds instead of after a training run;
2. it creates the evaluation environment, `make_vector_env(env_id, min(n_episodes, 64), **env_kwargs)`;
3. it evaluates random actions, the classical controller (when the environment has one) and your policies, first, so that an error in a policy shows before any training;
4. it trains every algorithm once per seed, with its preset (`train_preset`) or, without a preset, with its default configuration and `total_steps`, and evaluates the deterministic policy;
5. it returns a `Comparison`, which also keeps every trained algorithm and training log (the replay buffers of off-policy algorithms are released unless `keep_replay=True`).

Every evaluation resets the copies with the same seed and counts only the first episode of every copy, so all rows meet identical initial conditions and disturbances. More than 64 episodes run in chunks of 64, reset with `eval_seed`, `eval_seed + 64`, and so on; automatic resets after episodes of policy-dependent length could otherwise give different policies different episodes. With the default training seed 0 and evaluation seed 1 the numbers reproduce the published results of Table 12.10.

12.3.2 From the command line

```
marl-train compare PowerGrid-v0                # every preset algorithm, seed 0
marl-train compare PowerGrid-v0 --algos mappo ippo --seeds 0 1 2 --csv runs/pg.csv
marl-train compare LineMsg-v0 --markdown        # a table for a paper
marl-train compare Consensus-v0 --save-dir runs/baselines # keep the models
marl-train compare Formation-v0 --algos mappo --steps 200000 # no preset
```

`marl-train compare` checks the environment, the algorithm names, their action kinds and the `--set` fields before training (Table 12.3).

12.3.3 Recipes

Choose the baselines and the budget

`algorithms=["mappo", "maddpg"]` selects methods (by default every algorithm with a preset for the environment, `marl-train presets`); `seeds=(0, 1, 2, 3, 4)` adds seeds; `total_steps` and `num_envs` replace the presets' budget, for example for a quick check before a long run.

Another configuration of the environment

`env_kwargs={"n_buses": 32}` trains and evaluates every method on the variant. The presets' hyperparameters are kept; a larger variant may need a larger `total_steps`.

An algorithm without a preset

Pass `total_steps` to train it with its default configuration, or tune it yourself with `train(...)` and pass the trained algorithm in `policies`.

A policy written for a single environment

`compare` evaluates batched observations. Wrap a policy that maps one observation (`n_agents`, `obs_dim`) to one action with `per_copy(policy)`; batched policies are faster.

A method built on this package

A modified algorithm (a new loss, another critic) is an **Algorithm**: train it and pass the object in `policies`; its deterministic policy is evaluated like the baselines. For several seeds of your method, train one per seed and pass them under separate names, or register it (Section 12.10) and list it in `algorithms`.

Reuse the trained baselines

`report.algorithms[("mappo", 0)]` is the trained MAPPO of seed 0: save it, record an episode with `record_episode(env, algo.policy(), path)` or evaluate it on other scenarios; `report.logs` holds the training logs for learning curves (Section 14.4).

Tables and further analysis

`report.to_markdown()`, `report.to_csv(path)` and `report.records()` (one dictionary per row, for example for `pandas.DataFrame`); `report["mappo"].returns` gives the per-seed returns and `report.ranking()` orders the algorithms and your policies by mean return.

12.4 Design of the core

The package separates what all methods share, what distinguishes each method, and the settings tuned for the environments. Every algorithm has its own module, next to the `common` module of its family, and the algorithm modules contain only what is specific to them:

<code>src/marl_algorithms/</code>	
<code>__init__.py</code>	lazy exports (<code>train</code> , <code>compare</code> , <code>algorithms</code> , ...)
<code>__main__.py</code>	the <code>marl-train</code> command line
<code>registry.py</code>	<code>AlgorithmInfo</code> , <code>get_algorithm</code> , <code>make_algorithm</code> , <code>train</code>
<code>baselines.py</code>	<code>compare</code> , <code>Comparison</code> , <code>per_copy</code> : baselines
<code>core/</code>	shared by all methods
<code>spec.py</code>	<code>MultiAgentSpec</code> : per-agent view of the joint spaces
<code>runner.py</code>	<code>make_vector_env</code> , <code>VectorRunner</code> , <code>Transition</code>
<code>buffers.py</code>	<code>RolloutBuffer</code> , <code>compute_gae</code> , <code>ReplayBuffer</code>
<code>normalization.py</code>	<code>RunningMeanStd</code> , <code>ObservationNormalizer</code> , <code>RewardScaler</code>
<code>networks.py</code>	<code>mlp</code> , <code>policies</code> , <code>QMixer</code> , <code>PerAgent</code> , <code>agent_one_hot</code> , <code>soft_update</code>
<code>config.py</code>	<code>AlgorithmConfig</code> , <code>OnPolicyConfig</code> , <code>OffPolicyConfig</code>
<code>base.py</code>	<code>Algorithm</code> , <code>OnPolicyAlgorithm</code> , <code>OffPolicyAlgorithm</code> ,

	TrainingLog
algorithms/	one module per algorithm, grouped by family
ppo/	on-policy
common.py	PPOConfig and the shared PPO implementation
ippo.py	IPPO (decentralised critics)
mappo.py	MAPPPO (centralised critic)
ddpg/	off-policy actor-critic
common.py	DDPGConfig and the shared implementation
maddpg.py	MADDPG
matd3.py	MATD3 (twin critics, smoothing, delayed actors)
q_learning/	value-based
common.py	QLearningConfig and the shared implementation
iql.py	IQL (no mixing)
vdn.py	VDN and VDNMixer (additive mixing)
qmixon.py	QMIX (monotonic mixing with QMixer)
presets/	tuned settings for the env_lib environments
__init__.py	PRESETS, get_preset, list_presets, train_preset
ppo.py, ddp.py, q_learning.py	the tables of each family, with tuning notes

The docstring of every family package (for example `help(marl_algorithms.algorithms.ppo)`) describes the methods, the implementation choices and the references; the docstring of every class gives its losses.

12.4.1 The per-agent view: MultiAgentSpec

`MultiAgentSpec.from_env(env)` reads the agent structure of a single or vector environment (from `single_observation_space` and `single_action_space` when present) and `MultiAgentSpec.from_spaces(observation_space, action_space)` from the spaces. The spec is a frozen dataclass with the fields `n_agents`, `obs_dim`, `action_kind` ("continuous" or "discrete"), `action_dim` (1 for discrete actions), `n_actions` (discrete only), `action_low` and `action_high` (continuous only, shape `(n_agents, action_dim)`), `obs_shape`, `action_shape` and `action_dtype`. The observation space must be a `Box`; the action spaces are read as follows.

- `Box` of shape `(n, d)` with observations `(n, obs_dim)`: n agents with d -dimensional continuous actions; shape `(n,)`: n agents with one action each. Any other `Box` is a single agent whose action is the flattened box (the Kuramoto environments). The bounds must be finite; they may differ between agents and dimensions.
- `MultiDiscrete` and `MultiBinary`: one agent per entry, with the same number of choices for every agent; `MultiDiscrete` must start at 0.
- `Discrete`: only for a single agent. A joint `Discrete(2 ** n)` action of `LineMsg`, raises `TypeError`; `make_vector_env` therefore creates `LineMsg` with `action_space_type="multibinary"`.
- With several agents the observation must be stacked as `(n, obs_dim)`; with one agent it is flattened.

`spec.agent_obs(obs)` returns observations with any leading batch shape as `float32` arrays `(*batch, n_agents, obs_dim)`; `spec.env_action(actions)` converts per-agent actions, `(*batch, n_agents, action_dim)` floats (or `(*batch, n_agents)` when `action_dim` is 1) or `(*batch, n_agents)` integers, into the joint action and clips continuous actions to the bounds. The property `state_dim = n * obs_dim` is the size of the global state $s = [o_1, \dots, o_n]$ that the centralised critics and the QMIX mixer use: none of the environments exposes a separate state,

so the global state is the concatenation of all agents' observations. `describe()` returns a one-line summary such as "16 agents, 10-d observations, continuous 1-d per agent".

12.4.2 Collecting experience: VectorRunner and Transition

```
make_vector_env(env_id, num_envs=16, **env_kwargs)
# = env_lib.make_vec(env_id, num_envs, autoreset_mode="same_step", **env_kwargs)
```

creates the training copies; for LineMsg it sets `action_space_type="multibinary"` unless the argument is given. Environments without a native vector implementation (LineMsg, WirelessComm, Pistonball, AJLATT and the PyTorch Kuramoto ids) are simulated with Gymnasium's `SyncVectorEnv`.

`VectorRunner(envs, spec=None)` steps such a vector environment in per-agent form; it raises `ValueError` for any autoreset mode other than "same_step". `reset(seed=None)` resets all copies and returns observations (`B, n, obs_dim`); `step(actions)` applies per-agent actions to all copies and returns a `Transition` (Table 12.4), after which `runner.obs` holds the observations to act on next; `pop_episodes()` returns and clears the `(return, length)` pairs of the episodes finished so far.

Table 12.4: Fields of Transition (B copies, n agents).

Field	Shape	Content
<code>obs</code>	<code>(B, n, obs_dim)</code>	observations the actions were taken in
<code>actions</code>	<code>(B, n, action_dim)</code> or <code>(B, n)</code>	per-agent actions as passed to <code>step</code> (unclipped)
<code>reward</code>	<code>(B,)</code>	team rewards
<code>agent_rewards</code>	<code>(B, n)</code>	per-agent rewards from <code>info["agent_rewards"]</code> ; the team reward for every agent when the environment reports none
<code>next_obs</code>	<code>(B, n, obs_dim)</code>	successor observations; the <i>final</i> observation for copies whose episode ended
<code>terminated,</code> <code>truncated</code>	<code>(B,)</code>	episode-end flags; the property <code>done</code> is their disjunction

With same-step autoreset a finished copy is reset within the step, so the observation returned by the environment already belongs to the next episode. The runner therefore takes the final observation from `infos["final_obs"]` and the final per-agent rewards from `infos["final_info"]["agent_rewards"]`. Every transition is a real one (there is no reset step to skip, as with next-step autoreset), and the value of the final observation is available to bootstrap episodes that were truncated by the time limit. Only `terminated` stops the bootstrap in the algorithms; `truncated` only cuts the advantage recursion.

12.4.3 Buffers and generalised advantage estimation

Rollouts. `RolloutBuffer(length, num_envs, fields)` stores named arrays of shape `(T, B, ...)`, with `fields` a dictionary `{name: (trailing_shape, dtype)}`. `add(**values)` stores one vector step (every field must be given), `buffer[name]` returns the stored steps, `full` tells whether T steps are stored and `reset()` starts a new rollout in the same arrays.

GAE. The function

```
compute_gae(rewards, values, next_values, terminated, truncated, gamma,
            gae_lambda) -> (advantages, returns)
```

works on arrays of shape $(T, B, \dots)$; trailing dimensions such as agents are broadcast against the (T, B) flags. With $V_t = V(o_t)$, V'_t the value of the successor observation o'_t (the final observation when the episode ended), τ_t the **terminated** flag of step t and $done_t$ the flag that is set when the episode terminated or was truncated at step t ,

$$\delta_t = r_t + \gamma(1 - \tau_t)V'_t - V_t, \quad (12.1)$$

$$A_t = \delta_t + \gamma\lambda(1 - done_t)A_{t+1}, \quad A_T = 0, \quad (12.2)$$

$$R_t = A_t + V_t.$$

The recursion runs backwards from the last step of the rollout, where the bootstrap enters through V'_{T-1} in δ_{T-1} . A terminated episode does not bootstrap, a truncated one bootstraps from the value of its final observation, and neither passes advantages across the episode boundary. A test compares the function with this recursion written out step by step.

Replay. `ReplayBuffer(capacity, spec)` is a circular buffer of joint transitions. Its method `add(transition)` stores every copy of a vector transition, one entry per copy and step, so the capacity counts transitions rather than vector steps. The method `sample(batch_size, rng)` draws transitions uniformly with replacement using the algorithm's generator and returns a dictionary of NumPy arrays:

<code>obs, next_obs</code>	$(N, n, \text{obs_dim})$	float32	
<code>actions</code>	$(N, n, \text{action_dim})$	float32	(continuous actions)
	(N, n)	int64	(discrete actions)
<code>reward</code>	$(N,)$	float32	
<code>agent_rewards</code>	(N, n)	float32	
<code>terminated</code>	$(N,)$	float32	

Truncation is not stored, because the stored successor of a truncated transition is its final observation and the target bootstraps from it.

12.4.4 Normalisation

`RunningMeanStd(shape=())` keeps a running mean μ , variance σ^2 and count N (initially 10^{-4}) and includes a batch with mean μ_b , variance σ_b^2 and size N_b with the parallel update rule

$$\Delta = \mu_b - \mu, \quad \mu \leftarrow \mu + \Delta \frac{N_b}{N + N_b}, \quad \sigma^2 \leftarrow \frac{N\sigma^2 + N_b\sigma_b^2 + \Delta^2 N N_b / (N + N_b)}{N + N_b}, \quad N \leftarrow N + N_b. \quad (12.3)$$

`ObservationNormalizer(obs_dim, clip=10.0, enabled=True)` standardises observations feature by feature, $\hat{o} = \text{clip}((o - \mu)/\sigma, -10, 10)$, with statistics *shared by all agents*: every agent's observation of every copy is a sample of the same per-feature statistics, as befits homogeneous agents with shared parameters. `update(obs)` adds observations of any leading shape; with `enabled=False` the normaliser returns its input unchanged.

`RewardScaler(num_envs, gamma, enabled=True)` divides rewards by the running standard deviation of the discounted return. Every copy b keeps a return $G_b \leftarrow \gamma G_b + \bar{r}_b$, where $\bar{r}_b$ is the reward of copy b averaged over its trailing dimensions (the agents), the statistics include the new G_b , the returns of finished copies are reset to zero, and the rewards are returned as r/σ_G .

Rewards are scaled, not centred, so the optimal policy is unchanged, while value targets become of order one whether an environment’s rewards are of order 10^{-3} per step (PowerGrid) or 10^2 (Consensus).

12.4.5 Networks

The building blocks of `marl_algorithms.core.networks` take per-agent inputs with arbitrary leading dimensions (`..., features`):

```
mlp(in_dim, out_dim, hidden_sizes=(64, 64), activation="tanh", *,
    output_gain=1.0, layer_norm=False)
PerAgent(factory, n_agents, shared=True)
GaussianPolicy(in_dim, action_dim, low, high, hidden_sizes, activation,
               log_std_init=-0.5)
CategoricalPolicy(in_dim, n_actions, hidden_sizes, activation)
DeterministicPolicy(in_dim, action_dim, low, high, hidden_sizes,
                   activation="relu")
QMixer(n_agents, state_dim, embed_dim=32, hypernet_hidden=64)
agent_one_hot(batch_shape, n_agents, device)
soft_update(target, source, tau)
```

- `mlp` builds a multi-layer perceptron with orthogonal initialisation (gain $\sqrt{2}$ for ReLU-type activations, $5/3$ for `tanh`) and zero biases; the activation is `"tanh"`, `"relu"`, `"elu"` or `"gelu"`. The policy heads use `output_gain=0.01`, which makes the initial policies nearly uniform (categorical) or centred in the action range (Gaussian, deterministic).
- `PerAgent` with `shared=True` applies one network, built by `factory()`, to all agents in one call; with `shared=False` every agent has its own network and agent i ’s slice of the input goes through network i . `call(method, x, *args)` dispatches any method of the underlying networks in the same way (tensor arguments are sliced per agent, others, such as a `torch.Generator`, are passed unchanged).
- Agent identifiers: when parameters are shared and `agent_ids=True` (both defaults) and there is more than one agent, the one-hot identifier $e_i \in \{0, 1\}^n$ of agent i is appended to its input, $x_i = [\hat{o}_i, e_i]$, so that agents sharing one network can still behave differently. `Algorithm.agent_inputs(obs)` builds these inputs for all agents at once with `agent_one_hot`.
- `GaussianPolicy` is a diagonal Gaussian with a state-independent standard deviation, expressed in units of the action range: with the centre $c = (\text{high} + \text{low})/2$ and the half-range $h = (\text{high} - \text{low})/2$, $\pi_\theta(\cdot | x) = \mathcal{N}(c + h \odot f_\theta(x), \text{diag}(h \odot e^\ell)^2)$ with the learned vector ℓ initialised to `log_std_init`. Samples are unbounded; the log-density $\sum_k \log \mathcal{N}(a_k)$ is that of the unclipped sample, and the entropy is $\sum_k (\frac{1}{2} + \frac{1}{2} \log 2\pi + \log \sigma_k)$. A shared policy keeps the bounds of all agents, so agents with different action ranges can share it.
- `CategoricalPolicy` returns the logits of a categorical distribution; like `GaussianPolicy` it provides `sample(x, generator)` (actions and log-probabilities) and `log_prob(x, action)` (log-probabilities and entropy).
- `DeterministicPolicy` computes $\mu_\theta(x) = c + h \odot \tanh f_\theta(x)$, always within the bounds.
- `QMixer` is the monotonic mixing network of QMIX (Section 12.7), and `soft_update` performs Polyak averaging, $\theta' \leftarrow (1 - \tau)\theta' + \tau\theta$.

12.4.6 Base classes and training loops

Every algorithm derives from `Algorithm`:

```
Algorithm(spec, config=None, *, device="cpu", seed=None, **overrides)
```

`spec` is a `MultiAgentSpec` or an environment (single or vector) to read it from; `config` an instance of the class attribute `config_class` (defaults to `config_class()`); `overrides` replace configuration fields with `dataclasses.replace`. An environment whose action kind is not in the class attribute `action_kinds`, a configuration of the wrong class and an unknown field raise `TypeError`; invalid values raise `ValueError` when the configuration is created. Table 12.5 lists the fields shared by all algorithms.

Table 12.5: Configuration fields shared by the algorithms (`marl_algorithms.core.config`). `PPOConfig` derives from `OnPolicyConfig`, `DDPGConfig` and `QLearningConfig` from `OffPolicyConfig`; both derive from `AlgorithmConfig`.

Field	Default	Meaning
<i>AlgorithmConfig (all algorithms)</i>		
<code>hidden_sizes</code>	(64, 64)	widths of the hidden layers of every network
<code>activation</code>	"tanh"	hidden activation ("tanh", "relu", "elu", "gelu")
<code>gamma</code>	0.99	discount factor γ
<code>share_parameters</code>	True	one set of network weights for all agents
<code>agent_ids</code>	True	append one-hot agent identifiers to per-agent inputs when parameters are shared
<code>max_grad_norm</code>	10.0	gradient-norm clipping threshold
<i>OnPolicyConfig (IPPO, MAPPO)</i>		
<code>max_grad_norm</code>	0.5	overrides the default above
<code>rollout_length</code>	64	vector steps T per rollout ($T \cdot B$ transitions per update)
<code>lr</code>	$3 \cdot 10^{-4}$	Adam learning rate (of the actor for PPO)
<code>gae_lambda</code>	0.95	GAE parameter λ
<code>normalize_observations</code>	True	standardise observations with running statistics
<code>normalize_rewards</code>	True	scale rewards by the running standard deviation of the return
<i>OffPolicyConfig (MADDPG, MATD3, IQL, VDN, QMIX)</i>		
<code>activation</code>	"relu"	overrides the default above
<code>buffer_size</code>	200 000	replay capacity in transitions (one per copy and step)
<code>batch_size</code>	256	transitions per gradient step
<code>warmup_steps</code>	2 000	environment steps (summed over copies) with uniformly random actions before learning starts
<code>update_every</code>	1	vector steps between rounds of gradient steps
<code>gradient_steps</code>	1	gradient steps per round
<code>tau</code>	0.01	Polyak coefficient τ of the target networks
<code>reward_scale</code>	1.0	constant factor κ applied to rewards before learning

Seeding without global side effects. The seed initialises a `numpy.random.SeedSequence`, from which the algorithm derives its own NumPy generator `self.np_rng` (random warm-up actions, replay sampling, ϵ -greedy draws) and a seed for its own `torch.Generator` `self.torch_rng` (policy samples, exploration and target-smoothing noise, minibatch permutations). The networks are built inside `torch.random.fork_rng`, seeded from the same sequence, so network initiali-

sation is reproducible while PyTorch's global random state is restored afterwards. Apart from this temporary, forked seeding, no code of the package seeds or draws from the global NumPy or PyTorch generators. `Algorithm.learn` runs with the current number of PyTorch threads; `train`, `train_preset`, `compare` and `marl-train` use one thread by default (`threads=1`), which is several times faster for these small networks, and restore the previous setting when they return. Wrap your own calls of `learn` in `with torch_threads(1):` for the same effect. With the same seed and the same machine, training is identical (the tests check this for every algorithm); `seed=None` draws fresh entropy.

The on-policy loop. `OnPolicyAlgorithm.learn` resets the copies with the given seed and then repeats until `env_steps` reaches `total_steps`:

1. for `rollout_length` vector steps, call `_rollout_step(obs)` for exploratory actions and extra fields (for PPO the normalised observations and the log-probabilities), step the runner, pass the transition to `_observe_transition` and store it with the extra fields in a `RolloutBuffer`;
2. call `_update(buffer)` and record its statistics and the finished episodes in the log, then call the callback.

The buffer always holds the fields `obs`, `actions`, `reward`, `agent_rewards`, `next_obs`, `terminated` and `truncated`, plus the fields declared by `_rollout_fields()`.

The off-policy loop. `OffPolicyAlgorithm.learn` resets the copies and then, at every vector step, takes uniformly random actions within the bounds while `env_steps < warmup_steps` and `_explore(obs)` afterwards, steps the runner and adds the transition to the replay buffer (created on the first call), and records the episodes finished in this step in the log. Once the warm-up is over and the buffer holds at least `batch_size` transitions, every `update_every` vector steps it performs `gradient_steps` calls of `_update(batch)` on batches sampled from the replay buffer, records the statistics of the last one and calls the callback. With B copies the loop therefore performs `gradient_steps / (update_every · B)` gradient steps per environment step; the presets use one gradient step per vector step of 16 copies.

Public methods. Every algorithm provides the following methods and the counters `env_steps` and `num_updates`:

```
algo.learn(envs, total_steps, *, seed=None, log=None,
          callback=None) -> TrainingLog
algo.act(obs, deterministic=False) # (*batch, n, obs_dim) -> agent actions
algo.policy(deterministic=True) # env observation -> env action
algo.evaluate(env, n_episodes=10, *, seed=None, deterministic=True)
algo.state_dict(); algo.load_state_dict(state)
algo.save(path) -> Path
Algorithm.load(path, device="cpu") -> Algorithm # class method
```

The method `evaluate` runs `env_lib.evaluate` with the policy returned by `policy` and returns its `EvaluationResult`.

The training log. `TrainingLog` records every finished training episode as `(env_steps, team_return, length)` in `episodes` and the statistics of every update as `(env_steps, statistics)` in `updates`; the returns are those of the environment's team reward, before any reward scaling. `episode_steps` and `episode_returns` return them as arrays,

`mean_return(last=100)` averages the last episodes (NaN before the first), `curve(points=50)` bins the returns into equal intervals of environment steps for a learning curve, `summary()` gives a one-line summary, `to_csv(path)` writes the episodes as CSV (`env_steps`, `return`, `length`), and `wall_time` is the time from the creation of the log to its last record. Section 14.4 shows how to plot learning curves of several seeds with confidence bands.

12.5 PPO: IPPO and MAPPO

IPPO and MAPPO train decentralised stochastic actors $\pi_\theta(a_i | x_i)$ with the clipped surrogate objective of PPO (Schulman et al., 2017) and generalised advantage estimation (Schulman et al., 2016), and differ only in the critic:

- **IPPO** gives every agent a decentralised critic $V_\phi(x_i)$ on its own input $x_i = [\hat{o}_i, e_i]$, trained on the agent's own reward r_t^i (`reward_source="agent"`, the default of IPPO). Every agent is an independent PPO learner that treats the others as part of the environment.
- **MAPPO** gives every agent a centralised critic $V_\phi(s, i)$ on the global state $s = [\hat{o}_1, \dots, \hat{o}_n]$, the concatenation of all normalised observations, trained on the team reward r_t (`reward_source="team"`, the default of MAPPO). With shared parameters one network takes $[s, e_i]$, which amounts to one value head per agent over the shared state; without sharing agent i has its own network on s . The critic is used only in training, so execution stays decentralised.

`reward_source` selects the reward of the critics for both methods: `"team"` gives every critic the team reward, `"agent"` gives critic i agent i 's own reward. The presets of MAPPO on `Platoon-v0` and `Consensus-v0` use `"agent"` (Section 12.8).

Acting. In a rollout the observation normaliser first includes the new observations and then normalises them; the normalised observations are stored, so the update evaluates the policy on exactly the input that produced each action. Continuous actions are sampled unbounded and stored unclipped (the environment adapter clips them); the stored log-probability is that of the unclipped sample. The deterministic policy returns the mean, clipped to the bounds, or the most likely discrete action.

Advantages and value targets. After a rollout of T vector steps on B copies the critic values V_t^i of the stored inputs and $V_t'^i$ of the successor observations (normalised with the current statistics) give, for every agent and with the rewards r_t^i of the chosen source scaled by the `RewardScaler`, the GAE advantages A_t^i and targets $R_t^i = A_t^i + V_t'^i$ of (12.1) and (12.2). With `normalize_advantages` the advantages are standardised over the whole rollout (all steps, copies and agents), $\hat{A} = (A - \bar{A})/(\text{std } A + 10^{-8})$.

Losses. With the probability ratio $\rho_t^i = \pi_\theta(a_t^i | x_t^i)/\pi_{\theta_{\text{old}}}(a_t^i | x_t^i)$ and the means taken over the samples of a minibatch and all agents, the actor minimises

$$L^\pi(\theta) = -\text{mean}\left[\min(\rho_t^i \hat{A}_t^i, \text{clip}(\rho_t^i, 1 - \epsilon, 1 + \epsilon) \hat{A}_t^i)\right] - c_H \text{mean}[\mathcal{H}(\pi_\theta(\cdot | x_t^i))] \quad (12.4)$$

with $\epsilon = \text{clip_range}$ and $c_H = \text{ent_coef}$, and the critic minimises

$$L^V(\phi) = \text{mean}\left[\max\left(\ell(V_\phi - R), \ell(V_{\text{old}} + \text{clip}(V_\phi - V_{\text{old}}, -\epsilon_V, \epsilon_V) - R)\right)\right], \quad (12.5)$$

where V_ϕ is $V_\phi(x_t^i)$ for IPPO and $V_\phi(s_t, i)$ for MAPPO, V_{old} the value before the update, $\epsilon_V = \text{value_clip_range}$ (`None` removes the clipped term) and $\ell(e) = e^2/2$, or with `huber_delta =  $\eta$` the Huber loss $\ell(e) = e^2/2$ for $|e| \leq \eta$ and $\eta(|e| - \eta/2)$ otherwise.

Optimisation. A sample is one copy at one step with all its agents, because the centralised critic needs every agent’s observation; a rollout therefore holds TB samples. Each of the `n_epochs` epochs shuffles them with the algorithm’s generator and splits them into `num_minibatches` minibatches; on each minibatch the actor takes one Adam step on (12.4) and then the critic one Adam step on (12.5), with separate optimisers ($\epsilon_{\text{Adam}} = 10^{-5}$, learning rates `lr` and `critic_lr`) and separate gradient-norm clipping at `max_grad_norm`. With `target_kl` the epochs stop early once the mean approximate KL divergence of an epoch, $\text{mean}[(\rho - 1) - \log \rho]$, exceeds 1.5 times `target_kl`. With `anneal_lr` both learning rates decay linearly to zero over the steps of the current `learn` call. Every update records `policy_loss`, `value_loss`, `entropy`, `approx_kl`, `clip_fraction` (the fraction of $|\rho - 1| > \epsilon$), averaged over the minibatches, and `explained_variance`, `epochs`, `learning_rate`, `value_mean`, `action_std` (continuous actions) and `reward_std` (with reward scaling).

Stated simplifications. The implementation follows the PPO recipe of the references (including the practices studied by Andrychowicz et al., 2021) with four standard simplifications, stated in the module docstring: feed-forward instead of recurrent networks (the `env_lib` observations carry the recent history that matters, such as previous actions and rates of change); a state-independent Gaussian standard deviation with unbounded samples clipped by the environment adapter; reward scaling by the running standard deviation of the return instead of MAPPO’s value normalisation (PopArt or ValueNorm), both of which keep value targets of order one; and the concatenation of the local observations as MAPPO’s global state, not an environment-specific state.

Table 12.6: Fields of `PPOConfig`, in addition to those of `OnPolicyConfig` and `AlgorithmConfig` (Table 12.5).

Field	Default	Meaning
<code>n_epochs</code>	10	passes over each rollout
<code>num_minibatches</code>	4	minibatches per epoch (a sample is one copy at one step with all its agents)
<code>clip_range</code>	0.2	ratio clipping ϵ
<code>value_clip_range</code>	0.2	value clipping ϵ_V around the pre-update value; <code>None</code> disables it
<code>huber_delta</code>	<code>None</code>	Huber threshold η of the critic loss; <code>None</code> uses $e^2/2$
<code>ent_coef</code>	0.0	entropy weight c_H
<code>normalize_advantages</code>	<code>True</code>	standardise the advantages over the rollout
<code>critic_lr</code>	<code>None</code>	critic learning rate; <code>None</code> uses <code>lr</code>
<code>critic_hidden_sizes</code>	<code>None</code>	hidden widths of the critic; <code>None</code> uses <code>hidden_sizes</code>
<code>anneal_lr</code>	<code>False</code>	decay both learning rates linearly to zero
<code>target_kl</code>	<code>None</code>	stop the epochs once the mean approximate KL of an epoch exceeds $1.5 \times$ this value
<code>log_std_init</code>	-0.5	initial log standard deviation of the Gaussian policy, in units of half the action range
<code>reward_source</code>	<code>None</code>	"team" or "agent"; <code>None</code> selects "agent" for IPPO and "team" for MAPPO

12.6 MADDPG and MATD3

MADDPG (Lowe et al., 2017) extends DDPG (Lillicrap et al., 2016) to several agents: decentralised deterministic actors $\mu_i(o_i)$ are trained with the deterministic policy gradient (Silver et al., 2014) of centralised critics $Q_i(s, a_1, \dots, a_n)$ that see the global state and the joint action. MATD3 (Ackermann et al., 2019) carries the three corrections of TD3 (Fujimoto et al., 2018) over to the centralised critics. Both learn from a replay buffer and support continuous actions only.

Normalised action units. The actors output actions in $[-1, 1]$ through a tanh; the per-agent bounds of the environment map them to physical units, $a_i = c_i + h_i \odot \tilde{a}_i$, and actions from the replay buffer are mapped back, $\tilde{a}_i = (a_i - c_i) \oslash h_i$. Critics, exploration noise and target smoothing all work in these normalised units, so one set of hyperparameters suits environments with very different action ranges.

Critic inputs. With shared parameters (the default) one actor and one critic serve all agents, with the one-hot identifier e_i appended to the actor input $x_i = [\hat{o}_i, e_i]$ and to the critic input; the shared critic evaluated at agent i 's identifier is Q_i . With `share_parameters=False` every agent has its own actor and critic. By default (`critic_local_inputs=True`) the input of Q_i also repeats agent i 's own observation and action,

$$z_i(s, \tilde{a}) = [s, \tilde{a}_1, \dots, \tilde{a}_n, \hat{o}_i, \tilde{a}_i, e_i], \quad Q_i(s, \tilde{a}) = Q_\phi(z_i(s, \tilde{a})). \quad (12.6)$$

This is a documented deviation from the original critic input $[s, a_1, \dots, a_n]$. The input is still a function of (s, a) and i only (the “agent-specific global state” of Yu et al., 2022), but a shared critic no longer has to learn from the identifier which of the n blocks of s and a belong to agent i . According to the implementation notes, MADDPG does not learn on `Consensus-v0` (8 agents) without these inputs and approaches the Laplacian baseline with them. `critic_local_inputs=False` restores the original input.

Critic update. On a replay batch of N transitions (o, a, r, o', d) , with d the `terminated` flag (truncated episodes bootstrap from their stored final observations) and r_i agent i 's own reward (`reward_source="agent"`, the default, as in MADDPG) or the team reward (`"team"`), the target actions and the TD targets are

$$\begin{aligned} \tilde{a}'_j &= \mu_{\theta'}(x'_j) & (\text{MADDPG}), \\ \tilde{a}'_j &= \text{clip}(\mu_{\theta'}(x'_j) + \text{clip}(\varepsilon_j, -c, c), -1, 1), \quad \varepsilon_j \sim \mathcal{N}(0, \sigma^2 I) & (\text{MATD3}), \\ y_i &= \kappa r_i + \gamma (1 - d) \min_k Q'_{i,k}(s', \tilde{a}'), \end{aligned} \quad (12.7)$$

with the target actor $\mu_{\theta'}$, the target critics $Q'_{i,k}$ ($k = 1$ for MADDPG, $k \in \{1, 2\}$ for MATD3), $\kappa = \text{reward_scale}$, $\sigma = \text{target_noise}$ and $c = \text{target_noise_clip}$. The critics minimise

$$L^Q(\phi) = \sum_k \frac{1}{Nn} \sum_{b,i} \ell(Q_{i,k}(s_b, \tilde{a}_b) - y_{b,i}), \quad (12.8)$$

with $\ell(e) = e^2$ (`critic_loss="mse"`, as in the papers) or, with `critic_loss="huber"`, $\ell(e) = e^2$ for $|e| \leq 1$ and $2|e| - 1$ beyond, which bounds the gradient of rare, very large errors such as the trip penalty of `PowerGrid-v0`. The gradient norm is clipped at `max_grad_norm`.

Actor update. Actor i follows the deterministic policy gradient of its (first) critic, with its own action replaced by $\mu_\theta(x_i)$ and the other agents' actions taken from the replay batch:

$$L^\mu(\theta) = -\frac{1}{Nn} \sum_{b,i} Q_{i,1}(s_b, (\tilde{a}_{b,1}, \dots, \mu_\theta(x_{b,i}), \dots, \tilde{a}_{b,n})). \quad (12.9)$$

The replacement applies to the joint action and, with local inputs, to the own-action slot of (12.6). All agents are updated in one batched pass (row i of the batched critic input carries agent i 's replacement); the critic parameters are frozen during this step, so only the actor changes. After every actor update the target networks follow by Polyak averaging, $\theta' \leftarrow (1 - \tau)\theta' + \tau\theta$ and $\phi' \leftarrow (1 - \tau)\phi' + \tau\phi$. MADDPG updates the actor after every critic update; MATD3 updates the actor and the targets only every `policy_delay` critic updates, which together with the twin critics and the target smoothing of (12.7) are the three TD3 corrections.

Exploration. During training, $\tilde{a}_i = \text{clip}(\mu_\theta(x_i) + \sigma_t \xi_i, -1, 1)$ with $\xi_i \sim \mathcal{N}(0, I)$, drawn from the algorithm's generator. The noise scale σ_t is `exploration_noise` or, when `final_exploration_noise` is set, decays linearly with the environment steps t (summed over copies),

$$\sigma_t = \sigma_0 + \min(1, t/T_\sigma) (\sigma_1 - \sigma_0), \quad (12.10)$$

where σ_0 is `exploration_noise`, σ_1 is `final_exploration_noise` and T_σ is the length `noise_decay_steps` of the decay; the property `exploration_scale` returns the current value. The warm-up actions are uniform in the bounds.

Observation normalisation. Optional (`normalize_observations`): the statistics are computed once, from all observations in the replay buffer at the first gradient step (the warm-up data), and frozen afterwards, so that stored transitions and learned values stay consistent. Every update records `critic_loss`, `actor_loss` (of the last actor update), `q_mean`, `target_mean` and `noise_scale`.

Table 12.7: Fields of `DDPGConfig`, in addition to those of `OffPolicyConfig` and `AlgorithmConfig` (Table 12.5). Noise scales are in units of half the action range.

Field	Default	Meaning
<code>actor_lr, critic_lr</code>	10^{-3}	Adam learning rates
<code>critic_hidden_sizes</code>	None	hidden widths of the critics; None uses <code>hidden_sizes</code> (which always sets the actors)
<code>exploration_noise</code>	0.1	standard deviation σ_0 of the exploration noise
<code>final_exploration_noise</code>	None	when set, the noise decays linearly to this value σ_1
<code>noise_decay_steps</code>	0	length T_σ of the decay in environment steps
<code>reward_source</code>	"agent"	"agent": critic i learns from agent i 's reward; "team": from the team reward
<code>normalize_observations</code>	False	standardise observations with statistics of the warm-up data, frozen at the first gradient step
<code>critic_local_inputs</code>	True	append agent i 's own observation and action to the input of Q_i (12.6)
<code>critic_loss</code>	"mse"	"mse" or "huber"
<code>policy_delay</code>	2	MATD3 only: critic updates per actor and target update
<code>target_noise</code>	0.2	MATD3 only: standard deviation σ of the target smoothing noise
<code>target_noise_clip</code>	0.5	MATD3 only: clipping bound c of the smoothing noise

12.7 IQL, VDN and QMIX

The value-based methods learn per-agent *utilities* $Q_\theta(x_i) \in \mathbb{R}^{|\mathcal{A}|}$, one value per action of agent i , with the DQN machinery (Mnih et al., 2015): a replay buffer, target networks and ϵ -greedy exploration. Every agent acts greedily on its own utility, $a_i = \arg \max_a Q_\theta(x_i)[a]$, from its own observation. The three methods differ only in how the utilities are trained. With shared parameters (default) one utility network serves all agents, with the one-hot identifier appended to the observation as in the QMIX paper; `share_parameters=False` gives one network per agent. The observations enter the utility network unnormalised.

On a replay batch (o, a, r, o', d) the chosen utilities are $q_i = Q_\theta(x_i)[a_i]$. The greedy next actions are selected by the online network and evaluated by the target network θ^- (double Q-learning, van Hasselt et al., 2016, `double_q=True`),

$$\hat{a}'_i = \arg \max_a Q_{\theta^-}(x'_i)[a], \quad q'_i = Q_{\theta^-}(x'_i)[\hat{a}'_i], \quad (12.11)$$

or both by the target network with `double_q=False`. With $\kappa = \text{reward_scale}$, d the `terminated` flag and ℓ the Huber loss (smooth L1 with threshold 1, $\ell(e) = e^2/2$ for $|e| < 1$ and $|e| - 1/2$ otherwise; `loss="huber"`) or the squared error $\ell(e) = e^2$ (`loss="mse"`):

- **IQL** (Tan, 1993, here with DQN agents): every agent regresses its utility on its own reward r_i with its own target; there is no mixing and no centralised information,

$$y_i = \kappa r_i + \gamma(1 - d) q'_i, \quad L = \frac{1}{Nn} \sum_{b,i} \ell(q_{b,i} - y_{b,i}). \quad (12.12)$$

- **VDN** (Sunehag et al., 2018): the team value is the sum of the utilities, trained end to end on the team reward r with one target,

$$Q_{\text{tot}} = \sum_i q_i, \quad y = \kappa r + \gamma(1 - d) \sum_i q'_i, \quad L = \frac{1}{N} \sum_b \ell(Q_{\text{tot},b} - y_b). \quad (12.13)$$

- **QMIX** (Rashid et al., 2018): the team value is a monotonic function of the utilities, computed by a mixing network whose weights are generated from the global state s by hypernetworks,

$$\begin{aligned} Q_{\text{tot}}(q; s) &= w_2(s)^\top \text{elu}(W_1(s)^\top q + b_1(s)) + b_2(s), \\ W_1(s) &= |h_{W_1}(s)| \in \mathbb{R}^{n \times m}, \quad w_2(s) = |h_{w_2}(s)| \in \mathbb{R}^m, \\ y &= \kappa r + \gamma(1 - d) Q_{\text{tot}}^-(q'; s'), \quad L = \frac{1}{N} \sum_b \ell(Q_{\text{tot}}(q_b; s_b) - y_b), \end{aligned} \quad (12.14)$$

with $m = \text{mixer_embed_dim}$. The hypernetworks h_{W_1} and h_{w_2} are MLPs with one hidden layer of `hypernet_hidden` ReLU units, b_1 is a linear function of s and b_2 an MLP with one hidden layer of m ReLU units; the absolute values make the weights non-negative, and since `elu` is increasing, $\partial Q_{\text{tot}} / \partial q_i \geq 0$. The target mixer Q_{tot}^- is a Polyak-averaged or periodically copied twin of the mixer.

Because the VDN and QMIX mixing is monotonic in every utility, the greedy joint action $\arg \max_a Q_{\text{tot}}$ is obtained by every agent maximising its own utility (the individual-global-max property). The centralised target is therefore computed from per-agent maxima, the mixers are used only in training, and execution stays decentralised. The global state of QMIX is the concatenation of all agents' observations (Section 12.4.1).

Optimisation and exploration. One Adam optimiser (learning rate `lr`) updates the utility network and the mixer, with the gradient norm of all their parameters clipped at `max_grad_norm`. After every gradient step the target networks follow by Polyak averaging with `tau` (`target_update_interval=0`, the default), or the online weights are copied into them every K gradient steps (`target_update_interval = K > 0`, the original DQN scheme). While training, every agent of every copy independently takes a uniformly random action with probability

$$\epsilon(t) = \epsilon_{\text{start}} + \min(1, t/T_\epsilon) (\epsilon_{\text{end}} - \epsilon_{\text{start}}), \quad (12.15)$$

where t counts environment steps (summed over copies) from the start of training, including the warm-up, and $T_\epsilon = \text{epsilon_decay_steps}$ (0 uses ϵ_{end} from the start); `epsilon_at(t)` and the property `epsilon` return the value. Every update records `td_loss`, `q_mean` (the trained value, Q_{tot} or q_i , in units of the scaled reward), `target_mean`, `grad_norm` (before clipping) and `epsilon`. The methods `q_values(obs, *, target=False)`, `mix(agent_q, obs, *, target=False)`, `td_target(batch)` and `td_loss(batch)` expose the pieces of the update.

Stated simplifications. The VDN and QMIX papers use recurrent (LSTM or GRU) agent networks trained on whole episodes sampled from an episode replay, to cope with partial observability. Here the agent networks are feed-forward and trained on single transitions from a transition replay: the observations of the discrete environments (the local message and queue windows of `LineMsg` and `WirelessComm`, the piston and ball features of `Pistonball`) are Markov enough for the demonstrations, and the feed-forward version is much cheaper on a CPU. Further standard choices: one-step TD targets, double Q-learning by default, Adam instead of RMSprop, and Polyak-averaged target networks by default. IQL with environments that report no per-agent rewards gives every agent the team reward, and with a single agent it is (double) DQN.

Table 12.8: Fields of `QLearningConfig` (IQL, VDN, QMIX), in addition to those of `OffPolicyConfig` and `AlgorithmConfig` (Table 12.5).

Field	Default	Meaning
<code>lr</code>	$5 \cdot 10^{-4}$	Adam learning rate of the utility network and the mixer
<code>epsilon_start</code> , <code>epsilon_end</code>	1.0, 0.05	exploration probability at the start and after the decay
<code>epsilon_decay_steps</code>	50 000	environment steps T_ϵ of the linear decay
<code>double_q</code>	True	select the next action with the online network, evaluate it with the target network
<code>target_update_interval</code>	0	0: Polyak averaging with <code>tau</code> after every gradient step; $K > 0$: copy the online weights every K gradient steps
<code>loss</code>	"huber"	TD loss, "huber" (threshold 1) or "mse"
<code>mixer_embed_dim</code>	32	QMIX: width m of the mixing layer
<code>hypernet_hidden</code>	64	QMIX: hidden width of the weight hypernetworks

12.8 Presets and results

The package `marl_algorithms.presets` holds a table `PRESETS[algorithm][env_id]` of tuned settings, collected from one module per family (`presets.ppo`, `presets.ddpg` and `presets.q_learning`, each with the notes from tuning). A preset fixes the environment steps ("`total_steps`"), the number of copies ("`num_envs`"), configuration overrides ("`config`") and environment arguments ("`env_kwargs`", empty for all current presets). The presets were tuned to train in at most about two minutes on one CPU core and to show clear learning. Table 12.9 lists all 17 of them, as printed by `marl-train presets`, with their configuration overrides.

Table 12.9: Presets (`list_presets()`); “Steps” is the preset’s `total_steps`, “Copies” its `num_envs`.

Algorithm	Environment	Steps	Copies	Configuration overrides
IPPO	PowerGrid-v0	200 000	32	<code>rollout_length=64, n_epochs=5, lr=1e-3, gamma=0.95, log_std_init=-1.5</code>
IPPO	Platoon-v0	450 000	32	<code>rollout_length=64, n_epochs=5, lr=3e-4, gamma=0.99, log_std_init=-1.0, normalize_observations=False, value_clip_range=None</code>
IPPO	Consensus-v0	400 000	32	<code>rollout_length=64, n_epochs=5, lr=1e-3, gamma=0.95, normalize_observations=False</code>
IPPO	LineMsg-v0	100 000	32	<code>rollout_length=50, n_epochs=5, lr=1e-3, ent_coef=0.01</code>
MAPPO	PowerGrid-v0	250 000	32	as IPPO on PowerGrid-v0
MAPPO	Platoon-v0	450 000	32	as IPPO on Platoon-v0, and <code>reward_source="agent"</code>
MAPPO	Consensus-v0	400 000	32	<code>rollout_length=64, n_epochs=5, lr=1e-3, gamma=0.9, reward_source="agent", normalize_observations=False</code>
MAPPO	LineMsg-v0	100 000	32	as IPPO on LineMsg-v0
MADDPG	PowerGrid-v0	64 000	16	<code>critic_hidden_sizes=(128, 128), batch_size=64, warmup_steps=3200, reward_source="team", critic_loss="huber"</code>
MADDPG	Consensus-v0	96 000	16	<code>critic_hidden_sizes=(128, 128), batch_size=64, warmup_steps=3200, gamma=0.95, reward_scale=0.01, normalize_observations=True, exploration_noise=0.2, final_exploration_noise=0.02, noise_decay_steps=96000</code>
MATD3	PowerGrid-v0	64 000	16	as MADDPG, and <code>exploration_noise=0.3, final_exploration_noise=0.05, noise_decay_steps=64000</code>
MATD3	Consensus-v0	96 000	16	as MADDPG on Consensus-v0
IQL, VDN, QMIX	LineMsg-v0	25 000	16	<code>batch_size=128, lr=1e-3, epsilon_decay_steps=12500</code>
VDN	WirelessComm-v1	120 000	16	<code>batch_size=128, lr=1e-3, gamma=0.8, epsilon_decay_steps=30000</code>
QMIX	WirelessComm-v1	100 000	16	<code>batch_size=128, lr=1e-3, gamma=0.8, epsilon_decay_steps=45000</code>

The source code documents the reasons for these settings. On **PowerGrid-v0** the PPO policies need little initial exploration noise (`log_std_init=-1.5`): with more noise the mean policy learns to compensate its own previous actions, which it observes, and performs poorly when executed without noise. On **Platoon-v0** and **Consensus-v0** the critics learn each agent’s own reward, which isolates the effect of an agent’s own command, and the observations are used unnormalised, because they are already of order one (Platoon) or in arena units (Consensus) and running statistics dominated by the large errors of early training would hide the small errors that matter once the task is nearly solved. MADDPG and MATD3 use the team reward on **PowerGrid-v0**, where each bus sees only about 1/16 of the benefit of its frequency support but pays its full control cost under the per-agent rewards, and the Huber loss keeps the rare trip transitions from dominating the critic updates. On **WirelessComm-v1** packets are delivered in the step they are sent, so a short horizon ($\gamma = 0.8$) suffices. IQL has no **WirelessComm-v1** preset, for the reason discussed below.

Results. Table 12.10 reports the results of `benchmarks/benchmark_marl.py`, which applies one protocol to every preset: train with `train_preset(algorithm, env_id, seed=0)` on one CPU core (`OMP_NUM_THREADS=1`, one PyTorch thread), then evaluate uniformly random actions, the trained deterministic policy and the baseline controller of the environment with `env_lib.evaluate` on the same 64 episodes (seed 1) of a 64-copy vector environment created with `make_vector_env`. The script appends one JSON record per preset to the file given by `--out` (default `renders/marl_benchmark.jsonl`); `--only ALGO:ENV ...` selects presets, `--shard K/N` runs every N -th preset starting at K , so that shards can run in parallel processes (for example pinned to different cores with `taskset`), and `--report FILE` prints the table of a result file, as Markdown with `--markdown`.

```
python benchmarks/benchmark_marl.py --out marl.jsonl           # all presets
python benchmarks/benchmark_marl.py --only mappo:PowerGrid-v0 qmix:LineMsg-v0
taskset -c 0 python benchmarks/benchmark_marl.py --shard 0/2 --out marl.jsonl &
taskset -c 1 python benchmarks/benchmark_marl.py --shard 1/2 --out marl.jsonl &
python benchmarks/benchmark_marl.py --report marl.jsonl --markdown
```

Discussion. Every preset learns: each trained policy is far better than random actions, and several match their baseline.

- *LineMsg.* All five discrete-action methods learn the optimal “always relay” policy (return 95.0, the same as the baseline); the value-based methods need a quarter of the environment steps of the PPO methods and 9 to 16 seconds.
- *Consensus.* IPPO and MAPPO come within about 2 and 5 % of the Laplacian protocol, MADDPG and MATD3, with a quarter of the environment steps, within about 11 and 14 %. The Laplacian protocol is a strong reference here: it is the classical solution of the consensus problem on a connected graph.
- *PowerGrid.* MAPPO reaches -0.89 against -0.79 for droop control, IPPO -2.37 . MADDPG and MATD3 improve the return from -629 to about -5.7 but remain far from droop control. Fitting the actions of the trained deterministic policies to the local frequency deviation ω_i (measured for this chapter over 200 steps on 16 copies) gives slopes of -0.09 (MADDPG) and -0.14 (MATD3), against the droop gain of -0.25 , with correlations of only -0.15 and -0.12 : the actors have learned a partial droop feedback, weaker than droop control, combined with a sizeable part of the action that does not follow the local frequency deviation and costs control effort.

Table 12.10: Results of `benchmarks/benchmark_marl.py`: mean team return of 64 evaluation episodes (seed 1, 64-copy vector environment) after training with the preset and seed 0; higher is better. “CPU” is the processor time of training on one core. “Env steps” are the steps actually collected, a multiple of the steps per rollout (on-policy) or of the number of copies (off-policy). The baselines are droop control (PowerGrid), cooperative adaptive cruise control (Platoon), the Laplacian protocol (Consensus), always relay (LineMsg) and the collision-free schedule (WirelessComm).

Environment	Algorithm	Env steps	CPU [s]	Random	Trained	Baseline
PowerGrid-v0	IPPO	200 704	69	−629.3	−2.37	−0.79
	MAPPO	251 904	106	−629.3	−0.89	−0.79
	MADDPG	64 000	70	−629.3	−5.69	−0.79
	MATD3	64 000	87	−629.3	−5.63	−0.79
Platoon-v0	IPPO	450 560	88	−5350	−84.8	−46.4
	MAPPO	450 560	102	−5350	−65.0	−46.4
Consensus-v0	IPPO	401 408	83	−18041	−1550	−1518
	MAPPO	401 408	96	−18041	−1586	−1518
	MADDPG	96 000	72	−18041	−1681	−1518
	MATD3	96 000	83	−18041	−1737	−1518
LineMsg-v0	IPPO	100 800	26	43.2	95.0	95.0
	MAPPO	100 800	26	43.2	95.0	95.0
	IQL	25 008	9	43.2	95.0	95.0
	VDN	25 008	10	43.2	95.0	95.0
	QMIX	25 008	16	43.2	95.0	95.0
WirelessComm-v1	VDN	120 000	74	139.2	338.0	339.6
	QMIX	100 000	101	139.2	319.1	339.6

- *Platoon*. IPPO and MAPPO reduce the cost from about 5 350 to 85 and 65, which is still 83 and 40 % above the cost of cooperative adaptive cruise control. According to the tuning notes in the source, MAPPO with the team reward and normalised observations does not learn this task (every episode ends in a collision); the per-agent rewards and unnormalised observations of the preset are essential.
- *WirelessComm*. VDN comes within 0.5 % of the collision-free schedule. IQL has no preset here: trained with the VDN settings (120 000 steps), it reached only 152.3 on the same evaluation episodes, hardly better than random actions. This is a structural failure rather than a matter of tuning. The per-agent reward is 1 for a delivered packet and 0 otherwise, so a collision costs the transmitting agent nothing compared with staying idle; for an independent learner transmitting is never worse than waiting, the harm it does to the other agent's delivery is invisible in its own reward, and the agents keep transmitting and colliding. VDN and QMIX learn from the team reward, in which a collision costs a delivery, and learn to leave a contested access point to one agent. The QMIX result depends on the seed. With seed 0 it converges to a fixed assignment in which each of the nine access points is used by exactly one agent, the one to its upper right, and the seven agents of the first column and the last row never deliver a packet (6 collisions in the 64 evaluation episodes); the baseline additionally lets the border agents borrow idle access points. With the same preset and seeds 1 and 2, QMIX reached 344.1 and 332.7 on the same evaluation episodes (measured for this chapter), with 14 and 12 agents delivering packets.

The off-policy methods need three to four times fewer environment steps than the PPO methods on the continuous tasks, but spend more computation per step, so the training times are similar. The results come from single training runs (seed 0); a comparison of methods for a publication needs several seeds per method and confidence intervals, for example with `plot_learning_curves` of `plotkit` (Section 14.4).

12.9 The demonstration script

`examples/algorithms/marl_training_demo.py` trains one algorithm of every family with its preset, each on a different environment:

- MAPPO on `PowerGrid-v0`: continuous actions, 16 agents, on-policy;
- MADDPG on `Consensus-v0`: continuous actions, 8 agents, off-policy;
- QMIX on `LineMsg-v0`: discrete actions, 10 agents, value-based;
- VDN on `WirelessComm-v1`: discrete actions, 16 agents, value-based.

Each run trains on batched copies created with `make_vector_env` (native batches for `PowerGrid` and `Consensus`), then evaluates random actions, the trained policy and the baseline controller on the same seeded episodes (a vector environment of `min(episodes, 64)` copies, seed `seed + 1`). The script prints one line per run and a results table, and saves the learning curves (training returns over environment steps, with the random and baseline evaluation returns as reference lines) as a PNG file. The full suite takes a few minutes on one CPU core.

```
python examples/algorithms/marl_training_demo.py           # the suite, a few
minutes
python examples/algorithms/marl_training_demo.py --algo mappo --env PowerGrid-v0
python examples/algorithms/marl_training_demo.py --algo qmix --env LineMsg-v0 \
--gif renders/qmix.gif
python examples/algorithms/marl_training_demo.py --quick --plot '' # smoke test
```

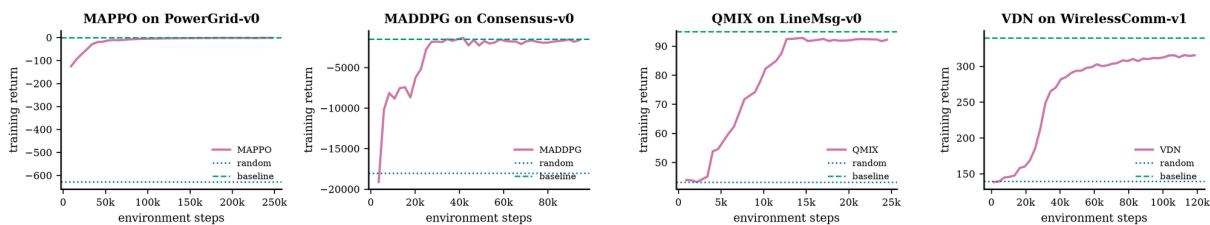
Table 12.11: Options of `examples/algorithms/marl_training_demo.py`.

Option	Default	Meaning
<code>--algo, --env</code>	the suite	train only this preset (both are required together)
<code>--quick</code>	off	smoke test: 2000 steps (PPO) or 1500 steps with a 500-step warm-up (off-policy) on 8 copies
<code>--seed</code>	0	training seed; the evaluation uses <code>seed + 1</code>
<code>--episodes</code>	64	evaluation episodes per policy
<code>--plot</code>	<code>renders/marl_training.png</code>	learning-curve figure; an empty string skips it
<code>--gif</code>	none	record the trained policy of a single run (<code>--algo/--env</code>)
<code>--threads</code>	1	PyTorch threads

Table 12.12 shows the results of the full suite with its default arguments and Figure 12.2 the learning curves it saves. The evaluation protocol is that of Table 12.10, and the returns agree with it to the digits shown there.

Table 12.12: Output of `examples/algorithms/marl_training_demo.py` with default arguments: training seed 0 on one CPU core (“Time” is the wall-clock training time), 64 evaluation episodes with seed 1 on a 64-copy vector environment.

Algorithm	Environment	Env steps	Time [s]	Random	Trained	Baseline
MAPPO	PowerGrid-v0	251 904	106.6	−629.3	−0.894	−0.786
MADDPG	Consensus-v0	96 000	76.1	−18041	−1681	−1518
QMIX	LineMsg-v0	25 008	16.6	43.2	95.0	95.0
VDN	WirelessComm-v1	120 000	78.8	139.2	338.0	339.6

**Figure 12.2:** Learning curves saved by `examples/algorithms/marl_training_demo.py` (default suite, seed 0): mean return of the exploratory training episodes over environment steps, with the evaluation returns of random actions (dotted) and of the baseline controller (dashed) for reference. The training returns include the random warm-up and the exploration (policy noise or ϵ -greedy actions) and are averaged over the few episodes that finish in each interval, so they are not directly comparable with the evaluation returns of the deterministic policy in Table 12.12.

12.10 Adding an algorithm

A new algorithm reuses the core and implements only what is specific to it. Inside the package it gets its own module in the folder of its family (or a new family folder with a `common.py` when it shares code with a sibling):

1. Define a configuration dataclass that derives from `OnPolicyConfig` or `OffPolicyConfig` (validate new fields in `__post_init__`, after calling the parent’s).

2. Subclass `OnPolicyAlgorithm` or `OffPolicyAlgorithm` and set the class attributes `name`, `action_kinds` and `config_class`.
3. Implement the hooks: `_build()` (networks and optimisers; it runs under the seeded, forked PyTorch generator), `act(obs, deterministic=False)`, `_modules()` (what save stores) and, for an on-policy method, `_rollout_step(obs)` and `_update(rollout)`, optionally `_rollout_fields()` and `_observe_transition(transition)`; for an off-policy method, `_explore(obs)` and `_update(batch)`. Extra state such as normalisers goes into `_extra_state()` and `_load_extra_state(state)`. Use `self.np_rng` and `self.torch_rng` for all randomness, `self.agent_inputs` for per-agent network inputs and `PerAgent` for shared or separate networks, and batch over copies and agents.
4. Register it: add an `AlgorithmInfo` record (name, "module:Class" entry point, family, action kinds, summary and reference) to the tuple in `marl_algorithms.registry`, and optionally a lazy export in `marl_algorithms/__init__.py`. `train`, `make_algorithm`, `Algorithm.load` and `marl-train` then find it. An unregistered class can still be trained directly (`MyAlgorithm(envs, seed=0).learn(envs, ...)`) and loaded with `MyAlgorithm.load(path)`.
5. Add presets as a table `{name: {env_id: preset}}` in a module of `marl_algorithms.presets` and merge it into `PRESETS` in `presets/__init__.py`, where `get_preset`, `compare` and `marl-train` presets look them up.
6. Add tests in `tests/algorithms/`: action shapes on several environments, `policy()` with `env_lib.evaluate`, the `TypeError` for an unsupported action kind, a short `learn` with shared and separate parameters, seed determinism, a `save/load` round trip and a loose learning check on a small task.

A minimal skeleton, an independent advantage actor-critic for discrete actions (one gradient step per rollout on the agents' own rewards):

```
from dataclasses import dataclass
from typing import ClassVar

import torch
from torch import nn

from marl_algorithms.core.base import OnPolicyAlgorithm
from marl_algorithms.core.buffers import compute_gae
from marl_algorithms.core.config import OnPolicyConfig
from marl_algorithms.core.networks import CategoricalPolicy, PerAgent, mlp

@dataclass
class A2CConfig(OnPolicyConfig):
    """Options of IA2C (this sketch ignores the normalisation options)."""

    ent_coef: float = 0.01

class IA2C(OnPolicyAlgorithm):
    """Independent advantage actor-critic (discrete actions)."""

    name: ClassVar[str] = "ia2c"
    action_kinds: ClassVar[tuple[str, ...]] = ("discrete",)
    config_class: ClassVar[type[A2CConfig]] = A2CConfig
```

```

def _build(self):
    cfg, spec = self.config, self.spec
    self.actor = PerAgent(
        lambda: CategoricalPolicy(self.agent_input_dim, spec.n_actions,
                                cfg.hidden_sizes, cfg.activation),
        spec.n_agents, cfg.share_parameters).to(self.device)
    self.critic = PerAgent(
        lambda: mlp(self.agent_input_dim, 1, cfg.hidden_sizes,
                    cfg.activation),
        spec.n_agents, cfg.share_parameters).to(self.device)
    self.params = [*self.actor.parameters(), *self.critic.parameters()]
    self.optimizer = torch.optim.Adam(self.params, lr=cfg.lr)

def _modules(self):
    return {"actor": self.actor, "critic": self.critic,
           "optimizer": self.optimizer}

def act(self, obs, deterministic=False):
    inputs = self.agent_inputs(self.tensor(obs))
    with torch.inference_mode():
        if deterministic:
            return self.actor(inputs).argmax(-1).cpu().numpy()
        actions, _ = self.actor.call("sample", inputs, self.torch_rng)
    return actions.cpu().numpy()

def _rollout_step(self, obs):
    return self.act(obs), {} # no extra rollout fields

def _update(self, rollout):
    cfg = self.config
    inputs = self.agent_inputs(self.tensor(rollout["obs"]))
    values = self.critic(inputs).squeeze(-1) # (T, B, n)
    with torch.no_grad():
        next_inputs = self.agent_inputs(self.tensor(rollout["next_obs"]))
        next_values = self.critic(next_inputs).squeeze(-1)
    advantages, returns = compute_gae(
        rollout["agent_rewards"], values.detach().cpu().numpy(),
        next_values.cpu().numpy(), rollout["terminated"],
        rollout["truncated"], cfg.gamma, cfg.gae_lambda)
    actions = self.tensor(rollout["actions"], torch.int64)
    log_prob, entropy = self.actor.call("log_prob", inputs, actions)
    loss = -(log_prob * self.tensor(advantages)).mean()
            - cfg.ent_coef * entropy.mean()
            + 0.5 * (values - self.tensor(returns)).pow(2).mean()
    self.optimizer.zero_grad(set_to_none=True)
    loss.backward()
    nn.utils.clip_grad_norm_(self.params, cfg.max_grad_norm)
    self.optimizer.step()
    return {"loss": loss.item(), "entropy": entropy.mean().item()}

# Registration (in marl_algorithms/registry.py) and a preset (in a module of
# marl_algorithms/presets/, merged into PRESETS):
# AlgorithmInfo("ia2c", "my_package.ia2c:IA2C", "on-policy", ("discrete",),
#               "independent advantage actor-critic", "<reference>")
PRESETS = {"ia2c": {"LineMsg-v0": {"num_envs": 32, "total_steps": 100_000,
                                "config": {"lr": 1e-3, "env_kwargs": {}}}}

```

12.11 References

The source code of `marl_algorithms` cites the following publications.

- J. Ackermann, V. Gabler, T. Osa and M. Sugiyama (2019). *Reducing overestimation bias in multi-agent domains using double centralized critics*. NeurIPS Deep RL Workshop, arXiv:1910.01465.
- M. Andrychowicz et al. (2021). *What matters in on-policy reinforcement learning? A large-scale empirical study*. ICLR.
- C. S. de Witt, T. Gupta, D. Makoviichuk, V. Makoviyshuk, P. H. S. Torr, M. Sun and S. Whiteson (2020). *Is independent learning all you need in the StarCraft multi-agent challenge?* arXiv:2011.09533.
- S. Fujimoto, H. van Hoof and D. Meger (2018). *Addressing function approximation error in actor-critic methods*. ICML.
- T. P. Lillicrap et al. (2016). *Continuous control with deep reinforcement learning*. ICLR.
- R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel and I. Mordatch (2017). *Multi-agent actor-critic for mixed cooperative-competitive environments*. NeurIPS.
- V. Mnih et al. (2015). *Human-level control through deep reinforcement learning*. Nature 518, 529–533.
- T. Rashid et al. (2018). *QMIX: monotonic value function factorisation for deep multi-agent reinforcement learning*. ICML.
- J. Schulman, P. Moritz, S. Levine, M. Jordan and P. Abbeel (2016). *High-dimensional continuous control using generalized advantage estimation*. ICLR.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov (2017). *Proximal policy optimization algorithms*. arXiv:1707.06347.
- D. Silver et al. (2014). *Deterministic policy gradient algorithms*. ICML.
- P. Sunehag et al. (2018). *Value-decomposition networks for cooperative multi-agent learning based on team reward*. AAMAS.
- M. Tan (1993). *Multi-agent reinforcement learning: independent vs. cooperative agents*. ICML.
- H. van Hasselt, A. Guez and D. Silver (2016). *Deep reinforcement learning with double Q-learning*. AAAI.
- C. Yu, A. Velu, E. Vinitzky, J. Gao, Y. Wang, A. Bayen and Y. Wu (2022). *The surprising effectiveness of PPO in cooperative multi-agent games*. NeurIPS Datasets and Benchmarks.

Part III

Toolkit

Chapter 13

Toolkit Overview and Parameter Management

The `toolkit` package collects research utilities that are independent of the environments. This chapter describes its structure and then documents `toolkit.parakit`, the parameter-management module. The plotting and neural-network subpackages have their own chapters.

13.1 Structure of the toolkit

Table 13.1: Subpackages of `toolkit`.

Subpackage	Contents	Dependencies	Chapter
<code>toolkit.plotkit</code>	Learning curves, bars, histograms, heatmaps, style presets, palettes, figure export, gallery CLI	matplotlib (core)	14
<code>toolkit.neural_toolkit</code>	Policy, value and Q-networks, encoders, decoders, factories, <code>NetworkUtils</code> , tabular tools	PyTorch (<code>torch</code> extra)	15
<code>toolkit.parakit</code>	Save, load, validate and apply <code>argparse</code> defaults; Tk editor window	standard library; Tk for the window	13

The subpackages are imported lazily: `import toolkit` only defines `toolkit.__version__`, and the first access to `toolkit.plotkit`, `toolkit.neural_toolkit` or `toolkit.parakit` imports that subpackage. A script that only plots therefore never imports PyTorch, and `parakit`'s headless functions work on systems without Tk. The usual import style names the subpackage explicitly:

```
from toolkit.plotkit import plot_learning_curves, save_figure
from toolkit.parakit import apply_parameters, load_parameters
from toolkit.neural_toolkit import MLPPolicyNetwork, NetworkUtils # needs torch
```

All three subpackages follow the conventions of the environments: they never print, never configure logging (diagnostics go to `logging` loggers named after the module), never touch the global random state, and report problems with exceptions or `warnings` whose messages say how to fix them.

13.2 parakit: managing argparse parameters

Research scripts are usually configured through an `argparse.ArgumentParser`. `parakit` treats the parser as the single source of truth for the parameters, their types and their allowed values, and adds four operations on top of it: reading the current defaults, saving them to a JSON file, loading a file, and installing validated values as the new defaults. Because the values become *defaults*, flags given on the command line still override them. An optional Tk window (Section 13.2.6) lets users edit the values interactively.

13.2.1 Quick start

The typical workflow saves the configuration of a run next to its results and later restores it:

```
import argparse
from toolkit.parakit import (
    apply_parameters, get_parameters, load_parameters, save_parameters,
)

parser = argparse.ArgumentParser()
parser.add_argument("--lr", type=float, default=3e-4, help="learning rate")
parser.add_argument("--batch_size", type=int, default=64)
parser.add_argument("--optimizer", choices=["adam", "sgd"], default="adam")
parser.add_argument("--hidden", type=int, nargs="+", default=[256, 256])
parser.add_argument("--layer_norm", action="store_true")
parser.add_argument("--checkpoint", default=None)

# Save the current defaults into a directory (timestamped file name).
path = save_parameters(get_parameters(parser), "runs/params")

# Later: load the newest file of the directory and install it as defaults.
apply_parameters(parser, load_parameters("runs/params"))

# Values given as text are converted according to the parser.
apply_parameters(parser, {"lr": "1e-3", "hidden": "64 64", "layer_norm": "yes"})
args = parser.parse_args([]) # lr=0.001, hidden=[64, 64], layer_norm=True
args = parser.parse_args(["--lr", "0.01"]) # command-line flags still win
```

13.2.2 Headless API

Table 13.2 lists the functions; none of them needs Tk.

Tunable options. `tunable_actions` skips help, version and sub-parser actions and options whose destination or default is `argparse.SUPPRESS`. When several options share a destination, such as a `--flag/--no-flag` pair, the first one is used. Positional arguments are included and keyed by their name.

Errors. `apply_parameters` either applies all values or none. With `strict=True` (the default) a key that is not a parser destination raises a `ValueError` that lists the unknown keys and the valid names; with `strict=False` unknown keys are ignored. Otherwise every value is converted, and all conversion and validation failures are reported together in one `ValueError`, one line per parameter. The parser is left unchanged whenever an exception is raised.

Table 13.2: Headless functions of `toolkit.parakit`.

Function	Description
<code>get_parameters(parser)</code>	Dictionary <code>{dest: default}</code> of all tunable options (lists are copied).
<code>save_parameters(values, path_or_dir)</code>	Write <code>values</code> as JSON and return the <code>Path</code> of the file. A path with a suffix is used as the file name; a directory (existing, or a path without suffix) receives a new timestamped file. Parent directories are created and the write is atomic.
<code>load_parameters(path)</code>	Read a JSON object from a file, or from the newest <code>parameters_*.json</code> file of a directory. Raises <code>FileNotFoundError</code> or <code>ValueError</code> .
<code>apply_parameters(parser, values, strict=True, validation_callbacks=None)</code>	Convert and validate every value with <code>parse_value</code> , then call <code>parser.set_defaults</code> . Returns the parser.
<code>parse_value(action, text, validator=None)</code>	Convert one value (text or an already typed JSON value) for an <code>argparse</code> action; see Section 13.2.4.
<code>parse_bool(value)</code>	Convert <code>true/false</code> , <code>yes/no</code> , <code>on/off</code> , <code>1/0</code> (any case) or a boolean to <code>bool</code> .
<code>format_value(action, value)</code>	Text that <code>parse_value</code> converts back (lists, also nested, as JSON; <code>None</code> as <code>"None"</code>).
<code>describe_type(action)</code>	Short type description such as <code>"float"</code> , <code>"list[int]"</code> , <code>"list[list[int]]"</code> or <code>"str or None"</code> .
<code>tunable_actions(parser)</code>	Dictionary <code>{dest: action}</code> of the options <code>parakit</code> manages.

13.2.3 File format

A parameter file is a flat JSON object that maps each destination name (`action.dest`) to its value, written in UTF-8 with an indentation of four spaces:

```
{
    "lr": 0.001,
    "batch_size": 64,
    "optimizer": "adam",
    "hidden": [
        64,
        64
    ],
    "layer_norm": true,
    "checkpoint": null
}
```

- *File names.* Saving into a directory creates `parameters_YYYYmmdd_HHMMSS.json`; if that name exists (two saves in the same second), `_1`, `_2`, ... is appended. Loading from a directory reads the most recent file, ranked by the timestamp and counter in its name (so `_10` is newer than `_9`); other `parameters_*.json` names rank below the timestamped ones.
- *Values.* JSON numbers, strings, booleans, `null` and lists are stored as such. NumPy arrays and scalars are converted with `tolist()`, sets are stored as sorted lists and any other object as its `str()`.
- *Loading.* Values read from a file pass through the same conversion as text, so a file may contain `"64"` or `64` for an integer option, and `"yes"` or `true` for a flag. A file may contain a subset of the parameters; the other defaults are kept.

- *Atomic writes.* Files are written to a temporary file in the target directory and renamed, so an interrupted save never leaves a truncated file behind. The file receives the usual permissions of new files (0o666 minus the umask).

13.2.4 Parsing rules

`parse_value(action, text)` derives the conversion from the parser's own declarations, so the same rules apply to values typed in the editor window, loaded from JSON or passed to `apply_parameters`. Table 13.3 summarises them.

Table 13.3: Conversion rules of `parse_value`.

Declaration	Accepted input and result
<code>type=int</code> , <code>type=float</code>	Text is converted after removing surrounding blanks. Already typed values from JSON are checked: an <code>int</code> option accepts 3 and 3.0 but not 3.5; a <code>float</code> option accepts integers.
Other <code>type=</code> callables	Text is passed to the callable. A value that is not text (for example from JSON) is passed as it is and, if that fails, as text (booleans as <code>"true"/"false"</code>), so converters written for command-line strings, such as <code>lambda s: int(s, 0)</code> , survive a save and load.
No <code>type</code>	The type is inferred from the default (<code>bool</code> , <code>int</code> , <code>float</code> , otherwise <code>str</code>), or from the <code>choices</code> when all have the same type.
Booleans: <code>store_true</code> , <code>store_false</code> , <code>BooleanOptionalAction</code> , <code>type=bool</code> or a boolean default	<code>true/false</code> , <code>yes/no</code> , <code>on/off</code> , 1/0 in any case, or a JSON boolean. The string <code>"False"</code> gives <code>False</code> .
Lists: <code>nargs="+"</code> , <code>nargs="*"</code> , an integer, <code>append</code> and <code>extend</code> actions	A JSON list (<code>[1.0, 2.0]</code>), a Python literal list or tuple, or tokens separated by commas (if the text contains a comma) or by whitespace. Each element is converted; <code>"+"</code> requires at least one element and an integer <code>nargs</code> exactly that many, while <code>append</code> and <code>extend</code> accept any number. <code>append</code> with <code>nargs</code> holds a list of lists such as <code>[[1, 2], [3, 4]]</code> ; for an integer <code>nargs</code> a flat list is split into groups of that size.
Default <code>None</code> or <code>nargs="?"</code>	<code>None</code> , <code>null</code> (any case) and the empty string give <code>None</code> . For other options <code>None</code> is rejected, except that a string option accepts the text <code>"None"</code> literally.
<code>choices</code>	Enforced on the converted value (on every element of a list).
<code>count</code> action	Converted as an integer.

Declare types explicitly. Type inference from the default is a convenience of `parakit`, not of `argparse`. An option declared as `add_argument("--gamma", default=0.99)` is converted to `float` by `parakit`, but `parser.parse_args(["--gamma", "0.9"])` returns the *string* `"0.9"`. Give every option a `type=` so that both paths agree.

Validation callbacks. A validator receives the *converted* value and signals the result in one of the following ways: return `True/False`; return a tuple (`is_valid`, `message`) or the legacy form (`is_valid`, `converted_value`, `message`); return `None` (valid); or raise `ValueError` or `TypeError`, whose text becomes the error message. Validators are not called when the converted value is `None`, and a value returned by a validator is ignored: the result always follows the parser declaration.

```
def positive_even(value):
    return value > 0 and value % 2 == 0, "batch size must be a positive even number"

validators = {"batch_size": positive_even,
             "lr": lambda v: (0 < v <= 1, "lr must lie in (0, 1]")}
apply_parameters(parser, {"batch_size": "128", "lr": 0.01},
                 validation_callbacks=validators)

try:
    apply_parameters(parser, {"batch_size": 7, "optimizer": "rmsprop"},
                    validation_callbacks=validators)
except ValueError as exc:
    print(exc)
# Invalid parameter values:
# batch_size: batch size must be a positive even number
# optimizer: 'rmsprop' is not a valid choice; value must be one of: 'adam', 'sgd'
```

13.2.5 Recommended workflow in training scripts

The following pattern gives a script three configuration layers with a clear precedence: parser defaults, then an optional parameter file, then explicit command-line flags.

```
import argparse
import sys
from toolkit.parakit import apply_parameters, load_parameters, save_parameters

def build_parser():
    parser = argparse.ArgumentParser()
    parser.add_argument("--config", default=None, help="JSON file or directory")
    parser.add_argument("--lr", type=float, default=3e-4)
    parser.add_argument("--seed", type=int, default=0)
    return parser

def main(argv=None):
    parser = build_parser()
    pre, _ = parser.parse_known_args(argv)           # read --config first
    if pre.config is not None:
        apply_parameters(parser, load_parameters(pre.config), strict=False)
    args = parser.parse_args(argv)                   # flags override the file
    run_dir = f"runs/seed{args.seed}"
    save_parameters(vars(args), f"{run_dir}/config.json")
    return args

main(["--lr", "0.001"])
```

`AJLATTConfig.add_arguments(parser)` registers every AJLATT option on a parser (Chapter 10), so environment parameters can be managed in the same way.

13.2.6 The ParameterTuner window

ParameterTuner opens a Tk window with one input field per tunable option: a check box for booleans, a drop-down list for options with choices, and a text field otherwise. Each field shows the option's help text and its type as reported by `describe_type`. Values are validated with `parse_value` (and the optional validation callbacks), written to timestamped JSON files, and

installed as the parser's defaults when the window closes.

Table 13.4: Constructor arguments of `ParameterTuner`.

Argument	Default	Meaning
<code>parser</code>	<code>required</code>	The <code>argparse.ArgumentParser</code> to tune.
<code>save_delay</code>	<code>30000</code>	Auto-save interval in milliseconds.
<code>save_path</code>	<code>None</code>	Output directory for the JSON files; <code>None</code> uses the current working directory. Created on the first save.
<code>auto_save_enabled</code>	<code>True</code>	Start with auto-save switched on.
<code>validation_callbacks</code>	<code>None</code>	Per-destination validators (Section 13.2.4); callbacks for unknown names are ignored with a warning.
<code>inactivity_timeout</code>	<code>10</code>	Seconds without keyboard or mouse activity after which the values are saved and the window closes; <code>None</code> disables the automatic close.

Window behaviour. The window offers the following controls:

- *Save Parameters* validates all fields and writes a new file. Invalid values are listed in an error dialog and nothing is saved.
- *OK* saves and closes the window; if the values are invalid the window stays open.
- *Auto-save* (check box) saves every `save_delay` milliseconds while enabled; problems are shown in the status line instead of dialogs.
- After `inactivity_timeout` seconds without activity the window saves (if the values are valid) and closes. The default of ten seconds suits scripts that open the window at start-up and should continue unattended; pass a larger value or `None` for interactive editing.
- *Browse* selects another output directory and *Reset to Default* returns to the initial one. Closing the window with the window manager does not save.

`tune()` blocks until the window closes and returns the parser. The values of the last *successful* save (manual, OK, auto-save or inactivity save) become the parser's defaults; if nothing was saved, the parser is unchanged. The path of the last file is available as `tuner.last_saved_path`. The class method `ParameterTuner.tune_parameters(parser, ...)` creates a tuner, calls `tune()` and returns the parser; `ParameterAdjuster` is a backward-compatible alias of the class. The font of the window is the class attribute `font_family` (default "Times New Roman").

```
from toolkit.parakit import ParameterTuner

tuner = ParameterTuner(parser, save_path="runs/params", inactivity_timeout=None,
                       validation_callbacks=validators)
parser = tuner.tune()           # blocks until the window is closed
args = parser.parse_args()
print("saved to", tuner.last_saved_path)
```

Systems without Tk or without a display. `tune()` raises `ImportError` with installation instructions when Tk is missing (the text of the message is the constant `TK_MISSING_MESSAGE` of `toolkit.parakit`) and `RuntimeError` when no display is available, for example on a cluster node or over SSH without X forwarding. Scripts that should run everywhere catch both exceptions and fall back to the headless API:

```
try:
    ParameterTuner(parser, save_path="runs/params").tune()
except (ImportError, RuntimeError) as exc:
    print(f"Editor not available ({exc}); using the current defaults.")
```

The script `examples/toolkit/parakit_demo.py` demonstrates the complete workflow, including the window (`--gui`), loading a file or directory (`--load`) and command-line overrides (`--set KEY=VALUE`).

Chapter 14

plotkit: Publication-Quality Plots

`toolkit.plotkit` produces the figures that research papers on reinforcement learning need most often: learning curves aggregated over seeds with uncertainty bands, grouped bar charts with error bars, histograms and annotated heatmaps. It depends only on `matplotlib`, accepts data from NumPy, pandas, PyTorch, TensorFlow and JAX, and never changes the global `matplotlib` configuration unless explicitly asked to.

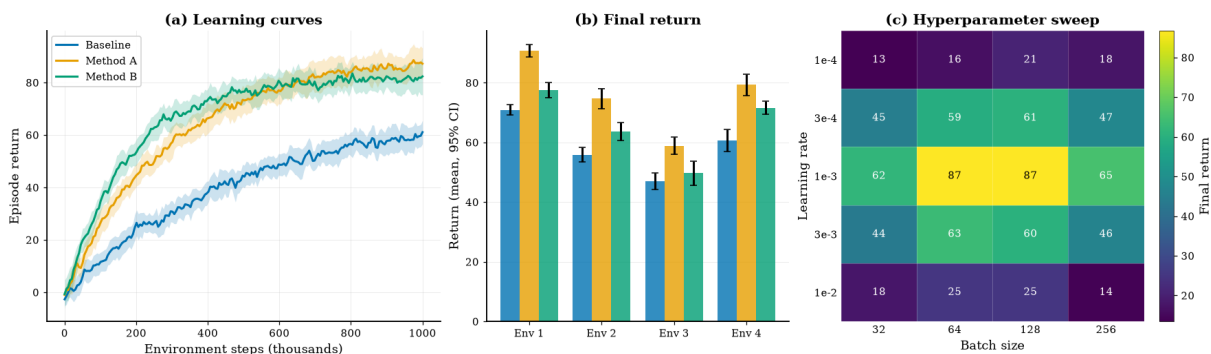


Figure 14.1: A three-panel summary figure made with `plotkit` (`examples/toolkit/plotkit_gallery.py`, synthetic data): (a) learning curves of three methods over eight seeds with 95 % confidence bands and EMA smoothing, (b) grouped bars with 95 % confidence intervals, (c) an annotated heatmap of a hyperparameter sweep. Each method keeps its Okabe-Ito colour in every panel.

14.1 Overview

All plot functions share the same behaviour:

- they draw into the `ax` argument when it is given and otherwise create a new figure of size `figsize`;
- they apply the `style` preset *locally*, inside a `matplotlib.rc_context`, so the caller's `rcParams` are unchanged afterwards;
- they return the `matplotlib.axes.Axes` they drew on, which can be customised further with ordinary `matplotlib` calls.

A non-default style also restyles the axes it draws on (spines, tick labels, axis-label fonts), so that axes created outside the style context match the preset. Pass `style="default"` to leave an existing axes exactly as it is.

Table 14.1: Public API of toolkit.plotkit.

Name	Purpose
<code>plot_shadow_curve</code>	Mean curves with a shaded band (std, SEM, 95 % CI, min-max or explicit) and optional smoothing.
<code>plot_learning_curves</code>	{method: runs} learning curves aggregated over seeds; runs may have different lengths.
<code>plot_line</code> , <code>plot_scatter</code>	Line and scatter plots of one or several series.
<code>plot_bar</code>	Bar charts; several series are grouped side by side, with error bars and value labels.
<code>plot_histogram</code>	Overlaid histograms on common bins.
<code>plot_heatmap</code> , <code>plot_gray_scale</code>	Annotated heatmaps with colorbar (seaborn-compatible arguments, matplotlib only).
<code>save_figure</code>	Save a figure to several formats at once.
<code>style_context</code> , <code>set_research_style</code> , <code>reset_style</code> , <code>STYLE_PRESETS</code>	Style presets applied locally or globally.
<code>get_palette</code> , <code>RESEARCH_COLORS</code> , <code>OKABE_ITO_COLORS</code> (and the <code>*_COLOR_LIST</code> variants)	Colour palettes, including a colour-blind-safe one.

```
import numpy as np
from toolkit.plotkit import plot_shadow_curve, save_figure

rng = np.random.default_rng(0)
returns = rng.normal(size=(5, 100)).cumsum(axis=1)      # 5 seeds x 100 steps
ax = plot_shadow_curve(returns, band="ci95", smoothing=10, labels="Agent",
                      xlabel="Episode", ylabel="Return")
save_figure(ax, "renders/first_curve", formats=("pdf", "png"))
```

14.2 Input rules

Every function converts its data inputs with the same rules:

- A 1-D array-like of numbers, including a flat Python list of scalars, is *one* series.
- A list or tuple of array-likes is *several* series, one per element; the elements may have different lengths.
- A 2-D array is read by the function: `plot_shadow_curve` and `plot_learning_curves` read it as *samples of one curve* (by default rows are samples such as seeds and columns are steps); `plot_line`, `plot_scatter`, `plot_bar` and `plot_histogram` read it as (`n_series`, `n_points`). For compatibility with matplotlib's `ax.plot(x, Y)`, `plot_line` reads a 2-D array whose columns, but not rows, have `len(x)` points as one series per column.
- A pandas `DataFrame` contributes one series per column, except in `plot_shadow_curve` and `plot_learning_curves`, where it is a sample matrix like a 2-D array (rows are samples).
- Dates and times (`datetime`, pandas `Timestamp`, NumPy `datetime64`) are valid x values and are drawn on a date axis; durations (`timedelta`, `timedelta64`) are converted to

seconds. Dates used as bar categories are labelled in ISO format.

- pandas **Series**, PyTorch tensors (on any device, with or without `requires_grad`), TensorFlow eager tensors and JAX arrays are converted to NumPy automatically. Masked arrays become NaN.
- A single `x` is shared by all series; a list of arrays (or a 2-D array) gives one `x` per series. When `x` is omitted, each series is plotted against `0, ..., n - 1`.

Length mismatches raise a `ValueError` that states both shapes and, when a 2-D input looks transposed, suggests transposing it. To pass a nested list as a single 2-D sample matrix to `plot_shadow_curve`, convert it with `np.asarray` first; otherwise it is read as several curves.

14.3 Common parameters

The parameters in Table 14.2 have the same meaning in all plot functions (the heatmap functions have no `labels`, `colors`, `legend` or `grid`).

Table 14.2: Parameters shared by the plot functions.

Parameter	Default	Meaning
<code>labels</code>	<code>None</code>	Legend labels, one per series; default "Curve 1", "Series 1", A wrong count gives a warning.
<code>colors</code>	<code>None</code>	<code>None</code> (the style's colour cycle), one colour, a list of colours (cycled), a palette name ("okabe_ito", "research", ...) or a matplotlib colormap name.
<code>ax</code>	<code>None</code>	Axes to draw on; a new figure is created when <code>None</code> .
<code>figsize</code>	(10, 6)	Size in inches of a new figure ((8, 6) for heatmaps).
<code>title, xlabel, ylabel</code>	<code>None</code>	Axes title and axis labels.
<code>legend</code>	<code>True</code>	Show a legend when there are several series or labels were given; a string sets the location, e.g. "lower right".
<code>grid</code>	<code>True</code>	Draw a major grid (only along the value axis for bars and histograms).
<code>style</code>	"research"	Style preset applied locally (Section 14.7), a matplotlib style name, a dict of <code>rcParams</code> , or <code>None</code> .
<code>**kwargs</code>		Passed to the underlying matplotlib call (<code>plot</code> , <code>scatter</code> , <code>bar</code> , <code>hist</code> , <code>imshow</code>); <code>color</code> and <code>label</code> are accepted as aliases of <code>colors</code> and <code>labels</code> .

14.4 Curves with uncertainty bands

14.4.1 `plot_shadow_curve`

```
plot_shadow_curve(y, x=None, y_std=None, labels=None, colors=None, alpha=0.2,
                 ax=None, axis=0, figsize=(10, 6), title=None, xlabel=None,
                 ylabel=None, legend=True, grid=True, style="research",
                 legend_labels=None, x_tick_labels=None, y_tick_labels=None,
                 *, band="std", smoothing=None, **kwargs) -> Axes
```

Each element of `y` gives one curve. A 2-D element holds samples of the curve (for example one row per seed) and is drawn as its mean with a band; a 1-D element is drawn as it is, with a band only when `y_std` is given.

Table 14.3: Parameters specific to `plot_shadow_curve`.

Parameter	Default	Meaning
<code>y</code>	required	One or several curves (see the input rules).
<code>x</code>	None	Shared x values or one array per curve.
<code>y_std</code>	None	Explicit half-width of the band: a scalar, one array for all curves, or one entry per curve (entries may be None). Overrides <code>band</code> .
<code>alpha</code>	0.2	Band opacity; 0 hides the band.
<code>axis</code>	0	Sample axis of 2-D inputs: 0 means rows are samples and columns are steps; use 1 for the transposed layout.
<code>band</code>	"std"	"std", "sem", "ci95", "minmax" or None (see below). Keyword only.
<code>smoothing</code>	None	int: trailing moving-average window; float in (0,1): EMA weight. Keyword only.
<code>legend_labels</code>	None	Alias of <code>labels</code> (takes precedence).
<code>x_tick_labels</code> , <code>y_tick_labels</code>	None	Custom tick labels: a mapping {position: label}, or a sequence (one label per x value of the first curve, otherwise spread evenly over the data range).

Bands. Let $y_{i,t}$ be sample i (for example seed i) at step t , and let $\mathcal{I}_t$ be the set of samples that are not NaN at step t , with $n_t = |\mathcal{I}_t|$. The drawn curve is the mean

$$\bar{y}_t = \frac{1}{n_t} \sum_{i \in \mathcal{I}_t} y_{i,t}, \quad (14.1)$$

and the band is $[\bar{y}_t - h_t, \bar{y}_t + h_t]$ with

$$\text{"std"} : \quad h_t = \sigma_t = \sqrt{\frac{1}{n_t} \sum_{i \in \mathcal{I}_t} (y_{i,t} - \bar{y}_t)^2}, \quad (14.2)$$

$$\text{"sem"} : \quad h_t = \frac{s_t}{\sqrt{n_t}}, \quad s_t = \sqrt{\frac{1}{n_t - 1} \sum_{i \in \mathcal{I}_t} (y_{i,t} - \bar{y}_t)^2}, \quad (14.3)$$

$$\text{"ci95"} : \quad h_t = z_{0.975} \frac{s_t}{\sqrt{n_t}}, \quad z_{0.975} \approx 1.96, \quad (14.4)$$

while "minmax" draws $[\min_i y_{i,t}, \max_i y_{i,t}]$ over $\mathcal{I}_t$. The "std" band uses the population standard deviation; the SEM-based bands use the sample standard deviation and are undefined (not drawn) where fewer than two samples exist. The "ci95" band is the normal approximation of a 95 % confidence interval of the mean; with few seeds (fewer than about ten) it is narrower than the exact Student- t interval, a fact worth stating in a figure caption. Because the statistics ignore NaN, runs of different length can be padded with NaN.

Smoothing. Smoothing is applied along the step axis *after* the statistics, with the same filter for the mean and both band edges. An integer $w \geq 1$ selects a trailing moving average with the same output length; the window shrinks at the start of the series:

$$\tilde{y}_t = \frac{1}{|W_t|} \sum_{s \in W_t} y_s, \quad W_t = \{s : \max(0, t - w + 1) \leq s \leq t, y_s \text{ finite}\}. \quad (14.5)$$

A float $\beta \in (0, 1)$ selects a debiased exponential moving average with smoothing weight β , the convention of TensorBoard's smoothing slider (larger values are smoother):

$$\tilde{y}_t = \frac{\sum_{s \leq t, y_s \text{ finite}} \beta^{t-s} y_s}{\sum_{s \leq t, y_s \text{ finite}} \beta^{t-s}}. \quad (14.6)$$

The normalisation removes the bias towards zero of the plain recursion $m_t = \beta m_{t-1} + (1 - \beta)y_t$, $m_{-1} = 0$; for finite data the result equals `pandas.Series(y).ewm(alpha=1 - beta).mean()`. Non-finite values are skipped. Note that smoothing the band edges does not turn them into a confidence band of the smoothed curve; it is a visual aid.

14.4.2 plot_learning_curves

```
plot_learning_curves(runs, x=None, *, band="ci95", smoothing=None, colors=None,
                    alpha=0.2, ax=None, figsize=(10, 6), title=None,
                    xlabel="Step", ylabel="Return", legend=True, grid=True,
                    style="research", **kwargs) -> Axes
```

`runs` maps a method name to its results: a 2-D array (`n_seeds`, `n_steps`), a 1-D array (a single seed), or a list of 1-D arrays of possibly different lengths, which are padded with NaN. `x` is either shared or a mapping `{method: x}`. The function is a thin layer over `plot_shadow_curve` with a 95 % confidence band by default, and always shows the legend when `legend` is true.

14.5 Lines, scatter plots, bars and histograms

`plot_line(x, y, ...)` and `plot_scatter(x, y, ...)` draw one or several series with the input rules of Section 14.2. `plot_scatter` uses `s=50` and `alpha=0.7` unless they are given; passing per-point values `c=...` (for a colormap) disables the series colour.

14.5.1 plot_bar

```
plot_bar(x, height, labels=None, colors=None, ax=None, figsize=(10, 6),
        title=None, xlabel=None, ylabel=None, legend=True, grid=True,
        style="research", *, width=0.8, yerr=None, value_labels=False,
        horizontal=False, capsize=3.0, **kwargs) -> Axes
```

14.5.2 plot_histogram

`plot_histogram(data, bins="auto", ...)` overlays one histogram per series; its keyword-only arguments are `density=False`, `alpha=0.45` and `histtype="stepfilled"`. The bin edges are computed once from the pooled data (`numpy.histogram_bin_edges`) so that all series share the same bins; NaN and infinite values are dropped, and `range=(lo, hi)` restricts the shared bins as in matplotlib. `density=True` normalises each histogram to unit area, and the default y-label is "Density" or "Count" accordingly. For the filled types the fill uses `alpha` while the outline stays opaque.

14.6 Heatmaps

Table 14.4: Parameters specific to `plot_bar`.

Parameter	Default	Meaning
<code>x</code>	required	Category labels (strings or numbers), or <code>None</code> for the indices. Bars are placed at <code>0, ..., n_categories - 1</code> .
<code>height</code>	required	One series (1-D), a list of series, or a 2-D array (<code>n_series</code> , <code>n_categories</code>). Several series are grouped side by side.
<code>width</code>	0.8	Total width of a group in category units; each bar is <code>width / n_series</code> wide.
<code>yerr</code>	<code>None</code>	Error bars: a scalar, one array per series (or one shared array), or the lower and upper errors of a single series as a <code>(2, n_categories)</code> array or a pair of lists.
<code>value_labels</code>	<code>False</code>	Annotate the bars: <code>True</code> (format <code>".3g"</code>), a format spec such as <code>".1f"</code> or <code>"d"</code> , a template such as <code>"{: .1%}"</code> , a printf-style format such as <code>"%.2f"</code> , or a callable.
<code>horizontal</code>	<code>False</code>	Horizontal bars, first category at the top.
<code>capsize</code>	3.0	Error-bar cap size in points.
<code>colors</code>	<code>None</code>	As usual; for a single series a list with one colour per category colours each bar.

```
plot_heatmap(data, xlabels=None, ylabels=None, cmap="viridis", annot=False,
             ax=None, figsize=(8, 6), title=None, xlabel=None, ylabel=None,
             cbar=True, cbar_label=None, style="research", *, fmt=None,
             vmin=None, vmax=None, center=None, robust=False, mask=None,
             nan_color=None, aspect="auto", square=False, linewidths=None,
             linecolor="white", annot_kws=None, cbar_kws=None, **kwargs) -> Axes
```

`plot_heatmap` draws a 2-D array with row 0 at the top (a 1-D array is drawn as one row). It is implemented with matplotlib's `imshow` and accepts the most common seaborn arguments, so existing seaborn calls usually work unchanged.

`plot_gray_scale(data, ...)` is `plot_heatmap` with `cmap="gray"` (black is low, white is high), convenient for images, occupancy grids and attention maps; it accepts all keyword arguments of `plot_heatmap`.

14.7 Styles

A style preset is a set of `rcParams` overrides applied on top of the active configuration. Table 14.6 summarises the presets; `STYLE_PRESETS[name]` is a read-only view of the exact values.

"research" is the paper-ready default of every function. "presentation" enlarges everything for slides and posters, "minimal" produces small, quiet figures, and "default" (or `None`) leaves the active configuration untouched. The `style` argument also accepts any name from `matplotlib.style.available` and a dictionary of `rcParams`.

Three functions apply styles outside the plot functions:

```
import matplotlib.pyplot as plt
from toolkit.plotkit import reset_style, set_research_style, style_context

# 1. Locally, for any matplotlib code (rcParams are restored on exit).
with style_context("presentation"):
    fig, ax = plt.subplots()
```

Table 14.5: Parameters specific to `plot_heatmap`.

Parameter	Default	Meaning
<code>xlabels, ylabels</code>	<code>None</code>	One tick label per column / row. <code>None</code> labels cells with their index (up to 30 cells, otherwise automatic integer ticks); <code>False</code> hides the ticks. Long x labels are rotated. seaborn's <code>xticklabels</code> / <code>yticklabels</code> are aliases.
<code>cmap</code>	<code>"viridis"</code>	Colormap name or object; use a diverging map (<code>"RdBu_r"</code>) with <code>center</code> .
<code>annot</code>	<code>False</code>	<code>True</code> writes each value into its cell; an array of the same shape writes its entries instead. Text is black or white, whichever contrasts better with the cell colour.
<code>fmt</code>	<code>None</code>	Annotation format: a format spec (<code>".2g"</code> , <code>"d"</code>), a template (<code>"{: .1%}"</code>), a printf-style format (<code>"%.2f"</code>) or a callable. Default <code>".0f"</code> when all numeric annotations are whole numbers, else <code>".2g"</code> .
<code>cbar, cbar_label</code>	<code>True, None</code>	Colorbar and its label (also <code>cbar_kws={"label": ...}</code>). The colorbar is available as <code>ax.images[-1].colorbar</code> .
<code>vmin, vmax</code>	<code>None</code>	Colour limits (default: data range).
<code>center</code>	<code>None</code>	Value mapped to the middle of the colormap; missing limits are chosen symmetrically around it.
<code>robust</code>	<code>False</code>	Use the 2nd and 98th percentiles for missing limits.
<code>mask</code>	<code>None</code>	Boolean array; cells where it is <code>True</code> are hidden.
<code>nan_color</code>	<code>None</code>	Colour of NaN and masked cells (default transparent).
<code>aspect, square</code>	<code>"auto"</code> , <code>False</code>	Cell aspect ratio; <code>square=True</code> equals <code>aspect="equal"</code> .
<code>linewidths, linecolor</code>	<code>None</code> , <code>"white"</code>	Lines between cells; by default 0.5 for matrices of up to 50 cells per side and none for larger ones.
<code>annot_kws, cbar_kws</code>	<code>None</code>	Text properties of the annotations (a <code>color</code> entry disables the automatic contrast colour) and arguments of <code>Figure.colorbar</code> .

Table 14.6: Style presets. Font sizes are in points.

	<code>"research"</code>	<code>"presentation"</code>	<code>"minimal"</code>	<code>"default"</code>
Font family	serif	sans-serif	sans-serif	unchanged
Base / label size	12 / 12	16 / 18	10 / 10	unchanged
Title size, weight	14, bold	20, bold	11, normal	unchanged
Tick / legend size	10 / 10	14 / 14	9 / 9	unchanged
Line width	2.0	3.0	1.5	unchanged
Axes line width	1.2	1.5	0.8 (dark grey)	unchanged
Top and right spines	hidden	hidden	hidden	unchanged
Legend frame	framed	framed, no edge	none	unchanged
Grid opacity	0.3	0.3	0.2	unchanged
Colour cycle	<code>research</code>	<code>research</code>	<code>research</code>	unchanged

```

ax.plot([0, 1, 2], [0, 1, 4])

# 2. Globally, as an explicit opt-in; also sets savefig.dpi=300 and
#     savefig.bbox="tight". reset_style() restores the previous rcParams.
set_research_style()
fig, ax = plt.subplots()
ax.plot([0, 1, 2], [0, 1, 4])
reset_style()

```

Library code should prefer the `style` argument or `style_context`; `set_research_style` is meant for the top level of a plotting script.

14.8 Colour palettes

`get_palette(name="research", n=None)` returns a list of hex colours:

- "research" (alias "tab10"): matplotlib's ten default colours, also available as the dictionary `RESEARCH_COLORS` ("blue", "orange", ...) and the list `RESEARCH_COLOR_LIST`. It is the colour cycle of all presets.
- "okabe_ito" (alias "colorblind"): the eight colours of Okabe and Ito (2008), designed to remain distinguishable for the common forms of colour-vision deficiency; available as `OKABE_ITO_COLORS` and `OKABE_ITO_COLOR_LIST`.
- Any matplotlib colormap name. Qualitative colormaps ("tab20", "Set2", ...) return their colours; continuous colormaps ("viridis", ...) are sampled at `n` evenly spaced points in [0.1, 0.9] (eight by default).

Palettes are cycled when `n` exceeds their length. A palette name can be passed directly as `colors` to every plot function.

Table 14.7: The Okabe-Ito palette in plotting order.

Name	Hex	Name	Hex
blue	#0072B2	reddish_purple	#CC79A7
orange	#E69F00	sky_blue	#56B4E9
bluish_green	#009E73	yellow	#F0E442
vermillion	#D55E00	black	#000000

Colour-blind guidance. About one in twelve men has a red-green colour-vision deficiency. Several pairs of the default `tab10` colours, notably its red and green, are hard to tell apart with protanopia or deuteranopia, so for papers we recommend `colors="okabe_ito"`. In addition:

- use the first six Okabe-Ito colours for curves; yellow has low contrast on white and black carries no hue, so they come last;
- keep the colour of a method identical in every panel and figure, by passing the same list of colours (Section 14.10);
- do not rely on colour alone: vary line styles or markers, or label the curves directly, so that the figure also works in greyscale print;

- for heatmaps use perceptually uniform colormaps ("viridis", "cividis") for magnitudes and a diverging map with `center` for signed quantities; avoid "jet" and red-green diverging maps.

14.9 Saving figures

```
save_figure(fig_or_ax, path, formats=None, dpi=300, transparent=False,
            **savefig_kwargs) -> list[Path]
```

`save_figure` accepts a `Figure`, a `SubFigure`, an `Axes` or an array of axes (the whole parent figure is saved), creates missing parent directories and returns the written paths. With `formats` given, the extension of `path` is replaced by (or, if it has no recognised extension, extended with) each format: with the path "results/curve" and `formats=("pdf", "png")`, the files `results/curve.pdf` and `results/curve.png` are written. Without `formats` the extension of `path` is used, or `rcParams["savefig.format"]` if it has none. `dpi` applies to raster formats and to rasterised parts of vector files. The defaults `bbox_inches="tight"` and `pad_inches=0.05` can be overridden through `**savefig_kwargs`.

For LaTeX documents, save vector PDF at the final size of the figure (set `figsize` to the column or text width, for example 3.5×2.5 inches for a two-column paper, and include it without scaling), so that the font sizes of the style preset are the sizes that appear in print. Some publishers require TrueType instead of Type 3 fonts in PDF files; set `pdf.fonttype` while saving:

```
import matplotlib as mpl

with mpl.rc_context({"pdf.fonttype": 42, "ps.fonttype": 42}):
    save_figure(ax, "renders/curve_truetype.pdf")
```

14.10 Recipes

14.10.1 Reinforcement-learning training curves

Episode returns logged for several seeds form a 2-D array (`n_seeds`, `n_episodes`). Smoothing with an EMA weight between 0.6 and 0.9 makes the trend visible without hiding the variability between seeds:

```
import numpy as np
from toolkit.plotkit import plot_shadow_curve

rng = np.random.default_rng(1)
episodes = np.arange(500)
returns = 100 * (1 - np.exp(-episodes / 150)) + rng.normal(0, 15, (10, 500))

ax = plot_shadow_curve(returns, x=episodes, band="ci95", smoothing=0.9,
                      labels="PP0", colors="okabe_ito",
                      xlabel="Episode", ylabel="Episode return",
                      title="Training progress (10 seeds, 95% CI)")
```

14.10.2 Comparing methods, including unfinished runs

`plot_learning_curves` takes one entry per method. Runs of different length, for example episodes that ended early or seeds that are still training, are passed as lists and padded with NaN. The following recipe combines an environment of `env_lib` with `plotkit`: it records the order parameter r of the Kuramoto model for three coupling strengths and eight seeds. Runs with strong coupling synchronise and terminate early, so the curves end at different steps, and the band of the strongest coupling is computed from the seeds that are still running.

```
import numpy as np
import env_lib
from toolkit.plotkit import plot_learning_curves

def order_parameter_trace(coupling, seed, steps=300):
    env = env_lib.KuramotoOscillatorEnv(n_oscillators=10, coupling_mode="constant",
                                         coupling_strength=coupling, max_steps=steps)
    obs, info = env.reset(seed=seed)
    trace = [info["order_parameter"]]
    action = np.zeros(env.action_space.shape, dtype=np.float32) # no control input
    for _ in range(steps):
        obs, reward, terminated, truncated, info = env.step(action)
        trace.append(info["order_parameter"])
        if terminated or truncated:
            break
    env.close()
    return np.asarray(trace)

runs = {f"K = {k}": [order_parameter_trace(k, seed) for seed in range(8)]
        for k in (0.1, 0.2, 0.5)}
ax = plot_learning_curves(runs, band="ci95", colors="okabe_ito",
                          xlabel="Step", ylabel="Order parameter r",
                          legend="lower right")
```

14.10.3 Several panels with consistent colours

Create the axes yourself, pass them as `ax`, and give every panel the same list of colours. Creating the figure inside `style_context` also applies the preset to anything drawn directly with matplotlib, such as a figure title.

```
import matplotlib.pyplot as plt
import numpy as np
from toolkit.plotkit import (get_palette, plot_bar, plot_learning_curves,
                             save_figure, style_context)

rng = np.random.default_rng(0)
methods = ["Baseline", "Method A", "Method B"]
colors = get_palette("okabe_ito", len(methods))
runs = {m: rng.normal(size=(8, 200)).cumsum(axis=1) + 0.1 * i * np.arange(200)
        for i, m in enumerate(methods)}
finals = np.array([r[:, -1].mean() for r in runs.values()]) # one per method

with style_context("research"):
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.8), layout="constrained")
    plot_learning_curves(runs, colors=colors, smoothing=0.8, ax=axes[0],
                        title="(a) Learning curves")
    plot_bar(methods, finals, colors=colors, ax=axes[1], value_labels=".1f",
```

```

        title="(b) Final return")                # one colour per bar
save_figure(fig, "renders/comparison", formats=("pdf", "png"))

```

14.10.4 Distinguishing series without colour

Keyword arguments apply to all series of one call. To give each method its own line style, call the function once per method on the same axes and pass the colours explicitly:

```

fig, ax = plt.subplots(figsize=(6, 4))
for method, color, linestyle in zip(methods, colors, ["-", "--", ":"]):
    plot_shadow_curve(runs[method], labels=method, colors=color, ax=ax,
                     band="sem", linestyle=linestyle)

```

14.10.5 Bar chart of final performance

```

from toolkit.plotkit import plot_bar

methods = ["DQN", "PPO", "SAC", "Ours"]
accuracy = np.array([71.2, 78.5, 80.1, 84.3])
std_err = np.array([1.8, 1.2, 1.5, 1.1])
ax = plot_bar(methods, accuracy, yerr=std_err, value_labels=".1f",
              colors="okabe_ito", ylabel="Success rate (%)")

# Several environments: height has shape (n_methods, n_environments).
scores = rng.uniform(40, 90, size=(3, 4))
ax = plot_bar(["Env 1", "Env 2", "Env 3", "Env 4"], scores,
              labels=["A", "B", "C"], yerr=rng.uniform(1, 4, size=(3, 4)))

```

14.10.6 Confusion and correlation matrices

```

from toolkit.plotkit import plot_heatmap

classes = ["left", "stay", "right"]
cm = np.array([[48, 5, 2], [4, 40, 6], [1, 7, 52]])
ax = plot_heatmap(cm, xlabels=classes, ylabels=classes, annot=True,
                  cmap="Blues", square=True, xlabel="Predicted",
                  ylabel="True", cbar_label="Count")

# Correlations: diverging colormap centred at zero, upper triangle hidden.
corr = np.corrcoef(rng.normal(size=(6, 200)))
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
ax = plot_heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0,
                  vmin=-1, vmax=1, mask=mask, square=True, cbar_label="r")

```

14.10.7 Distributions of returns

```

from toolkit.plotkit import plot_histogram

ax = plot_histogram([rng.normal(0, 1, 1000), rng.normal(1.5, 0.7, 1000)],
                    labels=["Policy A", "Policy B"], density=True,

```

```
colors="okabe_ito", xlabel="Return")
```

14.10.8 Plotting tensors and data frames

Tensors can be passed without conversion; the functions detach them and copy them to the CPU. A pandas DataFrame gives one series per column, which suits logs read with `pandas.read_csv`.

```
import torch
from toolkit.plotkit import plot_line

# three series of 100 points, shape (n_series, n_points)
losses = torch.rand(3, 100, requires_grad=True).cumsum(dim=1)
ax = plot_line(None, losses, labels=["actor", "critic", "entropy"])
```

14.11 Command-line gallery

The module is executable and shows every plot type with synthetic data. The same program is installed as the console script `plotkit-gallery`.

```
python -m toolkit.plotkit --demo all          # all eight demos in one window
# writes renders/plotkit/plotkit_heatmap.png and .pdf
plotkit-gallery --demo heatmap --style presentation \
    --save renders/plotkit --format png pdf
```

Table 14.8: Options of `python -m toolkit.plotkit` and `plotkit-gallery`.

Option	Default	Meaning
<code>--demo</code>	<code>all</code>	shadow, learning, heatmap, grayscale, line, bar, scatter, histogram, or all for a 2×4 grid of all demos.
<code>--style</code>	<code>research</code>	Style preset.
<code>--save DIR</code>	<code>none</code>	Save to <code>DIR/plotkit_<demo>.<format></code> instead of showing a window.
<code>--format</code>	<code>png</code>	One or more output formats.
<code>--dpi</code>	<code>150</code>	Resolution of raster output.
<code>--seed</code>	<code>0</code>	Seed of the synthetic data.

Without `--save` the figure is shown with `plt.show()`; when the active matplotlib backend is non-interactive (for example with `MPLBACKEND=Agg`), the figure is saved to `renders/plotkit/` instead. The script `examples/toolkit/plotkit_gallery.py` builds the three-panel figure of Figure 14.1 and is a good starting point for a paper figure; run it with `--help` for its options.

Chapter 15

neural_toolkit: Networks and Tabular Tools

`toolkit.neural_toolkit` provides configurable PyTorch modules for reinforcement learning: policy, value and Q-networks with MLP, convolutional, recurrent and transformer trunks, encoders and decoders (including a variational autoencoder), factories that build them by name, a collection of utilities for initialisation, optimisation and checkpoints, and NumPy-based tools for tabular methods. It does not implement learning algorithms; the modules are ordinary `torch.nn.Module` objects meant to be combined with your own training code.

The subpackage requires PyTorch (`pip install "[torch]"`). Importing it without PyTorch raises an `ImportError` that explains how to install it; the rest of `toolkit` does not need PyTorch.

15.1 Conventions

- *Batch first.* Inputs have a leading batch dimension B: feature vectors (B, D), images (B, C, H, W) and sequences (B, T, D).
- *One MLP convention.* Every fully connected stack in the toolkit consists of hidden blocks `Linear -> [LayerNorm] -> activation -> [Dropout]` followed by a linear output layer without activation.
- *Raw outputs.* Policies return unnormalised outputs of size `output_dim`: logits for a categorical policy, or distribution parameters (for example means) for a continuous one. Turning them into a distribution is left to the algorithm.
- *Devices.* Every constructor accepts `device` (default "cpu") and moves the module there at the end of construction; `device=None` leaves it where PyTorch created it. Inputs must be on the same device.
- *Validation.* Invalid sizes, dropout rates, activation names or cell types raise `ValueError` or `TypeError` at construction, with a message that lists the valid options.

15.2 Building blocks

Activations. `get_activation(name)` returns a new activation module for a case-insensitive name, or instantiates an `nn.Module` subclass passed instead of a name. The valid names are available as `ACTIVATION_NAMES`:

Name	Module	Name	Module
relu	nn.ReLU	gelu	nn.GELU
leaky_relu	nn.LeakyReLU	silu, swish	nn.SiLU
elu	nn.ELU	mish	nn.Mish
tanh	nn.Tanh	softplus	nn.Softplus
sigmoid	nn.Sigmoid	identity	nn.Identity

MLPs. `build_mlp` returns an `nn.Sequential`. Without `output_dim` only the hidden stack is built; its output width is `hidden_dims[-1]`.

```
build_mlp(input_dim, hidden_dims, output_dim=None, activation="relu", dropout=0.0,
          layer_norm=False, output_activation=None, bias=True) -> nn.Sequential
```

```
import torch
from toolkit.neural_toolkit import build_mlp, get_activation

mlp = build_mlp(4, [32, 32], output_dim=2, activation="tanh", layer_norm=True)
print(mlp(torch.zeros(8, 4)).shape)      # torch.Size([8, 2])
print(get_activation("swish"))           # SiLU()
```

Positional encoding. `PositionalEncoding(d_model, dropout=0.1, max_len=5000)` adds the sinusoidal table of Vaswani et al. (2017) to a batch-first sequence (B, T, d_model); position t receives

$$\text{PE}_{t,2i} = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \text{PE}_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/d}}\right), \quad (15.1)$$

for every batch element, followed by dropout. Sequences longer than `max_len` raise a `ValueError`. The table is a non-persistent buffer and is not stored in `state_dict`.

15.3 Network families

Table 15.1 lists all networks with their input and output shapes. Four classes have two names: the historical spellings `CNPolicyNetwork`, `RNPolicyNetwork`, `CNQNetwork` and `RNQNetwork` and the aliases with `CNN` and `RNN` refer to the same classes.

15.3.1 Constructor arguments

All classes share the arguments `activation="relu"`, `dropout=0.0`, `layer_norm=False` and `device="cpu"`; the transformer classes default to `dropout=0.1` and `layer_norm=True`. The remaining arguments depend on the trunk (Table 15.2).

`layer_norm=True` inserts `LayerNorm` after every hidden linear layer; in convolutional trunks it additionally inserts `BatchNorm2d` after every convolution.

15.3.2 Architectures

Convolutional trunk. `Conv2d -> [BatchNorm2d] -> activation -> [Dropout2d]` blocks, adaptive average pooling to `pooled_size × pooled_size` (1 for policies, value and Q-networks), flattening and an MLP with `fc_dims`, followed by the output layer. Attribute names: `conv_layers`, `adaptive_pool`, `fc_layers`.

Table 15.1: Networks, encoders and decoders with their shapes. B: batch, T: sequence length, A: action_dim, O: output_dim, L: latent_dim.

Class (alias)	Input	Output
<i>Policy networks</i> (input_dim, output_dim)		
MLPPolicyNetwork	(B, D)	(B, O)
CNPolicyNetwork (CNNPolicyNetwork)	(B, C, H, W)	(B, O)
RNPolicyNetwork (RNNPolicyNetwork)	(B, T, D) or (B, D)	((B, O), hidden)
TransformerPolicyNetwork	(B, T, D), mask (B, T)	(B, O)
<i>Value networks</i> (input_dim, output_dim=1)		
MLPValueNetwork	(B, D)	(B, O)
CNNValueNetwork	(B, C, H, W)	(B, O)
RNNValueNetwork	(B, T, D) or (B, D)	((B, O), hidden)
TransformerValueNetwork	(B, T, D), mask (B, T)	(B, O)
<i>Q-networks</i> (state_dim, action_dim); optional integer action (B,)		
MLPQNetwork	(B, D)	(B, A), or (B,) with action
MLPQNetwork, continuous critic	(B, D), (B, A)	(B, 1); see below
DuelingQNetwork	(B, D)	(B, A), or (B,) with action
CNQNetwork (CNNQNetwork)	(B, C, H, W)	(B, A), or (B,) with action
RNQNetwork (RNNQNetwork)	(B, T, D) or (B, D)	(q, hidden), q as above
TransformerQNetwork	(B, T, D), mask (B, T)	(B, A), or (B,) with action
<i>Encoders</i> (input_dim or input_channels, latent_dim)		
MLPEncoder	(B, D)	(B, L)
CNNEncoder	(B, C, H, W)	(B, L)
RNNEncoder	(B, T, D) or (B, D)	((B, L), hidden)
TransformerEncoder	(B, T, D), mask (B, T)	(B, L)
VariationalEncoder	(B, D)	(z, mu, logvar), each (B, L)
<i>Decoders</i> (latent_dim, output_dim or output_channels)		
MLPDecoder, VariationalDecoder	(B, L)	(B, O)
CNNDecoder	(B, L)	(B, C, S, S), $S = \text{initial_size} \cdot \prod \text{strides}$
RNNDecoder, TransformerDecoder	(B, L), seq_len	(B, T, O), T = seq_len or max_seq_len

Table 15.2: Trunk-specific constructor arguments and their defaults.

Trunk	Arguments
MLP	<code>hidden_dims=(256, 256)</code> . For <code>DuelingQNetwork</code> the shared trunk uses <code>hidden_dims[:-1]</code> and each stream one hidden layer of width <code>hidden_dims[-1]</code> .
Convolutional	<code>input_channels</code> , <code>conv_dims=(32, 64, 128)</code> , <code>fc_dims=(256, 256)</code> , <code>kernel_sizes</code> (int or one per layer, default 3), <code>strides</code> (default 1). Padding is <code>kernel_size // 2</code> . The feature map is globally average-pooled, so any image size works. <code>CNNEncoder</code> : <code>conv_dims=(32, 64, 128, 256)</code> , <code>fc_dims=(512, 256)</code> , <code>strides</code> 1 for the first layer and 2 afterwards, <code>pooled_size=4</code> (the map is pooled to 4×4).
Transposed conv.	<code>CNNDecoder</code> : <code>output_channels</code> , <code>fc_dims=(512, 256)</code> , <code>conv_dims=(256, 128, 64, 32)</code> , <code>kernel_sizes</code> (default 3), <code>strides</code> (default 2), <code>initial_size=4</code> , <code>output_activation="sigmoid"</code> ; the default output is 64×64 (property <code>output_size</code>).
Recurrent	<code>hidden_dim=256</code> , <code>num_layers=2</code> , <code>rnn_type="lstm"</code> (or "gru"), <code>bidirectional=False</code> (True for <code>RNNEncoder</code> ; not available for <code>RNNDecoder</code>), <code>fc_dims=(256,)</code> ; <code>RNNDecoder</code> : <code>max_seq_len=100</code> .
Transformer	<code>d_model=256</code> (divisible by <code>nhead</code>), <code>nhead=8</code> , <code>num_layers=6</code> , <code>dim_feedforward=1024</code> , <code>dropout=0.1</code> , <code>fc_dims=(256,)</code> , <code>max_len=5000</code> ; <code>TransformerDecoder</code> : <code>max_seq_len=100</code> , <code>max_len=None</code> (then <code>max(max_seq_len, 5000)</code>).

Recurrent trunk. A batch-first LSTM or GRU followed by an MLP head that reads the final hidden state of the last layer; for a bidirectional RNN the forward state after the last step and the backward state after the first step are concatenated, so both directions have seen the whole sequence. `forward(x, hidden=None)` accepts a sequence or a single step (B, D) and returns the new hidden state (`h_n` for a GRU, (`h_n`, `c_n`) for an LSTM), which can be passed to the next call:

```
from toolkit.neural_toolkit import RNNPolicyNetwork

policy = RNNPolicyNetwork(input_dim=5, output_dim=3, hidden_dim=32, rnn_type="gru")
hidden = None
for t in range(4):
    logits, hidden = policy(torch.randn(8, 5), hidden)
print(logits.shape, hidden.shape)  # (8, 3) and (num_layers, 8, 32)
```

Transformer trunk. A linear projection to `d_model`, the positional encoding, a batch-first `nn.TransformerEncoder`, mean pooling over the valid positions and an MLP head. The optional mask is a boolean (B, T) tensor in which True marks padding (PyTorch's `src_key_padding_mask` convention); padded positions are excluded from attention and pooling. Any sequence length up to `max_len` is accepted.

```
from toolkit.neural_toolkit import TransformerPolicyNetwork

policy = TransformerPolicyNetwork(input_dim=6, output_dim=4, d_model=64, nhead=4,
                                  num_layers=2, dim_feedforward=128)
x = torch.randn(2, 10, 6)  # two sequences of length 10
mask = torch.zeros(2, 10, dtype=torch.bool)
mask[1, 7:] = True  # second sequence has 7 steps
print(policy(x, mask).shape)  # torch.Size([2, 4])
```

Q-networks. Discrete Q-networks return one value per action, (B, A). Passing integer action indices of shape (B,) or (B, 1) as second argument returns the selected values $Q(s, a)$ with shape (B,), which is what temporal-difference losses need. `MLPQNetwork` with `continuous_action=True` is a critic for continuous actions (DDPG, TD3, SAC): it concatenates state and action and returns $Q(s, a)$ with shape (B, 1). `DuelingQNetwork` (Wang et al., 2016) combines a value stream $V(s)$ and an advantage stream $A(s, a)$ as

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'). \quad (15.2)$$

```
from toolkit.neural_toolkit import DuelingQNetwork, MLPQNetwork

q = DuelingQNetwork(state_dim=6, action_dim=3, hidden_dims=(64, 64))
states, actions = torch.randn(5, 6), torch.tensor([0, 1, 2, 1, 0])
print(q(states).shape, q(states, actions).shape)  # (5, 3) and (5,)

critic = MLPQNetwork(state_dim=6, action_dim=2, continuous_action=True)
print(critic(states, torch.randn(5, 2)).shape)  # (5, 1)
```

Variational autoencoder. `VariationalEncoder` returns (z, mu, logvar) with $z = \mu + \exp(\frac{1}{2} \log \sigma^2) \varepsilon$ and $\varepsilon \sim \mathcal{N}(0, I)$ drawn from PyTorch's default generator (the reparameterisation trick). The static method `kl_divergence(mu, logvar)` returns the per-sample KL term of the evidence lower bound,

$$\text{KL}(\mathcal{N}(\mu, \text{diag } \sigma^2) \parallel \mathcal{N}(0, I)) = -\frac{1}{2} \sum_j \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2 \right), \quad (15.3)$$

with shape (B,). `VariationalDecoder` is an MLP from z to the reconstruction.

```
import torch.nn.functional as F
from toolkit.neural_toolkit import VariationalDecoder, VariationalEncoder

encoder = VariationalEncoder(input_dim=64, latent_dim=8, hidden_dims=(128,))
decoder = VariationalDecoder(latent_dim=8, output_dim=64, hidden_dims=(128,))
x = torch.randn(32, 64)
z, mu, logvar = encoder(x)
reconstruction = F.mse_loss(decoder(z), x, reduction="none").sum(dim=1)
loss = (reconstruction + encoder.kl_divergence(mu, logvar)).mean()
```

Sequence decoders. `RNNDecoder` feeds the projected latent vector to a unidirectional LSTM or GRU at every time step and maps each output with a shared MLP head. `TransformerDecoder` is non-autoregressive: the queries are positional encodings of the output positions, the memory is the projected latent vector, and all positions are decoded in parallel. Both generate `max_seq_len` steps unless `forward(z, seq_len=...)` asks for another length.

15.4 Factories

Five factories build networks from a type name, which is convenient when the architecture is a command-line option. Names are case-insensitive; an unknown name raises a `ValueError` that lists the valid names. The remaining keyword arguments are passed to the constructor of the selected class.

Factory	Static method	Valid type names
PolicyFactory	<code>create_policy</code>	mlp, cnn, rnn, transformer
ValueFactory	<code>create_value_network</code>	mlp, cnn, rnn, transformer
QNetworkFactory	<code>create_q_network</code>	mlp, dueling, cnn, rnn, transformer
EncoderFactory	<code>create_encoder</code>	mlp, cnn, rnn, transformer, vae
DecoderFactory	<code>create_decoder</code>	mlp, cnn, rnn, transformer, vae

Each method takes the type name as its first argument followed by the keyword arguments of the constructor.

```
from toolkit.neural_toolkit import PolicyFactory, QNetworkFactory

policy = PolicyFactory.create_policy("mlp", input_dim=8, output_dim=4,
                                     hidden_dims=(64, 64))
q_net = QNetworkFactory.create_q_network("dueling", state_dim=8, action_dim=4)
```

15.5 NetworkUtils

NetworkUtils groups helper functions as static methods (Table 15.3).

15.6 Tabular tools

The tabular tools work on finite state and action spaces with NumPy arrays and do not need a GPU. Every table owns a `numpy.random.Generator` (argument `rng`: a generator, an integer seed, or `None` for fresh entropy) that is used for all sampling, so results are reproducible without touching the global random state. States and actions are integer indices; out-of-range indices raise `IndexError`.

The three table classes share one constructor:

```
QTable(state_space_size, action_space_size=None, initial_value=0.0,
        dtype=np.float32, rng=None)           # also ValueTable, PolicyTable
```

QTable

Action values $Q[s, a]$ (`action_space_size` is required). Entries are read and written with `get_value`, `set_value`, `update_value` and `get_q_values`. `get_max_q_value` and `get_max_action` return the greedy value and action (the lowest index among ties). `get_policy` samples an epsilon-greedy action, and the methods `get_epsilon_greedy_probs`, `get_softmax_probs` and `get_softmax_policy` give the corresponding probabilities or a softmax sample.

ValueTable

State values $V[s]$, with `get_values` for the whole array.

PolicyTable

A stochastic policy $\pi[s, a]$ that starts uniform. `set_value` and `update_value` change one probability and rescale the others so that the row still sums to one; `set_policy_probs`, `set_deterministic_policy`, `get_policy_probs` and `get_policy` (sampling) complete the interface.

Table 15.3: Static methods of `NetworkUtils`.

Method	Description
<i>Initialisation and parameters</i>	
<code>initialize_weights(module, method="xavier_uniform", gain=1.0)</code>	Initialise every linear and (transposed) convolution layer recursively; biases are set to zero. Methods: <code>xavier_uniform</code> , <code>xavier_normal</code> , <code>kaiming_uniform</code> , <code>kaiming_normal</code> , <code>orthogonal</code> , <code>uniform</code> ($U(-0.1, 0.1)$), <code>normal</code> ($\mathcal{N}(0, 0.1^2)$).
<code>count_parameters(model)</code>	Number of trainable parameters.
<code>count_parameters_by_layer(model)</code>	Trainable parameters owned directly by each sub-module.
<code>freeze_layers(model, names)</code> , <code>unfreeze_layers(model, names)</code>	Disable or enable gradients of parameters whose name contains any of <code>names</code> .
<i>Gradients</i>	
<code>get_grad_norm(model, norm_type=2.0)</code>	Total gradient norm.
<code>clip_grad_norm(model, max_norm, norm_type=2.0)</code>	Clip in place; returns the norm before clipping.
<i>Optimisation</i>	
<code>create_optimizer(model, optimizer_type="adam", lr=1e-3, weight_decay=0.0, skip_list=None, **kw)</code>	<code>adam</code> , <code>adamw</code> , <code>sgd</code> , <code>rmsprop</code> , <code>adagrad</code> , <code>adamax</code> . With <code>weight_decay > 0</code> the decay is applied to weights only (see <code>apply_weight_decay</code>).
<code>apply_weight_decay(model, weight_decay, skip_list=None)</code>	Parameter groups without decay for one-dimensional parameters (biases, normalisation scales) and for names in <code>skip_list</code> .
<code>create_scheduler(optimizer, scheduler_type="step", **kw)</code>	<code>step</code> , <code>multistep</code> , <code>exponential</code> , <code>cosine</code> , <code>cosine_warm_restart</code> , <code>plateau</code> , <code>linear</code> , <code>polynomial</code> ; <code>**kw</code> go to the PyTorch scheduler (for example <code>step_size</code> or <code>T_max</code>).
<i>Checkpoints</i>	
<code>save_checkpoint(model, optimizer, epoch, loss, filepath, **extra)</code>	Save model and optimizer state (<code>optimizer</code> may be <code>None</code>), epoch, loss and any extra entries; atomic write; returns the path.
<code>load_checkpoint(model, optimizer, filepath, map_location=None, weights_only=True, strict=True)</code>	Restore a checkpoint and return (<code>epoch</code> , <code>loss</code>). Tensors are loaded onto the device of the model by default, so GPU checkpoints load on CPU-only machines.
<i>Layer construction</i>	
<code>create_mlp_layers</code> , <code>create_conv_layers</code> , <code>create_transposed_conv_layers</code>	MLP and (transposed) convolution stacks following the conventions of Section 15.1.
<code>create_attention_layer(input_dim, num_heads=8, dropout=0.1)</code>	Batch-first <code>nn.MultiheadAttention</code> .
<code>create_positional_encoding(d_model, max_len)</code>	Frozen sinusoidal table of shape $(1, \text{max_len}, \text{d_model})$; <code>max_len=5000</code> .
<code>get_activation(activation)</code>	Same as the module-level function.
<i>Shape arithmetic</i>	
<code>compute_output_size(n, k, s=1, p=0, d=1)</code>	$\lfloor (n + 2p - d(k - 1) - 1)/s \rfloor + 1$
<code>compute_transposed_output_size(n, k, s=1, p=0, op=0, d=1)</code>	$(n - 1)s - 2p + d(k - 1) + op + 1$
<code>compute_conv_output_size((H, W), layers)</code>	Spatial size after a list of layer dictionaries.

DiscreteEnvironment

An abstract base class for small finite environments, constructed with (`state_space_size`, `action_space_size`, `rng=None`). Subclasses implement `reset()`, which returns the initial state, and `step(action)`, which returns (`state`, `reward`, `done`, `info`).

`update_value(state, value, action, learning_rate)` moves an entry towards a target: $x \leftarrow x + \alpha(\text{value} - x)$. The static methods of `DiscreteTools` implement the classic updates on top of it, with discount γ (`gamma=0.99`) and step size α (`alpha=0.1`). On a terminal transition (`done=True`) the bootstrap term is dropped.

$$\text{Q-learning: } Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)], \quad (15.4)$$

$$\text{SARSA: } Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)], \quad (15.5)$$

$$\text{Expected SARSA: } Q(s, a) \leftarrow Q(s, a) + \alpha\left[r + \gamma \sum_{a'} \pi(a' | s') Q(s', a') - Q(s, a)\right]. \quad (15.6)$$

For expected SARSA, π is a given `PolicyTable` or, if none is given, the epsilon-greedy policy of the Q-table with the given `epsilon`. `value_iteration_update(value_table, state, q_table)` sets $V(s) = \max_a Q(s, a)$ (the Q-table must already hold one-step look-ahead values), and `policy_iteration_update(policy_table, state, q_table)` makes $\pi(\cdot | s)$ greedy with respect to Q .

The exploration rules are static methods of `DiscreteTools` whose first two arguments are the Q-table and the state; they sample with the Q-table's generator.

epsilon_greedy_policy(q, s, epsilon)

A uniformly random action with probability ε , otherwise the greedy one. The corresponding probabilities (with ties sharing the greedy mass) are $\pi(a | s) = \varepsilon/|\mathcal{A}| + (1 - \varepsilon) \mathbf{1}[a \in G_s]/|G_s|$, where G_s is the set of greedy actions.

softmax_policy(q, s, temperature)

Also available as `boltzmann_policy`. Samples from $\pi(a | s) \propto \exp(Q(s, a)/\tau)$, computed in a numerically stable way.

ucb_policy(q, s, visit_counts, exploration_constant=1.0)

UCB1: $\arg \max_a Q(s, a) + c\sqrt{\ln N(s)/N(s, a)}$ with $N(s) = \sum_a N(s, a)$; untried actions are chosen first, at random among themselves. `visit_counts` has the shape of the Q-table.

thompson_sampling_policy(q, s, visit_counts, ...)

A heuristic Beta-Bernoulli Thompson sampler with $p = \text{sigmoid}(Q(s, a))$ and posterior $\text{Beta}(\alpha_0 + np, \beta_0 + n(1 - p))$, $n = N(s, a)$, where α_0 and β_0 are `prior_alpha` and `prior_beta` (default 1). It is exact only for Bernoulli rewards with Q in logit space.

The following example learns to walk right on a six-state chain. The reward is sparse, so the Q-table starts with optimistic values of 1, which makes the greedy policy try every action before settling:

```
import numpy as np
from toolkit.neural_toolkit import DiscreteEnvironment, DiscreteTools

class Chain(DiscreteEnvironment):
    """Chain 0..n-1: action 1 moves right, 0 moves left; reward 1 at the end."""

    def reset(self):
        self.current_state = 0
        return self.current_state
```

```

def step(self, action):
    move = 1 if action == 1 else -1
    last = self.state_space_size - 1
    self.current_state = int(np.clip(self.current_state + move, 0, last))
    done = self.current_state == last
    return self.current_state, float(done), done, {}

env = Chain(state_space_size=6, action_space_size=2, rng=0)
q = DiscreteTools.create_q_table(6, 2, initial_value=1.0, rng=0)  # optimistic
for episode in range(200):
    state = env.reset()
    for t in range(50):
        action = DiscreteTools.epsilon_greedy_policy(q, state, epsilon=0.1)
        next_state, reward, done, _ = env.step(action)
        DiscreteTools.q_learning_update(q, state, action, reward, next_state,
                                         gamma=0.9, alpha=0.5, done=done)

        state = next_state
    if done:
        break
print([q.get_max_action(s) for s in range(5)])  # [1, 1, 1, 1, 1]

```

15.7 Example: a DQN agent for LineMsg

The following script trains a small deep Q-network on the LineMsg environment of Chapter 5 with four agents, treating the 2^4 joint actions as one discrete action. It shows the pieces that a training loop typically needs: action-value selection with `q_net(s, a)`, a target network, an optimizer created by name, gradient clipping and checkpointing. On a laptop CPU it runs in a few seconds and the greedy policy reaches the return of the always-relay strategy (about 65 per episode, compared with about 30 for random actions).

```

from collections import deque

import numpy as np
import torch
import torch.nn.functional as F

import env_lib
from toolkit.neural_toolkit import MLPQNetwork, NetworkUtils

torch.manual_seed(0)
rng = np.random.default_rng(0)
env = env_lib.LineMsgEnv(num_agents=4, max_iter=50)
obs_dim = int(np.prod(env.observation_space.shape))  # 4 agents x 3 cells
n_actions = int(env.action_space.n)  # 2**4 joint actions

q_net = MLPQNetwork(obs_dim, n_actions, hidden_dims=(64, 64))
target_net = MLPQNetwork(obs_dim, n_actions, hidden_dims=(64, 64))
target_net.load_state_dict(q_net.state_dict())
optimizer = NetworkUtils.create_optimizer(q_net, "adam", lr=1e-3)
buffer = deque(maxlen=10_000)

obs, info = env.reset(seed=0)
for step in range(4000):
    epsilon = max(0.05, 1.0 - step / 2000)
    if rng.random() < epsilon:

```

```

        action = int(rng.integers(n_actions))
    else:
        with torch.no_grad():
            action = int(q_net(torch.as_tensor(obs.reshape(1, -1))).argmax())
    next_obs, reward, terminated, truncated, info = env.step(action)
    buffer.append((obs.ravel(), action, reward, next_obs.ravel(), terminated))
    obs = next_obs
    if terminated or truncated:
        obs, info = env.reset()

    if len(buffer) >= 500:
        batch = [buffer[i] for i in rng.integers(len(buffer), size=64)]
        s, a, r, s2, done = (torch.as_tensor(np.array(x)) for x in zip(*batch))
        with torch.no_grad():
            next_q = target_net(s2).max(dim=1).values
            target = r.float() + 0.99 * (1.0 - done.float()) * next_q
        loss = F.smooth_l1_loss(q_net(s, a), target)  # q_net(s, a): Q(s, a), (B,)
        optimizer.zero_grad()
        loss.backward()
        NetworkUtils.clip_grad_norm(q_net, max_norm=10.0)
        optimizer.step()
    if step % 500 == 0:
        target_net.load_state_dict(q_net.state_dict())

NetworkUtils.save_checkpoint(q_net, optimizer, epoch=step, loss=loss.item(),
                             filepath="checkpoints/dqn_linemsg.pt")

```

Evaluating the greedy policy and restoring the checkpoint later:

```

def evaluate(policy, episodes=10):
    returns = []
    for episode in range(episodes):
        obs, info = env.reset(seed=100 + episode)
        total, done = 0.0, False
        while not done:
            obs, reward, terminated, truncated, info = env.step(policy(obs))
            total += reward
            done = terminated or truncated
        returns.append(total)
    return float(np.mean(returns))

restored = MLPQNetwork(obs_dim, n_actions, hidden_dims=(64, 64))
epoch, last_loss = NetworkUtils.load_checkpoint(restored, None,
                                                "checkpoints/dqn_linemsg.pt")

def greedy(obs):
    with torch.no_grad():
        return int(restored(torch.as_tensor(obs.reshape(1, -1))).argmax())

greedy_return = evaluate(greedy)
random_return = evaluate(lambda o: int(rng.integers(n_actions)))
print(f"greedy {greedy_return:.1f}, random {random_return:.1f}")

```

Two larger examples ship with the repository, in `examples/toolkit/` (Chapter 16):

- `simple_transformer_example.py` trains a `TransformerPolicyNetwork` with REINFORCE on a memory task;

- `transformer_rl_example.py` trains transformer actor and critic networks with PPO on batched environments.

15.8 Changes in version 1.0

Version 1.0 fixed several defects that changed the computed functions. The most important ones are:

- The MLP stacks of earlier versions applied no activation between hidden layers, so deep “MLPs” were nearly linear. This affected every policy, value and Q-network, encoder and decoder.
- The positional encoding was indexed by batch position instead of sequence position.
- Convolutional networks with activations other than ReLU crashed; the dueling head was combined incorrectly; Q-networks ignored the `action` argument; `CNNEncoder` failed on 64×64 inputs; `CNNDecoder` indexed its kernel sizes incorrectly; bidirectional RNNs read a backward state that had seen only one time step.
- Unknown activation names now raise instead of falling back silently, and the `device` argument is honoured.
- Tabular tools use a per-table random generator instead of the global one; UCB, softmax and `PolicyTable.set_value` were corrected.

Checkpoint compatibility. Because the layer structure changed (activation modules now sit between the linear layers, so the indices of `nn.Sequential` entries differ), model checkpoints written by versions before 1.0 cannot be loaded into 1.0 networks: `load_state_dict` reports missing and unexpected keys. Even where the parameter shapes happen to match, the networks compute different functions. Retrain models with version 1.0; Appendix B lists the other API changes.

Part IV

Practice

Chapter 16

Examples, Benchmarks and Development

This chapter is a practical companion to the reference chapters. It lists the example scripts that ship with the repository and the `env-lib` command line, shows how to write evaluation loops and record episodes as GIF or MP4 files, explains the benchmark tools, and describes the testing, linting and continuous-integration workflow for contributors.

16.1 Example scripts

The directory `examples/` contains runnable, documented scripts, grouped by topic: `getting_started/` (Table 16.1), `environments/` (Table 16.2), `algorithms/` (Table 16.4) and `toolkit/` (Table 16.3); `examples/README.md` lists them with a typical invocation each. Every script has a `--help` option, a module docstring that explains what it demonstrates, and a `main()` function. Media files are written to `renders/` (relative to the working directory), which Git ignores. Run the scripts from the repository root after installing the package with the extras they need (`pip install ".[all]"` covers all of them).

Table 16.1: First steps (`examples/getting_started/`).

Script	What it shows
<code>quickstart.py</code>	Creates every registered environment with <code>env_lib.make</code> , takes a few random steps and prints the observation shape and return; <code>--render DIR</code> saves one PNG frame per environment.
<code>workflow_demo.py</code>	The workflow of Chapter 11 in six steps: lists the environments with continuous actions, creates a native vector environment, evaluates random actions and the baseline on all copies in parallel, flattens the spaces for single-agent libraries, adapts the environment to the PettingZoo Parallel API and records a GIF of the baseline (<code>--env</code> , <code>--num-envs</code> , <code>--episodes</code> , <code>--save</code> , <code>--no-save</code>).

The environment examples share a set of options: `--seed`, `--steps` (the episode length), `--episodes` (evaluation episodes per policy; `kuramoto_demo.py` and `consensus_demo.py` run a single episode and have no such option), `--save PATH` (record an episode; the suffix `.gif` or `.mp4` selects the format; `power_grid_demo.py` and `platoon_demo.py` call it `--render PATH`, and the former also accepts `--save`), `--render-mode` (human opens a window) and `--theme dark|light`. `workflow_demo.py` takes an environment id (`--env`) instead, and `quickstart.py` only `--steps` and `--render DIR`. The transformer examples, `marl_training_demo.py` and

Table 16.2: Environment examples (`examples/environments/`).

Script	What it shows
<code>kuramoto_demo.py</code>	Drives an oscillator network to synchronisation with a phase-alignment controller and a coupling ramp; NumPy or batched PyTorch backend (<code>--backend</code>), several topologies.
<code>linemsg_demo.py</code>	Compares random, duty-cycled and always-relay policies on LineMsg (<code>--policy</code> selects the one to show or record).
<code>wireless_comm_demo.py</code>	Compares random access, slotted ALOHA and a collision-free rotating schedule on WirelessComm.
<code>pistonball_demo.py</code>	Compares a hand-written heuristic with random actions; <code>--manual</code> enables keyboard control in a window.
<code>consensus_demo.py</code>	Compares random actions with the distributed Laplacian controller for consensus or formation control on any topology.
<code>power_grid_demo.py</code>	Compares no control, random injections and decentralised droop control on PowerGrid-v0 under the same disturbances; reports the return, frequency nadir, largest deviation, settling time and trips (<code>--buses</code> , <code>--topology</code> , <code>--gain</code> , <code>--noise</code>).
<code>platoon_demo.py</code>	Compares random actions, sensor-only ACC and cooperative ACC on Platoon-v0 and prints the analytic string-stability gain of both controllers; reports collisions, the smallest gap and the peak spacing errors of the first and last follower (<code>--vehicles</code> , <code>--topology</code> , <code>--scenario</code> , <code>--headway</code>).
<code>ajlatt_demo.py</code>	Compares random actions with the packaged encircling baseline (<code>env_lib.baseline_policy</code> , implemented by <code>env_lib.baselines.ajlatt_encircle</code> , computed from the observation alone) for multi-robot target tracking and reports tracking error, covariance and collisions.

Table 16.3: Toolkit examples (`examples/toolkit/`).

Script	What it shows
<code>plotkit_gallery.py</code>	The three-panel paper figure of Figure 14.1: learning curves with confidence bands, grouped bars and a sweep heatmap, saved as PDF and PNG.
<code>parakit_demo.py</code>	Saving, loading and applying <code>argparse</code> parameters with validation callbacks; <code>--gui</code> opens the Tk editor, <code>--load</code> reads a file or directory, <code>--set KEY=VALUE</code> overrides one value.
<code>simple_transformer_example.py</code>	A <code>TransformerPolicyNetwork</code> trained with REINFORCE on a small memory task (requires PyTorch).
<code>transformer_rl_example.py</code>	PPO with transformer actor and critic networks on batched environments (requires PyTorch; <code>--device auto</code> uses a GPU when available).

Table 16.4: Learning examples (examples/algorithms/; require PyTorch).

Script	What it shows
marl_training_demo.py	marl_algorithms on the environments (Chapter 12): MAPPO on PowerGrid-v0, MADDPG on Consensus-v0, QMIX on LineMsg-v0 and VDN on WirelessComm-v1, each trained with its tuned preset on batched copies and then compared with random actions and the baseline controller on the same seeded episodes; prints a results table and saves the learning curves (--algo, --env, --quick, --episodes, --plot, --gif, --seed, --threads).
baseline_comparison.py	Your method against the baselines (Section 12.3): a distributed droop controller that also uses the neighbours' mean frequency deviation, compared on PowerGrid-v0 with MAPPO and IPPO trained over two seeds, the classical droop controller and random actions, on the same episodes with marl_algorithms.compare; prints the table and saves it as CSV and a bar chart (--algorithms, --seeds, --episodes, --quick, --csv, --plot, --threads).

baseline_comparison.py accept --quick for a run of a few seconds, used as a smoke test in continuous integration.

```
python examples/getting_started/quickstart.py --render renders/quickstart
python examples/environments/kuramoto_demo.py --topology ring --save
    renders/kuramoto.gif
python examples/environments/linemsg_demo.py --policy relay --save
    renders/linemsg.gif
python examples/environments/wireless_comm_demo.py --policy schedule --save
    renders/wireless.gif
python examples/environments/pistonball_demo.py --episodes 20 --save
    renders/pistonball.gif
python examples/environments/consensus_demo.py --task formation --save
    renders/formation.gif
python examples/environments/power_grid_demo.py --topology ring --render
    renders/grid.gif
python examples/environments/platoon_demo.py --scenario stop_and_go --render
    renders/platoon.gif
python examples/environments/ajlatt_demo.py --map obstacles05 --save
    renders/ajlatt.gif
python examples/getting_started/workflow_demo.py --env Consensus-v0 --num-envs 128
    --episodes 128
python examples/toolkit/plotkit_gallery.py --formats pdf png
python examples/toolkit/simple_transformer_example.py --quick
python examples/algorithms/marl_training_demo.py --algo qmix --env LineMsg-v0
```

The full suite of marl_training_demo.py (without --algo) trains four presets and takes a few minutes on one CPU core; its results are in Table 12.12 and Figure 12.2. With --quick every run takes only a few updates, so the trained returns only show that the pipeline works (times and returns depend on the machine):

```
Training MAPPO on PowerGrid-v0 ...
    2,048 env steps in 4.4 s; trained return -63.31
Training MADDPG on Consensus-v0 ...
    1,504 env steps in 2.3 s; trained return -1.278e+04
Training QMIX on LineMsg-v0 ...
    1,504 env steps in 1.9 s; trained return 82.17
```

```
Training VDN on WirelessComm-v1 ...
1,504 env steps in 1.7 s; trained return 151.6
```

followed by a table with the columns of Table 12.12 (random -629.3 , trained -63.31 and baseline -0.7858 for MAPPO in this quick run) and the path of the learning-curve figure.

Table 16.5 lists the results that the comparison scripts print with their default arguments. They are useful as sanity checks after modifying an environment and as simple baselines; the environment chapters explain the policies.

Table 16.5: Output of the comparison scripts with default arguments (mean episode return, with the standard deviation over episodes where the script prints one).

Script (setting)	Result
<code>linemsg_demo.py</code> (10 agents, 20 episodes)	random 44.1 ± 4.1 , duty-cycled 85.2 ± 3.0 , always relay 95.0
<code>wireless_comm_demo.py</code> (6×6 , 20 episodes)	random 385 ± 12 , ALOHA 398 ± 16 , rotating schedule 911 ± 17 with no collisions
<code>pistonball_demo.py</code> (20 pistons, 5 episodes)	heuristic 889 ± 22 (all episodes reach the goal), random -342 ± 40 (none)
<code>consensus_demo.py</code> (ring, 8 agents)	Laplacian controller solves the task after 99 steps (return -1968); random actions do not (return -19906)
<code>power_grid_demo.py</code> (16 buses, 3 episodes)	droop -0.98 ± 0.10 (nadir -0.019 Hz, settled after 2.95 s); no control -103.86 ± 13.64 (nadir -0.114 Hz, never settled); random -1020.10 ± 726.37 with one trip
<code>platoon_demo.py</code> (8 followers, 5 episodes)	CACC -62.1 without collisions over all 600 steps; ACC -824.6 and random -4824.8 , both colliding in every episode; string-stability gain 1.000 (CACC) and 1.377 (ACC)
<code>ajlatt_demo.py</code> (obstacles04, 2 episodes)	baseline -7333 and -7113 with no collisions and target covariance trace about 2; random actions -34171 and -20871 with 63 and 53 collisions
<code>workflow_demo.py</code> (Formation-v0, 32 copies, 64 episodes)	baseline -1608.2 ± 635.7 , all episodes successful after 92.7 steps on average; random -18076.8 ± 5566.8 , none successful

16.2 The env-lib command line

The console script `env-lib` (equivalently `python -m env_lib`) is the zero-code way to use every environment: it lists and describes them, runs and records an episode of the baseline controller or of random actions, evaluates both with confidence intervals, and benchmarks the single, native vector and `SyncVectorEnv` implementations. Constructor arguments are passed after `--kwargs` as `key=value` pairs. Section 11.3 documents all sub-commands and options.

```
env-lib list --continuous           # continuous-control ids
env-lib describe Platoon-v0        # spaces, layout, parameters, baseline
env-lib run PowerGrid-v0           # one episode of the baseline
env-lib run Platoon-v0 --kwargs scenario=stop_and_go --steps 300 \
  --theme light --gif renders/platoon.gif # record a GIF
env-lib evaluate PowerGrid-v0 --episodes 32 --num-envs 32
env-lib bench Platoon-v0 --num-envs 256 --steps 200
```

The recording command prints a summary of the episode (agent rows shortened here):

```
Platoon-v0 (scenario='stop_and_go')
```

```

policy          baseline: cooperative adaptive cruise control
(env_lib.platoon_env.cacc_policy)
seed            0
episode         300 steps, stopped (--steps) (29.17 s)
team return     -26.8019
agent return    mean -3.3502, min -3.9278 (agent_0), max -2.8442 (agent_7)
  agent         return
  agent_0       -3.9278
  ...
  agent_7       -2.8442
final info      leader_speed=18.9 leader_acceleration=0.0009988 min_gap=11.41
  collision=False breakup=False error_amplification=0.338 step=300 time=30
gif             renders/platoon.gif (301 frames)

```

and the evaluation compares random actions with the baseline on 32 copies simulated in parallel:

```

PowerGrid-v0: 32 episodes per policy, seed 0, 32 parallel copies
policy  mean return      std  95% CI +-  length  terminated  success    s
random   -537.1770  404.5420  145.8530  198.1         6%         -  0.22
baseline   -0.8164   0.6262   0.2258   200.0         0%         -  0.15

```

The last column is the wall-clock time of each evaluation in seconds.

16.3 Writing an evaluation loop

All environments share the Gymnasium interface, so one evaluation function serves all of them. The packaged `env_lib.evaluate` (Section 11.6) does this with confidence intervals and runs vector environments in parallel; writing the loop yourself is useful when an experiment needs other statistics. The loop below evaluates a policy over several seeded episodes and reports the team return and the per-agent returns from `info["agent_rewards"]`. `np.sum` and `np.any` make it work also for AJLATT, whose reward and `terminated` are per-robot arrays.

```

import numpy as np
import env_lib

def evaluate(env, policy, episodes=5, seed=0):
    """Mean team return and mean per-agent returns over seeded episodes."""
    team, per_agent = [], []
    for episode in range(episodes):
        obs, info = env.reset(seed=seed + episode)
        total, agents = 0.0, 0.0
        while True:
            obs, reward, terminated, truncated, info = env.step(policy(obs))
            total += float(np.sum(reward))
            agents = agents + info["agent_rewards"]
            if np.any(terminated) or truncated:
                break
        team.append(total)
        per_agent.append(agents)
    return np.mean(team), np.mean(per_agent, axis=0)

env = env_lib.make("WirelessComm-v1")
env.action_space.seed(0) # reproducible random actions
mean_return, agent_returns = evaluate(env, lambda obs: env.action_space.sample())
print(f"{mean_return:.1f}", agent_returns.shape) # 16 agents on the 4x4 grid

```

```
env.close()
```

`env_lib.baseline_policy(env)` returns the baseline controller of any environment as a function of the observation, which can be passed to this loop directly. Policies that need more than the observation can close over the environment, for example `lambda obs: env.unwrapped.laplacian_policy()` for the consensus environment. Use `env.unwrapped` to reach environment-specific methods and attributes through the wrappers that `gymnasium.make` adds.

16.4 Recording episodes

The module `env_lib.utils` provides two functions to turn rendered frames into animations:

```
record_episode(env, policy=None, path=None, *, max_steps=None, seed=None,
              fps=None, render_every=1) -> list[np.ndarray]
save_animation(frames, path, fps=30.0, *, loop=0) -> Path
```

`record_episode` requires an environment created with `render_mode="rgb_array"`. It resets the environment with `seed`, renders the initial state, then steps with `policy(observation)` (uniform random actions from `env.action_space` when no policy is given) until the episode terminates or is truncated, or `max_steps` steps have been taken, rendering every `render_every`-th step. It returns the list of (H, W, 3) uint8 frames and, if `path` is given, writes them with `save_animation` at `fps` frames per second (default: the environment's `metadata["render_fps"]`).

`save_animation` chooses the format from the file suffix: `.gif` is written with Pillow, which is always available; `.mp4`, `.webm`, `.avi`, `.mov` and `.mkv` require the `video extra` (`imageio` and `imageio-ffmpeg`). For video formats the frames are cropped to even width and height, which most codecs require. Parent directories are created. `loop` is the GIF loop count (0 repeats forever).

```
import env_lib
from env_lib.utils import record_episode, save_animation, set_theme

set_theme("light")          # renderers created from now on use the light theme
env = env_lib.ConsensusEnv(task="formation", formation_shape="wedge",
                           render_mode="rgb_array")
frames = record_episode(env, policy=lambda obs: env.laplacian_policy(),
                       path="renders/formation.gif", seed=0, render_every=2)
save_animation(frames, "renders/formation.mp4", fps=15) # needs the video extra
env.close()
set_theme("dark")          # back to the default theme
```

`set_theme("dark"|"light")` selects the colour theme of all renderers created afterwards: the dark theme (default) suits screens and slides, the light theme suits papers and printed material. Custom themes can be added with `register_theme`.

For full control, for example to record only part of an episode or to annotate frames, collect the frames yourself with `env.render()`:

```
import env_lib
from env_lib.utils import save_animation

env = env_lib.make("KuramotoOscillator-v0", render_mode="rgb_array")
obs, info = env.reset(seed=1)
frames = []
```

```

for step in range(300):
    obs, reward, terminated, truncated, info = env.step(env.action_space.sample())
    if step % 5 == 0:                                # every fifth step
        frames.append(env.render())
    if terminated or truncated:
        break
save_animation(frames, "renders/kuramoto.gif", fps=20)
env.close()

```

GIF files grow quickly with the number of frames and the frame size; rendering every second or fifth step and using MP4 for long episodes keeps the files small. The first `render()` call of an environment builds its figure and takes noticeably longer than later calls.

16.5 Benchmarks

Single environments. The script `benchmarks/benchmark_envs.py` measures the mean wall-clock time of `env.step` (with pre-sampled random actions and no rendering) and the additional time of `env.render()` in "rgb_array" mode, for representative configurations of every environment. Environments whose optional dependencies are missing are reported as skipped. The script sets `MPLBACKEND=Agg` and `SDL_VIDEODRIVER=dummy` itself, so it runs headless.

```

python benchmarks/benchmark_envs.py                    # all environments
python benchmarks/benchmark_envs.py --steps 500 --only kuramoto ajlatt
python benchmarks/benchmark_envs.py --markdown > renders/benchmarks.md

```

Table 16.6: Options of `benchmarks/benchmark_envs.py`.

Option	Default	Meaning
<code>--steps</code>	300	Timed steps per case.
<code>--render-steps</code>	40	Timed rendered steps per case; 0 skips the rendering measurement.
<code>--seed</code>	0	Seed of the environment and of the action space.
<code>--only</code>	all	Subset of the groups <code>kuramoto</code> , <code>linemsg</code> , <code>wireless</code> , <code>pistonball</code> , <code>consensus</code> , <code>ajlatt</code> , <code>power_grid</code> , <code>platoon</code> .
<code>--markdown</code>	off	Print a Markdown table instead of plain text.
<code>--torch-threads</code>	PyTorch default	Number of PyTorch threads for the PyTorch Kuramoto case.

Table 16.7 shows typical results. The “before 1.0” column was measured with the implementations that preceded version 1.0; the version 1.1 columns are the output of `OMP_NUM_THREADS=1 python benchmarks/benchmark_envs.py --torch-threads 1` on one core of an Intel Xeon at 2.8 GHz (Python 3.11, NumPy 2.4). Timings on a shared machine vary by 10 to 20 per cent between runs; the ratios are more informative than the absolute numbers.

Before version 1.0 the Kuramoto "rgb_array" renderer failed with matplotlib 3.10 and later, Pistonball drew its screen on every step even without rendering, and an AJLATT frame took about 110 ms. Rendering now costs more than a simulation step in every environment, which is why it is performed only on request (Section 17.11). The chapters of the environments added in version 1.1 report their step and frame times (Sections 8.7, 8.8, 9.8 and 9.9).

Vector environments. `benchmarks/benchmark_vector.py` measures the throughput of `env_lib.make_vec(id, n)`, once with the native implementation and once with Gymnasium’s

Table 16.7: Time per step and per rendered "rgb_array" frame in milliseconds. A dash marks a configuration that did not exist before version 1.0.

Configuration	Step, before 1.0	Step, 1.1	Frame, 1.1
Kuramoto NumPy, $N = 10$	0.17	0.075	28
Kuramoto NumPy, $N = 50$	2.36	0.093	44
Kuramoto NumPy RK4, $N = 50$	7.1	0.12	53
Kuramoto PyTorch, $N = 50$, 8 systems	10.3	0.45	42
LineMsg, 10 agents	0.019	0.017	16
WirelessComm, 6×6	0.154	0.057	14
WirelessComm, 12×12	0.56	0.064	18
Pistonball, 20 pistons	1.11	0.078	3
Consensus, 8 agents	–	0.082	19
Formation, 32 agents, proximity graph	–	0.28	29
PowerGrid, 16 buses	–	0.25	36
Platoon, 8 followers	–	0.13	38
AJLATT, obstacles04, 4 robots	90	4.7	33
AJLATT, obstacles05, 4 robots	67	4.7	34

`SyncVectorEnv`, for $n \in \{1, 16, 64, 256, 1024\}$ (Sync is skipped above `--sync-max`, default 256) and prints environment steps, agent-steps per second and the speed-up; `--ids` selects environments, `--num-envs` the batch sizes, `--markdown` prints a Markdown table and `--quick` runs a short smoke test. Every case runs at least 300 batched steps (`--min-steps`), so automatic resets are included. `env-lib bench ID` measures the throughput of an environment in environment steps per second: the single environment, its native vector implementation (if any) and `SyncVectorEnv` over single environments (and `AsyncVectorEnv` with `--async`), all stepped with a fixed random action. `--num-envs` sets the number of copies, `--steps` the number of timed steps and `--kwargs` the constructor arguments. Table 11.3 collects reference values, and Section 17.11 discusses batch sizes and threads.

```
env-lib bench Consensus-v0 --num-envs 256 --steps 200
env-lib bench PowerGrid-v0 --num-envs 1024 --kwargs n_buses=64
env-lib bench LineMsg-v0 --num-envs 64 --async      # no native implementation
```

Controllers. `benchmarks/benchmark_baselines.py` evaluates uniformly random actions and the classical controller on every registered id (and on `KuramotoOscillator-v0` with `reward_mode="penalty"` and AJLATT without collision termination and with `reward_mode="bounded"`, Sections 4.4 and 10.4) on the same seeded episodes: the first episode of every copy of a vector environment with `--episodes` copies (64 by default) reset with `--seed` (0); `--only` selects ids and `--markdown` prints a table.

Algorithms. `benchmarks/benchmark_marl.py` trains every preset of `marl_algorithms` (seed 0, one thread) and evaluates random actions, the trained policy and the baseline controller on the same 64 seeded episodes; `--only` selects presets, `--shard K/N` splits the presets between parallel processes, `--out` names the JSON-lines result file and `--report FILE` (with `--markdown`) prints the table. Section 12.8 describes the protocol and reports the results.

16.6 Testing, linting and continuous integration

16.6.1 Running the tests

The test suite lives in `tests/` (`tests/envs/` for the environments, `tests/algorithms/` for `marl_algorithms`, `tests/toolkit/` for the toolkit) and uses `pytest`. After an editable installation with the `dev` extra (Section 2.3):

```
pytest -q                                # whole suite (about two minutes)
pytest tests/envs/test_consensus.py -q    # one file
pytest -k "seed or determinism" -q        # tests whose name matches
pytest --cov=env_lib --cov=marl_algorithms --cov=toolkit \
    --cov-report=term-missing
```

The suite contains 1462 tests, 240 of them for `marl_algorithms`. `tests/conftest.py` sets `MPLBACKEND=Agg` and `SDL_VIDEODRIVER=dummy`, so rendering tests run without a display. Tests of optional features are skipped when the dependency is missing (`pytest.importorskip`). The marker `slow` is reserved for long-running tests and can be deselected with `-m "not slow"`.

The tests check, for every environment, the shapes, dtypes and bounds of the spaces, that observations lie in `observation_space`, determinism under seeding, reward and termination rules against simple reference implementations written in the tests, backward-compatible arguments, the registered ids, rendered frames and, where the multi-agent interface permits it, Gymnasium's `check_env`. For the native vector environments they check that copy 0 reproduces a single environment started from the same state, the three autoreset modes and the info masks; for the workflow layer they check the catalogue, the command line, the baseline controllers, the wrappers (including PettingZoo's `parallel_api_test` when PettingZoo is installed) and the evaluation statistics. For `marl_algorithms` they check the per-agent view of every kind of action space, the runner's final observations and per-agent rewards, generalised advantage estimation against the recursion written out, the replay buffer, the running statistics and the networks (including the monotonicity of the QMIX mixer), and for every algorithm the action shapes and bounds, `policy()` with `env_lib.evaluate`, the rejection of unsupported action kinds and invalid configurations, short training runs with shared and separate parameters, the losses and targets against their definitions, seed determinism, `save/load` round trips, small learning checks, the presets and the `marl-train` command line. For the toolkit they cover input handling and numerical results of `plotkit`, network shapes and the version 1.0 fixes of `neural_toolkit`, and the conversion rules of `parakit`.

16.6.2 Linting and formatting

The code base is formatted and linted with `ruff`; the configuration (line length 100, target Python 3.9, rule sets E, F, W, I, B, UP, NPY) is in `pyproject.toml`.

```
ruff check src tests examples benchmarks    # lint
ruff check --fix src tests examples benchmarks # apply automatic fixes
ruff format src tests examples benchmarks    # format
ruff format --check src tests examples benchmarks # verify formatting only
```

16.6.3 Continuous integration

The workflow `.github/workflows/ci.yml` runs on pushes to the `main` and `development` branches and on pull requests. It has three jobs:

lint

On Python 3.11: `ruff check` and `ruff format --check` on `src`, `tests`, `examples` and `benchmarks`.

package

On Python 3.12: builds the source distribution and the wheel, checks them with `twine check --strict`, installs the wheel into a fresh virtual environment and runs the quick start and `env-lib list` there.

test

On Python 3.9, 3.10, 3.11, 3.12 and 3.13 with `MPLBACKEND=Agg` and `SDL_VIDEODRIVER=dummy`: installs the CPU build of PyTorch and the package with the `pistonball`, `video` and `dev` extras, runs `pytest -q`, and smoke-tests the examples and the command line: `quickstart.py --steps 5`, `simple_transformer_example.py --quick --no-plot`, `workflow_demo.py --no-save`, `power_grid_demo.py --episodes 2 --steps 50`, `platoon_demo.py --episodes 2 --steps 100`, `env-lib list`, `env-lib evaluate Formation-v0 --episodes 8 --num-envs 8`, `benchmarks/benchmark_envs.py --steps 20 --render-steps 2`, `benchmarks/benchmark_vector.py --quick`, `marl-train list`, `marl_training_demo.py --quick --plot` ", `baseline_comparison.py --quick` (one seed, 16 episodes) and `marl-train compare LineMsg-v0` with IQL and VDN on a small budget.

Running the same commands locally before opening a pull request avoids failed CI runs.

16.6.4 Conventions for contributions

Contributions follow the conventions that the existing code relies on:

- Python 3.9 compatible code with `from __future__ import annotations`, type hints on public functions and NumPy-style docstrings (Parameters, Returns, Raises, Examples).
- No global side effects in library code: no `print`, no `logging.basicConfig`, no calls to the global NumPy or PyTorch random generators, no changes to `matplotlib.rcParams` and no `matplotlib.use`. Randomness comes from `self.np_random` or an explicitly owned generator.
- Arguments are validated early with `ValueError` or `TypeError` and actionable messages; user-facing problems are reported with `warnings.warn(..., stacklevel=2)`; diagnostics go to `logging.getLogger(__name__)`.
- Environments follow the contract of Chapter 3: Gymnasium API, `float32` observations inside `observation_space`, `info["agent_rewards"]`, their own episode limit, and no drawing work unless a render mode is set. Renderers subclass `env_lib.utils.MatplotlibRenderer`, build their artists once and use the colours of the active theme.
- Deprecated arguments keep working and emit a `DeprecationWarning` that names the replacement; every behavioural change is recorded in `CHANGELOG.md`.
- Plain ASCII text in code, output and documentation, without emojis or decorative symbols.

A new environment needs the following additions:

1. an entry in `ENV_SPECS` in `src/env_lib/registration.py`, with a string entry point so that registration does not import optional dependencies, and the metadata of Table 3.4 (family, observation and action types, `requires`, `limit_kwarg` and, for a native batched

implementation, `vector_entry_point`); the catalogue and the command line pick the entry up automatically;

2. a lazy export in `src/env_lib/__init__.py`;
3. a baseline controller that is a pure function of the observation, registered in `src/env_lib/baselines.py` so that `baseline_policy` finds it;
4. for a native vector environment, a subclass of `env_lib.utils.BatchedVectorEnv` that accepts the constructor arguments of the single environment plus `num_envs` and `autoreset_mode` (Appendix A);
5. tests in `tests/envs/` and an example script in `examples/environments/`;
6. a chapter in this manual and an entry in `CHANGELOG.md`.

Section 12.10 lists the corresponding steps for a new learning algorithm in `marl_algorithms`.

Chapter 17

Troubleshooting

This chapter collects the problems that users most often run into, grouped by symptom, together with their causes and solutions. If a problem is not covered here, run the failing code with warnings enabled (`python -W default ...`), check the installation with the commands of Section 2.4, and report the issue with the full error message and the output of `pip list` at https://github.com/EricDMW/My_Tool_Box/issues.

17.1 Rendering on machines without a display

Symptom. Rendering fails or hangs on a compute cluster, in a container, over SSH or in continuous integration, often with messages such as `cannot connect to X server`, `no display name` or `TclError`.

Solution. Use `render_mode="rgb_array"`, which draws off-screen and never needs a display: the matplotlib renderers draw on an Agg canvas that is independent of the active matplotlib backend, and the Pistonball renderer draws on pygame surfaces without initialising the display. In addition, select non-interactive back ends for any other plotting code:

```
export MPLBACKEND=Agg           # matplotlib: off-screen rendering
export SDL_VIDEODRIVER=dummy    # pygame: no window system
python train.py
```

The same settings can be made at the top of a script, before matplotlib or pygame is imported:

```
import os

os.environ.setdefault("MPLBACKEND", "Agg")
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
```

Frames returned by `env.render()` can then be recorded to GIF or MP4 (Section 16.4) and inspected later. The test suite, the benchmark script and the CI workflow use exactly these settings.

17.2 No window appears in human mode

Symptom. With `render_mode="human"` nothing is shown, and a `RuntimeWarning` states that the active matplotlib backend is non-interactive.

Cause. "human" mode opens a window through matplotlib, which needs an interactive backend such as TkAgg or QtAgg and a display. When matplotlib falls back to Agg (because no GUI toolkit is installed, no display is available, or `MPLBACKEND=Agg` is set), the environment warns once and renders nothing. Pistonball's window is opened by pygame and needs a display, but no matplotlib backend.

Solution. Check which backend is active:

```
python -c "import matplotlib; print(matplotlib.get_backend())"
```

If it prints `agg` (any case), install a GUI toolkit and select it:

- TkAgg needs Tk: `sudo apt-get install python3-tk` (Debian, Ubuntu), `sudo dnf install python3-tkinter` (Fedora), `brew install python-tk` (Homebrew) or `conda install tk`.
- QtAgg needs a Qt binding: `pip install PyQt6` (or PySide6).

Then unset `MPLBACKEND` or set it explicitly, for example `export MPLBACKEND=TkAgg`. In a Jupyter notebook, "human" mode is not the right tool: render with `"rgb_array"` and display the frames with `matplotlib.pyplot.imshow` or save an animation. Over SSH, either enable X forwarding (`ssh -X`) or record the episode and copy the file.

17.3 Missing optional dependencies

Optional dependencies are imported only when a feature needs them, and the resulting error names the extra that provides them (Table 17.1).

Table 17.1: Errors caused by missing optional dependencies.

Error message (abbreviated)	Fix
PistonballEnv requires pymunk (and pygame for rendering)	<code>pip install ".[pistonball]"</code>
Pistonball rendering requires pygame	<code>pip install ".[pistonball]"</code>
toolkit.neural_toolkit requires PyTorch	<code>pip install ".[torch]"</code>
KuramotoOscillatorEnvTorch requires PyTorch when creating a KuramotoOscillatorTorch-* id	<code>pip install ".[torch]"</code>
Video export requires imageio and imageio-ffmpeg	<code>pip install ".[video]"</code>
The parakit GUI requires Tk (the 'tkinter' module)	system package <code>python3-tk</code> or <code>conda install tk</code> (Section 2.2.4)
No module named 'pytest' or 'ruff'	<code>pip install -e ".[all,dev]"</code>
<code>to_parallel(...)</code> is not an instance of PettingZoo's <code>ParallelEnv</code> (<code>env_lib.wrappers.PETTINGZOO_AVAILABLE</code> is False)	<code>pip install pettingzoo</code> (not an extra; Section 2.2.3)

Note that `pip install ".[dev]"` installs only the development tools; the test suite skips the tests of optional features that are not installed, so a complete test run needs `".[all,dev]"`. If an extra is installed but the error persists, the package was probably installed into a different Python environment than the one running the code; compare `which python` with `pip -V`, and use `python -m pip install ...` to be sure.

17.4 “Out of date” warnings for -v0 environment ids

Symptom. Creating `WirelessComm-v0` or `KuramotoOscillator-v0` with `gymnasium.make` prints

```
DeprecationWarning: WARN: The environment WirelessComm-v0 is out of date.
You should consider upgrading to version `v1`.
```

Cause and solution. `Gymnasium` assumes that a higher version suffix is a newer revision of the same environment. In `env_lib` the suffixes denote preset configurations: `WirelessComm-v1` is the 4×4 grid, not an improved `WirelessComm-v0`. The warning is harmless and can be ignored. `env_lib.make`, which otherwise behaves like `gymnasium.make`, suppresses it. To silence it when calling `gymnasium.make` directly:

```
import warnings
import gymnasium as gym
import env_lib # registers the ids

warnings.filterwarnings("ignore", message=r".*is out of date",
                        category=DeprecationWarning)
env = gym.make("WirelessComm-v0")
```

17.5 AJLATT deprecation warnings

Symptom. Creating an AJLATT environment prints warnings such as AJLATT parameter `'num_Robot'` is deprecated; use `'num_robots'` or parameter `'figID'` no longer has any effect and is ignored.

Cause. The environment is configured by the `AJLATTConfig` dataclass since version 1.0. The legacy keyword names of the former `argparse`-based configuration are still accepted by `AJLATTEnv(...)`, `env_lib.ajlatt_env(...)`, `env_lib.ajlatt_env.make(...)` and `AJLATTConfig.from_kwargs`, but translated with a `DeprecationWarning`.

Solution. Rename the arguments as in Table 17.2; Appendix B shows complete before-and-after examples.

Unknown names raise a `TypeError` in `AJLATTEnv(...)` and `AJLATTConfig.from_kwargs`, and are ignored with a warning by the backward-compatible factory `make`. Python hides `DeprecationWarning` outside the `__main__` module by default; run `python -W default::DeprecationWarning script.py` to see every translation when migrating a code base.

17.6 PyTorch, GPUs and devices

Installing a CPU or a CUDA build. The `torch` extra installs the default PyTorch wheel of the platform, which on Linux is a large CUDA build. For CPU-only machines install the CPU wheel first (`pip install torch --index-url https://download.pytorch.org/whl/cpu`);

Table 17.2: Legacy AJLATT parameters and their replacements.

Legacy argument	Replacement
num_Robot, num_Target	num_robots, num_targets
T_steps	max_episode_steps
maxmum_run	max_episode_steps; the episode is limited to maxmum_run + 1 steps
SIGPERCENT	range_noise_proportional
useupdate	use_update
fix_seed	seed=<int> in the configuration, or <code>env.reset(seed=...)</code>
render=True	render_mode="human" (or "rgb_array")
figID, record, ros, log_dir, episode_num, env_name, ...	no effect; remove them

for a specific CUDA version use the command shown at <https://pytorch.org/get-started/>. Check the result with

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

The Kuramoto PyTorch backend. `KuramotoOscillatorEnvTorch` accepts `device="cpu"`, `"cuda"`, `"cuda:<index>"`, a `torch.device`, or `"auto"`, which selects the first GPU when CUDA is available and the CPU otherwise (the registered id `KuramotoOscillatorTorch-v2` uses `"auto"`). Requesting `"cuda"` on a machine without CUDA raises a `ValueError` that suggests `"auto"` or `"cpu"`. Observations returned by `reset` and `step` are always NumPy arrays; the batched accessors `get_batch_observations()` and `get_batch_rewards()` return tensors on the environment's device. A GPU pays off for large batches of systems; for a single small system the CPU is faster because of the transfer and launch overhead.

neural_toolkit devices. The networks take a `device` argument (default `"cpu"`) that is passed to `Module.to`; `"auto"` is not accepted there. Choose the device explicitly and move the inputs to it:

```
import torch
from toolkit.neural_toolkit import MLPPolicyNetwork

device = "cuda" if torch.cuda.is_available() else "cpu"
policy = MLPPolicyNetwork(8, 4, device=device)
logits = policy(torch.randn(32, 8, device=device))
```

`NetworkUtils.load_checkpoint` loads tensors onto the device of the model by default, so checkpoints written on a GPU load on a CPU-only machine. Checkpoints written before version 1.0 cannot be loaded (Section 15.8).

17.7 Video export

Symptom. Saving a `.mp4` (or `.webm`, `.avi`, `.mov`, `.mkv`) file fails with `Video export requires imageio and imageio-ffmpeg`, or `imageio` reports that it cannot find an `ffmpeg` executable.

Solution. Install the video extra, `pip install "[video]"`. The `imageio-ffmpeg` package (version 0.5.1 or newer; older versions fail with recent `setuptools`) ships its own `ffmpeg` binary, so no system installation is needed. `env-lib run --gif out.mp4` checks for the extra before the rollout. GIF export uses Pillow, a dependency of the base installation, and is the fallback when a video codec is unavailable. `save_animation` rejects other suffixes with a `ValueError`; frames must all have the same shape (H, W, 3) or (H, W, 4).

17.8 Vector environments

A Sync instead of a native vector environment. `env-lib.make_vec` uses a native batched implementation only for the ids that register one: the NumPy KuramotoOscillator-* ids, Consensus-v0, Formation-v0, PowerGrid-v0 and Platoon-v0. For every other id it returns Gymnasium's `SyncVectorEnv`, which steps one Python environment per copy, so its cost grows linearly with `num_envs`. `env-lib list --native` lists the native ids, and `env-lib.get_spec(id).vector_entry_point` is `None` for the others; forcing `vectorization_mode="vector_entry_point"` for such an id raises `gymnasium.error.Error` (Cannot create vectorized environment ... because it doesn't have a vector entry point defined). A `SyncVectorEnv` does not have the methods of the native classes, so that the call `envs.laplacian_policy()`, for example, raises an `AttributeError`; the controller returned by `env-lib.baseline_policy(envs)` works with both kinds of vector environment.

AJLATT in Sync and Async vector environments. `SyncVectorEnv` and `AsyncVectorEnv` store one scalar reward per copy, but AJLATT returns per-robot reward arrays. `env-lib.make_vec` therefore wraps every copy in `TeamReward` (the per-robot rewards stay in `infos["agent_rewards"]`). If you pass your own wrappers, include `TeamReward`; otherwise a step fails with `ValueError: setting an array element with a sequence`. `gymnasium.make_vec` does not add the wrapper.

Where is the last observation of an episode? It depends on the autoreset mode (Table 11.2). With the default "next_step" mode, the observation returned together with `terminated` or `truncated` is the last one of the episode; on the `next_step()` the copy ignores its action, returns reward 0, both flags `False`, the first observation of the new episode and, in `infos`, the rows of its reset info. With "same_step" the returned observation already belongs to the new episode, and the last observation and info are in `infos["final_obs"]` (an object array with `None` for the copies that did not finish) and `infos["final_info"]`, masked by `infos["_final_obs"]` and `infos["_final_info"]`. With "disabled" nothing is reset; reset the finished copies with `envs.reset(options={"reset_mask": terminated | truncated})`. When you store transitions for learning, skip the "next_step" reset step (its observation is not a successor of the previous one); `env-lib.evaluate` does this.

Errors when selecting the autoreset mode. Pass the mode as `env-lib.make_vec(id, n, autoreset_mode="same_step")`; this works for native, Sync and Async environments. Passing `vector_kwargs` to a native environment raises `gymnasium.error.Error` (Custom vector environment can be passed arguments only through kwargs), and `gymnasium.make_vec` with Sync or Async environments accepts only `gymnasium.vector.AutoresetMode` members or their values ("SameStep"), not "same_step". Gymnasium 1.0 has no `AutoresetMode`, and its Sync and Async environments always use next-step autoreset.

Copies do not match single environments. A native vector environment draws the initial states of all copies from one generator, so copy i does not reproduce a single environment reset with seed $s + i$, whereas `SyncVectorEnv` seeds copy i with $s + i$. To compare a copy with a single environment, start both from the same state through the reset options (Table 3.7), or use `vectorization_mode="sync"`.

ResetNeededError from a partial first reset. The first `reset()` of a native vector environment must reset every copy; `reset(options={"reset_mask": mask})` with some entries `False` before that raises `ResetNeededError`.

17.9 Warnings about unknown reset options

Symptom. `reset()` prints a `UserWarning` such as `ConsensusEnv.reset(): ignoring unknown reset option(s) ['position']; supported: ['positions', 'velocities']`.

Cause. Since version 1.1 the environments with reset options (Kuramoto, Consensus, PowerGrid, Platoon and their vector versions) ignore keys they do not know, with this warning, instead of raising `ValueError`; the values of known keys are still validated strictly (Section 3.5.6). This lets generic tools pass options of their own, as PettingZoo's `parallel_api_test` does.

Solution. Check the key against the supported ones printed in the message: a misspelt key, as in the example, means that the option had no effect and the initial state was sampled as usual. If a tool passes options on purpose, silence only its keys, for example `warnings.filterwarnings("ignore", message=r".*reset option.*'options'", category=UserWarning)`. `LineMsg`, `WirelessComm`, `Pistonball` and `AJLATT` have no reset options and ignore options without a warning.

17.10 Reproducibility and seeding

The environments are deterministic given their seeds and never use global random state, but a complete experiment contains several independent sources of randomness. Seed each of them:

- *Environment.* Call `env.reset(seed=s)` once at the start of a run; later calls to `env.reset()` without a seed continue the same random stream, so a whole sequence of episodes is reproducible. Resetting with the same seed every episode repeats the same initial state.
- *Action sampling.* `env.action_space.sample()` uses the space's own generator, which `reset` does not seed. Call `env.action_space.seed(s)`. This also applies to `record_episode` without a policy.
- *Construction-time randomness.* Some environments draw random structure in the constructor, independent of `reset`: the Erdős-Rényi graph of `ConsensusEnv` (`graph_seed`; with the default `None` two instances may differ, even in their observation size), the random topology of the Kuramoto environments (`topology_seed=42`), the PowerGrid network (`network_seed=0`) and the Platoon vehicles (`vehicle_seed=0`); with `network_seed=None` or `vehicle_seed=None` every instance draws fresh entropy. `AJLATT` accepts a default seed in its configuration (`seed=...`), used by the first `reset()` without a seed.

- *Vector environments.* `envs.reset(seed=s)` makes a batch reproducible; the copies of a native vector environment share one generator (Section 17.8).
- *Networks.* Call `torch.manual_seed(s)` before creating networks and before training. `VariationalEncoder` samples its noise from PyTorch’s default generator. On GPUs some operations are non-deterministic; see the PyTorch documentation on reproducibility (`torch.use_deterministic_algorithms`).
- *Tabular tools.* Pass `rng=s` to the tables; each table owns its generator.
- *Your own code.* Use a `numpy.random.default_rng(s)` generator instead of the global `np.random` functions.

Seeded trajectories are reproducible for a fixed version of the package and its dependencies. Across versions, results can change where bugs were fixed; Appendix B lists these changes. The AJLATT environment can reproduce its behaviour before version 1.0 closely with `ci_solver="slsqp"`.

17.11 Performance

- *Render only when needed.* Rendering a frame costs 3 to 36 ms, which is far more than a simulation step (Table 16.7). Train with `render_mode=None`, and record evaluation episodes separately, with `render_every` greater than one for long episodes. "human" mode additionally limits the frame rate, so never train in "human" mode.
- *Create environments once.* Construction loads maps, builds graphs and allocates buffers; reuse an environment across episodes with `reset()` instead of creating a new one per episode. The first `render()` call builds the figure and is slower than later calls.
- *Choose the integrator.* For the Kuramoto environments, Euler integration is about twice as fast as RK4; use RK4 when accuracy at large `dt` matters.
- *Batch on the GPU.* The PyTorch Kuramoto environment simulates `n_agents` systems per step. One batched environment is much cheaper than many separate ones, especially on a GPU.
- *AJLATT.* Keep the default `ci_solver="newton"`, which is several times faster than "slsqp"; the cost per step grows with the number of robots because every robot fuses its neighbours’ estimates.
- *Small networks on the CPU.* PyTorch uses several threads by default. For the small networks typical of these environments, and when several training processes share a machine, thread contention can make training much slower. `marl_algorithms.train`, `train_preset` and `compare` already use one thread (`threads=1`); elsewhere limit the threads with `marl_algorithms.torch_threads(1)`, `torch.set_num_threads(1)` or the environment variable `OMP_NUM_THREADS=1` and `compare`.
- *Simulate batches natively.* For many copies of an environment with a native vector implementation, `env_lib.make_vec` advances all of them with one set of array operations; a `SyncVectorEnv` steps them one by one, and an `AsyncVectorEnv` pays inter-process communication on every step, which only an expensive step (AJLATT, Pistonball) outweighs.
- *Choose the batch size by measurement.* The throughput of a native vector environment (environment steps per second) grows with the number of copies while the fixed cost per call dominates, then levels off and can drop once the arrays no longer fit into the CPU caches: for PowerGrid with 128 buses, 1024 copies gave a lower throughput than 64 (Table 8.7; Platoon behaves similarly, Table 9.8). Run `env-lib bench ID --num-envs B` for a few

values of `B` with your constructor arguments (`--kwarg`) and pick the smallest batch near the maximum.

- *Pin the BLAS threads.* Some kernels of the native vector environments use matrix products (the coupling terms of Kuramoto and of PowerGrid with small networks), which NumPy passes to its BLAS library; depending on the installation, BLAS starts one thread per core. For the small matrices of these environments the threads bring little, and when several processes share a machine (parallel runs, `AsyncVectorEnv` workers) they compete for the cores. Start Python with `OMP_NUM_THREADS=1` (for OpenBLAS also `OPENBLAS_NUM_THREADS=1`, for MKL `MKL_NUM_THREADS=1`) and compare with `env-lib bench`; pinned threads also make timings more reproducible.
- *Measure.* `env-lib bench` measures the throughput of the single, native vector and Sync (and Async) implementations, `benchmarks/benchmark_envs.py` (Section 16.5) measures step and frame times on your machine, and `python -m cProfile -s cumtime script.py` shows where time goes in your own code.

17.12 Training with marl_algorithms

ImportError: “requires PyTorch”. The algorithms need PyTorch, which the base installation does not include (`marl-train` reports “PyTorch is not installed” and exits with status 1). Install the extra, `pip install "my-tool-box[torch]"`, or a PyTorch build of your choice (Section 2.2.2).

TypeError when creating an algorithm. Every method supports one kind of action (Table 12.1): MADDPG and MATD3 need continuous actions, IQL, VDN and QMIX discrete ones, while IPPO and MAPPO accept both. A joint `Discrete` action over several agents, such as `LineMsg` with its default `action_space_type`, is not a per-agent action and is rejected; `make_vector_env` therefore creates `LineMsg` with `action_space_type="multibinary"`. Pass the same argument when you build a `LineMsg` environment yourself.

ValueError: VectorRunner needs same-step autoreset. The training loops bootstrap from the true last observation of an episode, which a vector environment only provides in same-step mode. Create the environment with `marl_algorithms.make_vector_env` or pass `autoreset_mode="same_step"` to `env-lib.make_vec`.

marl-train reports an unknown algorithm, environment or field. `marl-train` checks its arguments before training and exits with status 2 and a message on standard error. Algorithm names are those of `marl-train list`, environment ids those of `env-lib list`, and the keys of `--set` the fields of the algorithm’s configuration class (Section 12.2.2).

Results differ from the published ones. A run is determined by its seed, its configuration and the software it runs on: with the same seed and PyTorch thread count, the same library versions and the same machine, training repeats exactly. Other PyTorch or BLAS builds, another number of threads or another processor change the rounding of floating-point sums, and reinforcement learning amplifies such differences, so the returns in Section 12.8 are reproduced to within the variation between seeds rather than digit for digit. Compare several seeds before concluding that a change helps.

Training is slower than the reported times. The published times use one PyTorch thread, the default of `train`, `train_preset`, `compare`, `marl-train` and the demonstration script. A direct call of `algo.learn` uses PyTorch's setting; wrap it in `with marl_algorithms.torch_threads(1):`. With PyTorch's default thread count, the small networks of these methods often train more slowly, especially when several runs share a machine.

The return does not improve. Start from a preset (`marl-train presets`), which fixes the learning rates, reward scale and exploration schedule for its environment; the defaults of the configuration classes are generic. Without a preset, normalise the observations, scale the rewards so that the team return per step is of order one, and check the learning curve of `TrainingLog.curve` before increasing `total_steps`. An off-policy method whose budget ends before `warmup_steps` warns that it takes no gradient step at all.

The agent learns to end episodes early (Kuramoto, AJLATT). Two families reward a behaviour other than the intended one, in their default reward settings, if a learner maximises the return literally. The Kuramoto ids other than the `FreqSync` ones pay the order parameter $r \in [0, 1]$ at every step (`reward_mode="dense"`) and end the episode with `sync_bonus` (10) once r exceeds `sync_threshold` (0.99), so an agent that stays just below the threshold for the whole episode (up to `max_steps`, 1000) earns far more than one that synchronises. AJLATT pays negative rewards (`reward_mode="cost"`) and ends the episode at the first collision, so an agent can avoid further penalties by colliding. The registered ids keep these defaults for compatibility; `env-lib describe` prints a caution. For training, choose another reward mode: for Kuramoto `reward_mode="penalty"` (synchronising sooner earns more), `"progress"` or `"terminal"`, or `terminate_on_sync=False` (Section 4.4); for AJLATT `reward_mode="bounded"` or `terminate_on_collision=False`, which keeps the collision penalties (Section 10.4). With `marl_algorithms`, pass them as `env_kwargs`. Evaluate with the metric you care about (`info["synchronized"]`, `info["tracking_cost"]`) rather than the return alone.

ValueError when evaluating or loading an algorithm. `algo.evaluate` and `marl-train evaluate` check that the environment has the number of agents, observation size and action space the algorithm was trained for, and name the difference if not; a model trained on a variant (for example `n_buses=32`) is evaluated on the same variant. Checkpoints record the environment id and keyword arguments of `train` in `algo.metadata`, and `marl-train evaluate` prints a note when they differ from the ones given. `Algorithm.load` raises `FileNotFoundError` for a missing path and `ValueError` for a file that is not a checkpoint of this package; load only files you trust, since checkpoints are pickled. `device="cuda"` (or `"mps"`) without such a device raises `ValueError` when the algorithm is created.

17.13 Other common errors

`step()` raises before the first `reset()`

Every environment, including the native vector environments, requires `reset()` before the first `step()`. Otherwise it raises `env_lib.ResetNeededError`, which can be caught either as a `RuntimeError` or as Gymnasium's `ResetNeeded` error. Environments created with `gymnasium.make` raise Gymnasium's own `ResetNeeded` from the `OrderEnforcing` wrapper.

An episode is longer or shorter than expected

Each environment enforces its own episode limit through a constructor argument (`max_steps`, `max_iter`, `max_cycles` or `max_episode_steps`; `env_lib.get_spec(id).limit_kwarg`

names it). Pass it to the constructor, or pass `max_episode_steps` to `env_lib.make` or `env_lib.make_vec`, which set the same argument; giving both with different values raises `ValueError`. Only `gymnasium.make` called directly with `max_episode_steps` (and `gymnasium.make_vec` in the Sync or Async mode) adds a second `TimeLimit` wrapper, and the shorter of the two limits applies; `gymnasium.make_vec` passes the argument on to a native vector class, which rejects it with `TypeError`. In version 1.0, `env_lib.make` behaved like `gymnasium.make` here, and for AJLATT the value did not reach the environment at all (Section B.2).

AttributeError for an environment method

`gymnasium.make` and `env_lib.make` return a wrapped environment. Methods and attributes that only the environment class defines are reached through `env.unwrapped`; for example, the consensus controller is `env.unwrapped.laplacian_policy()`. The function `env_lib.baseline_policy` needs no such access: it accepts wrapped, flattened and vectorised environments.

TypeError from baseline_policy

Keyword arguments set the gains of the controller of the environment's family; an unknown name raises `TypeError` with the list of valid names (`env-lib` `baselines` lists the controllers). A wrapper that changes the spaces beyond a reshape, or an environment that is not part of `env_lib`, raises `TypeError` as well.

render() returns None with a warning

The environment was created without a render mode. Pass `render_mode="rgb_array"` (or `"human"`) to the constructor or to `make`.

Actions are rejected

Actions must match `env.action_space` in shape. Pistonball in discrete mode expects one value in $\{0, 1, 2\}$ per piston, not a single integer; AJLATT expects an array of shape `(num_robots, 2)`. Appendix B describes the changes from earlier versions.

The parakit window does not open

`ParameterTuner.tune()` raises `ImportError` without Tk and `RuntimeError` without a display; use the headless API (Section 13.2.6).

Figures look different from the examples

`plotkit` applies its style locally and does not change global settings. If other code changed `matplotlib.rcParams` (or called `set_research_style` without `reset_style`), the preset is applied on top of those settings; call `matplotlib.rcParamsdefaults()` to start from the `matplotlib` defaults.

Appendix A

API Reference

This appendix lists the public classes and functions of version 1.2 with their signatures, grouped by module. Signatures omit type annotations; defaults are shown as in the source code. The chapters give the semantics, and the docstrings (`help(obj)` in Python) give full details. Names that start with an underscore and modules not listed here are internal and may change without notice; the one exception are the documented hooks of `BatchedVectorEnv` (Section A.5.4), which subclasses implement.

A.1 `env_lib`

A.1.1 Package functions

```
env_lib.make(env_id, **kwargs) -> gym.Env
env_lib.make_vec(env_id, num_envs=1, vectorization_mode=None, *,
                 autoreset_mode=None, vector_kwargs=None, wrappers=None,
                 **kwargs) -> gym.vector.VectorEnv
env_lib.get_spec(env_id) -> EnvSpec
env_lib.list_envs() -> list[str]
env_lib.register_envs()
env_lib.__version__           # "1.2.1"
env_lib.ResetNeededError     # exception class
```

`make` creates a registered environment (a thin wrapper around `gymnasium.make` that suppresses the “out of date” warning of `-v0` ids); keyword arguments go to the environment constructor. `max_episode_steps=N` sets the environment’s own limit argument (`EnvSpec.limit_kwarg`) instead of adding a `TimeLimit` wrapper; it raises `ValueError` if the limit argument is also given with a different value. `make_vec` creates `num_envs` copies as one `Gymnasium` vector environment with the same keyword arguments: `vectorization_mode=None` selects the native batched implementation (`EnvSpec.vector_entry_point`) when there is one and `gymnasium.vector.SyncVectorEnv` otherwise, and `"sync"`, `"async"` and `"vector_entry_point"` force a mode. `autoreset_mode` (`"next_step"`, `"same_step"`, `"disabled"`, `Gymnasium`’s spellings or `AutoresetMode` members) works for every mode. `vector_kwargs` (further arguments of the `Sync` or `Async` class) and `wrappers` (applied to every copy) are only accepted by the `Sync` and `Async` modes; for environments with per-agent reward arrays (`EnvSpec.per_agent_rewards`, `AJLATT`) `TeamReward` is applied when no `wrappers` are given. `get_spec` returns the registration record of an id and raises `KeyError` for an unknown one. `list_envs` returns the 18 registered ids in the order of Table A.1. `register_envs` registers all ids with `Gymnasium`; it runs automatically on `import env_lib` and is idempotent. `ResetNeededError` is raised by every environment, including the

native vector environments, when `step()` or `render()` is called before `reset()`; it subclasses both `RuntimeError` and `gymnasium.error.ResetNeeded`. The environment classes below, and `baseline_policy`, `evaluate`, `rollout`, `describe` and `EnvInfo`, are exported lazily by `env_lib`, so optional dependencies are imported only when a name is used.

A.1.2 Registry records

Module `env_lib.registration`; Section 3.4.

```
EnvSpec(id, entry_point, kwargs={}, description='', family='',
        observation_type='continuous', action_type='continuous',
        requires=None, vector_entry_point=None, limit_kwarg='max_steps',
        per_agent_rewards=False, disable_env_checker=False) # frozen dataclass
ENV_SPECS # list[EnvSpec] in list_envs() order
```

Table 3.4 describes the fields. `EnvSpec` is not Gymnasium’s class of the same name, which `env.spec` returns.

Table A.1: Registered environment ids, their preset arguments and the vector implementation used by `make_vec`.

Id	Class and preset arguments	Vector
KuramotoOscillator-v0	KuramotoOscillatorEnv() (10 oscillators)	native
KuramotoOscillator-v1	n_oscillators=5	native
KuramotoOscillator-Constant-v0	n_oscillators=6, coupling_mode="constant"	native
KuramotoOscillator-FreqSync-Constant-v0	as above, reward_type="frequency_synchronization", target_frequency=2.0	native
KuramotoOscillatorTorch-v0	KuramotoOscillatorEnvTorch() (10 oscillators)	sync
KuramotoOscillatorTorch-v1	n_oscillators=8, n_agents=4, device="cpu"	sync
KuramotoOscillatorTorch-v2	n_oscillators=10, device="auto", integration_method="rk4"	sync
KuramotoOscillatorTorch-Constant-v0	n_oscillators=6, coupling_mode="constant"	sync
KuramotoOscillatorTorch-FreqSync-Constant-v0	as above, frequency synchronisation, target_frequency=2.0	sync
LineMsg-v0	LineMsgEnv()	sync
WirelessComm-v0	WirelessCommEnv() (6 × 6 grid)	sync
WirelessComm-v1	grid_x=4, grid_y=4	sync
Pistonball-v0	PistonballEnv()	sync
Consensus-v0	ConsensusEnv()	native
Formation-v0	ConsensusEnv(task="formation")	native
PowerGrid-v0	PowerGridEnv() (16 buses)	native
Platoon-v0	PlatoonEnv() (8 followers)	native
AJLATT-v0	AJLATTEnv() (Gymnasium’s passive environment checker disabled)	sync

A.1.3 Common environment interface

Every environment class implements the Gymnasium methods

```

reset(*, seed=None, options=None) -> tuple[observation, dict]
step(action) -> tuple[observation, reward, terminated, truncated, dict]
render() -> np.ndarray | None          # (H, W, 3) uint8 in "rgb_array" mode
close()                                # idempotent

```

and has the attributes `action_space`, `observation_space`, `render_mode`, `metadata` and `np_random`. The info dictionary always contains "agent_rewards", a float64 array of shape (n_agents,). Unknown keys of options are ignored with a UserWarning (Section 3.5.6). Chapter 3 describes the conventions.

The native vector environments (`KuramotoOscillatorVectorEnv`, `ConsensusVectorEnv`, `PowerGridVectorEnv`, `PlatoonVectorEnv`) implement the Gymnasium vector API through `BatchedVectorEnv`:

```

reset(*, seed=None, options=None) -> tuple[observations, dict]
step(actions) -> tuple[observations, rewards, terminations, truncations, dict]
render() -> np.ndarray | None          # frame of copy 0
close()

```

with the attributes `num_envs`, `single_observation_space`, `single_action_space`, `observation_space`, `action_space` (batched), `autoreset_mode`, `render_mode`, `metadata` and `np_random` (one generator for all copies). `options` accepts "reset_mask" and the reset options of the single environment. Their constructors take the keyword arguments of the single environment plus `num_envs` and `autoreset_mode`. Rewards have shape (num_envs,); every info value has a leading num_envs dimension and a mask "_key"; infos["agent_rewards"] has shape (num_envs, n_agents).

A.1.4 Kuramoto oscillator classes

Module `env_lib.kos_env`; Chapter 4.

```

KuramotoOscillatorEnv(n_oscillators=10, n_agents=1, dt=0.01, max_steps=1000,
                      coupling_range=(0.0, 5.0), control_input_range=(-1.0, 1.0),
                      natural_freq_range=(0.5, 2.0), device='cpu',
                      render_mode=None, integration_method='euler',
                      reward_type='order_parameter', noise_std=0.0,
                      topology='fully_connected', adj_matrix=None,
                      target_frequency=1.0, coupling_mode='dynamic',
                      constant_coupling_matrix=None, coupling_strength=1.0,
                      normalize_coupling=False, topology_seed=42,
                      sync_threshold=0.99, sync_bonus=10.0)

KuramotoOscillatorEnvTorch(...same arguments..., device='cpu', render_agent=0)

KuramotoOscillatorVectorEnv(num_envs=1, *, autoreset_mode='next_step',
                             ...keyword arguments of KuramotoOscillatorEnv...)

```

`KuramotoOscillatorEnv` simulates one network with NumPy. `KuramotoOscillatorEnvTorch` simulates `n_agents` independent systems in a batch on device ("cpu", "cuda", "cuda:<k>", a `torch.device` or "auto"); `render_agent` selects the rendered system. Additional methods of the PyTorch class:

Method	Description
<code>get_batch_observations()</code>	Tensor of the observations of all systems, (<code>n_agents</code> , <code>obs_dim</code>), on the device.
<code>get_batch_rewards(dphases_dt=None, include_bonus=False)</code>	Per-system rewards of the current state, (<code>n_agents</code> ,).

`KuramotoOscillatorVectorEnv` (module `env_lib.kos_env.vector`) is the native vector environment of the NumPy ids: `observations` (`num_envs`, `obs_dim`), `actions` (`num_envs`, `action_dim`). It copies the configuration attributes of the single environment (`n_oscillators`, `coupling_indices`, `topology_matrix`, ...) and exposes the batched state as the read-only properties `phases`, `natural_frequencies`, `control_inputs` (each (`num_envs`, `N`)), `coupling_strengths` ((`num_envs`, `M`); `None` in constant mode) and `step_count` ((`num_envs`,)). `env_lib.kos_env.register()` is deprecated; registration happens on import.

A.1.5 LineMsgEnv and WirelessCommEnv

Modules `env_lib.linemsg_env` and `env_lib.wireless_comm_env`; Chapter 5.

```
LineMsgEnv(num_agents=10, n_obs_neighbors=1, max_iter=50, render_mode=None,
           action_space_type='discrete')

WirelessCommEnv(grid_x=6, grid_y=6, ddl=2, packet_arrival_probability=0.8,
                 success_transmission_probability=0.8, n_obs_neighbors=1,
                 max_iter=50, render_mode=None)
```

Both support the render modes "human", "rgb_array" and "ansi" (text). `seed(seed=None)` is kept for backward compatibility and deprecated in favour of `reset(seed=...)`. `WirelessCommEnv` also exposes `access_point_mapping(i, j, agent_action)`, which returns the access point (row, column) used by agent (`i`, `j`) (or (`None`, `None`)), and `check_transmission_fail(ap_x, ap_y, access_point_profile)`, which is `True` unless exactly one agent uses the access point. `env_lib.wireless_comm_env.register()` is deprecated.

A.1.6 PistonballEnv

Module `env_lib.pistonball_env`; Chapter 6. Requires the `pistonball` extra.

```
PistonballEnv(n_pistons=20, time_penalty=-0.1, continuous=True, random_drop=True,
               random_rotate=True, ball_mass=0.75, ball_friction=0.3,
               ball_elasticity=1.5, max_cycles=125, render_mode=None,
               movement_penalty=0.0, movement_penalty_threshold=0.01, kappa=1,
               terminated_condition=True, leftmost_piston_reward=0.0,
               termination_reward=0.5)

ManualPolicy(env, agent_id=0, show_obs=False)
```

Member	Description
<code>observable_mask(ball_x=None)</code>	Boolean (<code>n_pistons</code> ,) mask of the pistons within <code>kappa</code> of the piston under the ball.
<code>get_local_reward_for_piston(i, prev_x, curr_x)</code>	Reference implementation of one piston's local reward.
<code>get_local_reward(prev_position, curr_position)</code>	The ball term of the local reward.
<code>renderer</code>	The <code>PistonballRenderer</code> , or <code>None</code> before the first <code>render()</code> .
<code>seed(seed=None)</code>	Deprecated; use <code>reset(seed=...)</code> .
<code>draw()</code> , <code>draw_background()</code> , <code>draw_pistons()</code> , <code>enable_render()</code>	Deprecated no-ops kept for compatibility; use <code>render()</code> .

`ManualPolicy` is a keyboard controller for "human" mode: calling it returns a joint action in which the selected piston follows the keys (W/S or Up/Down move it, A/D or Left/Right select another piston); its `running` attribute becomes `False` when the user presses Escape or closes the window. The module also exports `PistonballLayout` (screen geometry) and `PistonballRenderer` (the pygame renderer).

A.1.7 ConsensusEnv and ConsensusVectorEnv

Module `env_lib.consensus_env`; Chapter 7.

```
ConsensusEnv(*, n_agents=8, task='consensus', topology='ring', dynamics='single',
             dt=0.1, max_steps=200, arena_size=10.0, max_control=1.0,
             control_cost=0.01, noise_std=0.0, tolerance=0.05, success_bonus=10.0,
             formation_shape='circle', formation_radius=3.0, comm_radius=4.0,
             edge_probability=0.3, graph_seed=None, max_neighbors=None,
             damping=0.5, render_mode=None)

ConsensusVectorEnv(num_envs=1, *, autoreset_mode='next_step',
                  ...keyword arguments of ConsensusEnv...)
```

Member	Description
<code>laplacian_policy(gain=1.0, velocity_gain=None)</code>	Distributed feedback $u = -\text{gain } L(x - d)$ (plus velocity feedback for double integrators), clipped to the action bounds.
<code>positions</code> , <code>velocities</code>	Copies of the current states, (<code>n_agents</code> , 2).
<code>formation_offsets</code>	Offsets d_i , (<code>n_agents</code> , 2) (zeros for consensus).
<code>adjacency</code> , <code>laplacian</code>	Current communication graph and its Laplacian $L = D - A$.
<code>algebraic_connectivity</code>	Fiedler value of the current Laplacian.
<code>task_error</code> , <code>step_count</code>	Current task error and step within the episode.
<code>observation_layout</code>	Column slices of one observation row.

`ConsensusVectorEnv` (module `env_lib.consensus_env.vector`) is the native vector environment of Consensus-v0 and Formation-v0. It has the same members with a leading `num_envs` dimension (`positions` (`num_envs`, `n_agents`, 2), `adjacency` (`num_envs`, `n_agents`, `n_agents`), `task_error` (`num_envs`,), ...; `formation_offsets` is shared), and `laplacian_policy()` returns the actions of all copies, (`num_envs`, `n_agents`, 2).

Module-level helpers and constants:

```
make_topology(topology, n_agents, *, edge_probability=0.3, rng=None,
              max_attempts=1000) -> np.ndarray      # adjacency of a static graph
proximity_adjacency(positions, comm_radius) -> np.ndarray  # disk graph
make_formation(shape, n_agents, radius=1.0) -> np.ndarray  # offsets  $d_i$ 
algebraic_connectivity(adjacency) -> float  # second smallest eigenvalue of  $L$ 
is_connected(adjacency) -> bool

TOPOLOGIES = ('ring', 'line', 'star', 'complete', 'erdos_renyi', 'proximity')
TASKS = ('consensus', 'formation')
DYNAMICS = ('single', 'double')
FORMATION_SHAPES = ('circle', 'line', 'grid', 'wedge')
```

A.1.8 PowerGridEnv and PowerGridVectorEnv

Module `env_lib.power_grid_env`; Chapter 8.

```
PowerGridEnv(*, n_buses=16, topology='small_world', topology_kwargs=None,
             network_seed=0, randomize_network=False, inertia_range=(2.0, 8.0),
             damping_range=(2.0, 6.0), susceptance_range=(0.3, 0.8),
             line_loading=0.5, nominal_frequency=50.0, u_max=0.1, dt=0.05,
             substeps=2, max_steps=200, initial_disturbances=1,
             disturbance_rate=0.1, disturbance_magnitude=(0.05, 0.2),
             load_increase_prob=0.5, noise_std=0.0, noise_tau=1.0,
             frequency_limit=0.5, w_freq=1.0, w_rocof=0.1, w_u=10.0,
             trip_penalty=100.0, neighbourhood=1, render_mode=None)

PowerGridVectorEnv(num_envs=1, *, autoreset_mode='next_step', render_mode=None,
                  **kwargs)      # kwargs: the arguments of PowerGridEnv

droop_policy(observation, gain=DEFAULT_DROOP_GAIN) -> np.ndarray

TOPOLOGIES          # the topologies of env_lib.utils.graphs.make_graph
OBSERVATION_FEATURES # ('omega', 'rocof', ..., 'capacity', 'inertia')
OBSERVATION_SCALE    # feature -> fixed scale factor
DEFAULT_DROOP_GAIN = 0.25
```

Member of <code>PowerGridEnv</code>	Description
<code>observation_layout</code> , <code>observation_scale</code>	Column slice and scale factor of every feature.
<code>adjacency</code> , <code>edges</code> , <code>susceptance</code>	Line graph (n, n) , lines $(E, 2)$, weighted adjacency B_{ij} .
<code>inertia</code> , <code>inertia_constants</code> , <code>damping</code>	M_i , H_i and D_i .
<code>nominal_injections</code> , <code>equilibrium</code>	P_i^0 and the equilibrium angles θ^* .
<code>u_max</code> , <code>omega_limit</code>	Control bounds and trip limit (rad/s).
<code>theta</code> , <code>omega</code> , <code>disturbance</code> , <code>line_flows</code>	Current state, ΔP_i and line flows.
<code>time</code> , <code>step_count</code> , <code>positions</code>	Simulated time, steps, plotting layout.
<code>n_buses</code> , <code>n_agents</code> , <code>obs_dim</code> , <code>max_steps</code> , <code>dt</code> , <code>config</code>	Configuration.

`PowerGridVectorEnv` exposes `observation_layout`, `observation_scale`, `u_max` and, batched over the copies, `adjacency`, `inertia`, `damping`, `nominal_injections`, `equilibrium`, `theta`,

omega and disturbance. `droop_policy` computes $u_i = \text{clip}(-k\omega_i, -u_{\max,i}, u_{\max,i})$ from each agent's observation row for any leading batch shape.

A.1.9 PlatoonEnv and PlatoonVectorEnv

Module `env_lib.platoon_env`; Chapter 9.

```
PlatoonEnv(*, n_followers=8, topology='predecessor', scenario='mixed', dt=0.1,
           max_steps=600, headway=0.6, standstill_distance=2.0, accel_min=-6.0,
           accel_max=3.0, tau_range=(0.2, 0.4), length_range=(4.0, 5.0),
           vehicle_seed=0, randomize_vehicles=False, init_spacing_noise=0.1,
           init_speed_noise=0.05, spacing_weight=1.0, speed_weight=0.1,
           control_weight=0.02, jerk_weight=0.01, collision_penalty=500.0,
           breakup_distance=100.0, breakup_penalty=250.0, render_mode=None)

PlatoonVectorEnv(num_envs=1, *, autoreset_mode='next_step', render_mode=None,
                 **kwargs)           # kwargs: the arguments of PlatoonEnv

cacc_policy(obs, *, k_p=CACC_GAINS['k_p'], k_d=CACC_GAINS['k_d'],
            k_a=CACC_GAINS['k_a'], accel_min=-6.0, accel_max=3.0) -> np.ndarray
string_stability_gain(k_p=CACC_GAINS['k_p'], k_d=CACC_GAINS['k_d'],
                     k_a=CACC_GAINS['k_a'], *, headway=0.6, time_constant=0.3,
                     dt=0.1, n_frequencies=2000) -> float
platoon_adjacency(topology, n_followers) -> tuple[np.ndarray, np.ndarray]
PlatoonConfig.from_kwargs(**kwargs) -> PlatoonConfig # frozen dataclass

TOPOLOGIES = ('predecessor', 'predecessor_leader', 'bidirectional', 'none')
SCENARIOS = ('mixed', 'cruise', 'stop_and_go', 'random')
CACC_GAINS = {'k_p': 0.5, 'k_d': 0.6, 'k_a': 0.65}
OBSERVATION_FEATURES # the 13 column names, 'spacing_error', ...
FEATURE_SCALES       # feature -> scale (observation = value / scale)
```

Both classes have the read-only properties `config`, `n_followers`, `n_agents`, `topology`, `scenario`, `dt`, `max_steps`, `headway`, `standstill_distance`, `obs_dim`, `adjacency` (boolean (n, n), rows are receivers), `leader_links` (n,) and `observation_layout`. `PlatoonEnv` also exposes copies of the state with the leader first (positions, velocities, accelerations, time_constants, lengths, leader_command), per follower (gaps, spacing_errors) and `step_count`. `cacc_policy` is the decentralised CACC law, `string_stability_gain` the peak gain of its string-stability transfer function (1 means string stable, `inf` an unstable vehicle loop), and `platoon_adjacency` returns the V2V graph and the leader links of a topology.

A.1.10 AJLATTEnv and AJLATTConfig

Module `env_lib.ajlatt_env`; Chapter 10.

```
AJLATTEnv(config=None, render_mode=None, **overrides)
AJLATTConfig(map_name='obstacles04', num_robots=4, num_targets=1,
             max_episode_steps=120, sampling_period=0.5, ..., ci_solver='newton',
             target_policy='auto', ..., seed=None) # dataclass, about 50 fields
TeamRewardWrapper(env) # gymnasium.Wrapper
make(figID=0, *, render_mode=None, **kwargs) -> AJLATTEnv # legacy factory
available_maps() -> list[str]
load_grid_map(map_name, margin2wall=0.5) -> GridMap
load_map(map_name) -> tuple[np.ndarray, list[float], list[float]]
```

AJLATTEnv is configured by an AJLATTConfig and/or keyword overrides of its fields. TeamRewardWrapper returns the summed team reward and a scalar terminated flag and keeps the per-robot values in `info["agent_rewards"]` and `info["agent_terminated"]`; it is equivalent to `env_lib.wrappers.TeamReward(env, reduce="sum")`. `make` (also callable as `env_lib.ajlatt_env(...)`) accepts the legacy parameter names with a `DeprecationWarning` and ignores unknown names with a warning.

Member	Description
<i>AJLATTEnv</i>	
<code>single_action_space</code> ,	Per-robot spaces; the joint spaces have a leading
<code>single_observation_space</code>	<code>num_robots</code> axis.
<code>observation_layout()</code>	Dictionary of slices of the per-robot observation vector.
<code>episode_statistics()</code>	Dictionary of per-step traces of the current episode (<code>robot_cov_trace</code> , <code>target_cov_trace</code> , <code>robot_error</code> , <code>target_error</code> , <code>reward</code>), each (<code>steps + 1</code> , <code>num_robots</code>).
<code>config</code> , <code>MAP</code>	The configuration and the loaded <code>GridMap</code> .
<i>AJLATTConfig</i>	
<code>from_kwargs(strict=True, **kwargs)</code>	Build from keyword arguments, translating legacy names.
<code>add_arguments(parser, prefix="")</code>	Register every scalar field as <code>--<prefix><name></code> on an <code>argparse</code> parser.
<code>from_namespace(namespace, prefix="", strict=False)</code>	Build from parsed arguments.
<code>to_dict()</code> , <code>from_dict(data, strict=True)</code>	Plain-dictionary round trip.
<code>replace(**changes)</code>	Validated copy with some fields replaced.
<code>validate()</code> , <code>observation_dim</code> ,	Range checks, per-robot observation size
<code>field_names()</code>	$7 + 4(n_R - 1) + 5$, field list.

A.1.11 AJLATT maps

Module `env_lib.ajlatt_env.maps`; Chapter 10.

```

GridMap(map_path, margin2wall=0.5)
GridMap.from_array(occupancy, *, mapmin=(0.0, 0.0), mapres=0.4, margin2wall=0.5,
                  name='custom') -> GridMap
DynamicMap(map_dir_path, map_name, map_path=None, margin2wall=0.5)
DynamicMap.generate_map(chosen_idx=None, rot_angs=None, rng=None) -> np.ndarray
resolve_map_path(map_name) -> Path
MAP_DIR                                # directory of the bundled maps

```

GridMap method	Description
<code>is_blocked(start_pos, end_pos)</code>	Whether the straight line between two positions crosses an obstacle.
<code>is_blocked_batch(start_pos, end_pos)</code>	Vectorised <code>is_blocked</code> for arrays of shape <code>(n, 2)</code> or <code>(n, 3)</code> .
<code>get_closest_obstacle(odom, ang_res=0.05, fov=pi, r_max=3.0, return_pt=False)</code>	Range and bearing of the closest obstacle cell in a field of view.
<code>get_front_obstacle(odom, r_max=3.0)</code>	Range to the first obstacle straight ahead, or <code>None</code> .
<code>is_collision(pos, margin=None)</code>	Out of bounds or within <code>margin</code> of an occupied cell.
<code>in_bound(pos)</code>	At least <code>margin2wall</code> inside the map boundary.
<code>se2_to_cell(pos), cell_to_se2(cell)</code>	Conversions between positions and cell indices.

The submodules `maps.builder` (`build_map`, `rasterize`, `load_spec`, `save_map`) and `maps.plotting` (`plot_map`) implement the console scripts `ajlatt-build-map` and `ajlatt-plot-map`. The geometry helpers `bresenham2D`, `bresenham_batch`, `se2_to_cell`, `cell_to_se2` and `coord_change2g` are exported as well.

A.2 env_lib.catalog

Section 11.2.

```

catalog(*, family=None, action_type=None, observation_type=None,
        native_vector=None, available_only=False, inspect=True) -> Catalog
describe(env_id) -> str
clear_cache()
EnvInfo(id, family, description, observation_type, action_type, requires,
        native_vector, available, n_agents=None, observation_shape=None,
        action_shape=None, kwargs={}, entry_point='', vector_entry_point=None,
        observation_space=None, action_space=None, note=None) # frozen dataclass
EnvInfo.matches(kind, *, action=True) -> bool
Catalog                                     # list subclass of EnvInfo records
Catalog.ids() -> list[str]
Catalog.get(env_id) -> EnvInfo             # KeyError if absent
Catalog.to_markdown() -> str

```

`catalog` returns the registered environments that match all given filters (`action_type` and `observation_type` take "continuous" or "discrete", otherwise `ValueError`); `str(catalog)` is an aligned text table. `describe` raises `KeyError` for an unknown id. `clear_cache` forgets the cached availability checks and inspection results. The module is callable: `env_lib.catalog(...)` calls `catalog(...)` whether or not the submodule has been imported, and `env_lib.describe` and `env_lib.EnvInfo` are exported at the top level.

A.3 env_lib.baselines

Section 11.5.

```

baseline_policy(env, **kwargs) -> BaselinePolicy
list_baselines() -> dict[str, str]         # family -> description
family_of(env) -> str

```

```

BaselinePolicy(function, parameters, *, family, description,
               observation_shapes=None, action_shapes=None)
    # attributes function, parameters, family, description, name
    # __call__(observation) -> action (NumPy array, or tensor for a tensor input)

laplacian_consensus(observation, *, gain=1.0, velocity_gain=0.0,
                   max_control=math.inf) -> np.ndarray
kuramoto_feedback(observation, *, n_oscillators, n_couplings=0,
                  control_low=-1.0, control_high=1.0, coupling=5.0,
                  target_frequency=None, gain=3.0) -> np.ndarray
linemsg_relay(observation, *, joint_discrete=True) -> np.ndarray
wireless_schedule(observation, *, grid_x, grid_y, ddl,
                  n_obs_neighbors=1) -> np.ndarray
pistonball_ramp(observation, *, continuous=True, gain=8.0) -> np.ndarray
ajlatt_encircle(observation, *, radius=1.8, gain=1.2, max_linear_velocity=2.0,
                 max_angular_velocity=math.pi / 4,
                 heading_column=-2) -> np.ndarray

```

`baseline_policy` accepts a single or vector environment (native, Sync or Async), wrapped or not, a `ParallelEnvAdapter` (the returned policy then maps observation dictionaries to action dictionaries) or a registered id, and binds the controller's parameters from the environment. Keyword arguments override the gains: `gain` and `velocity_gain` (consensus), `gain` and `coupling` (Kuramoto), `gain` (Pistonball and PowerGrid), `radius` and `gain` (AJLATT), `k_p`, `k_d`, `k_a`, `accel_min` and `accel_max` (Platoon; the bounds default to the action bounds); `LineMsg` and `WirelessComm` take none. It raises `TypeError` for environments without a baseline, for unknown keyword arguments and for wrappers that change the spaces beyond a reshape, and `NotImplementedError` if the module of the baseline cannot be imported. `BaselinePolicy.name` is "baseline:<family>". The `PowerGrid` and `Platoon` controllers are `env_lib.power_grid_env.droop_policy` and `env_lib.platoon_env.cacc_policy`. `family_of` returns the family of an environment or id (`TypeError` for other environments).

A.4 env_lib.wrappers

Section 11.4.

```

FlattenJointSpaces(env)                                # gymnasium.Wrapper
FlattenJointSpaces.observation(observation) -> np.ndarray # 1-D float32
FlattenJointSpaces.action(action)                  # flat action -> joint action shape
TeamReward(env, reduce='sum', source='reward')      # gymnasium.Wrapper
ParallelEnvAdapter(env, *, agent_prefix='agent')
to_parallel(env_or_id, **kwargs) -> ParallelEnvAdapter
PETTINGZOO_AVAILABLE                                # bool: pettingzoo importable

```

Member of ParallelEnvAdapter	Description
<code>possible_agents, agents, n_agents</code>	Agent names "agent_0", ...; live agents (empty after the end of an episode); their number.
<code>observation_space(agent), action_space(agent)</code>	Per-agent spaces (also as the dictionaries <code>observation_spaces, action_spaces</code>).
<code>reset(seed=None, options=None)</code>	Observation and info dictionaries.
<code>step(actions)</code>	Observation, reward, termination, truncation and info dictionaries; <code>RuntimeError</code> after the end of an episode, <code>ValueError</code> for a missing action.
<code>state(), state_space</code>	Joint observation and its space.
<code>stack_observations(observations)</code>	Joint observation from a dictionary.
<code>split_actions(joint_action, agents=None)</code>	Action dictionary from a joint action.
<code>render(), close(), metadata, render_mode</code>	Forwarded to or taken from the wrapped environment.

`FlattenJointSpaces` and `ParallelEnvAdapter` raise `TypeError` for unsupported spaces (and the adapter for a vector environment); `TeamReward` raises `ValueError` for an unknown `reduce` or `source`. `to_parallel` creates the environment with `env_lib.make` when given an id and raises `TypeError` for keyword arguments together with an environment instance.

A.5 env_lib.utils

A.5.1 Rendering and recording

```
record_episode(env, policy=None, path=None, *, max_steps=None, seed=None,
              fps=None, render_every=1) -> list[np.ndarray]
save_animation(frames, path, fps=30.0, *, loop=0) -> Path
set_theme(theme)                                # "dark" (default) or "light"
get_theme(theme=None) -> Theme
register_theme(theme)
Theme(name, background, panel, grid, text, muted, accent, positive, negative,
      warning, categorical, sequential_cmap='viridis', diverging_cmap='RdBu_r')
DARK, LIGHT                                     # the built-in themes
FigureWindow(render_mode, *, figsize=(8.0, 6.0), dpi=100, fps=30.0, title=None,
             theme=None)
MatplotlibRenderer(render_mode, *, figsize=(8.0, 6.0), dpi=100, fps=30.0,
                  title=None, theme=None, blit=True)
figure_to_rgb(fig) -> np.ndarray
style_axes(ax, theme, *, title=None, xlabel=None, ylabel=None, grid=True,
          show_spines=False, fontsize=9.0) -> Axes
with_alpha(color, alpha) -> tuple[float, float, float, float]
```

`record_episode` and `save_animation` are described in Section 16.4. `Theme` is a dataclass of colours with `color(index)` for the cycled categorical colours; `set_theme` affects renderers created afterwards. `FigureWindow` owns one matplotlib figure (off-screen Agg canvas in "rgb_array" mode, a window in "human" mode) with `draw()`, `close()` and `is_open`. `MatplotlibRenderer` is the base class of the environment renderers: subclasses implement `_build(fig)` (create the artists once) and `_update(...)` (update their data), and users call `render(...)`, `reset()` and `close()`. `figure_to_rgb`, `style_axes` and `with_alpha` are helpers for custom renderers.

A.5.2 Evaluation and trajectories

Module `env_lib.utils.evaluation`, also exported by `env_lib.utils` and, for `evaluate` and `rollout`, by `env_lib`; Section 11.6.

```

evaluate(env, policy=None, *, n_episodes=10, seed=None, max_steps=None,
         deterministic_seeds=True) -> EvaluationResult
rollout(env, policy=None, *, n_steps=None, n_episodes=None,
        seed=None) -> Trajectory

EvaluationResult(returns, lengths, terminated, agent_returns=None,
                 successes=None, env_id=None, policy=None)  # frozen dataclass
EvaluationResult.summary() -> dict[str, Any]
    # derived fields: n_episodes, mean_return, std_return, ci95, mean_length,
    #                  termination_rate, success_rate

Trajectory(observations, actions, rewards, next_observations, terminated,
           truncated, episode_ids, agent_rewards=None, env_id=None)  # dataclass
Trajectory.save(path) -> Path  # compressed .npz
Trajectory.load(path) -> Trajectory  # class method
Trajectory.episode(index) -> Trajectory
Trajectory.episode_returns() -> np.ndarray
Trajectory.n_episodes  # property; len(trajectory) = steps

```

`evaluate` raises `ValueError` for `n_episodes < 1` or `max_steps < 1`; `rollout` raises `TypeError` for a vector environment and `ValueError` for non-positive limits.

A.5.3 Graph utilities

Module `env_lib.utils.graphs` (also `env_lib.utils.graphs`); Section 3.7.

```

make_graph(topology, n, *, rng=None, edge_probability=0.3, neighbors=4,
           rewire_probability=0.1, radius=None, connected=True,
           max_attempts=1000, return_positions=False)
    # -> bool (n, n) adjacency, or (adjacency, positions (n, 2))
is_connected(adjacency) -> bool
laplacian(adjacency) -> np.ndarray
algebraic_connectivity(adjacency) -> float
k_hop_adjacency(adjacency, k, *, include_self=False) -> np.ndarray
degree(adjacency) -> np.ndarray
neighbor_lists(adjacency) -> list[np.ndarray]
edge_list(adjacency) -> np.ndarray  # (n_edges, 2), i < j
grid_shape(n) -> tuple[int, int]
circular_layout(n, radius=1.0) -> np.ndarray
spring_layout(adjacency, *, iterations=200, seed=0,
              initial=None) -> np.ndarray
TOPOLOGIES = ('ring', 'line', 'star', 'complete', 'grid', 'erdos_renyi',
              'small_world', 'random_geometric')

```

`make_graph` raises `ValueError` for an unknown topology, invalid parameters, fewer than two nodes, or when no connected random graph is found in `max_attempts` samples; `algebraic_connectivity` needs at least two nodes.

A.5.4 Batched vector environments

Module `env_lib.utils.vector`; `BatchedVectorEnv` is also exported by `env_lib.utils`.

```
BatchedVectorEnv(num_envs, single_observation_space, single_action_space, *,
                 autoreset_mode='next_step', render_mode=None)
AUTORESET_MODES = ('next_step', 'same_step', 'disabled')

# Hooks implemented by subclasses
_reset_envs(mask, options)           # re-initialise the copies where mask is True
_step_envs(actions, active) -> tuple [rewards, terminated, truncated, infos]
_observe() -> np.ndarray              # (num_envs, *single_observation_space.shape)
_reset_infos(mask) -> dict            # optional; the info returned by reset()
```

`BatchedVectorEnv` derives from `gymnasium.vector.VectorEnv` and implements `reset` and `step` on top of the hooks: seeding of the shared generator `self.np_random`, the "reset_mask" option (removed before `options` reaches `_reset_envs`), action validation (reshaping when the number of entries matches, `ValueError` for a wrong size or for NaN and infinite values), the three autoreset modes (`autoreset_mode` also accepts "NextStep"-style names and `gymnasium.vector.AutoresetMode` members), `infos["final_obs"]` and `infos["final_info"]` in same-step mode, the "_key" masks of every info key and `ResetNeededError`. `_step_envs` returns arrays of shape `(num_envs,)` and a dictionary of arrays with a leading `num_envs` dimension; `active` marks the copies whose results are used (under next-step autoreset the others are reset right afterwards). `render()` returns `None` without a render mode and raises `NotImplementedError` otherwise unless a subclass overrides it. A subclass that serves as a `vector_entry_point` must accept the keyword arguments of the single environment plus `num_envs` and `autoreset_mode`, because `gymnasium.make_vec` calls it as `VectorClass(num_envs=..., **registered_kwargs, **user_kwargs)`. A minimal subclass:

```
import numpy as np
from gymnasium import spaces
from env_lib.utils import BatchedVectorEnv

class Integrators(BatchedVectorEnv):
    """n agents with  $x' = u$ , reward  $-x_i^2$  per agent, 50 steps per episode."""

    def __init__(self, num_envs=1, *, autoreset_mode="next_step", n_agents=4):
        obs = spaces.Box(-np.inf, np.inf, (n_agents, 1), np.float32)
        act = spaces.Box(-1.0, 1.0, (n_agents, 1), np.float32)
        super().__init__(num_envs, obs, act, autoreset_mode=autoreset_mode)
        self.x = np.zeros((self.num_envs, n_agents, 1))
        self.t = np.zeros(self.num_envs, dtype=np.int64)

    def _reset_envs(self, mask, options):
        self.x[mask] = self.np_random.uniform(-1.0, 1.0, self.x[mask].shape)
        self.t[mask] = 0

    def _step_envs(self, actions, active):
        self.x += 0.1 * np.clip(actions, -1.0, 1.0)
        self.t += 1
        agent_rewards = -self.x[..., 0] ** 2  # (num_envs, n_agents)
        terminated = np.zeros(self.num_envs, dtype=bool)
        infos = {"agent_rewards": agent_rewards}
        return agent_rewards.sum(axis=1), terminated, self.t >= 50, infos

    def _observe(self):
```

```

        return self.x.astype(np.float32)

envs = Integrators(8)
obs, infos = envs.reset(seed=0)
obs, rewards, terminated, truncated, infos = envs.step(-obs)
print(obs.shape, rewards.shape)           # (8, 4, 1) (8,)
print(infos["agent_rewards"].shape)       # (8, 4)

```

A.6 The env-lib command line

Module `env_lib.__main__`; Section 11.3. The console script `env-lib` and `python -m env_lib` call `env_lib.__main__.main(argv=None) -> int`, which returns the exit status (0 on success, 2 for usage errors such as an unknown id).

```

env-lib list [--continuous | --discrete] [--family F [F ...]] [--native]
             [--available] [--markdown] [--no-inspect]
env-lib describe ENV_ID
env-lib baselines
env-lib run ENV_ID [--steps N] [--policy {baseline,random}] [--seed S]
                [--gif PATH] [--fps F] [--theme {dark,light}]
                [--kwarg KEY=VALUE [KEY=VALUE ...]]
env-lib evaluate ENV_ID [--episodes N] [--policy {both,baseline,random}]
                    [--num-envs N] [--seed S] [--kwarg KEY=VALUE [...]]
env-lib bench ENV_ID [--num-envs N] [--steps N] [--seed S] [--async]
                    [--kwarg KEY=VALUE [...]]

```

Defaults: run steps until the episode ends (`--steps` caps it), with `--policy` `baseline`, `--seed` 0 and `--theme` `dark`; `evaluate` uses `--episodes` 10, `--policy` `both`, `--num-envs` 1 and `--seed` 0; `bench` uses `--num-envs` 32, `--steps` 200 and `--seed` 0. `--kwarg` values are parsed as Python literals (plain text otherwise) and may be repeated.

A.7 marl_algorithms

Chapter 12. The package requires PyTorch. The top-level package exports lazily the training entry points, the registry, the preset functions, the baselines (`compare`, `Comparison`, `ComparisonRow`, `per_copy`), the core classes `Algorithm`, `OnPolicyAlgorithm`, `OffPolicyAlgorithm`, `TrainingLog`, `MultiAgentSpec`, `VectorRunner`, `Transition` and `make_vector_env`, the seven algorithm classes and the three configuration classes; the preset tables, buffers, normalisers, networks and base configurations live in their modules. Without PyTorch, accessing any of these names raises an `ImportError` that names the `torch` extra.

A.7.1 Training, registry and presets

Modules `marl_algorithms.registry` and `marl_algorithms.presets` (both exported by the package); Sections 12.2 and 12.8.

```

train(algorithm, env_id, total_steps, *, num_envs=16, seed=0,
      env_kwargs=None, device='cpu', callback=None, threads=1,
      **config) -> tuple[Algorithm, TrainingLog]
make_algorithm(name, env_or_spec, **kwargs) -> Algorithm
get_algorithm(name) -> type[Algorithm]      # case-insensitive

```

```

list_algorithms() -> list[AlgorithmInfo]      # registry order
AlgorithmInfo(name, entry_point, family, action_kinds, summary,
               reference)                    # frozen dataclass
marl_algorithms.__version__                 # "1.2.1"

# marl_algorithms.presets
get_preset(algorithm, env_id) -> dict | None  # a copy
list_presets() -> list[tuple[str, str]]      # (algorithm, env_id) pairs
train_preset(algorithm, env_id, *, seed=0, total_steps=None, num_envs=None,
              env_kwargs=None, device='cpu', callback=None, threads=1,
              **config) -> tuple[Algorithm, TrainingLog]
PRESETS                                     # {algorithm: {env_id: preset}}, all families

```

`train` creates the copies with `make_vector_env`, builds the algorithm with `make_algorithm` and trains it with `learn`, passing the seed to both; it closes the copies, sets `log.env_id` and records the environment id and keyword arguments in `algo.metadata`. It trains with `threads` PyTorch threads (`None` keeps the current setting) and restores the previous setting afterwards, as `torch_threads` does; an unknown algorithm is reported before any environment is created. `make_algorithm` passes its keyword arguments (`config`, `device`, `seed` and configuration overrides) to the class. `get_algorithm`, `get_preset` and `train_preset` raise `KeyError` for an unknown algorithm, and `train_preset` also when the pair has no preset. A preset is a dictionary with the keys `"total_steps"`, `"num_envs"`, `"config"` and `"env_kwargs"`; `train_preset` merges its `env_kwargs` into the preset's. `PRESETS` collects the tables of the modules `marl_algorithms.presets.ppo`, `.ddpg` and `.q_learning`.

A.7.2 Baselines

Module `marl_algorithms.baselines` (exported by the package); Section 12.3.

```

compare(env_id, algorithms=None, *, seeds=(0,), policies=None, n_episodes=64,
        eval_seed=1, total_steps=None, num_envs=None, env_kwargs=None,
        include_random=True, include_baseline=True, keep_replay=False,
        device='cpu', threads=1, verbose=False, **config) -> Comparison
per_copy(policy) -> policy                    # single-environment policy -> batched

Comparison(env_id, env_kwargs, n_episodes, eval_seed, rows, algorithms, logs)
report[name] -> ComparisonRow                 # KeyError for an unknown name
report.ranking(kinds=("algorithm", "policy")) -> list[ComparisonRow]
report.records() -> list[dict]; report.to_markdown() -> str
report.to_csv(path) -> Path; str(report)      # aligned table
report.algorithms[(name, seed)] -> Algorithm; report.logs[(name, seed)]

ComparisonRow(name, kind, returns, seeds=(), env_steps=(), train_seconds=())
row.mean, row.std                            # over seeds; std is NaN for one value
# records(): std_return is None for rows with one value

```

`algorithms=None` selects every algorithm with a preset for the environment and an empty sequence `none`; `policies` maps names to batched policies or trained algorithms. Rows are in the order references (`"random"`, `"baseline"`), `algorithms`, `policies`, with kind `"reference"`, `"algorithm"` or `"policy"`. Evaluation uses only the first episode of every copy after a seeded reset, in chunks of at most 64 episodes (reset with `eval_seed`, `eval_seed + 64`, ...), so every row meets the same episodes. `compare` checks all arguments before it trains anything: it raises `KeyError` for an unknown algorithm and `ValueError` for an algorithm listed twice, one that does not support the environment's actions, one without a preset when `total_steps` is not

given, a configuration override that an algorithm rejects, invalid environment arguments or ones that differ from a preset's, a policy name that repeats another row's name, empty or repeated `seeds`, `eval_seed=None`, or a non-positive `n_episodes`, `total_steps` or `num_envs`. The classical controller row is omitted for environments without one; the replay buffers of trained off-policy algorithms are released unless `keep_replay=True`.

A.7.3 Agent spec and experience collection

Modules `marl_algorithms.core.spec` and `marl_algorithms.core.runner`; Sections 12.4.1 and 12.4.2.

```
MultiAgentSpec(n_agents, obs_dim, action_kind, action_dim, n_actions,
               action_low, action_high, obs_shape, action_shape,
               action_dtype) # frozen dataclass
MultiAgentSpec.from_env(env) -> MultiAgentSpec # class method
MultiAgentSpec.from_spaces(obs_space, act_space) -> MultiAgentSpec
spec.continuous # property, bool
spec.state_dim # property, n * obs_dim
spec.agent_obs(obs) -> np.ndarray # (*batch, n, obs_dim)
spec.env_action(actions) -> np.ndarray # joint action, clipped
spec.describe() -> str

make_vector_env(env_id, num_envs=16, **env_kwargs) -> gym.vector.VectorEnv

VectorRunner(envs, spec=None)
runner.reset(seed=None) -> np.ndarray # (B, n, obs_dim)
runner.step(actions) -> Transition
runner.pop_episodes() -> list[tuple[float, int]] # (return, length)
runner.obs, runner.envs, runner.spec, runner.num_envs

Transition(obs, actions, reward, agent_rewards, next_obs, terminated,
           truncated) # dataclass
transition.done # terminated / truncated
```

`MultiAgentSpec.from_spaces` raises `TypeError` for unsupported spaces (a non-Box observation, a joint `Discrete` action over several agents, unequal or shifted `MultiDiscrete` choices, infinite continuous bounds, observations not stacked per agent), and `agent_obs` raises `ValueError` for observations of the wrong shape. `VectorRunner` raises `ValueError` unless the vector environment uses "same_step" autoreset, and `step` raises `RuntimeError` before `reset`.

A.7.4 Buffers, normalisation, networks and configurations

Modules of `marl_algorithms.core`, which also exports the buffers, normalisers and configuration classes; Sections 12.4.3 to 12.4.5.

```
RolloutBuffer(length, num_envs, fields) # fields: {name: (shape, dtype)}
buffer.add(**values); buffer.reset(); buffer[name]; name in buffer
buffer.full # property
compute_gae(rewards, values, next_values, terminated, truncated, gamma,
            gae_lambda) -> tuple[np.ndarray, np.ndarray] # (adv., returns)
ReplayBuffer(capacity, spec)
replay.add(transition); len(replay)
replay.sample(batch_size, rng) -> dict[str, np.ndarray]

RunningMeanStd(shape=()) # update(x), mean, var, count, std
ObservationNormalizer(obs_dim, clip=10.0, enabled=True) # update(obs), norm(obs)
```

```

RewardScaler(num_envs, gamma, enabled=True) # scaler(reward, done) -> reward
# all three: state_dict(), load_state_dict(state)

# marl_algorithms.core.networks
mlp(in_dim, out_dim, hidden_sizes=(64, 64), activation='tanh', *,
    output_gain=1.0, layer_norm=False) -> nn.Sequential
agent_one_hot(batch_shape, n_agents, device) -> torch.Tensor
soft_update(target, source, tau)
GaussianPolicy(in_dim, action_dim, low, high, hidden_sizes=(64, 64),
    activation='tanh', log_std_init=-0.5)
CategoricalPolicy(in_dim, n_actions, hidden_sizes=(64, 64), activation='tanh')
# both: sample(x, generator=None) -> (action, log_prob)
#       log_prob(x, action) -> (log_prob, entropy)
DeterministicPolicy(in_dim, action_dim, low, high, hidden_sizes=(64, 64),
    activation='relu')
QMixer(n_agents, state_dim, embed_dim=32, hypernet_hidden=64) # (q, state)
PerAgent(factory, n_agents, shared=True) # forward(x, *args)
per_agent.call(method, x, *args)

# marl_algorithms.core.config (dataclasses)
AlgorithmConfig(hidden_sizes=(64, 64), activation='tanh', gamma=0.99,
    share_parameters=True, agent_ids=True, max_grad_norm=10.0)
OnPolicyConfig(..., max_grad_norm=0.5, rollout_length=64, lr=0.0003,
    gae_lambda=0.95, normalize_observations=True,
    normalize_rewards=True)
OffPolicyConfig(..., activation='relu', buffer_size=200000, batch_size=256,
    warmup_steps=2000, update_every=1, gradient_steps=1,
    tau=0.01, reward_scale=1.0)
Config.field_names() -> list[str] # class method
config.to_dict() -> dict[str, Any]

```

RolloutBuffer.add raises RuntimeError when the buffer is full and KeyError when a field is missing; ReplayBuffer raises ValueError for a non-positive capacity and sample RuntimeError when empty; mlp raises ValueError for an unknown activation. The configuration classes validate their fields in `__post_init__` and raise ValueError for invalid values.

A.7.5 Base classes and the training log

Module `marl_algorithms.core.base`; Section 12.4.6.

```

Algorithm(spec, config=None, *, device='cpu', seed=None, **overrides)
Algorithm.name, Algorithm.action_kinds, Algorithm.config_class # class attr.
algo.act(obs, deterministic=False) -> np.ndarray
algo.learn(envs, total_steps, *, seed=None, log=None,
    callback=None) -> TrainingLog
algo.policy(deterministic=True) -> Callable[[Any], np.ndarray]
algo.evaluate(env, n_episodes=10, *, seed=None,
    deterministic=True) -> EvaluationResult
algo.state_dict() -> dict[str, Any]
algo.load_state_dict(state)
algo.save(path) -> Path
Algorithm.load(path, device='cpu') -> Algorithm # class method
algo.check_env(env) # ValueError if the spaces do not match algo.spec
algo.metadata # dict saved with the checkpoint (env_id, env_kwargs)
torch_threads(threads) # context manager; None leaves the setting unchanged
algo.spec, algo.config, algo.device, algo.seed, algo.np_rng, algo.torch_rng

```

```

algo.env_steps, algo.num_updates
# helpers for subclasses
algo.uses_agent_ids, algo.agent_input_dim          # properties
algo.agent_inputs(obs) -> torch.Tensor
algo.tensor(array, dtype=torch.float32) -> torch.Tensor
algo.random_actions(batch) -> np.ndarray
# hooks: _build(), _modules(), _extra_state(), _load_extra_state(state)

OnPolicyAlgorithm(Algorithm)      # config_class OnPolicyConfig
# hooks: _rollout_step(obs) -> (actions, extras), _update(rollout) -> stats,
#       _rollout_fields() -> fields, _observe_transition(transition)
OffPolicyAlgorithm(Algorithm)     # config_class OffPolicyConfig; algo.replay
# hooks: _explore(obs) -> actions, _update(batch) -> stats

TrainingLog(algorithm, env_id=None)
log.episodes          # list of (env_steps, team_return, length)
log.updates           # list of (env_steps, statistics)
log.env_steps         # steps taken when training last returned
log.wall_time, log.algorithm, log.env_id
log.episode_steps, log.episode_returns          # properties, arrays
log.mean_return(last=100) -> float
log.curve(points=50) -> tuple[np.ndarray, np.ndarray] # (steps, mean returns)
log.summary() -> str
log.to_csv(path) -> Path
log.record_episodes(env_steps, episodes); log.record_update(env_steps, stats)
log.record_steps(env_steps)          # called by learn() when it returns

```

The constructor raises `TypeError` for an unsupported action kind, a configuration of the wrong class or unknown override fields. `callback(algorithm, log)` is called after every update (on-policy) or every round of gradient steps (off-policy) and stops training by returning `False`. `Algorithm.load` selects the class through the registry unless it is called on the class that saved the file; it raises `FileNotFoundError` for a missing file and `ValueError` for a file that is not a checkpoint of this package. `evaluate` calls `check_env`, which raises `ValueError` when the environment's number of agents, observation size or action space differs from the one the algorithm was built for (and warns when only the bounds of a continuous action space differ). `learn` requires a positive integer `total_steps`; an off-policy `learn` whose budget ends before `warmup_steps` warns that no gradient step will be taken. A device of "cuda" or "mps" that is not available raises `ValueError` in the constructor.

A.7.6 Algorithms and their configurations

Packages `marl_algorithms.algorithms.ppo` (modules `common`, `ippo`, `mappo`), `.ddpg` (`common`, `mad-dpg`, `matd3`) and `.q_learning` (`common`, `iq1`, `vdn`, `qm1x`); every family package exports its classes and its configuration. Sections 12.5 to 12.7. All seven classes take the constructor arguments of `Algorithm`.

```

IPPO, MAPPO          # OnPolicyAlgorithm; PPOConfig; continuous and discrete
MADDPG, MATD3        # OffPolicyAlgorithm; DDPGConfig; continuous
IQL, VDN, QMIX       # OffPolicyAlgorithm; QLearningConfig; discrete

# IPPO, MAPPO
algo.values(obs) -> torch.Tensor      # critic values (... , n) of normalised obs
algo.critic_lr, algo.critic_input_dim # properties
algo.reward_source      # "agent" or "team", resolved
algo.obs_normalizer, algo.reward_scaler

```

```

# MADDPG, MATD3
algo.exploration_scale          # property, current noise scale
MADDPG.n_critics, .smooth_targets, .delayed_policy    # 1, False, False
MATD3.n_critics, .smooth_targets, .delayed_policy    # 2, True, True
# IQL, VDN, QMIX
algo.epsilon_at(env_steps) -> float; algo.epsilon    # property
algo.q_values(obs, *, target=False) -> torch.Tensor  # (... , n, n_actions)
algo.mix(agent_q, obs, *, target=False) -> torch.Tensor
algo.td_target(batch) -> torch.Tensor
algo.td_loss(batch) -> tuple[torch.Tensor, dict[str, float]]
VDNMixer()                    # q_learning.vdn module

PPOConfig(<OnPolicyConfig fields>, n_epochs=10, num_minibatches=4,
          clip_range=0.2, value_clip_range=0.2, huber_delta=None,
          ent_coef=0.0, normalize_advantages=True, critic_lr=None,
          critic_hidden_sizes=None, anneal_lr=False, target_kl=None,
          log_std_init=-0.5, reward_source=None)
DDPGConfig(<OffPolicyConfig fields>, actor_lr=0.001, critic_lr=0.001,
           critic_hidden_sizes=None, exploration_noise=0.1,
           final_exploration_noise=None, noise_decay_steps=0,
           reward_source='agent', normalize_observations=False,
           critic_local_inputs=True, critic_loss='mse', policy_delay=2,
           target_noise=0.2, target_noise_clip=0.5)
QLearningConfig(<OffPolicyConfig fields>, lr=0.0005, epsilon_start=1.0,
                epsilon_end=0.05, epsilon_decay_steps=50000, double_q=True,
                target_update_interval=0, loss='huber', mixer_embed_dim=32,
                hypernet_hidden=64)

```

Tables 12.5, 12.6, 12.7 and 12.8 describe the fields. The PPO act raises `ValueError` for observations whose trailing shape is not `(n_agents, obs_dim)`.

A.7.7 The marl-train command line

Module `marl_algorithms.__main__`; Section 12.2.2. The console script `marl-train` and `python -m marl_algorithms` call `main(argv=None) -> int`; `build_parser()` returns the `argparse` parser.

```

marl-train list
marl-train presets
marl-train run ALGORITHM ENV_ID [--steps N] [--num-envs N] [--seed S]
                [--no-preset] [--set KEY=VALUE [KEY=VALUE ...]]
                [--env-kwarg KEY=VALUE [KEY=VALUE ...]] [--eval-episodes N]
                [--reports N] [--save PATH] [--csv PATH] [--threads N]
marl-train compare ENV_ID [--algos ALGO [ALGO ...]] [--seeds S [S ...]]
                [--steps N] [--num-envs N] [--episodes N] [--eval-seed S]
                [--set KEY=VALUE [KEY=VALUE ...]]
                [--env-kwarg KEY=VALUE [KEY=VALUE ...]] [--markdown]
                [--csv PATH] [--save-dir DIR] [--threads N]
marl-train evaluate CHECKPOINT ENV_ID [--episodes N] [--seed S]
                [--env-kwarg KEY=VALUE [KEY=VALUE ...]]

```

Defaults: `run` uses the preset's steps and copies (100 000 and 16 without a preset), `--seed` 0, `--eval-episodes` 64, `--reports` 10 and `--threads` 1; `compare` uses every algorithm with a preset, `--seeds` 0, the presets' budgets, `--episodes` 64, `--eval-seed` 1 and `--threads` 1; `evaluate` uses `--episodes` 64 and `--seed` 1. `KEY=VALUE` values are parsed as Python literals

(plain text otherwise). Command-line usage errors (including a malformed KEY=VALUE pair and a non-positive count), and an unknown environment, algorithm, configuration field or checkpoint, invalid environment arguments or configuration values, an algorithm that does not support the environment's actions, or (for `compare`) one without a preset and without `--steps`, end the command before any training with a message on standard error and exit status 2. Without PyTorch, every sub-command reports that PyTorch is not installed and exits with status 1.

A.8 toolkit.plotkit

Chapter 14. All plot functions return the `matplotlib.axes.Axes` they drew on.

```
plot_shadow_curve(y, x=None, y_std=None, labels=None, colors=None, alpha=0.2,
                  ax=None, axis=0, figsize=(10, 6), title=None, xlabel=None,
                  ylabel=None, legend=True, grid=True, style='research',
                  legend_labels=None, x_tick_labels=None, y_tick_labels=None,
                  *, band='std', smoothing=None, **kwargs)
plot_learning_curves(runs, x=None, *, band='ci95', smoothing=None, colors=None,
                     alpha=0.2, ax=None, figsize=(10, 6), title=None,
                     xlabel='Step', ylabel='Return', legend=True, grid=True,
                     style='research', **kwargs)
plot_line(x, y, labels=None, colors=None, ax=None, figsize=(10, 6), title=None,
          xlabel=None, ylabel=None, legend=True, grid=True, style='research',
          **kwargs)
plot_scatter(x, y, labels=None, ...)      # same arguments as plot_line
plot_bar(x, height, labels=None, colors=None, ax=None, figsize=(10, 6),
         title=None, xlabel=None, ylabel=None, legend=True, grid=True,
         style='research', *, width=0.8, yerr=None, value_labels=False,
         horizontal=False, capsize=3.0, **kwargs)
plot_histogram(data, bins='auto', labels=None, colors=None, ax=None,
               figsize=(10, 6), title=None, xlabel=None, ylabel=None,
               legend=True, grid=True, style='research', *, density=False,
               alpha=0.45, histtype='stepfilled', **kwargs)
plot_heatmap(data, xlabels=None, ylabels=None, cmap='viridis', annot=False,
              ax=None, figsize=(8, 6), title=None, xlabel=None, ylabel=None,
              cbar=True, cbar_label=None, style='research', *, fmt=None,
              vmin=None, vmax=None, center=None, robust=False, mask=None,
              nan_color=None, aspect='auto', square=False, linewidths=None,
              linecolor='white', annot_kws=None, cbar_kws=None, **kwargs)
plot_gray_scale(data, xlabels=None, ylabels=None, ax=None, figsize=(8, 6),
                title=None, xlabel=None, ylabel=None, style='research',
                **kwargs)

save_figure(fig_or_ax, path, formats=None, dpi=300, transparent=False,
            **savefig_kwargs) -> list[Path]
style_context(style='research')      # context manager
set_research_style()
reset_style()
get_palette(name='research', n=None) -> list[str]

STYLE_PRESETS      # read-only; 'research', 'presentation', 'minimal', 'default'
RESEARCH_COLORS, RESEARCH_COLOR_LIST      # tab10, dict and list
OKABE_ITO_COLORS, OKABE_ITO_COLOR_LIST      # colour-blind-safe, dict and list
```

`python -m toolkit.plotkit` and the console script `plotkit-gallery` run the gallery (Section 14.11). The module `toolkit.plotkit.core` re-exports the same names for code written against

earlier versions.

A.9 toolkit.neural_toolkit

Chapter 15. Requires PyTorch. All networks are `torch.nn.Module` subclasses; the shapes are given in Table 15.1.

A.9.1 Layers

```
get_activation(name) -> nn.Module
build_mlp(input_dim, hidden_dims, output_dim=None, activation='relu',
          dropout=0.0, layer_norm=False, output_activation=None,
          bias=True) -> nn.Sequential
PositionalEncoding(d_model, dropout=0.1, max_len=5000)
ACTIVATION_NAMES    # ('elu', 'gelu', 'identity', 'leaky_relu', 'mish', 'relu',
                    # 'sigmoid', 'silu', 'softplus', 'swish', 'tanh')
```

A.9.2 Policy and value networks

```
MLPPolicyNetwork(input_dim, output_dim, hidden_dims=(256, 256),
                  activation='relu', dropout=0.0, layer_norm=False, device='cpu')
CNPolicyNetwork(input_channels, output_dim, conv_dims=(32, 64, 128),
                 fc_dims=(256, 256), kernel_sizes=None, strides=None,
                 activation='relu', dropout=0.0, layer_norm=False, device='cpu')
RNPPolicyNetwork(input_dim, output_dim, hidden_dim=256, num_layers=2,
                  rnn_type='lstm', bidirectional=False, fc_dims=(256,),
                  activation='relu', dropout=0.0, layer_norm=False, device='cpu')
TransformerPolicyNetwork(input_dim, output_dim, d_model=256, nhead=8,
                          num_layers=6, dim_feedforward=1024, dropout=0.1,
                          fc_dims=(256,), activation='relu', layer_norm=True,
                          device='cpu', max_len=5000)
CNNPolicyNetwork = CNPolicyNetwork; RNPPolicyNetwork = RNPPolicyNetwork

MLPValueNetwork, CNNValueNetwork, RNNValueNetwork, TransformerValueNetwork
# same arguments as the policy networks, with output_dim=1 by default
```

A.9.3 Q-networks

```
MLPQNetwork(state_dim, action_dim, hidden_dims=(256, 256), activation='relu',
             dropout=0.0, layer_norm=False, device='cpu', continuous_action=False)
DuelingQNetwork(state_dim, action_dim, hidden_dims=(256, 256),
                 activation='relu', dropout=0.0, layer_norm=False, device='cpu')
CNQNetwork(input_channels, action_dim, conv_dims=(32, 64, 128),
            fc_dims=(256, 256), kernel_sizes=None, strides=None,
            activation='relu', dropout=0.0, layer_norm=False, device='cpu')
RNQNetwork(state_dim, action_dim, hidden_dim=256, num_layers=2, rnn_type='lstm',
            bidirectional=False, fc_dims=(256,), activation='relu', dropout=0.0,
            layer_norm=False, device='cpu')
TransformerQNetwork(state_dim, action_dim, d_model=256, nhead=8, num_layers=6,
                     dim_feedforward=1024, dropout=0.1, fc_dims=(256,),
                     activation='relu', layer_norm=True, device='cpu',
```

```

        max_len=5000)
CNNQNetwork = CNQNetwork; RNNQNetwork = RNNQNetwork
# forward(state, action=None[, hidden or mask]): (B, A), or Q(s, a) of shape (B,)

```

A.9.4 Encoders and decoders

```

MLPEncoder(input_dim, latent_dim, hidden_dims=(256, 256), activation='relu',
            dropout=0.0, layer_norm=False, device='cpu')
CNNEncoder(input_channels, latent_dim, conv_dims=(32, 64, 128, 256),
            fc_dims=(512, 256), kernel_sizes=None, strides=None,
            activation='relu', dropout=0.0, layer_norm=False, device='cpu',
            pooled_size=4)
RNNEncoder(input_dim, latent_dim, hidden_dim=256, num_layers=2, rnn_type='lstm',
            bidirectional=True, fc_dims=(256,), activation='relu', dropout=0.0,
            layer_norm=False, device='cpu')
TransformerEncoder(input_dim, latent_dim, d_model=256, nhead=8, num_layers=6,
                   dim_feedforward=1024, dropout=0.1, fc_dims=(256,),
                   activation='relu', layer_norm=True, device='cpu',
                   max_len=5000)
VariationalEncoder(input_dim, latent_dim, hidden_dims=(256, 256),
                   activation='relu', dropout=0.0, layer_norm=False,
                   device='cpu') # forward -> (z, mu, logvar)
VariationalEncoder.kl_divergence(mu, logvar) -> Tensor # static, shape (B,)

MLPDecoder(latent_dim, output_dim, hidden_dims=(256, 256), activation='relu',
            dropout=0.0, layer_norm=False, device='cpu')
CNNDecoder(latent_dim, output_channels, fc_dims=(512, 256),
            conv_dims=(256, 128, 64, 32), kernel_sizes=None, strides=None,
            initial_size=4, activation='relu', dropout=0.0, layer_norm=False,
            device='cpu', output_activation='sigmoid') # property output_size
RNNDecoder(latent_dim, output_dim, hidden_dim=256, num_layers=2,
            rnn_type='lstm', max_seq_len=100, fc_dims=(256,), activation='relu',
            dropout=0.0, layer_norm=False, device='cpu')
TransformerDecoder(latent_dim, output_dim, d_model=256, nhead=8, num_layers=6,
                   dim_feedforward=1024, max_seq_len=100, dropout=0.1,
                   fc_dims=(256,), activation='relu', layer_norm=True,
                   device='cpu', max_len=None)
VariationalDecoder(latent_dim, output_dim, hidden_dims=(256, 256),
                   activation='relu', dropout=0.0, layer_norm=False,
                   device='cpu')
# RNNDecoder and TransformerDecoder: forward(z, seq_len=None)

```

A.9.5 Factories

```

PolicyFactory.create_policy(policy_type, **kwargs) # mlp cnn rnn transformer
ValueFactory.create_value_network(network_type, **kwargs) # mlp cnn rnn transformer
QNetworkFactory.create_q_network(network_type, **kwargs) # + dueling
EncoderFactory.create_encoder(encoder_type, **kwargs) # + vae
DecoderFactory.create_decoder(decoder_type, **kwargs) # + vae

```

The abstract base classes `BasePolicyNetwork`, `BaseValueNetwork`, `BaseQNetwork`, `BaseEncoder` and `BaseDecoder` are exported for subclassing and type checks.

A.9.6 NetworkUtils

All methods are static; Table 15.3 describes them.

```

initialize_weights(module, method='xavier_uniform', gain=1.0)
count_parameters(model) -> int
count_parameters_by_layer(model) -> dict[str, int]
freeze_layers(model, layer_names)
unfreeze_layers(model, layer_names)
get_grad_norm(model, norm_type=2.0) -> float
clip_grad_norm(model, max_norm, norm_type=2.0) -> float
create_mlp_layers(input_dim, hidden_dims, output_dim, activation='relu',
                  dropout=0.0, layer_norm=False, bias=True) -> nn.Sequential
create_conv_layers(input_channels, conv_dims, kernel_sizes=None, strides=None,
                  padding=None, activation='relu', dropout=0.0,
                  layer_norm=False) -> nn.Sequential
create_transposed_conv_layers(input_channels, conv_dims, kernel_sizes=None,
                              strides=None, padding=None, output_padding=None,
                              activation='relu', dropout=0.0,
                              layer_norm=False) -> nn.Sequential
create_attention_layer(input_dim, num_heads=8, dropout=0.1) -> nn.Module
create_positional_encoding(d_model, max_len=5000) -> nn.Parameter
get_activation(activation) -> nn.Module
apply_weight_decay(model, weight_decay, skip_list=None) -> list[dict]
create_optimizer(model, optimizer_type='adam', lr=0.001, weight_decay=0.0,
                 skip_list=None, **kwargs) -> torch.optim.Optimizer
create_scheduler(optimizer, scheduler_type='step', **kwargs)
save_checkpoint(model, optimizer, epoch, loss, filepath, **extra) -> Path
load_checkpoint(model, optimizer, filepath, map_location=None,
               weights_only=True, strict=True) -> tuple[int, float]
compute_output_size(input_size, kernel_size, stride=1, padding=0,
                    dilation=1) -> int
compute_transposed_output_size(input_size, kernel_size, stride=1, padding=0,
                               output_padding=0, dilation=1) -> int
compute_conv_output_size(input_size, conv_layers) -> tuple[int, int]

```

A.9.7 Tabular tools

```

QTable(state_space_size, action_space_size=None, initial_value=0.0,
       dtype=np.float32, rng=None)
ValueTable(...same arguments...)
PolicyTable(...same arguments...)           # rows start uniform
BaseDiscreteTable                           # abstract base of the three tables
DiscreteEnvironment(state_space_size, action_space_size, rng=None) # abstract

DiscreteTools.create_q_table(state_space_size, action_space_size,
                             initial_value=0.0, rng=None) -> QTable
DiscreteTools.create_value_table(state_space_size, initial_value=0.0,
                                 rng=None) -> ValueTable
DiscreteTools.create_policy_table(state_space_size, action_space_size,
                                  rng=None) -> PolicyTable
DiscreteTools.q_learning_update(q_table, state, action, reward, next_state,
                                gamma=0.99, alpha=0.1, done=False)
DiscreteTools.sarsa_update(q_table, state, action, reward, next_state,
                           next_action, gamma=0.99, alpha=0.1, done=False)
DiscreteTools.expected_sarsa_update(q_table, state, action, reward, next_state,

```

```

        policy_table=None, gamma=0.99, alpha=0.1,
        epsilon=0.0, done=False)
DiscreteTools.value_iteration_update(value_table, state, q_table, gamma=0.99)
DiscreteTools.policy_iteration_update(policy_table, state, q_table)
DiscreteTools.epsilon_greedy_policy(q_table, state, epsilon) -> int
DiscreteTools.softmax_policy(q_table, state, temperature) -> int
DiscreteTools.boltzmann_policy(q_table, state, temperature) -> int
DiscreteTools.ucb_policy(q_table, state, visit_counts,
        exploration_constant=1.0) -> int
DiscreteTools.thompson_sampling_policy(q_table, state, visit_counts,
        prior_alpha=1.0, prior_beta=1.0) -> int

```

A.10 toolkit.parakit

Chapter 13, Section 13.2.

```

get_parameters(parser) -> dict[str, Any]
save_parameters(values, path_or_dir) -> Path
load_parameters(path) -> dict[str, Any]
apply_parameters(parser, values, strict=True,
        validation_callbacks=None) -> argparse.ArgumentParser
parse_value(action, text, validator=None) -> Any
parse_bool(value) -> bool
format_value(action, value) -> str
describe_type(action) -> str
tunable_actions(parser) -> dict[str, argparse.Action]

ParameterTuner(parser, save_delay=30000, save_path=None, auto_save_enabled=True,
        validation_callbacks=None, inactivity_timeout=10)
ParameterTuner.tune() -> argparse.ArgumentParser
ParameterTuner.tune_parameters(parser, ...same arguments...)    # class method
ParameterAdjuster = ParameterTuner
TK_MISSING_MESSAGE                                             # str: how to install Tk

```

Appendix B

Migration from Earlier Versions

This appendix explains what code written for earlier versions of the package has to change. Section B.1 covers the step from version 1.1 to 1.2, which adds a package and moves the example scripts. Section B.2 covers the step from version 1.0 to 1.1, which adds features and changes little existing behaviour. Section B.3 covers code written for the versions before 1.0, when the package consisted of two separate projects. The complete list of changes is in `CHANGELOG.md`.

B.1 From 1.1 to 1.2

Version 1.2 adds the import package `marl_algorithms` (Chapter 12) and the `marl-train` command. The packages `env_lib` and `toolkit` are unchanged apart from their version number, so code written for version 1.1 runs without changes. Three points are new:

- `marl_algorithms` needs PyTorch, which the base installation does not include; install it with `pip install "my-tool-box[torch]"` (Section 2.2.2). Importing `marl_algorithms` itself works without PyTorch; using its algorithms or training functions raises an `ImportError` that names this command.
- The former folder `classsical_algorithm_project` held only a submodule pointer without code and was removed; its role is taken by `marl_algorithms`, whose MAPPO implementation replaces the `light_mappo` submodule.
- The example scripts moved into topic folders: `examples/getting_started/` (`quickstart.py`, `workflow_demo.py`), `examples/environments/` (the environment demonstrations), `examples/algorithms/` and `examples/toolkit/` (the `plotkit`, `parakit` and transformer examples). Scripts or notes that call them by path need the new folder.

Version 1.2.1 is a maintenance release. It requires Gymnasium 1.1 or newer (Gymnasium 1.0 lacks the same-step autoreset mode that the training loops use) and `imageio-ffmpeg` 0.5.1 or newer for video export. The names that need an optional extra (`KuramotoOscillatorEnvTorch`, `PistonballEnv`, the module `env_lib.pistonball_env` and `toolkit.neural_toolkit`) are still attributes of their packages but no longer part of `__all__`, so `from env_lib import *` works on a base installation; import them by name. `train`, `train_preset` and `compare` take `threads=1` and restore the previous PyTorch thread count when they return, where version 1.2 left the setting to the caller. Environments can be pickled and deep-copied after rendering, and several errors that were raised late or without a useful message are now reported early (Chapter 17). The Kuramoto and AJLATT environments gain reward settings (`reward_mode` and others, Sections 4.4 and 10.4) whose defaults reproduce the rewards of version 1.2 exactly; their step infos gain the keys `"synchronized"` and `"tracking_cost"`.

B.2 From 1.0 to 1.1

Version 1.1 adds the environments `PowerGrid-v0` and `Platoon-v0`, native vector environments, the catalogue and the `env-lib` command line, wrappers, baseline controllers, evaluation tools and graph utilities (Chapter 11). No public name was removed, and class names, module paths, constructor arguments and their defaults are unchanged. The following changes can affect code written for version 1.0.

Gymnasium 1.0 or newer. The package requires `gymnasium>=1.0` (version 1.0 accepted 0.29). The vector API of Gymnasium changed in 1.0: a finished copy of a vector environment is by default reset on the *next* call to `step()`, whereas Gymnasium 0.29 reset it within the same step, and in same-step mode the last observation is returned in `infos["final_obs"]` instead of Gymnasium 0.29's `infos["final_observation"]`. Code that ran vector environments with Gymnasium 0.29 has to follow these conventions (Section 11.1); single environments are not affected.

Unknown reset options warn instead of raising. In version 1.0 the Consensus, Formation and Kuramoto environments raised `ValueError` for a key of `reset(options=...)` that they did not know. They now ignore such keys with a `UserWarning`, as do the new `PowerGrid` and `Platoon` environments, so that generic tools can pass options of their own; the values of known keys are still validated and raise `ValueError` when invalid (Section 3.5.6). Code or tests that relied on the error to catch a misspelt key can turn the warning into an error:

```
import warnings
import numpy as np
import env_lib

env = env_lib.make("Consensus-v0")
with warnings.catch_warnings():
    warnings.filterwarnings("error", message=r".*unknown reset option")
    try:
        env.reset(seed=0, options={"position": np.zeros((8, 2))}) # misspelt
    except UserWarning as err:
        print(err)
```

max_episode_steps sets the environment's own limit. In version 1.0, `env_lib.make(id, max_episode_steps=N)` passed the argument on to `gymnasium.make`, which added a `TimeLimit` wrapper on top of the environment's own limit: the shorter of the two applied, so `env_lib.make("Consensus-v0", max_episode_steps=500)` still ended episodes after 200 steps, and for `AJLATT` the value never reached the configuration. `env_lib.make` and `env_lib.make_vec` now set the environment's limit argument (`max_steps`, `max_iter`, `max_cycles` or `max_episode_steps`; see `env_lib.get_spec(id).limit_kwarg`) instead, so the value given is the episode length and no `TimeLimit` wrapper is added. Consequently `env.spec.max_episode_steps` is `None`; read the limit from the environment, for example `env.unwrapped.max_steps`. Passing both `max_episode_steps` and a different value of the limit argument raises `ValueError`. `gymnasium.make` called directly behaves as before.

Native vector environments. The ids `Consensus-v0` and `Formation-v0` and the NumPy Kuramoto ids now register a vector entry point, as do the new `PowerGrid` and `Platoon`

ids. For these ids, `gymnasium.make_vec(id, n)`, which returned a `SyncVectorEnv` in version 1.0, and `env_lib.make_vec(id, n)` therefore return the native `ConsensusVectorEnv` or `KuramotoOscillatorVectorEnv`. They follow the same dynamics and the Gymnasium conventions of `SyncVectorEnv` (next-step autoreset, where the reset step ignores the copy's action and returns reward 0, both flags `False` and the info of the reset; batched infos with `"_key"` masks), but differ in the following points:

- All copies draw from one generator: copy i no longer reproduces a single environment reset with seed $s + i$ (Section 11.1).
- There is no `envs.envs` list of single environments; the batched state is available through properties such as `envs.positions` (Appendix A).
- `render()` returns the frame of copy 0 instead of a tuple with one frame per copy.
- The autoreset mode is a keyword argument of `make_vec` (`autoreset_mode="same_step"`, for every vectorisation mode), not an entry of `vector_kwargs`.

Pass `vectorization_mode="sync"` to keep the `SyncVectorEnv` of version 1.0.

AJLATT example controller. The function `encircle_policy(env, radius, gain)` of `examples/ajlatt_demo.py` was removed from the example. Its controller is part of the package as `env_lib.baselines.ajlatt_encircle` (a pure function of the observation) and is returned, with its parameters bound to the environment, by `env_lib.baseline_policy(env)`:

```
import env_lib
from env_lib import AJLATTEnv

env = AJLATTEnv(num_robots=4)
# Before (examples/ajlatt_demo.py of version 1.0): action = encircle_policy(env)
policy = env_lib.baseline_policy(env, radius=1.8, gain=1.2) # the old defaults
obs, info = env.reset(seed=0)
obs, reward, terminated, truncated, info = env.step(policy(obs))
```

The old function read the robots' pose and target estimates from the environment's internal filters; the packaged controller reads the same quantities from the observation, in the robot's frame. Its results are therefore close to, but not identical with, those of version 1.0: the two default episodes of `ajlatt_demo.py` return -7333.0 and -7113.1 instead of -7334.3 and -7170.4 .

The catalogue module is callable. `env_lib.catalog` is both a module and a function: `env_lib.catalog(...)` returns the catalogue whether or not the submodule has been imported, and its other names remain module attributes (`env_lib.catalog.describe`, `env_lib.catalog.EnvInfo`, `env_lib.catalog.Catalog`). `from env_lib.catalog import catalog` imports the function itself.

B.3 From versions before 1.0

Before version 1.0 the code lived in two separate projects, `envlib_project/env_lib` and `toolkit_project/toolkit`, each with its own `setup.py`, and several environments had their own installation files. Version 1.0 turns them into one distribution with a stable API, fixes a number of bugs and changes some behaviour. This section explains, module by module, what code written for these earlier versions has to change.

Most user code keeps working: class names, module paths, constructor arguments and their defaults are unchanged, and deprecated arguments are translated with a `DeprecationWarning`. Python shows these warnings only for code in `__main__` by default; run your scripts once with `python -W default::DeprecationWarning` to list everything that needs attention.

B.3.1 Checklist

1. Uninstall the old distributions and install `my-tool-box` (Section B.3.2).
2. Unpack (`observation`, `info`) from every `reset()` call and use the five-tuple of `step()`, including for AJLATT.
3. Create AJLATT through `env_lib.AJLATTEnv` with the new parameter names and adapt to the joint action and observation spaces (Section B.3.7).
4. For Pistonball in discrete mode, pass one value in $\{0, 1, 2\}$ per piston (Section B.3.6).
5. Treat `terminated` and `truncated` separately, and set episode limits through the constructor arguments (Section B.3.3).
6. Retrain `neural_toolkit` models; old checkpoints cannot be loaded (Section B.3.9).
7. If your figures relied on side effects of `plotkit` on the global `matplotlib` configuration, call `set_research_style()` explicitly (Section B.3.8).

B.3.2 Installation

Before, each project (and optionally each environment) was installed from its own folder:

```
cd envlib_project && pip install -e .
cd ../toolkit_project && pip install -e .
cd ../envlib_project/env_lib/pistonball_env && pip install -e .  # and similar
```

After, one installation from the repository root provides both import packages; optional features are selected with extras (Chapter 2):

```
pip uninstall -y env_lib toolkit
pip uninstall -y ttenv kos_env linemsg_env pistonball_env wireless_comm_env
pip install -e ".[all]"          # or ".[all,dev]" for development
```

The import names `env_lib` and `toolkit` are unchanged. TensorFlow, seaborn, pandas and scikit-learn are no longer required. Importing `env_lib` or `toolkit` no longer imports PyTorch, pygame, pymunk or Tk; the subpackages and environment classes are loaded on first use, so `import toolkit; toolkit.plotkit.plot_line(...)` and `from env_lib import PistonballEnv` continue to work.

B.3.3 Changes common to all environments

Reset and step. All environments follow the Gymnasium API. `reset(*, seed=None, options=None)` returns (`observation`, `info`), and `step(action)` returns (`observation`, `reward`, `terminated`, `truncated`, `info`). The `seed` argument must be passed by keyword.

```
# Before (AJLATT and old-style loops)
# obs = env.reset()
# obs, reward, done, info = env.step(action)
```

```
# After
obs, info = env.reset(seed=0)
obs, reward, terminated, truncated, info = env.step(env.action_space.sample())
done = bool(np.any(terminated)) or truncated
```

Seeding. `env.seed(s)` is deprecated where it existed (LineMsg, WirelessComm, Pistonball) and emits a `DeprecationWarning`; use `env.reset(seed=s)`. No environment seeds the global NumPy or PyTorch generators any more (the Kuramoto environments used to call `np.random.seed(42)` and `torch.manual_seed(42)`); code that relied on these side effects for its own random numbers must seed its own generator.

Episode limits and termination. The registered ids no longer add a `TimeLimit` wrapper. Previously, for example, `KuramotoOscillator-v0` was truncated after 1000 steps by the wrapper even if a larger `max_steps` was passed to `gym.make`; now the constructor argument (`max_steps`, `max_iter`, `max_cycles` or `max_episode_steps`) is the only limit, and reaching it sets `truncated=True`. Task success (for example synchronisation of the oscillators) sets `terminated=True`; previously some environments reported both cases as `terminated`.

Registration. Environments are registered when `env_lib` is imported; the `register()` functions of `env_lib.kos_env` and `env_lib.wireless_comm_env` are deprecated no-ops that call the central registration. `env_lib.make(id)` and `env_lib.list_envs()` are new. The id `AJLATT-v1`, an unusable alias of `AJLATT-v0`, was removed.

Observations, rewards and rendering. Observations are `float32` and lie in `observation_space`; per-agent rewards are available in `info["agent_rewards"]`. Rendering happens only on request: in "human" mode the environment renders itself in `reset()` and `step()`, so explicit `env.render()` calls in "human" loops are no longer needed, and "rgb_array" works without a display. `step()` before the first `reset()` now raises an error instead of resetting implicitly.

B.3.4 Kuramoto oscillators

- *PyTorch backend.* A sign error made the coupling repulsive ($\sin(\theta_i - \theta_j)$ instead of $\sin(\theta_j - \theta_i)$). Results obtained with `KuramotoOscillatorEnvTorch` before 1.0 are therefore not comparable with new ones. Both backends now produce the same trajectories.
- *Normalisation.* Division of the coupling sum by N is opt-in for both backends (`normalize_coupling=True`). Code that relied on the normalisation of the old PyTorch backend must set this flag.
- *Coupling matrix.* The NumPy backend used to skip entries $K_{ij} \leq 0$; the full matrix is used now. Default settings are not affected.
- *Constant coupling.* `coupling_mode="constant"` without a matrix now uses `coupling_strength * adjacency`, which makes the registered `*-Constant-v0` ids usable (they raised before).
- *Episode end.* Synchronisation ($r > \text{sync_threshold}$) sets `terminated`; `sync_threshold` and `sync_bonus` are constructor arguments. The PyTorch backend accepts `device="auto"`, and its phase coherence and bonus rules now match the NumPy backend.

B.3.5 LineMsg and WirelessComm

Seeded trajectories are bit-identical to the previous implementations, so existing results remain valid. The text output of the old renderers is available as `render_mode="ansi"; "human"` and `"rgb_array"` show the new dashboards. LineMsg accepts actions either as an integer in `Discrete(2**N)` or as an array of N bits, and `action_space_type="multibinary"` supports more than 62 agents. WirelessComm validates its arguments and actions and reports per-agent outcomes (idle, success, collision, lost) in `info["outcomes"]`.

B.3.6 Pistonball

Discrete actions. The discrete action space is `MultiDiscrete([3] * n_pistons)`: one value per piston, 0 moves the piston down, 1 keeps it still and 2 moves it up. Earlier versions declared the same array format as `Box(0, 2, (n_pistons,), int32)`, and some earlier documentation used a single base-3 integer in `Discrete(3**n_pistons)`. Code that sends arrays keeps working; code that encodes a single integer must decode it:

```
import numpy as np
import env_lib

env = env_lib.PistonballEnv(n_pistons=5, continuous=False)
obs, info = env.reset(seed=0)

# Before: one integer for all pistons (base-3 encoding)
code = 13
# After: one value per piston; decode an old integer action like this
action = np.array([(code // 3**i) % 3 for i in range(env.n_pistons)]) # [1 1 1 0 0]
obs, reward, terminated, truncated, info = env.step(action)
```

Other changes. Actions are validated (shape, range, NaN). The screen is only drawn when rendering is requested; the methods `draw()`, `draw_background()`, `draw_pistons()` and `enable_render()` are deprecated no-ops, and the PNG sprites were replaced by a vector renderer. The environment works with pymunk 6 and 7.

B.3.7 AJLATT

AJLATT was rewritten as a Gymnasium environment configured by a dataclass. The old factory called `argparse.parse_args()` internally, which failed in any script with its own command-line arguments.

Creating the environment. The legacy factory still works; the class and the configuration dataclass are the preferred interface:

```
import env_lib
from env_lib import AJLATTConfig, AJLATTEnv

# Before (still works, with DeprecationWarnings for the legacy names)
env = env_lib.ajlatt_env(map_name="obstacles04", num_Robot=4, T_steps=100)

# After
env = AJLATTEnv(map_name="obstacles04", num_robots=4, max_episode_steps=100)
```

```
# Or with an explicit configuration object
config = AJLATTConfig(map_name="obstacles05", num_robots=3)
env = AJLATTEnv(config, render_mode="rgb_array")
```

Table 17.2 lists the renamed parameters. Rendering is off by default; the old `render=True` flag becomes `render_mode="human"`, and `render_mode="rgb_array"` records frames without a display.

Spaces, rewards and termination. The spaces are *joint*: `action_space` has shape `(num_robots, 2)` (linear and angular velocity of each robot) and `observation_space` shape `(num_robots, obs_dim)`, both `float32`. The per-robot spaces used by the old version are available as `single_action_space` and `single_observation_space`. The reward and `terminated` are per-robot arrays; wrap the environment in `TeamRewardWrapper` for a scalar team reward and termination flag.

```
import numpy as np
from env_lib import AJLATTEnv
from env_lib.ajlatt_env import TeamRewardWrapper

env = AJLATTEnv(num_robots=4)
print(env.single_action_space.shape, env.action_space.shape)    # (2,) (4, 2)

obs, info = env.reset(seed=0)                                   # before: obs = env.reset()
actions = np.stack([env.single_action_space.sample() for _ in range(4)])
obs, reward, terminated, truncated, info = env.step(actions)
print(reward.shape, terminated.shape)                           # (4,) (4,)

team_env = TeamRewardWrapper(AJLATTEnv(num_robots=4))
obs, info = team_env.reset(seed=0)
obs, team_reward, done, truncated, info = team_env.step(actions) # float, bool
```

Behaviour changes. The following changes affect results:

- The target-velocity entries of the observation contain the commanded target velocity instead of the configured constant.
- Covariance intersection uses a Newton solver by default; its objective is never worse than that of the previous SLSQP solver. Pass `ci_solver="slsqp"` to reproduce results of earlier versions (to about 10^{-6}).
- Target policies are per environment instance and selected with `target_policy`; they used to be shared class attributes that were not reset between episodes. Unknown map names raise an error instead of prompting with `input()`.
- Covariance traces were appended to lists that were never cleared (a memory leak). Per-episode traces are returned by `env.episode_statistics()`.
- Dynamic maps are regenerated at every `reset()` and no longer need `scikit-image`.

Training scripts. The PPO, LLM and debugging options that used to live in the `AJLATT` parameter module were removed from the environment package. To expose the environment options on a script's own parser, use `AJLATTConfig.add_arguments`:

```
import argparse
```

```

from env_lib import AJLATTConfig, AJLATTEnv

parser = argparse.ArgumentParser()
parser.add_argument("--lr", type=float, default=3e-4)           # your own options
AJLATTConfig.add_arguments(parser, prefix="env.")              # --env.num_robots, ...
args = parser.parse_args(["--env.num_robots", "3"])
config = AJLATTConfig.from_namespace(args, prefix="env.")
env = AJLATTEnv(config)

```

Maps. The map tools moved to `env_lib.ajlatt_env.maps` and are available as the console scripts `ajlatt-build-map` and `ajlatt-plot-map` instead of the scripts `build_map.py` and `plot_map.py`. The old builder wrote transposed grids; rebuild custom maps from their specification with the new builder. Maps whose grid is one row shorter than the header declares are padded with a wall row and a warning (three bundled maps had this problem and were fixed).

B.3.8 plotkit

The function names and positional arguments are unchanged; new options are keyword-only arguments appended to the signatures.

- *No global side effects.* The plot functions used to call `plt.style.use("default")` and change `rcParams`, which also affected later, unrelated figures. They now apply their style locally. Code that relied on the side effect should call `set_research_style()` once (and `reset_style()` to undo it). `set_research_style` no longer sets `figure.dpi=300`, which made on-screen figures very large; it still sets `savefig.dpi=300`.
- *Fixed input handling.* `plot_line(x, [y1, y2])` plots both series (it plotted only the first), and a flat list of numbers is one series (it was treated as several). Several bar series are grouped side by side instead of overlapping.
- *New features.* `band` and `smoothing` in `plot_shadow_curve`, NaN-robust statistics, `plot_learning_curves`, `plot_histogram`, `save_figure`, style presets, the Okabe-Ito palette and the gallery command. The heatmap functions no longer need `seaborn`.

```

from toolkit.plotkit import plot_line, reset_style, set_research_style

# Before: calling any plotkit function changed the global style as a side effect.
# After: opt in explicitly when plain matplotlib code should use the research style.
set_research_style()
ax = plot_line([0, 1, 2], [[0, 1, 4], [0, 2, 3]], labels=["a", "b"]) # two series
reset_style()

```

B.3.9 neural_toolkit

Retrain models. Earlier MLP stacks applied no activation between hidden layers, and several other defects changed the computed functions (Section 15.8). The layer structure of every network changed, so checkpoints written before 1.0 cannot be loaded; retrain and save new checkpoints.

API differences. Besides the fixes, the following details changed:

- Unknown activation names raise `ValueError` instead of silently falling back to a default, and the `device` argument is honoured (modules are moved to it at construction).
- Q-networks honour the `action` argument: `q(s, a)` returns $Q(s, a)$ with shape `(B,)` for integer actions.
- `NetworkUtils.load_checkpoint` loads with `weights_only=True` and maps tensors to the model's device by default. Checkpoints that contain arbitrary Python objects in the extra entries need `weights_only=False` (only for trusted files). `save_checkpoint` accepts `optimizer=None` and extra keyword entries, and returns the path.
- The method `create_positional_encoding` of `NetworkUtils` returns the frozen table as an `nn.Parameter` of shape `(1, max_len, d_model)`; use `PositionalEncoding` for a module that adds it to its input.
- The tabular tools draw random numbers from a per-table generator (`rng` argument) instead of the global NumPy state.

Names from earlier documentation. The documentation of earlier versions described some names that never existed in the code. Table B.1 lists the actual equivalents.

Table B.1: Names used in pre-1.0 documentation and their actual equivalents.

Documented before 1.0	Use
<i>Static methods of NetworkUtils</i>	
<code>count_trainable_parameters</code>	<code>count_parameters</code> (counts trainable parameters)
<code>init_weights</code>	<code>initialize_weights</code>
<code>clip_gradients</code>	<code>clip_grad_norm</code>
<code>get_device, to_device</code>	plain PyTorch: <code>torch.device(...)</code> , <code>module.to(device)</code>
<i>Encoder arguments</i>	
<code>output_dim</code> of <code>CNNEncoder</code> and <code>RNNEncoder</code>	<code>latent_dim</code>

B.3.10 parakit

`ParameterTuner` (and its alias `ParameterAdjuster`) keeps its constructor and `tune_parameters` class method, but its behaviour was corrected:

- `tune()` installs the accepted values as the parser's defaults, as it was documented to do; previously the returned parser was unchanged.
- Boolean strings (the text `"False"` used to be stored as a string, which is true in a boolean context), lists (`nargs`), `None` values and `store_true` options are parsed correctly.
- Validation callbacks receive the *converted* value. Callbacks written for the old convention, which received text and returned `(is_valid, converted_value, message)`, keep working as long as they also accept an already converted value (for example `float(value)` of a float); their converted value is ignored.
- `import toolkit.parakit` works without Tk; the window imports Tk only when it opens. The library no longer calls `logging.basicConfig`, so configure logging in your script if you want to see its log messages.

- New headless functions (`get_parameters`, `save_parameters`, `load_parameters`, `apply_parameters`, `parse_value`) make the GUI optional (Section 13.2).

```
import argparse
from toolkit.parakit import apply_parameters, load_parameters, save_parameters

parser = argparse.ArgumentParser()
parser.add_argument("--learning_rate", type=float, default=1e-3)

# Before: validators received text and returned a 3-tuple
def validate_lr_old(value):
    lr = float(value)
    return (0 < lr <= 1), lr, "learning rate must be in (0, 1]"

# After: validators receive the converted value; a bool or (ok, message) suffices
def validate_lr(value):
    return 0 < value <= 1, "learning rate must be in (0, 1]"

path = save_parameters({"learning_rate": 0.01}, "runs/params")
apply_parameters(parser, load_parameters(path),
                 validation_callbacks={"learning_rate": validate_lr_old})
apply_parameters(parser, {"learning_rate": "0.02"},
                 validation_callbacks={"learning_rate": validate_lr})
```